Web-based Piano Music Tutoring using Artificial Intelligence

Tsz-ying Cheng*[1], Chi-wang Cheung†[1], and Chuck-jee Chau‡[2]

[1]Department of Computer Science and Engineering, The Chinese University of Hong Kong, Shatin, N.T., Hong Kong

Abstract

This project focuses on building a web application that would help users improve their piano-playing ability and musicality through performing analysis on different piano pieces or performances and then presenting the analytics in an understandable and meaningful way. The application has three major functionalities: performance analysis, emotion analysis and music piece analysis.

The performance analysis function is designed to provide feedback on users' piano practices. It transcribes users' performances from practice recordings, compares them against the music score, and offers scoring and improvement suggestions. The emotion analysis function examines the emotional changes in a music recording, utilizing a valence-arousal graph. As for the music piece analysis functions, they classify the music style, recreate it in a different style, and generate piano fingerings to facilitate users in playing the piece effectively.

*ctychristy@link.cuhk.edu.hk
†hugocheung1210@link.cuhk.edu.hk
‡chuckjee@cse.cuhk.edu.hk

Table of Contents

Table of Contents **2**

1 Introduction **4**

2 Background **6**
2.1 General Music Terms 6

3 Web Application **12**
3.1 Requirements 12
3.2 System Architecture 15
3.3 Storage 18
3.4 User Experience Design 21
3.5 Content 28

4 Performance Analysis **51**
4.1 Function Overview 51
4.2 Sample Music Pieces and Recordings 51
4.3 Dynamics Extraction 55
4.4 Piano Fingering Extraction 57
4.5 Tempo Detection 77
4.6 Pitch Detection 86
4.7 Analysis Result Unification 102
4.8 Performance Metrics 103

5 Emotion Classification **109**
5.1 Related Works 109
5.2 Dataset 110
5.3 Feature extraction 111
5.4 Model Training 113
5.5 Result 116
5.6 Visualization of result 120
5.7 Conclusion 122

6 Music Piece Analysis **123**
6.1 Style Classification 124
6.2 Style Transfer 135
6.3 Piano Fingering 146

7 Conclusion **160**
7.1 Summary 160
7.2 Evaluation 160

8 References **162**

9 Appendix **169**
9.1 Python Application with Graphical User Interface (Term One) 169

Work Distribution

Sections 1, 3, 4.4 and 6.3 are authored by CHENG Tsz Ying.

Sections 4.3, 4.5, 4.6, 5, 6.1 and 6.2 are authored by CHEUNG Chi Wang.

All remaining sections are co-authored by all members.

1 Introduction

The major goal of the system is to help users improve their ability to play the piano and musicality through performing analysis on different piano pieces or performances, and then present the analytics in an understandable and meaningful way. It is implemented as a web application that contains three major functionalities, namely performance analysis, emotion analysis and music piece analysis.



Figure 1: Application overview.

The performance analysis function analyzes users' piano-performance video, extracts information including pitches, fingering, tempo and dynamics, and then presents them along with the video in an intuitive and easily interpretable manner. All past performance scores of each piece are also listed on an overview page, which allows users to understand their practice progress and improvements along the way.

The emotion analysis function takes a piano music snippet and analyzes the change in emotions conveyed throughout the music. It helps users to build up a better ability to understand the underlying emotions and messages of the music, and how they can adjust their performances to alter the emotions expressed.

The music piece analysis function takes a MIDI file of a music piece and analyze it by first classifying which music style it is, which music genre it belongs to, and a set of optimal piano fingerings for users to play the piece. Users can also choose to transfer the music piece into another style, such as from classical to baroque. The aim of this is to enhance the musicality of users and also facilitate them to play a new piece and understand it more.

We aimed to present the analytics and feedback in a way that is easily understandable even for beginners. Therefore, we have utilized elements such as line and polar charts, and piano scores for visual representation of data. Also, some simple representations of results are provided, such as some scoring out of 100, and textual feedback corresponding to the performance. It gives a quick and comparable overview of how well the performance is.

Hybrid Development We adopted a hybrid of the waterfall and agile development approaches in our development (Figure 2). This includes a requirement analysis and project planning from the waterfall model, and the development-design-implementation-testing cycles from the agile approach.

This is because the project involves a web application and three major analysis functions that come with data and charts in various formats. Therefore, careful requirement analysis and planning ahead are crucial for arranging our content appropriately and enabling a smooth development process. Meanwhile, the project time is limited and many functions are developed in parallel. An agile approach at later stages allows us the flexibility to promptly make appropriate changes to our design and implementations.

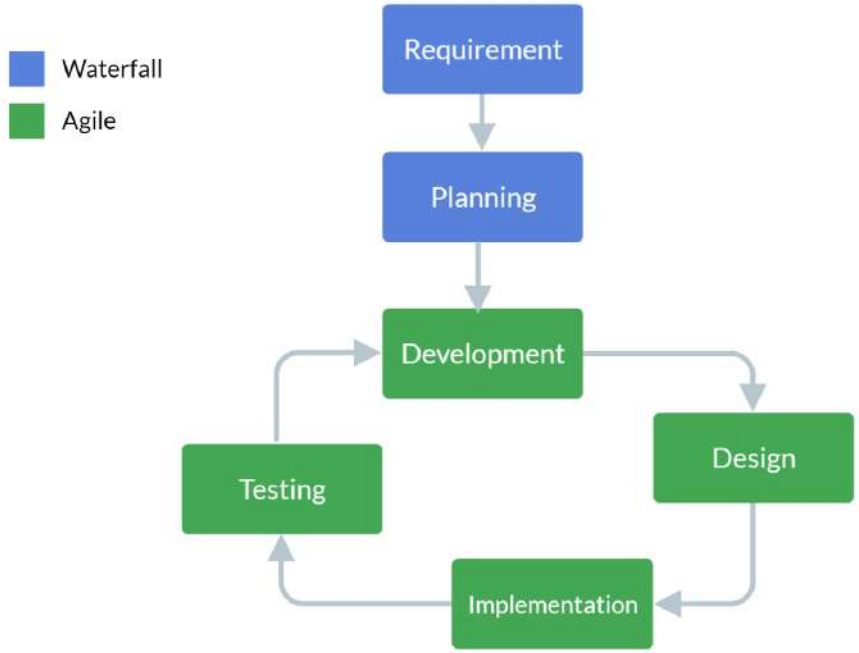


Figure 2: Hybrid Development Approach.

2 Background

2.1 General Music Terms

2.1.1 Pitch, Notes, Key

Pitch is a perceptual property of sound in which the listener can assign musical tones based on a musical scale. Pitch can be quantified as frequency and perceived in a logarithmic manner. However, the pitch is not necessarily an objective property but it can be a subjective psychoacoustical attribute of sound.

Notes in music is a concept of building blocks of music. A note is composed of various information such as pitch class, the duration of the note, the sound level of the note, etc. Notes can also be visually presented by writing in musical notation.

Key is the overall use of pitches that form the basis of a musical composition. The key can be specified in major mode or minor mode, where the major key often represents a brighter sound for the music but music with the minor key often is considered as "serious or melancholic".

2.1.2 Piano

The piano is the instrument we are focusing on in this project. A pianist creates musical notes by pressing keys on a piano keyboard (Figure 3). A keyboard is formed by a sequence of keys, where each piano key corresponds to a note. The pitches of notes increase from left to right of the keyboard.



Figure 3: A standard design of a piano keyboard.

The white keys correspond to "natural notes" while black keys are the "sharps" or "flats" (shifting the pitch by a half step) of their neighbouring white keys. A pianist usually plays with both hands, where the left and right hands are responsible for the keys lower and higher than middle C (C4) respectively.

2.1.3 Piano Fingering

Piano fingering refers to the choice of which fingers to use when playing each note on the piano. A set of good piano fingering helps pianists navigate the keyboard and play the notes efficiently and comfortably. In this project, indices would be used to denote fingers.

The indices are thumb(1), index(2), middle(3) ring(4), and little (5) fingers (Figure 4). A sequence of N fingerings is denoted as finger1-finger2-...-fingerN (e.g. 1-2-3) in this project.

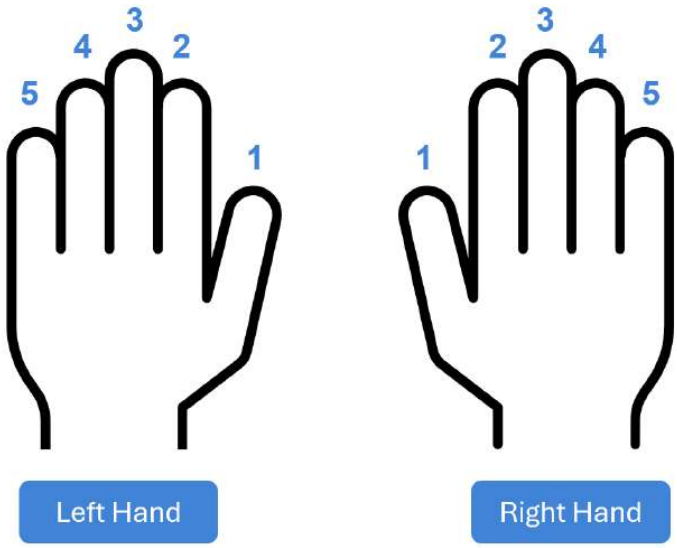


Figure 4: The indices of fingerings.

2.1.4 Amplitude, Decibel

Amplitude is an essential concept in signal processing that represents the strength or magnitude of a wave. In music, the amplitude of a sound wave also refers to the sound level and strength of the sound wave. Amplitude is measured in decibels (dB), which refers to the sound pressure level or intensity.

2.1.5 Monophonic, Polyphonic

The words monophonic and polyphonic come from Greek where "mono" means one, "poly" means many, and "phonic" means voice. In terms of music, monophonic music only allows one note to be expressed every time while polyphonic music is enabled to play multiple notes at one time. In later sections of our project, we will dive into monophonic pitch detection and polyphonic pitch detection.

2.1.6 Fourier Transform

Fourier Transform is a frequently used method in signal processing. In the field of audio signal processing, as the sound wave is stored as a discrete-time signal in the digital system, therefore discrete Fourier Transform (DFT) is performed to transform the sound wave from the time domain (amplitude vs. time) into the frequency domain (magnitude vs. frequency). It is essential in extracting the frequency information from the audio. The discrete Fourier Transform formula is expressed as:

$$X_k := \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i k n}{N}}$$

where X_k series is called the DFT coefficients and $x_n := x_0, x_1, ..., x_{N-1}$.

Apart from only presenting the data in the frequency domain, we can also include the time information by using a Short-time Fourier Transform (STFT). STFT breaks the process into multiple DFTs in time segments and hence the result is a spectrogram where the magnitude is presented in color dimension. Based on the auditory-based mel-frequency scale, the mel-spectrogram or in other words, the power spectrogram provides better resolution for lower frequencies than the spectrogram and therefore a mel-spectrogram as shown in Figure 5 is used instead of the normal spectrogramn in our project.

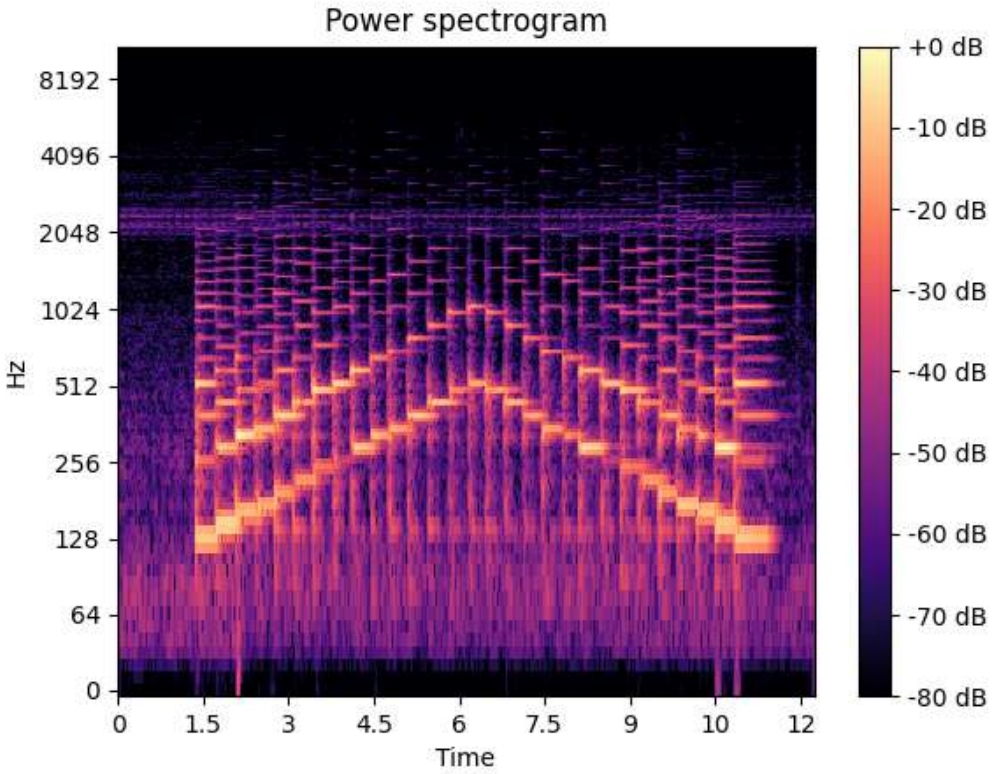


Figure 5: Sample spectrogram.

2.1.7 Piano Score

A Piano score is a printed form of a piano musical work that presents the musical notation to a musician about how to play the music piece. A piano score is composed of multiple parts such as musical staff, clefs, notes, time signature, dynamics marking, etc.

The fundamental building block of musical notation is the five-line musical staff. The lines and spaces on the staff represent different notes. Bar lines running perpendicular to the staff differentiate musical measures or in other words "bars". Double bar lines separate sections within the musical piece and bolded double bar lines represent the end of the piece. A five-line musical staff with different bar lines is shown in Figure 6.

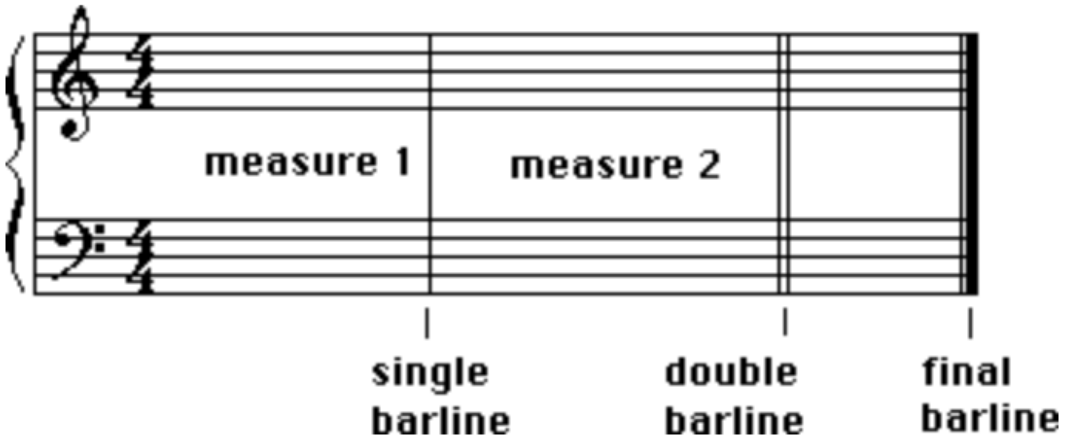
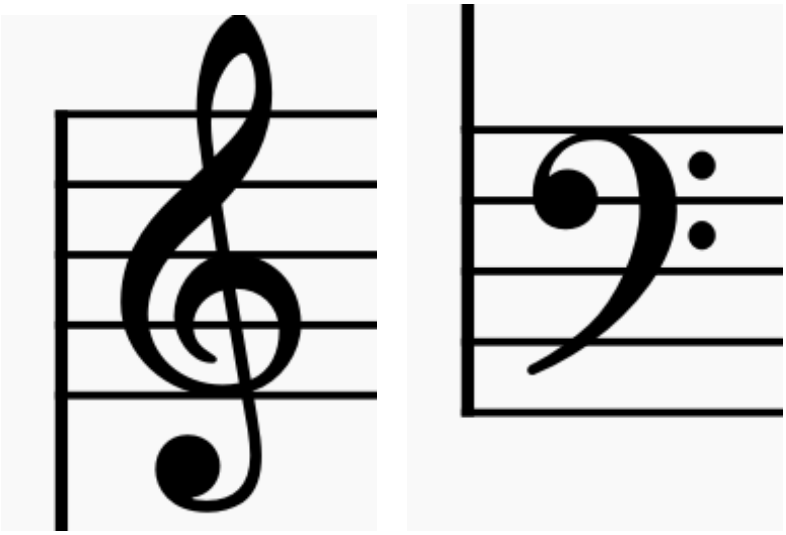


Figure 6: A five-line staff with different types of bar lines [1].

The lines and spaces of the different clef represent different notes. There are two main musical clef in piano score. A treble clef is also called a G clef and the right hand typically plays the notes on the treble clef in piano music. A bass clef is also called an F clef and the left hand typically plays the notes on the bass clef in piano music. For better illustration, we have computed the images of two kinds of clefs which are shown in Figure 7.



(a) A treble clef (b) A bass clef

Figure 7: Two main types of clef.

2.1.8 Musical Instrument Digital Interface

Musical Instrument Digital Interface (MIDI) is a kind of symbolic representation of musical data. In our project, MIDI files of different pieces are used to extract the ground truth in tempo analysis and pitch detection. The basic message of MIDI data consists of "Note On/Off", "Velocity", and "Note Number". "Note On" denotes the starting point of a note playing, whereas "Note Off" is the ending point of a note playing. "Velocity" refers to how fast is the key being pressed. The note number can be calculated from:

$$\text{Note number} = 69 + 12 * \log_2(\frac{\text{frequency}}{440 \text{ Hz}})$$

2.1.9 Music Eras

In classical music history, there are 6 music eras, namely Medieval, Renaissance, Baroque, Classical, Romantic, and 20th/21st Century (Modern), with each fitting into an approximate time frame. In our project, we mainly need to know about some differences between Baroque, Classical, Romantic, and Modern era music.

Baroque music is known for its complex polyphonic textures with multiple melodic lines. It follows specific structures like the fugue, suite, and concerto grosso. While for music in the Classical era, the structure and melodies are much simpler compared to that of other music eras. Moreover, the composition is rather symmetric and balanced for better clarity. On the other hand, Romantic music emphasizes expressiveness and emotion. These could be represented and observed in the frequent changes in dynamics and tempo. Lastly, for music in the Modern era, the format and structure are much more innovative with experimental approaches. It is also focusing more on the timbre and texture of the music.

2.1.10 Arousal and Valence

The valence and arousal model, proposed by James A. Russell [2], is a framework used to describe and measure the emotional experiences of individuals. It provides a way to understand emotions based on two primary dimensions: valence and arousal.

Valence refers to the pleasantness or unpleasantness of an emotional state. It ranges from positive valence (indicating positive emotions such as happiness, joy, or contentment) to negative valence (representing negative emotions like sadness, anger, or disgust). The valence dimension allows us to assess the overall emotional quality of an experience. Arousal, on the other hand, pertains to the level of physiological and psychological activation associated with an emotional state. It spans from low arousal (indicating calmness or relaxation) to high arousal (representing excitement, fear, or intense stimulation). The arousal dimension captures the intensity or energy level of an emotional experience.

By combining valence and arousal, the model creates a two-dimensional emotional space. Different emotions can be plotted within this space based on their valence and arousal levels, allowing for a visual representation of emotional states.

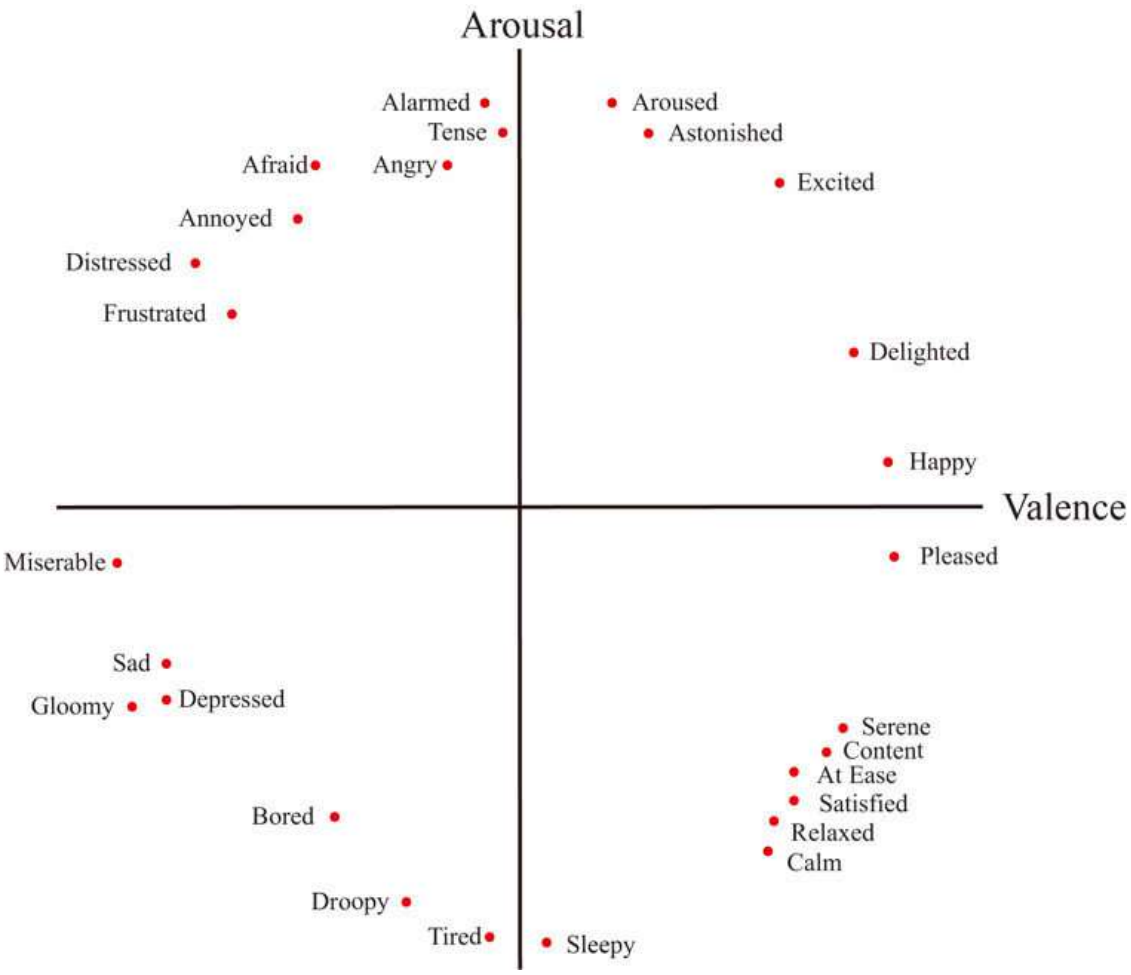


Figure 8: Model of valence and arousal[2].

The valence and arousal model has been widely used in various fields, including psychology, neuroscience, and human-computer interaction, to understand and measure emotional responses. It provides a useful framework for analyzing and comparing different emotional states based on their valence and arousal characteristics.

3 Web Application

This section covers the architecture, functionality and implementation of the web application. For technical details for the analysis functions, please refer to sections 4, 5 and 6 . All parts of this section are written in term two.

The web application serves as an interface between the analysis functions and the user. Figure 9 shows the page structure of the application, where each leaf node represents a page.

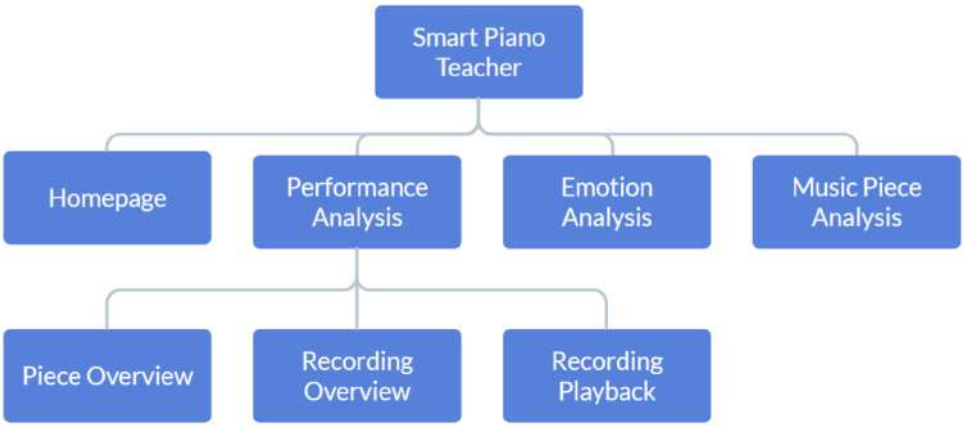


Figure 9: Overview of all pages in the web application.

3.1 Requirements

3.1.1 Target User

The target users of the application are piano learners whose main goal is to improve their ability to play the piano through receiving feedback for their piano practices.

The users are expected to be able to carry out piano practice on their own. This means that they shall have the basic ability to understand musical notations of piano scores and play them on a piano. On the other hand, they are likely amateurs or intermediate learners who require more advanced pianists, most often piano teachers, to provide feedback on their practice and guidance to their learning.

Advanced pianists would already possess the ability to perform in-depth analysis of piano music, and independently design and carry out practices. Therefore, they are not the target users of our application.

The targeted users may look for an affordable and accessible alternative to piano teachers that offers feedback and guidance on improving their piano-playing ability. The feedback provided should be meaningful and useful, akin to that of a piano teacher, by identifying both strengths and weaknesses in performances, and offering suggestions for enhancement,

such as adjusting tempo or modifying fingering techniques for challenging sections of the piece.

3.1.2 Background Study

This section further investigates what is included in piano skills, and explores what functions may be provided to improve one's piano skills.

In his work that evaluates the evolution of Western piano education, Arshinova summarized piano skills into a model: "I see-I play-I hear" and identified three main tasks of piano teachers [3].

"I see" refers to the ability to read musical notations and improve music literacy, which allows for the subjective interpretation and evaluation of music pieces. I play refers to the ability to first accurately reconstruct music, then develop into the ability to convey emotions and personal style in performances. In order to accurately express the music with intended pitch, tempo and dynamics, one would need to have motor skills, which include good posture, finger control and musical memory. I hear refers to one's aural skills, which is the ability to hear and correct mistakes.

This can be briefly grouped into one's ability to perform a piano piece, and musicality to support an in-depth interpretation of music pieces.

As suggested by Coffman [4], physical practice is the most effective compared to mental practice and knowledge of results on piano performance, for it results in superior psychomotor skill improvement. Therefore, the application should facilitate users to carry out physical practices. Thus, the first step would be to determine what to practice.

In his work, Dyal found out that a balanced program of piano lessons, which includes the variety of music pieces selected, is a main factor contributing to a successful piano experience [5]. A good selection of pieces, which includes music variety and an appropriate level of difficulty is important.

During piano practice, pianists often engage in repetitive practice of individual sections. According to Maynard, advanced musicians exhibit a greater ability to identify and target specific passages for practice compared to less advanced musicians [6]. This indicates that advanced musicians are better at recognizing areas of weakness in their performance. It involves the ability to evaluate performance quality and accurately identify errors that require attention. Similarly, Duke has highlighted the significance of a pianist's ability to discover errors and correct them efficiently on the effectiveness of piano practices [7].

Meanwhile, less skilled pianists would require others to evaluate their performances. Therefore, the application should be capable of providing feedback to piano practices. Kim et al. proposed the use of a feedback model in piano education, which includes goal-setting, finding gaps and giving feedback [8].

A good goal should satisfy the SMART framework, which is to be specific, measurable, attainable, relevant, and time-bound, according to George T. Doran [**?**]. Our application can facilitate users' goal-setting by providing some metrics for the performances that are specific and measurable. Such metrics should also be relevant and reflect the gaps (errors) discovered in the performance.

It is important to note that the feedback should not only contain the evaluation but also solid suggestions to improve their practice methods. A good practice procedure can significantly improve the practice's effectiveness and efficiency, as from the findings of Dyal [5] and Duke [7].

Apart from providing good feedback for individual practice, motivating users to perform practices consistently is also critical. Rubin-Rabason compared the effectiveness of retention of mass and distributed practice and found that distributed practice is more effective in improving one's piano performance [9]. This illustrates the importance of motivating users to carry out consistent practices.

3.1.3 Requirement Summarization

Based on the study above, this part summarizes some requirements for the system, and how the application can satisfy these features.

Piano Piece Selection The application should facilitate users to select what piano piece to practice. The performance analysis function is available for 10 selected piano pieces of varied difficulties and musical styles. They could be some good initial pieces to learn. Also, the music piece analysis functions help visualize the piano score and analyze its musical style, which helps users to know more about the piece and determine if they would like to practice it.

Evaluation and Error Identification The application should be capable of evaluating a piano performance, and provide useful insights which the user can understand and improve based on that. The performance analysis function provides a series of metrics, together with scoring on four different criteria and performance transcription with wrong notes highlighting. These locate errors for users and help them identify areas of improvement.

Practice Method Suggestions The application should provide suggestions or tips for improving the practice procedure for users to follow. Apart from scores on various criteria, the comments on performance analysis would include suggestions or methods users can adapt to refine their practices.

Clear Goal and Metrics The application should identify specific and measurable goals for users to aim for when refining their performances. The quantization of performance using scores and metrics provides users with a clear and specific goal, which is to improve the scores across all criteria.

Motivate Consistent Practice In the performances' comments, some motivational sentences and words of encouragement are included. Also, the overall scoring of all practices is presented on the piano piece's overview page. It visualizes the users' progress and improvements and fosters a sense of accomplishment, fueling their motivation to progress further by practising more.

Nurture User Musicality The application should provide functions that can help users build their musicality. The music piece analysis and emotion analysis functions can enhance one's sensitivity to the styles of different musical eras and emotions conveyed in music. It encourages users to explore piano pieces out of the provided set and build up their ability to evaluate and compare music pieces.

3.2 System Architecture

3.2.1 Term One Architecture

In the first term, we used Python to build an application with a graphical user interface (GUI) (Figure 10).

This is because we only aimed for a simple application prototype for proof-of-concept of the overall flow by that time, so using the same programming language allows the easiest integration. The application proved that the concept is feasible, and together with the increase in the number of functions, we have decided to switch to the current architecture. More details of the Python application can be found in the appendix (Section 9.1).

3.2.2 Current Architecture

The MERN stack (MongoDB [10], Express [11], React [12], and Node.js [13]) is used for building the frontend and most backend components of the web application, while Python is used for performing analysis. The construction of this architecture is an extension of the skills acquired from the course CSCI2720 (Building Web Applications). The system architecture is illustrated in Figure 11.

Frontend The user interacts with the system purely through the frontend interface. The frontend interface is built using React [12], a JavaScript library that conveniently combines HTML, CSS and JavaScript in one set of syntax, and supports features including component-based UI elements and dynamic data rendering.

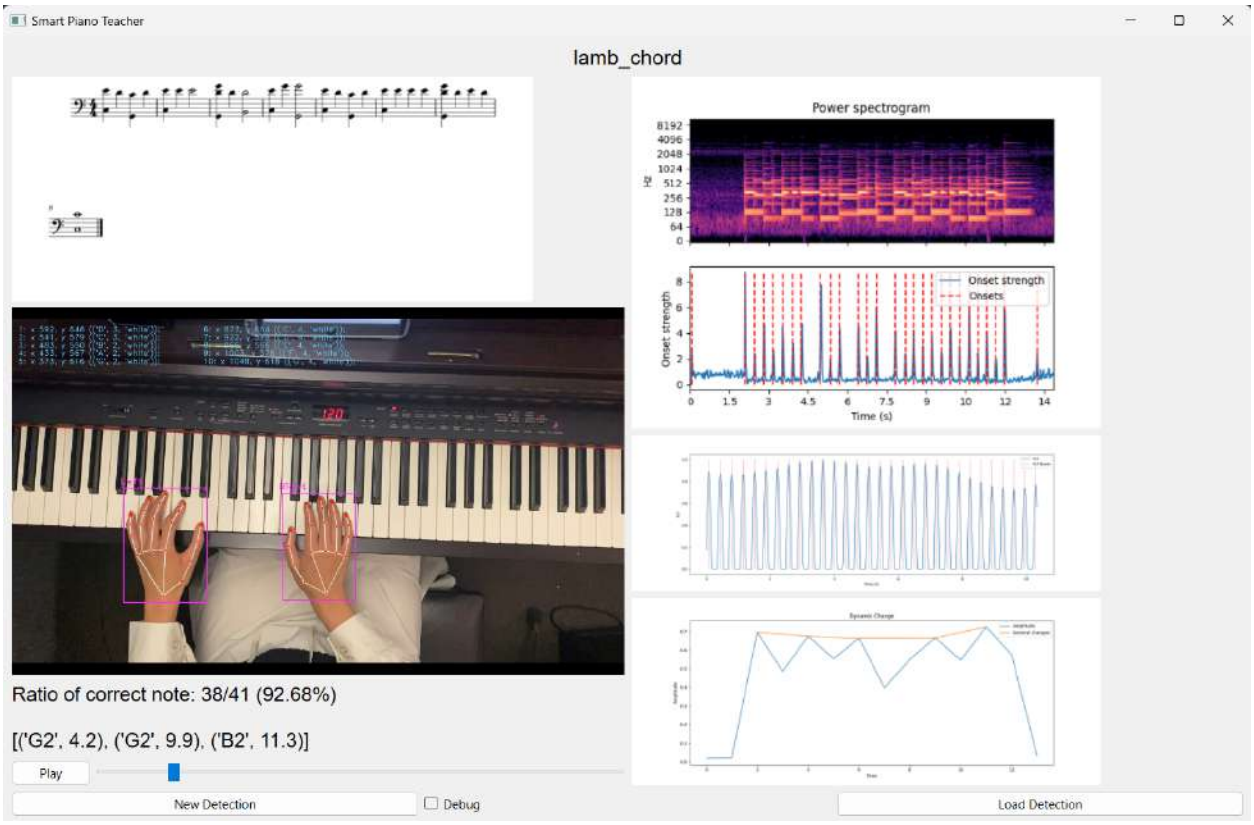


Figure 10: GUI application developed in Term 1.

The frontend interacts with the server via API calls. The web application passes messages by triggering requests and receiving responses from the server. The API calls are sent and received using a React library called Axios [14]. When it uploads media files to the server storage, it would also do it through an API call by appending the media as form data. When it retrieves a media file from the server storage, it can access it through the endpoints exposed by the server.

API Server The API server acts as the main entry point for the backend components. The server storage refers to the 'assets' folder in the server's file system, while the MongoDB and analysis component needs to be accessed from the server too.

The server is built using Node.js [13], a JavaScript runtime environment for executing server-side code, and supports easy import of libraries, which significantly enhances the development efficiency and product quality.

The server interacts with the MongoDB database through Mongoose [15], a React library that supports the definition of database schemas and CRUD (create, read, update, delete) actions on a MongoDB database. When the server receives a request from the frontend to start an analysis function, it executes the Python analytics scripts as subprocesses and

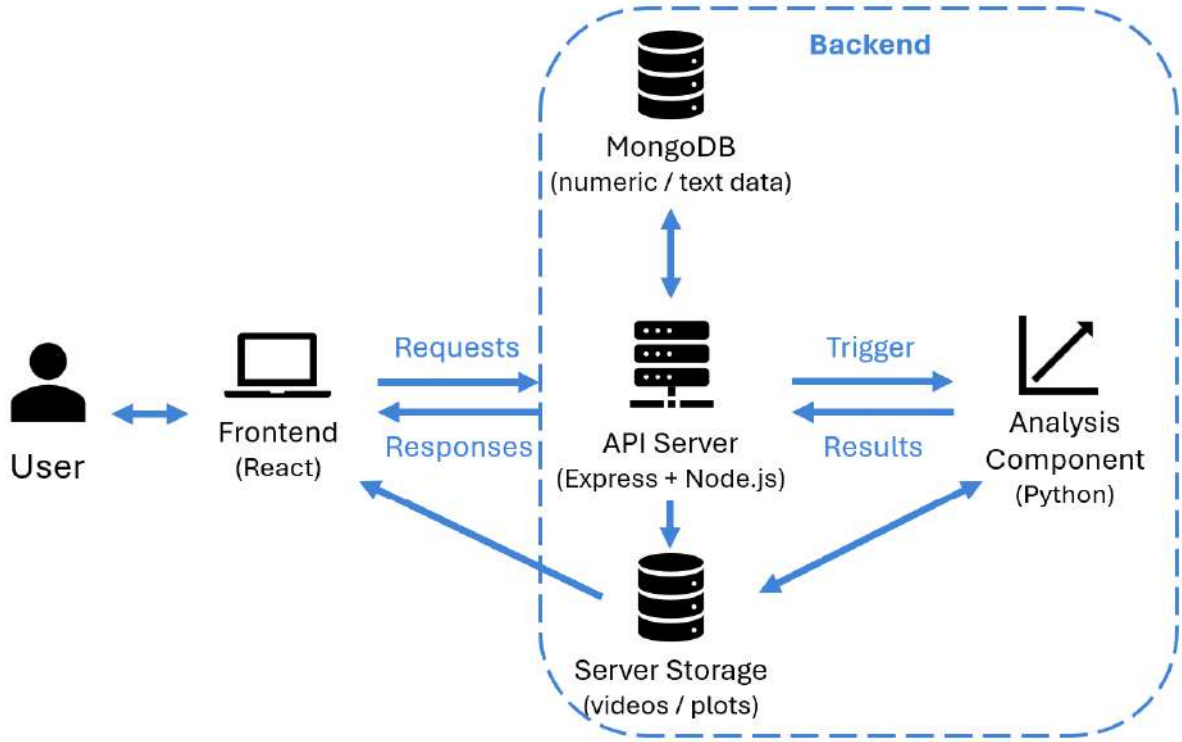


Figure 11: Overall system architecture.

receives the results either through API calls from the Python scripts or by parsing the script's standard output.

Analysis Component The analysis components are Python scripts stored inside the server file system. It is initiated from the API server as its subprocesses and receives the relevant input and output file paths, or other input for the computation as arguments. Since it is also located in the server file system, the media files can be directly retrieved using appropriate relative paths. Meanwhile, all the numerical and text results, such as scores and comments for the piece, are stored in the MongoDB database through API calls to the API server.

This part is implemented in Python because the analytics require video and audio processing, as well as some machine learning applications, in which relevant libraries and pipelines are more readily available in Python.

For basic scientific and numerical computing, Numpy [16] and Scipy [17] are used due to their wide range of functions and tools. For the data preparation task, we use MoviePy [18] and FFmpeg [19] to extract audio from the recorded video. Librosa [20] is specifically designed for audio analysis and signal processing tasks and therefore it is adopted for most of the audio feature extraction and analysis tasks. While OpenCV [21] is used for image processing tasks. By using Matplotlib [22], various graphs can be plotted respectively for

better presentation.

For MIDI processing, Mido [23] is used for reading and extracting MIDI data from the .mid files. For visualization of the detected pitch notes, music21 [24] is used to convert the polyphonic pitches into a musicXML file which is a file type for the musical score. By using the built-in command line function of MuseScore [25], we can convert the musicXML file into an image of the music score.

Also, for better organization and flexibility to integrate all the analysis functions, an object-oriented design is adopted. All required Python packages are loaded to a requirements.txt file and can be installed using pip [26]. However, note that FFmpeg [19], MuseScore [25], and CREPE [27] need to be installed in the system as well for the detection to run normally.

3.3 Storage

As explained previously, both server file storage and MongoDB database are used to store data. The server storage is mainly for multimedia files, while the MongoDB database is for numerical and textual data. Knowledge regarding database schema design and organization is learnt from the course CSCI3170 (Introduction to Database).

3.3.1 Server Storage

The server storage stores multimedia files which are too large to be stored in MongoDB database. It contains videos and audio of recordings uploaded by users, MIDI and PDF files of pieces, and some intermediate text files and graphical plots in bitmap image format generated from the analysis service. The folders and their contents are shown in Table 1.

Folder	Description
scores	It stores all MIDI and PDF files of the piano pieces available for recording analysis.
recordings	It contains sub-folders named as the recording ID it corresponds to. Each sub-folder stores the uploaded recording and all intermediate files generated during recording analysis, the list of files is shown in Table 2.
emotion	It stores the audio (waveform) of the music to be analyzed and the resulting plot image. The folder is cleared whenever a new music is uploaded for analysis.
style	It stores the MIDI file of the music to be analyzed using music piece analysis, and the piano score and audio (waveform) of both the input and output piece. The folder is cleared whenever a new music is uploaded for analysis.

Table 1: Server storage content.

Filename	Explanation
raw_video.mp4	Raw input video.
audio.wav	Audio extracted from the input video.
background.jpg	Piano frame selected for piano key detection.
keyboard.jpg	Background piano frame annotated with bounding box and pitch of all piano keys.
hand_detection.txt	List of pitches with fingers placed on it at each timestamp (per 0.1 seconds).
key_finger_map.txt	Detailed map of what note each finger is placed on at each timestamp.
onset_detection.png	Onset plotted with a mel-spectrogram.
pitch_detection.txt	Detected pitch by the algorithm.
tempo_analysis.png	Graph of PLP function.
dynamics_analysis.png	Graph of dynamics change.
note_info.csv	A csv file listing out each note's onset time, pitch and correctness (correct, false positive or false negative).
note_info_finger.csv	Note info with the hand and fingering indices added for each note.
overall_accuracy.txt	Error rate of playing and wrongly played notes with timestamps.
output.mid	MIDI of the transcribed piano performance.
output_score.xml	Transcribed piano score in musicXML format.
output_score.png	Image of piano score generated from output_score.xml.
output_score_LR.xml	Transcribed piano score in musicXML format, with the left and right-hand parts separated.

Table 2: Files stored for each recording (sorted in generated order).

3.3.2 MongoDB Database

MongoDB [10] is used as the database for storing all text and numeric data. It is a no-SQL database, which provides more flexibility on data schema and supports semi-structured data compared to traditional SQL-based databases.

In the server, the library Mongoose[15] is used to connect to and interact with the database. An auto-increment plugin is also used to assign a unique numerical ID to each newly created object automatically.

There are two models in the database, namely Recordings and Pieces. The Recordings model is used to store metadata and numeric information of each recording. The Pieces model is used to store the information of pieces and locations of the relevant score files on server storage.

Pieces This model is used to store the information of each piano piece available for recording analysis. It contains information including name, composer and a short description of the piece, as well as the file names of its MIDI and piano score in PDF. It also contains an ID to uniquely identify each piece. The ER diagram of the model is shown in Figure 12.



Figure 12: ER Diagram of Pieces model.

Recordings This model is used to store information on all recordings uploaded by the users. It includes metadata of the recording, such as the corresponding piece ID, and creation time. As well as an assigned unique recording ID, and some of the analysis results that include charts, comments and scores in different metrics. The ER diagram of the model is shown in Figure 13.

All items in "charts" are "chart" objects. This self-defined class consists of three attributes: xAxis, yAxis, and verticalLines. The xAxis and yAxis are an array of numbers corresponding to the x (time) and y (value) axes of each data point. While verticalLines marks x values where a vertical line needs to be plotted (e.g. onset time). Note that the charts are resampled from the audio sample rate (44100 or 22050) to 10 (one sample per 0.1 second), so they would not overwhelm the request size and database while being sufficiently smooth for displaying on the web application.

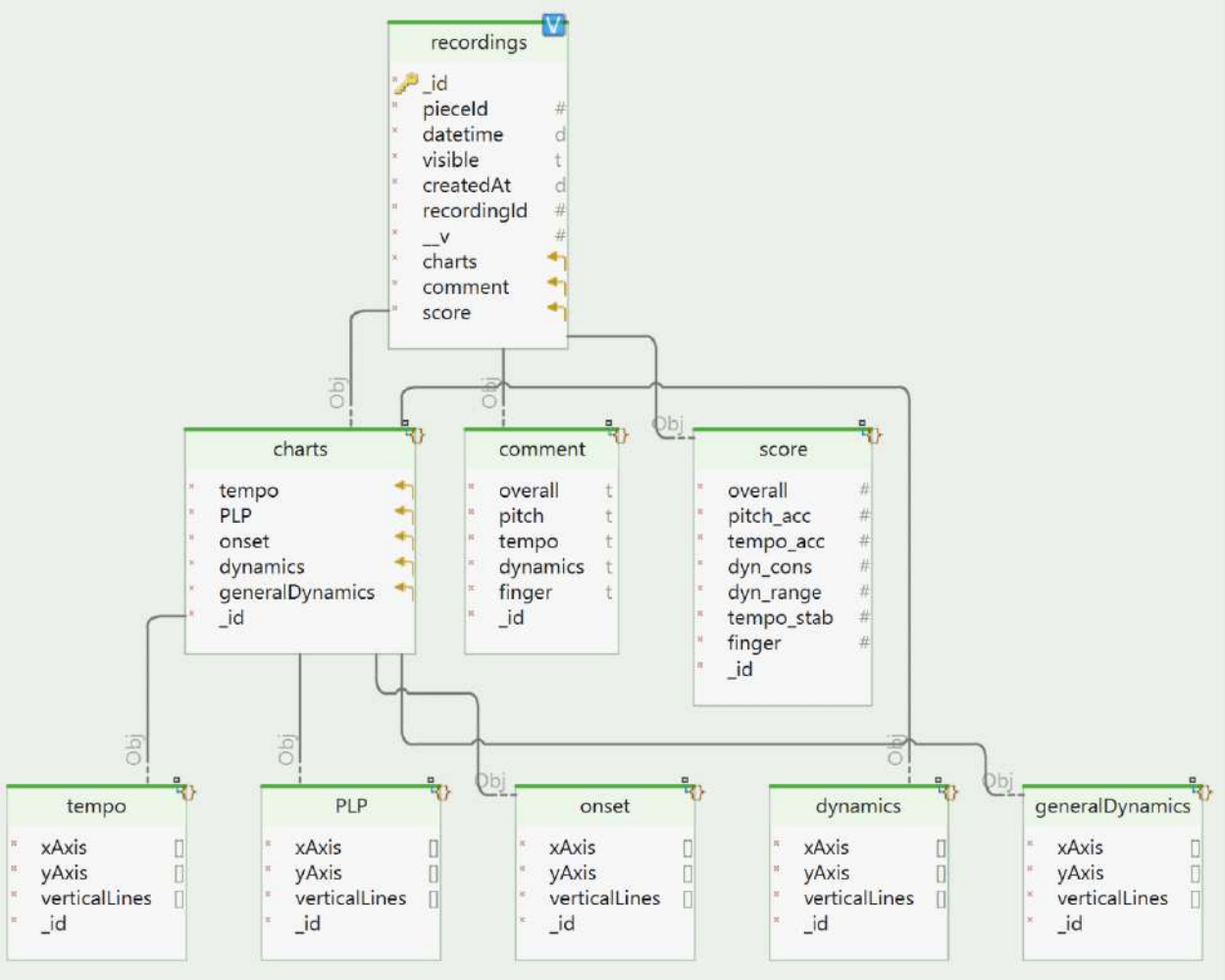


Figure 13: ER Diagram of Recordings model.

The recordings and pieces model is in a one-to-many relationship. Each recording needs to be associated with exactly one piece, but each piece may be associated with any number of recordings.

3.4 User Experience Design

This section focuses on user experience design, it includes the design of user action flow and the user interface. Many relevant knowledge and methodologies are learnt in the course SEEM3510 (Human-Computer Interaction).

3.4.1 User Flow

This section establishes the expected user flow and the functionality and arrangement of each page.

User Actions Table 3 shows an overview of the user actions available for the web application's users.

Action	Description
Performance Analysis	Users can upload performance recordings of any available piano pieces and review the analysis results of the recordings.
Emotion Analysis	Users can upload a music recording (waveform) and view the emotion expressed throughout the song.
music piece analysis	Users can upload a music piece (MIDI) to view its piano score with suggested fingerings and the musical era it belongs to. They can also convert the song to the style of another musical era.

Table 3: User actions available.

Performance Analytics Hierarchy There are many analytics available for each performance, and multiple performances are available for each piece. Details of the analytics available can be found in Section 3.5.3. The page would be clogged and hard to read if all of the information were shown on the same page. Therefore, we decided to present them with a decreasing level of abstractness. As shown in Figure 14, the scope of the presented information gets smaller and increasingly detailed from left to right.

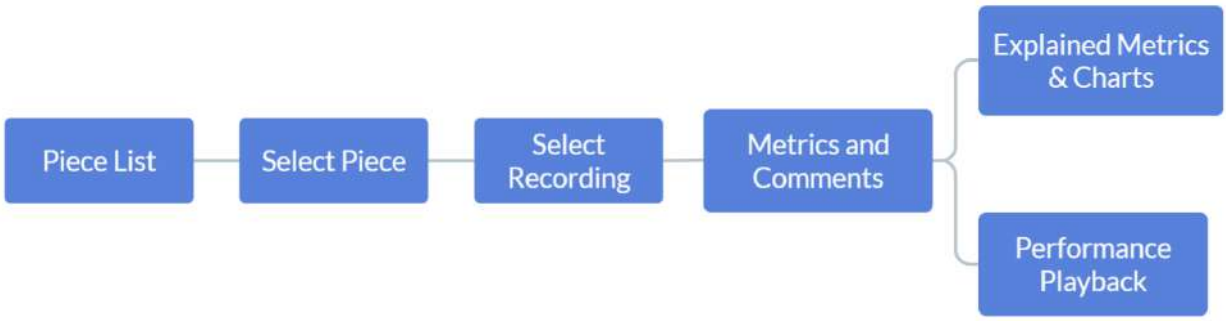


Figure 14: Hierarchy in presenting recording analysis results.

Page Arrangement Combining the information above, 6 pages are present in our web application. An overview and description for each page are provided in Figure 15 and Table 4. The recording analysis information is presented in 3 pages (piece overview, recording overview, recording playback) following the order in Figure 14. While the remaining two analysis functions are relatively simple, so one page is assigned for each of them to avoid unnecessarily complex navigation.

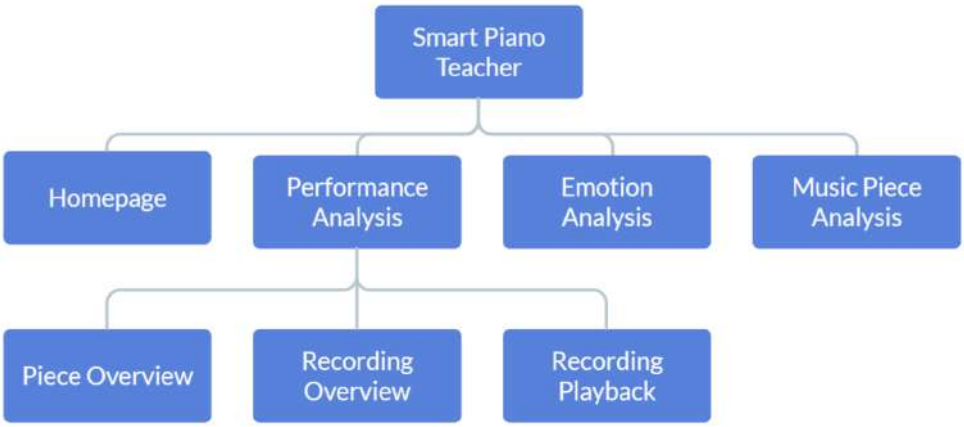


Figure 15: Overview of all pages in the web application.

Page	Description
Homepage	Entry point of the application that welcomes the user and links to the piece overview page.
Piece Overview	Lists out
Recording Overview	Users can upload a music recording (waveform) and view the emotion expressed throughout the song.
Recording Playback	Users can upload a music recording (waveform) and view the emotion expressed throughout the song.
Music Piece Analysis	Users can upload a music piece (MIDI) to view its piano score with suggested fingerings and the musical era it belongs to. They can also convert the song to the style of another musical era.
Emotion Analysis	Users can upload a music piece (MIDI) to view its piano score with suggested fingerings and the musical era it belongs to. They can also convert the song to the style of another musical era.

Table 4: Pages available in the web application.

User Flow Figure 16 shows the overall user flow of the application.

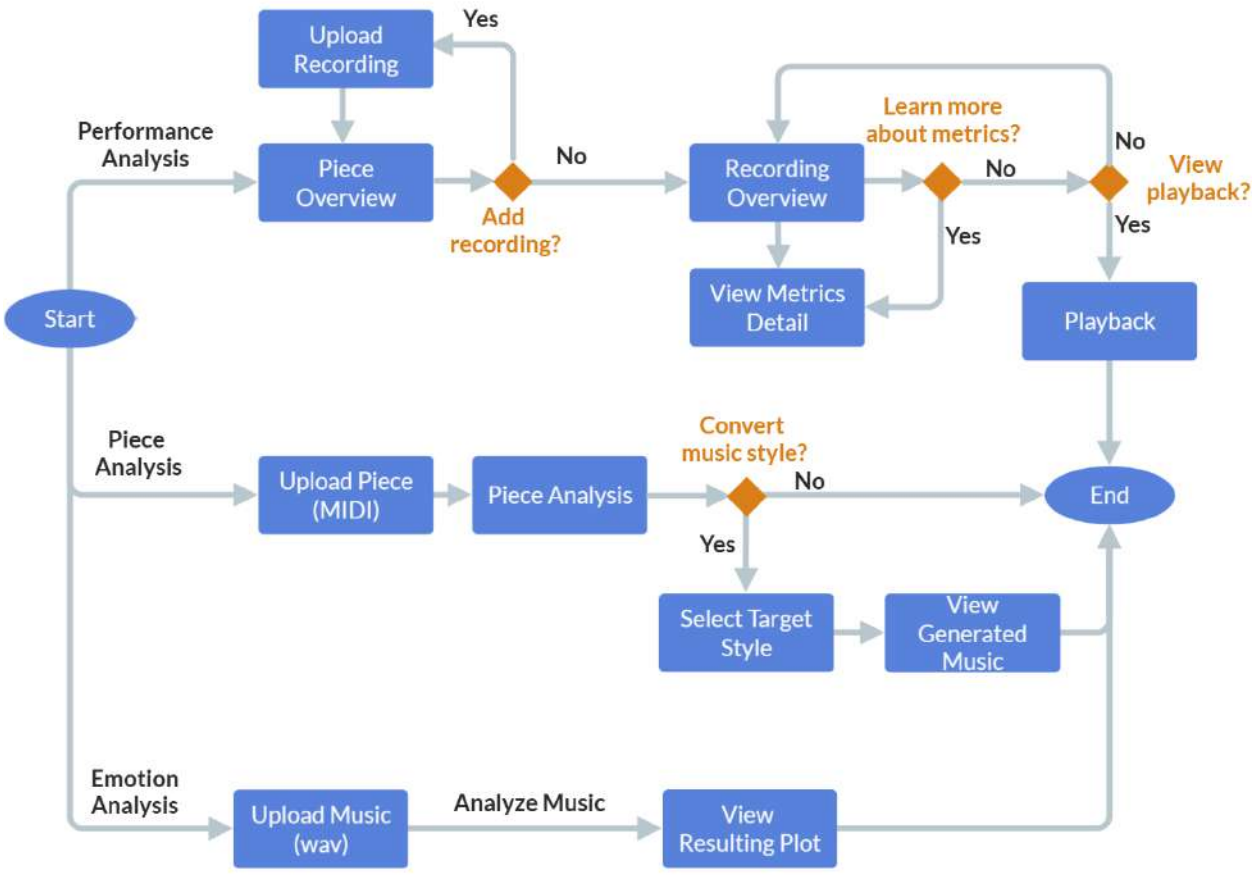


Figure 16: Application user flow diagram.

3.4.2 Interface Design

In the planning stage, we have adopted a progressive design method, in which we first started with very rough sketches and boxes, then proceeded to more refined wireframes and software-generated interactive mock-ups.

Due to time constraints, and the fact that visual aesthetics is not the focus of the application, we proceeded to the development stage once the page layouts and transitions were established. Note that further modifications to the design have been made at the later stages of development.

Layout Sketches It is hand-drawn on iPad, including very rough sketches and wireframes of the pages' overall layout. The navigation and transition of pages were also determined in this stage. Some samples of the sketches are shown in Figure 17 and Figure 18.

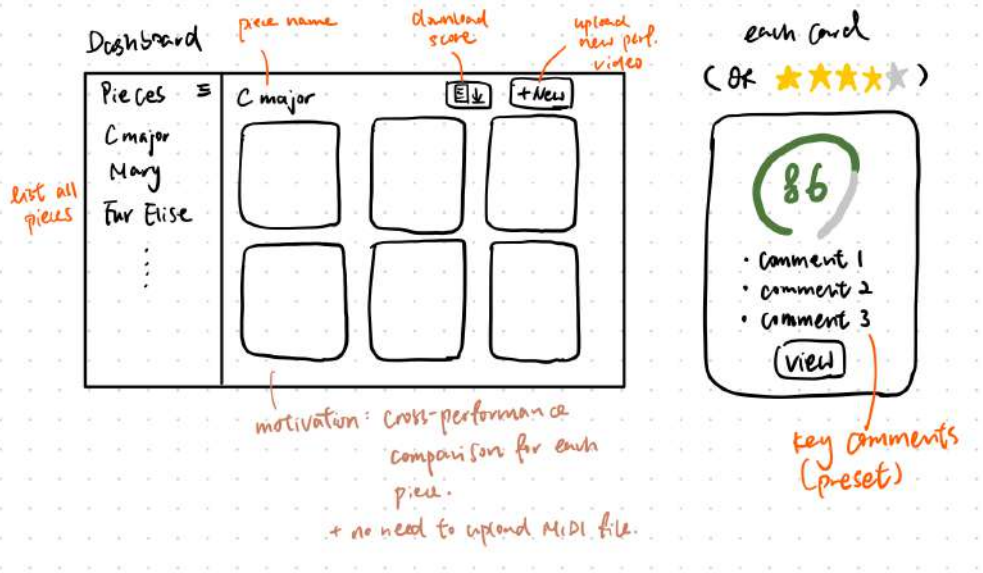


Figure 17: Sketch of the piece overview page.

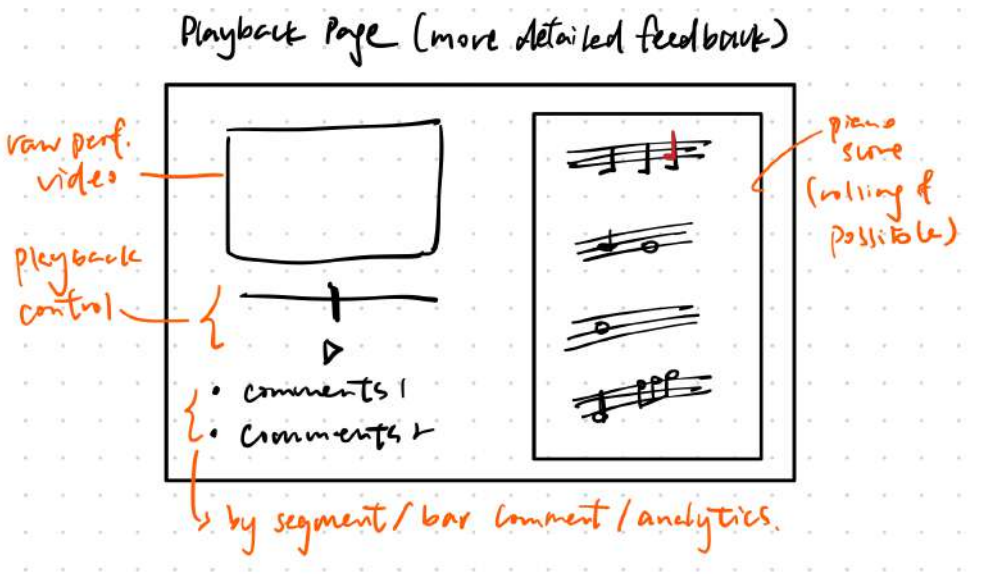


Figure 18: Sketch of the recording playback page.

Wireframes It is a more delicate prototype prepared using Figma, in which the overall layout and content of each component are drafted at this stage. It also includes interactive transitions simulating the page transition triggered by clicking on each button. The content and layouts are consistent with the sketches drawn previously. Some samples of the designed wireframes are shown in Figure 19 and Figure 20.

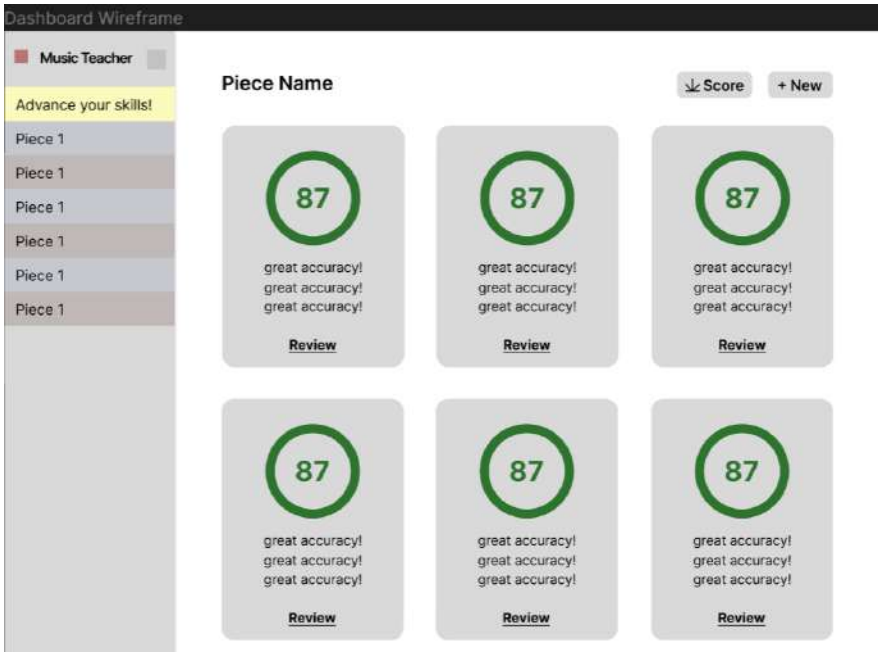


Figure 19: Wireframe of the piece overview page.

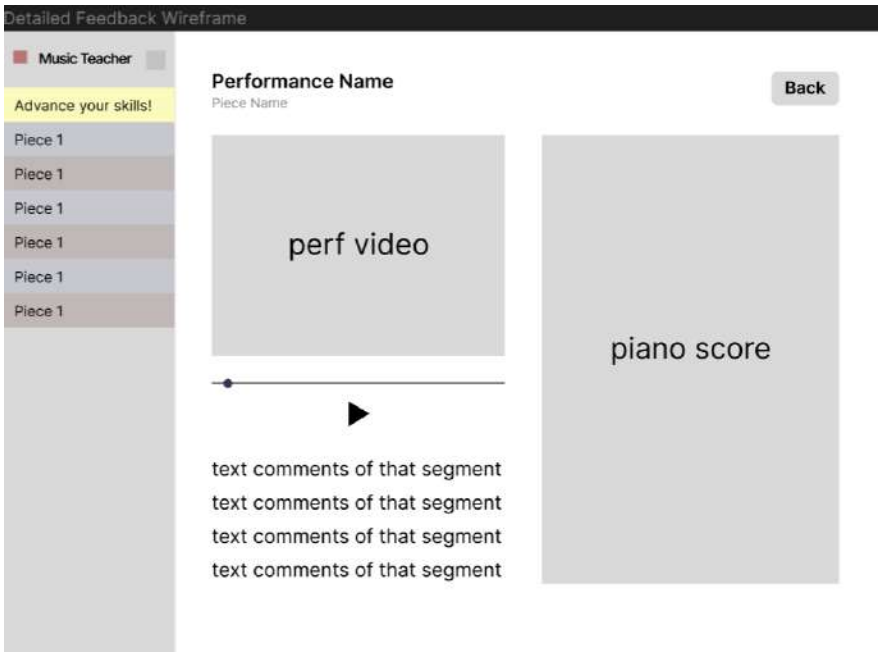


Figure 20: Wireframe of playback page.

Final Design A simplistic and modern design is used with black-and-white as the main colour tone, except for charts and display of scores. The visual elements are built mainly using Tailwind [28]. It is a CSS framework with many predefined classes to allow the incorporation of some complex CSS and HTML settings by simply adding a few class tags. A sample page of the final design is shown in Figure 21.

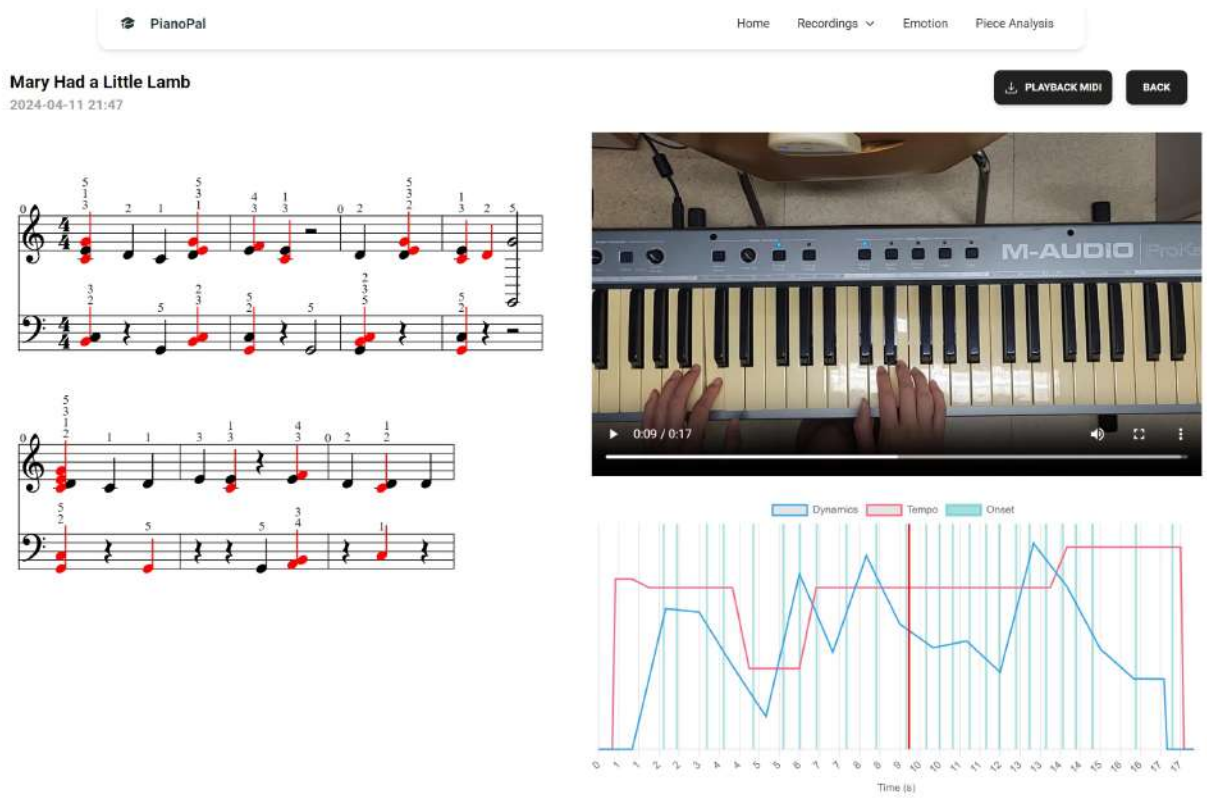
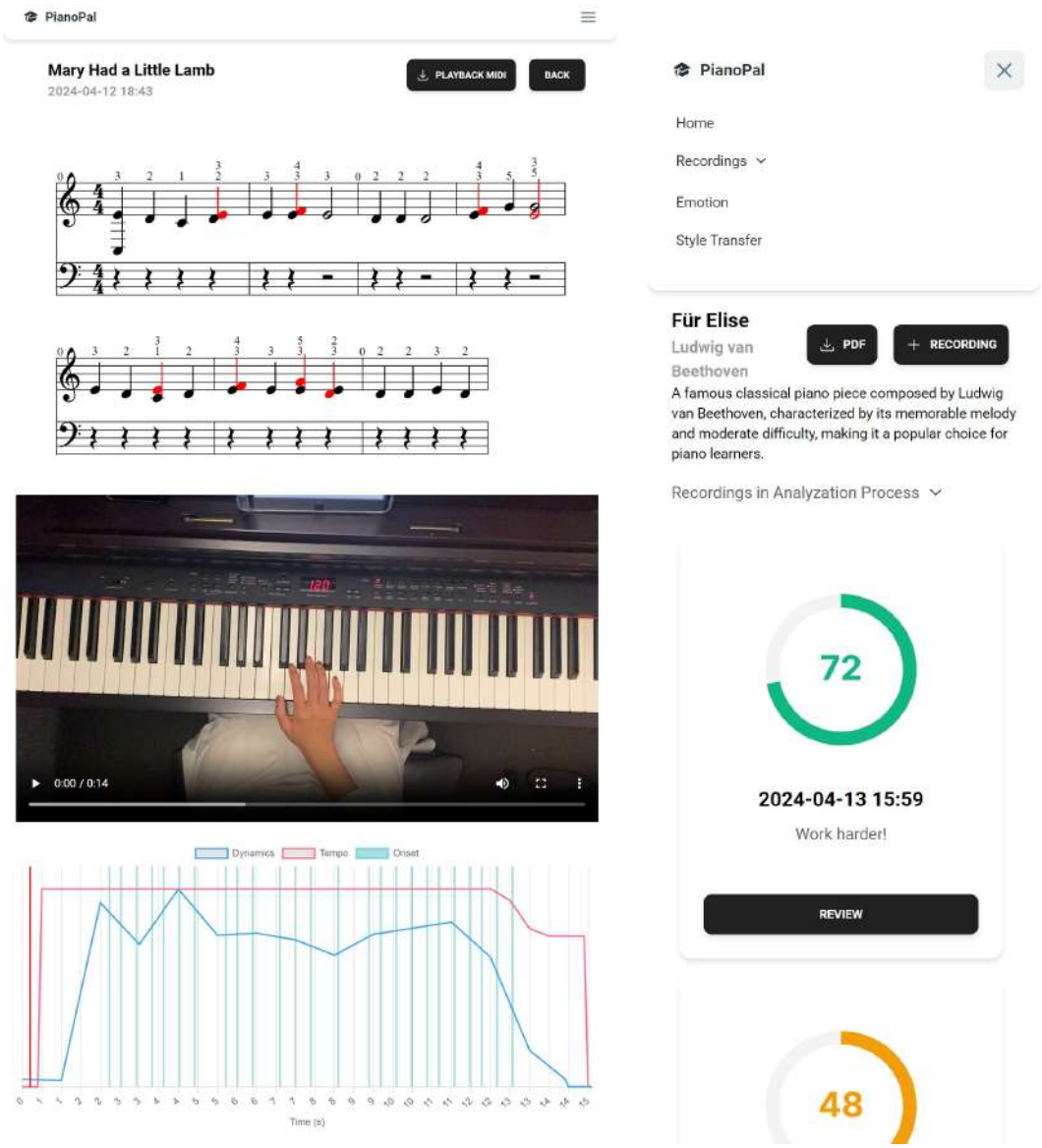


Figure 21: Example page of the final design.

Responsiveness The web application incorporates a responsive design, as illustrated in Figure 22. The interface dynamically adapts its layout and component size to provide a consistent and satisfactory user experience across devices of various dimensions, including computers, tablets and mobile phones.



(a) Interface on tablet. (b) Interface on mobile phone.

Figure 22: Responsive design on different devices.

3.5 Content

3.5.1 Homepage

The homepage (Figure 23) serves as the entry point of the application and the start of user actions. It contains a welcome message and a brief description of the application. The "GET STARTED" button links to the first piece overview, which is the starting point of the recording analysis action flow.



Figure 23: Homepage.

3.5.2 Navigation Bar

In order to facilitate ease of navigation between different functions, a navigation bar is present on all pages (Figure 24). It includes quick links to the homepage, piece overview, emotion analysis and music piece analysis pages.

Since there are many pieces available for recording analysis, a second-level menu is present for users to select the desired piece. The menu is unfolded when the pointer hovers over the "Recordings" button and displays the piece name and composer of each piece. Each menu item links to the corresponding piece's overview page upon clicking.

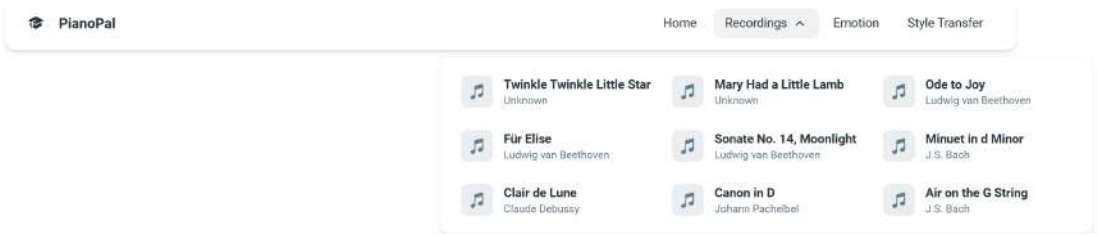


Figure 24: Navigation bar.

The pieces displayed in the menu are retrieved from the database every time the navigation bar is loaded (i.e. when a new page is opened).

3.5.3 Performance Analysis

The main goal of this part is to automatically provide objective feedback on piano performances that are relevant and useful for users to become better pianists. It should be able to extract data and perform quantitative analysis on relevant features, then convert them into qualitative feedback that is meaningful and easily understandable. Figure 25 gives an overview of how different components in the system interact to support the actions of uploading new performance videos and retrieving recording analysis. More details of each action will be presented in subsequent sections.

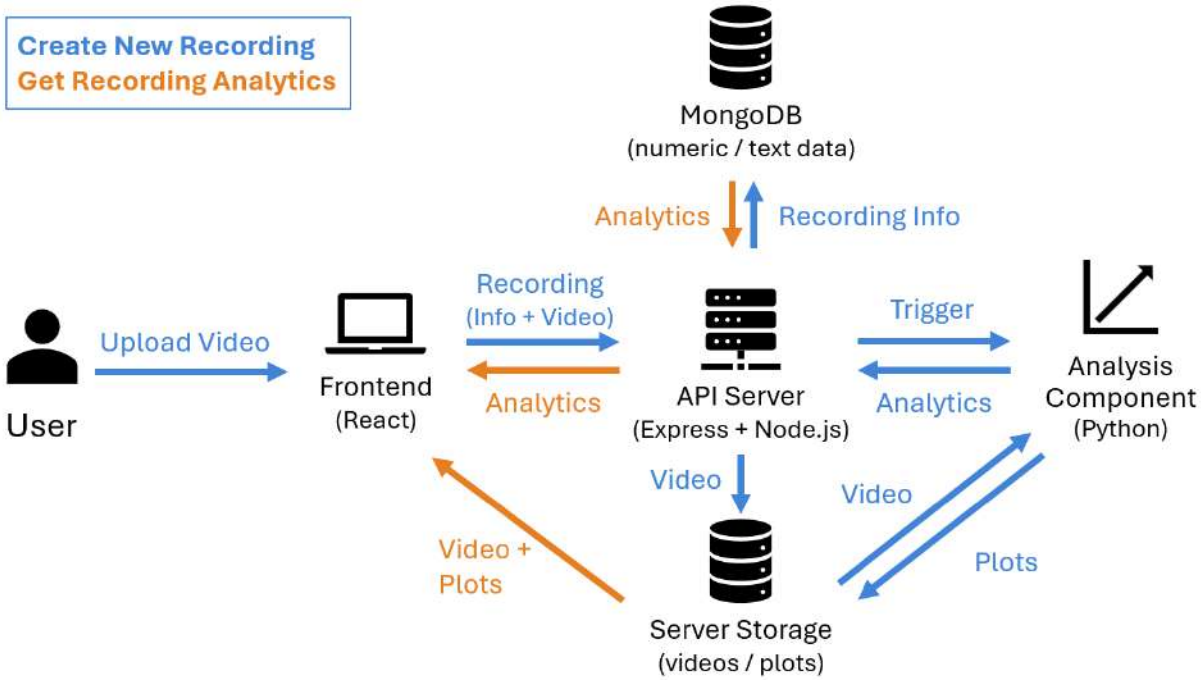


Figure 25: System flowchart for getting and adding recordings.

Page Loading

All pages in this function would display performance analysis results (in different levels of detail) upon loading. The sequence of execution to retrieve and present data from the database is shown in Figure 26.

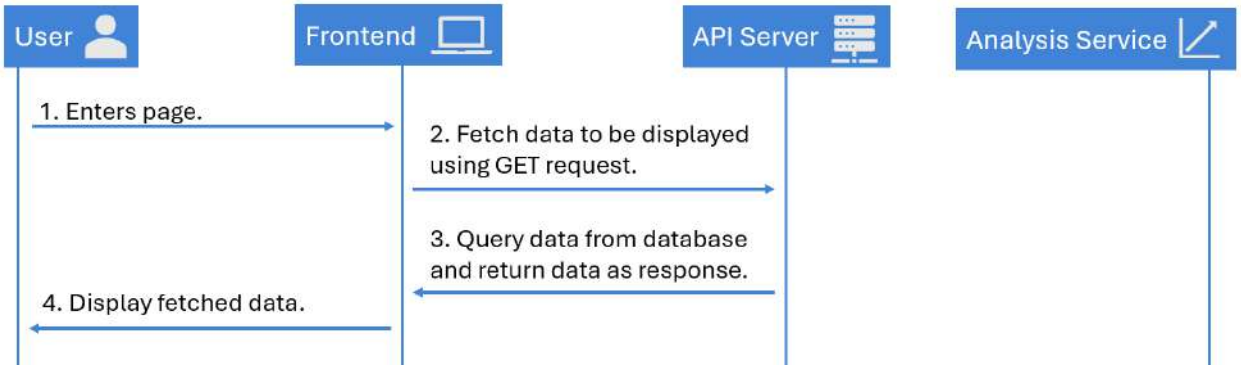


Figure 26: Sequence diagram of how pages are loaded.

Recording Analytics

The list of analytics available is shown in Figure 27. For each recording, three primary types of analytics are available: performance transcription, charts, and metrics with comments.

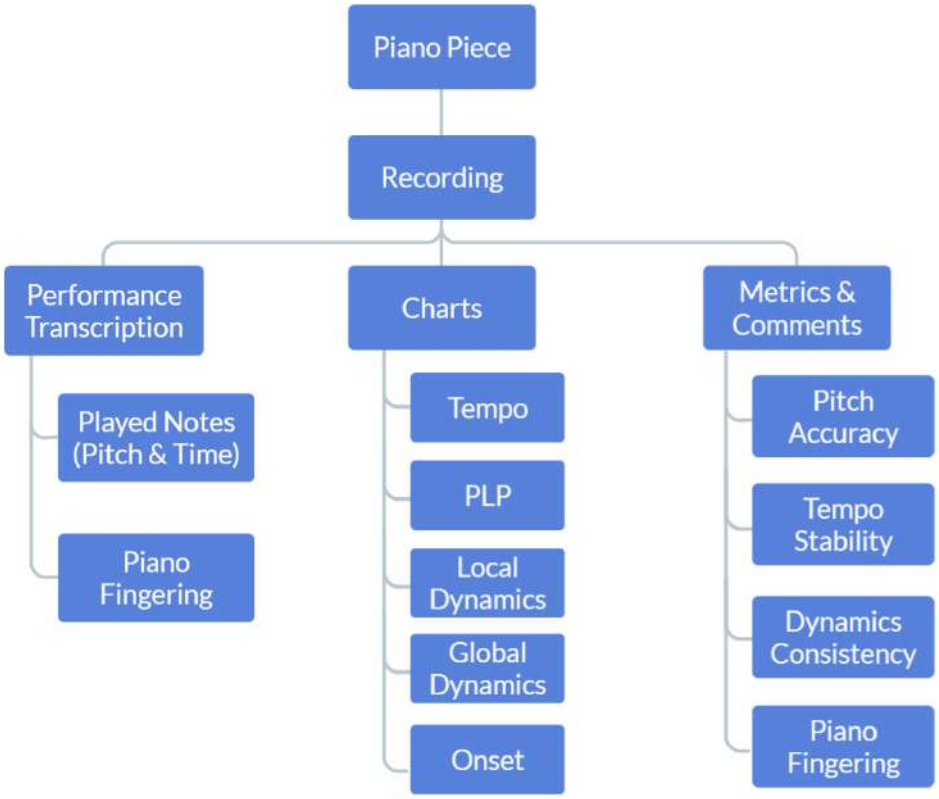


Figure 27: Details of analytics available.

Performance transcription refers to the process of extracting detailed data from the recordings. It captures various aspects of the performance, such as note pitch, time, and piano fingering. This raw data provides a detailed representation of how the piano piece was played, enabling further analysis and interpretation.

After the performance transcription process, the system further analyzes the extracted data to generate charts. These charts present music-relevant information in a visual format. They may include visualization of note distributions, tempo variations, volume dynamics, pitch contours, and other musical characteristics. These charts provide a graphical representation of the recorded performance, allowing users to gain insights into specific aspects of the interpretation.

Metrics and comments provide more summarized results derived from the charts generated in the previous step. They offer a concise and intuitive understanding of the recorded performance. Each criterion or aspect, including pitch accuracy, tempo stability, dynamics consistency and piano fingering, is evaluated and assigned a single score. Additionally, some textual comments are added for a better understanding of the results.

As explained previously in Section 3.4.1, there are three pages (piece overview, recording overview, recording playback) that present the results in a decreasing level of abstractness (Figure 28).

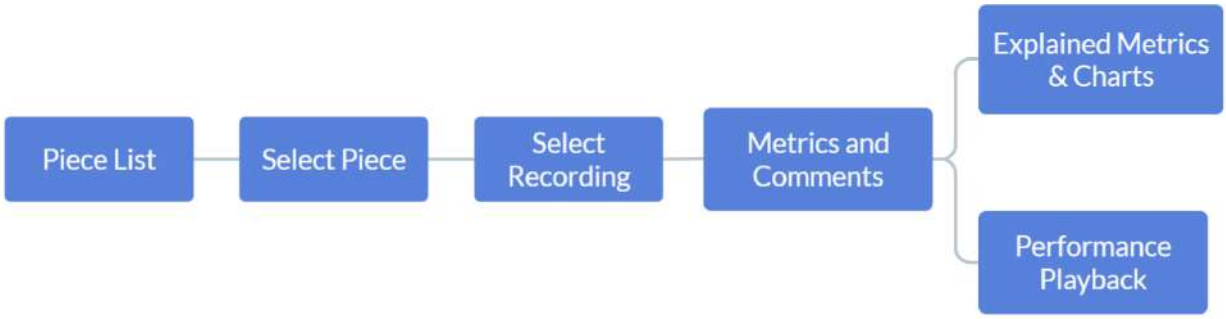


Figure 28: Hierarchy of analytics presentation.

Piece Overview

This page is the entry point for the entire performance analysis component of the web application. It gives an overview of the information and practice results of the selected piece. Its user flow chart and interface are shown in Figure 29 and Figure 30 respectively.

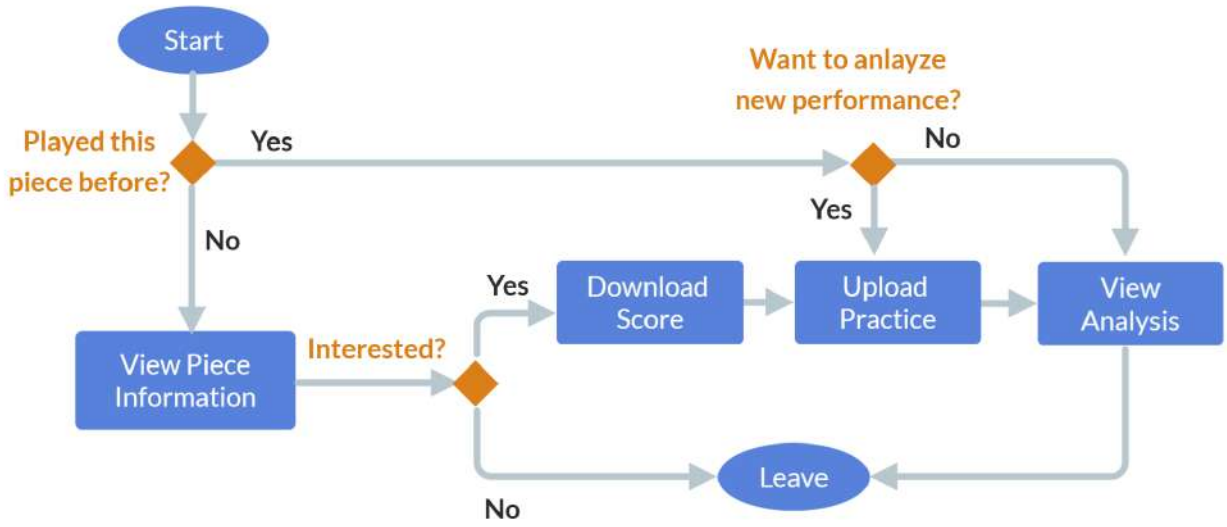


Figure 29: User flow of piece overview.

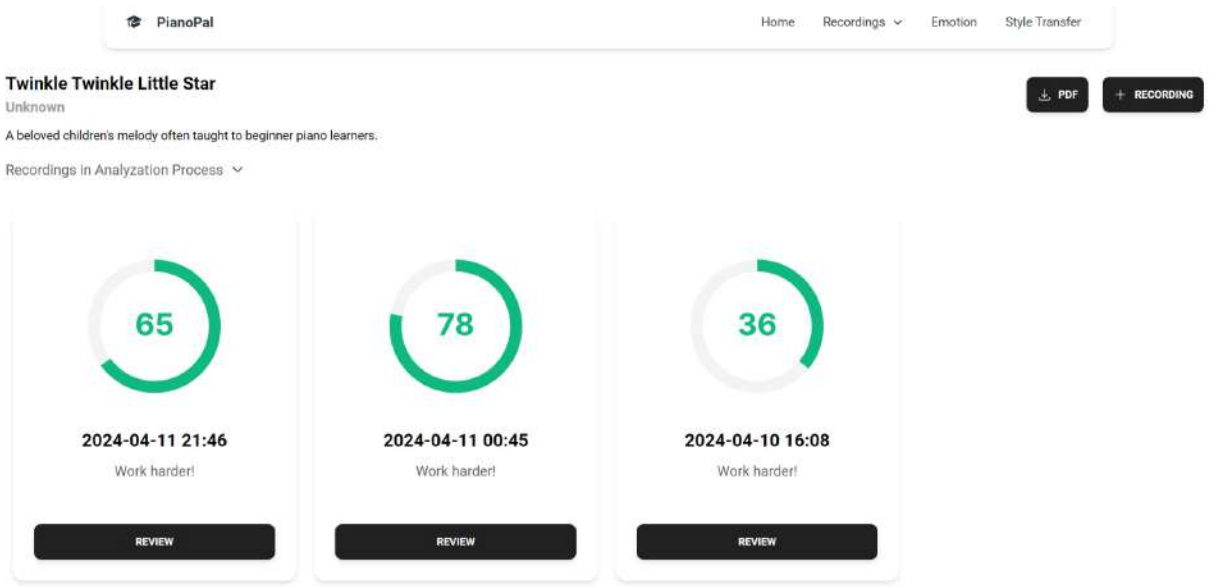


Figure 30: Interface of piece overview.

In the upper part, some introductory information about the piece, including the name, composer and a short description, is displayed. This helps users to have a basic knowledge of the piece. If the user is interested in playing it, they can download a PDF file of the piano score by clicking the "PDF" button at the top right. If the user wants to upload a new practice recording, the "RECORDING" button at the top right can be used.

The bottom part of the page shows all uploaded recordings associated with that piece. It lets users have a quick overview of their practice record and performance. The recordings are sorted in descending order of recorded time, so users can easily review the progress and compare the results of their practices. Right above the cards, there is also a foldable list named "Recordings in Analyzation Process", which would show the recording date and time of all recordings that are still under analysis.

Each recording's information is shown using a card, which contains the performance's overall score, its recorded date and time, overall comment, and a button "REVIEW" linking to the recording overview page. The recorded date and time help uniquely identify which practice it is, while the score and comments show how well it is done.

The overall score element is designed to be visual and interpreted intuitively. Firstly, it is colour-coded, a score over 70 out of 100 is shown in green, a score between 30 and 69 is shown in yellow, and a score below 30 is shown in red. Secondly, the value of the score is placed in the middle and bolded, which is the most eye-catching among all texts on the card. Thirdly, a circular bar surrounding the score also reflects the score by colouring the portion of the score value out of 100. These elements combined allow users to quickly grasp how well the practice is done.

Add Recording

When a user clicks on the "+RECORDING" button, a modal pop-up box is displayed as in Figure 31. The user can select the time the practice is recorded and select the video file, a preview of the video will be immediately shown. The user needs to drag the slider to a frame containing an empty keyboard for piano key detection for piano fingering tracking. Also, suppose the video is not in the same orientation as shown in the interface. In that case, the user can select the angle that needs to be rotated to convert the video back to the correct orientation. After that, the "UPLOAD" button can be clicked to upload the video.

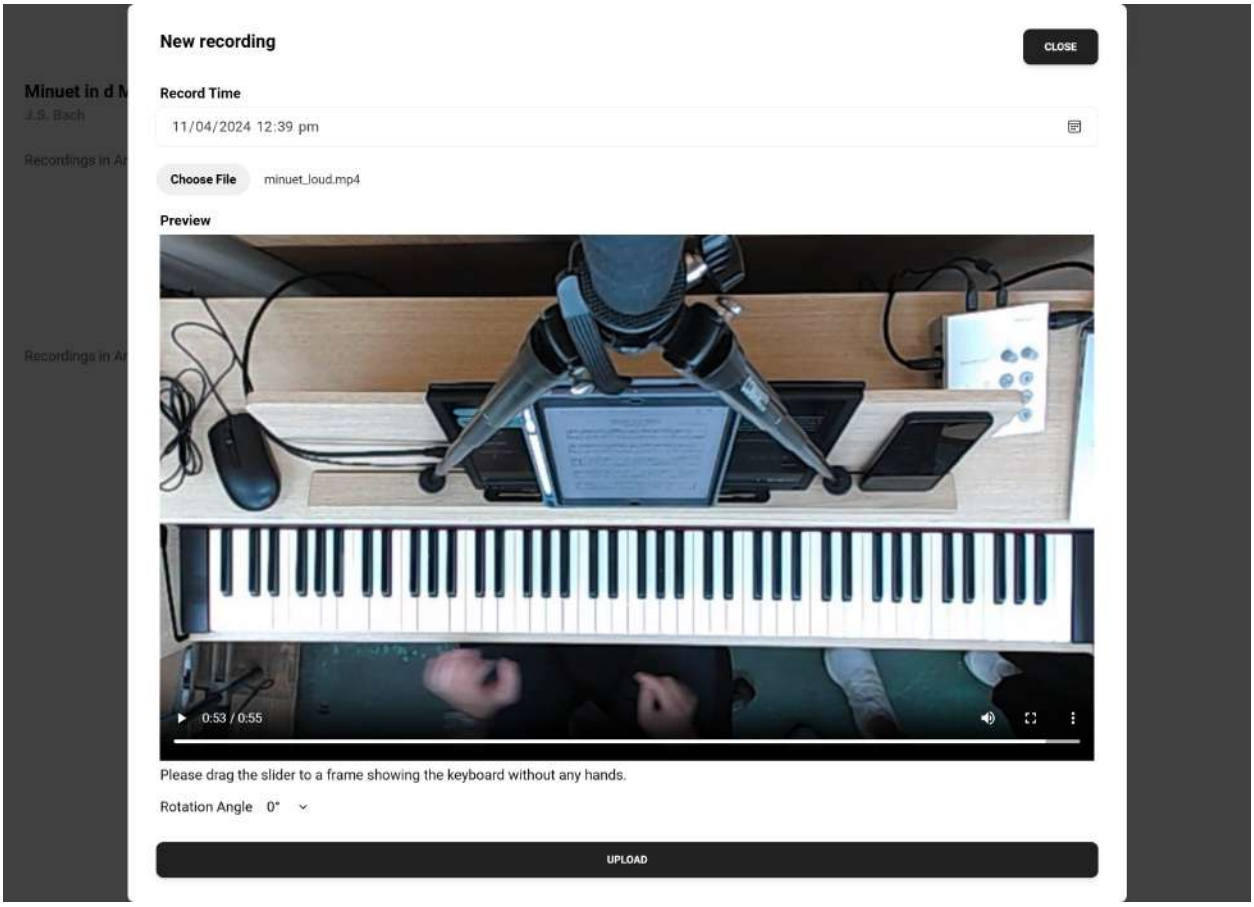


Figure 31: Interface of add recording.

This function requires the most user input in the entire application. Therefore, several design decisions were made to minimize possible input errors. Firstly, the corresponding input elements are used for all input fields, which constrains the possible input to valid values only. This includes the use of a calendar with time for record time, a file selector for selecting video, a progress bar for frame selection and a dropdown list for rotation angle (0, 90, 180, 270). Secondly, the video preview helps users to make sure the intended recording is selected. Finally, the "CLOSE" button allows the user to reverse his or her action and close the modal box if it is accidentally opened.

After the "UPLOAD" button is clicked, the process of uploading and analyzing the recording starts. The sequence diagram of the entire add-recording process is shown in Figure 32.

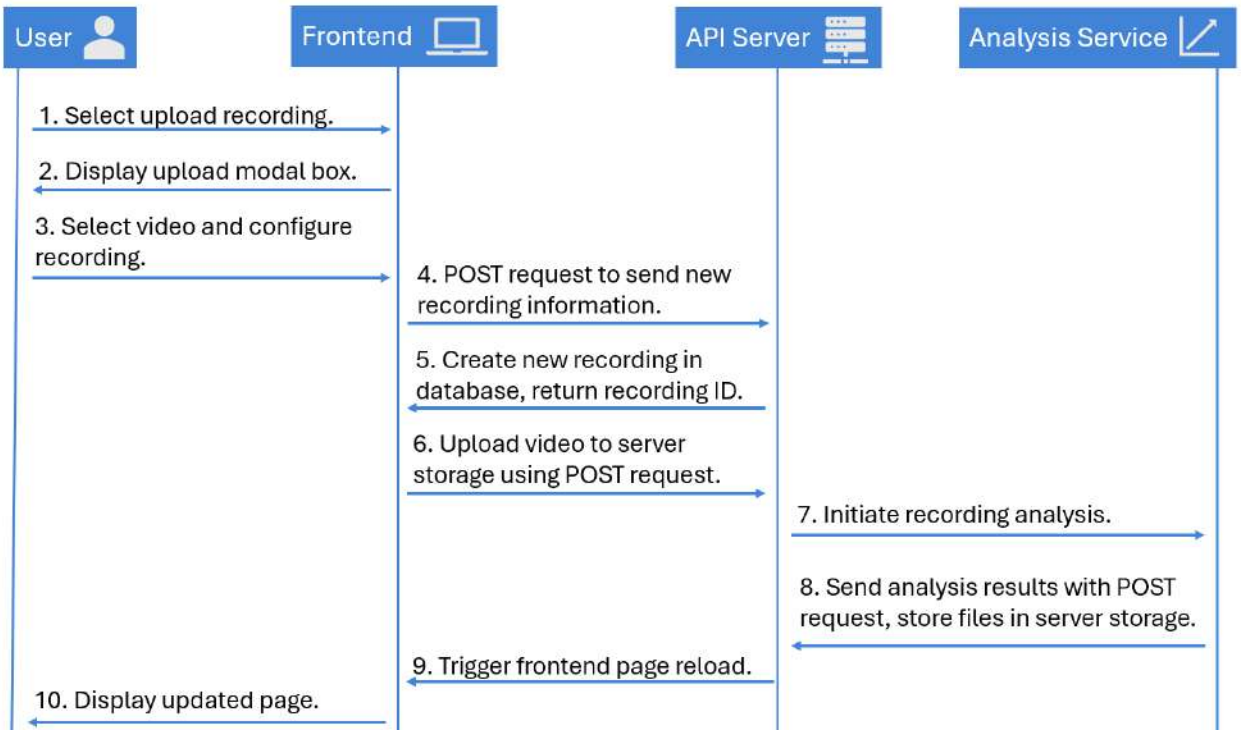


Figure 32: Sequence diagram of add recording.

After the upload action is triggered, the web page will first send an API request to the server with the recording's information (corresponding piano piece ID, recording time and orientation configured by the user). The server would receive the request and create a new recording object in the MongoDB database, then return the recording ID.

Afterwards, the server will immediately send a second request that contains the recording ID and the video. The video will be stored on the server storage, and the server will initiate a Python script to start analysing the video after the video is uploaded. When the analysis is done, the Python script will send a request to the API server containing numerical and textual results, while storing the generated media in server storage directly.

When the server receives the request from the Python script, it updates the state of the recording from "analyzing" to "ready", sends a signal to the frontend and triggers it to reload the recording analysis results. The process of sending the signal to the frontend is done by establishing a bi-directional socket connection between the frontend and the API server using Socket.IO [29].

Recording Overview

The recording overview page displays the metrics and comments on the performance. Its interface and user flow chart is shown in Figure 33 and Figure 34 respectively.

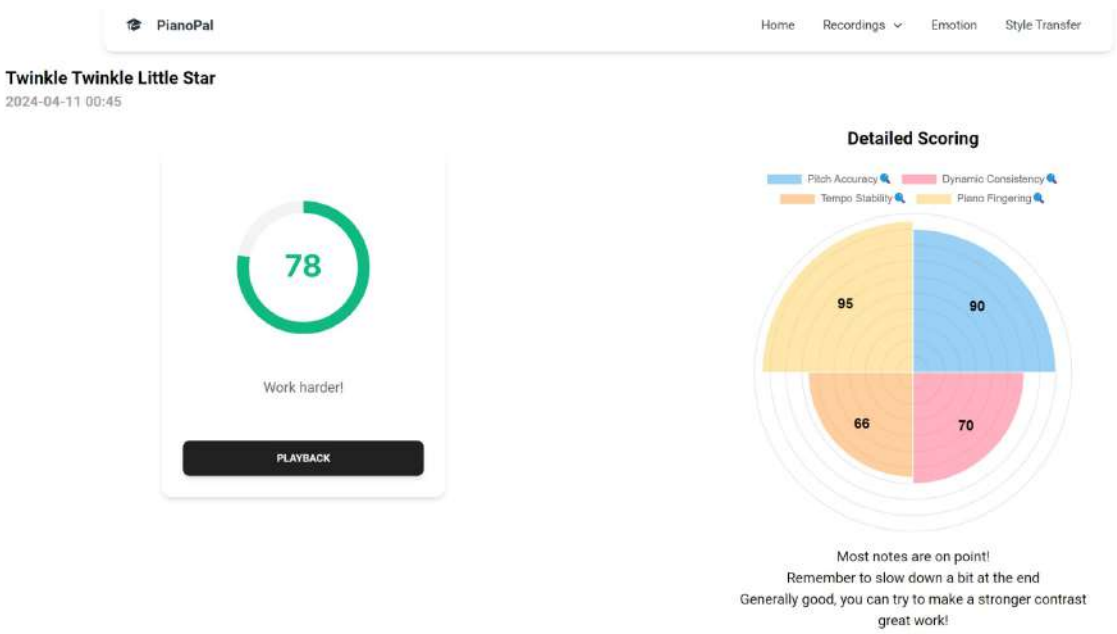


Figure 33: Interface of recording overview.

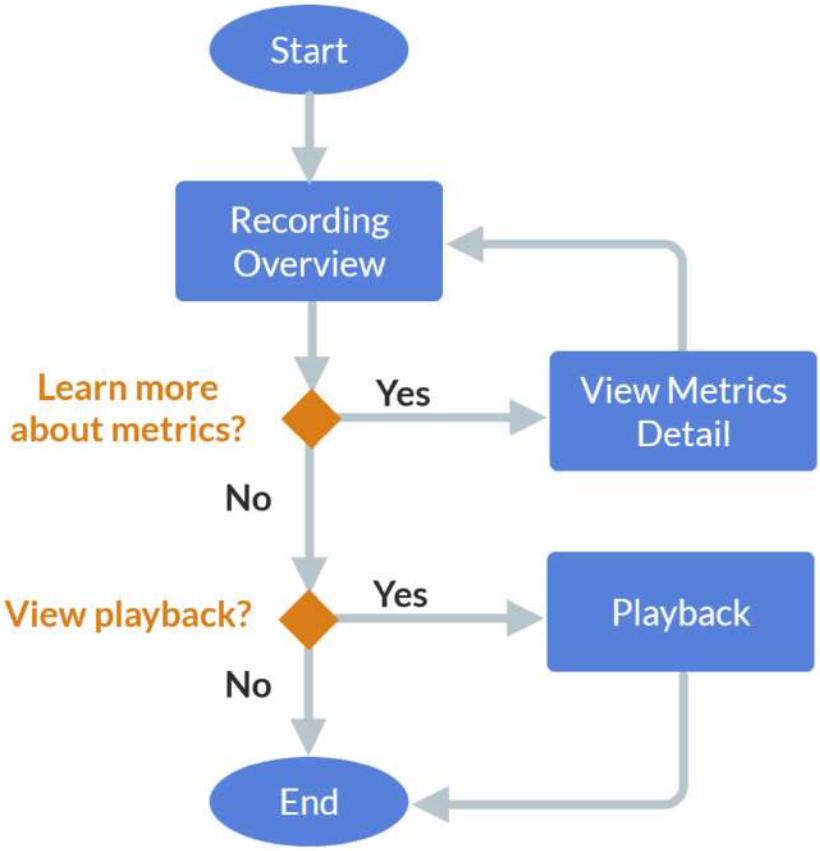


Figure 34: User flow of recording overview.

The top part of the user interface is a header that shows the piece name and recording date and time to help users identify the performance. While in the bottom part, the left half shows the overall score and comment, and a button linking to the recording playback page. The design resembles the cards in the piece overview page for consistency.

Whereas the right half shows the scoring of each metric presented in a polar chart. Upon clicking on the legend item, a modal pop-up box will appear, displaying more details of the metrics. The metrics available are pitch accuracy, dynamic consistency, tempo stability and piano fingering. More details of the metrics calculation can be found in Section 4.8.

The polar chart is chosen since it provides a good visual overview of the scores of various metrics. It is plotted using Chart.js [30], a library available on React for plotting different types of graphs. It is used to plot all charts in this web application.

Recording Metrics Detail

The metrics detail includes a text description of the metrics and some relevant charts (if applicable). Figure 35 shows a metric detail for the metric "dynamics consistency".

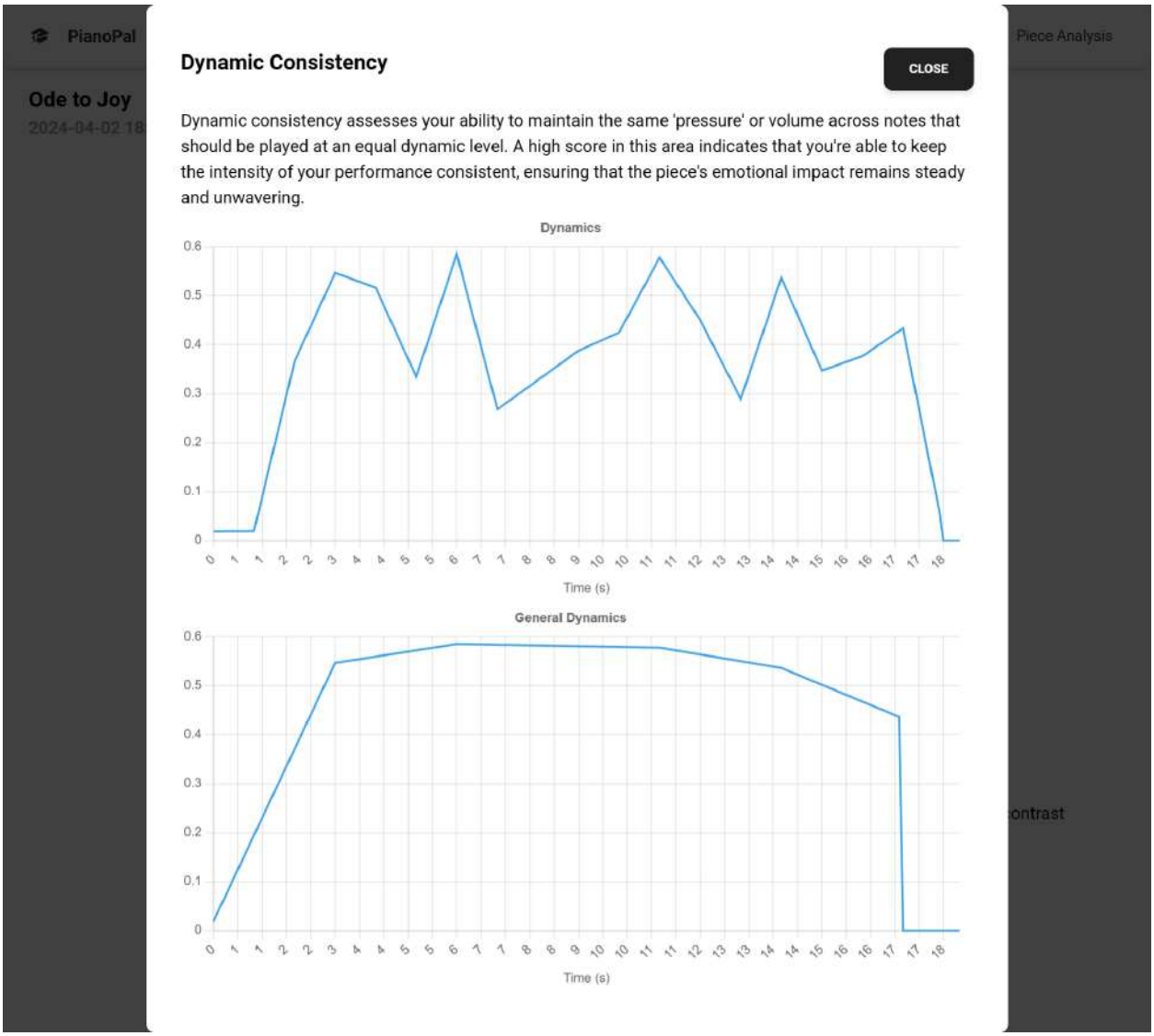


Figure 35: Sample interface of metrics detail modal box.

These detailed descriptions and charts are placed inside a pop-up modal box to keep the recording overview interface simple and easy to read. This is because normal users may be confused or overwhelmed by too many analytics and charts on more technical metrics such as predominant local pulses and onsets. Some advanced users who look for more objective analytics on their performance for a more thorough understanding may seek such information through these modal boxes.

Recording Playback

The recording playback page shows the performance video with the transcribed score and fingering together with the change of metric values along the video. Its user interface is shown in Figure 36.

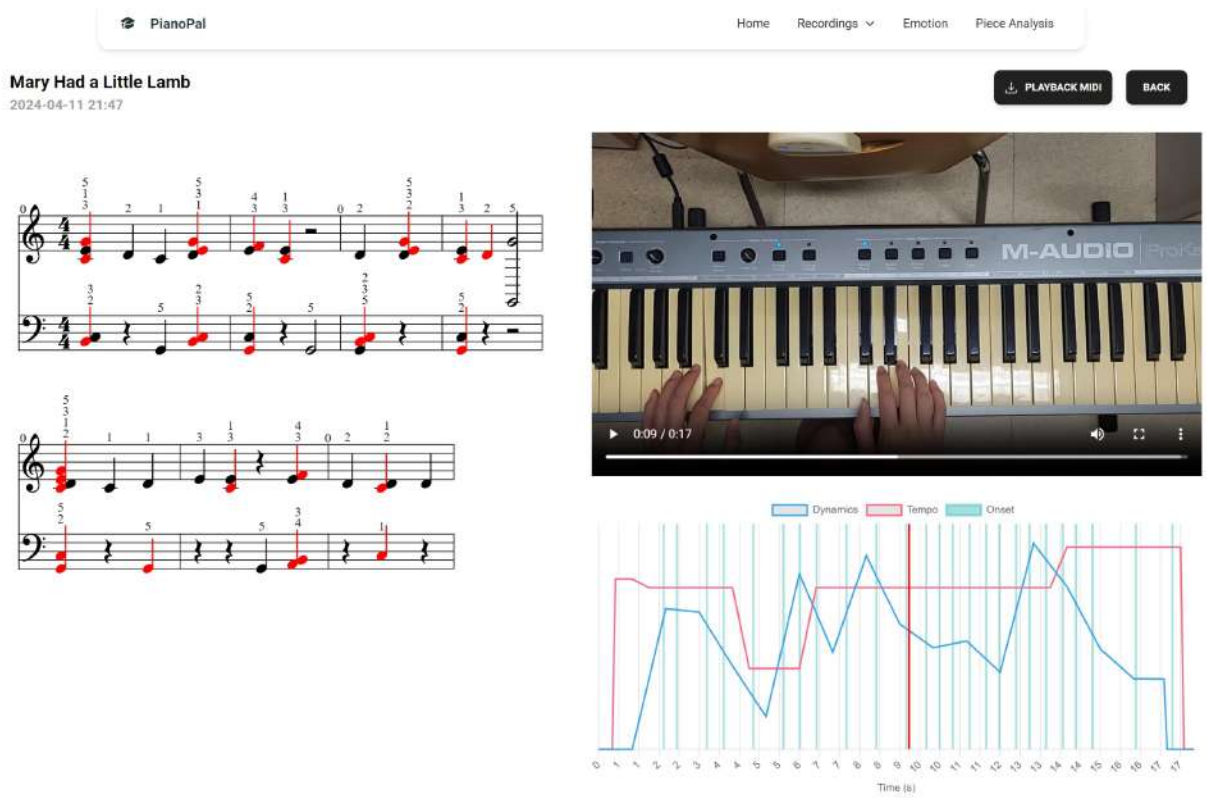


Figure 36: Interface of recording playback.

The top part of the page shows the piece name and recording date time for users to identify the performance video. The "PLAYBACK MIDI" button at the top right allows users to download their transcribed performance in MIDI format. While the "BACK" button links to the recording overview page.

The left part contains the transcribed piano score of the performance. Meanwhile, the right half contains the user-uploaded video (with corrected orientation) and a chart showing the onset, tempo and dynamics throughout the performance.

A sample of the transcribed score is shown in Figure 37. The pitch, timing, length of notes and fingerings are shown in the music score notation. It also utilizes the piano fingering results to separate the detected notes into two parts played by the left and right hands. If a note is wrongly played, it is denoted in red. Details on how the performance is transcribed can be found in Sections 4.6 and 4.4.

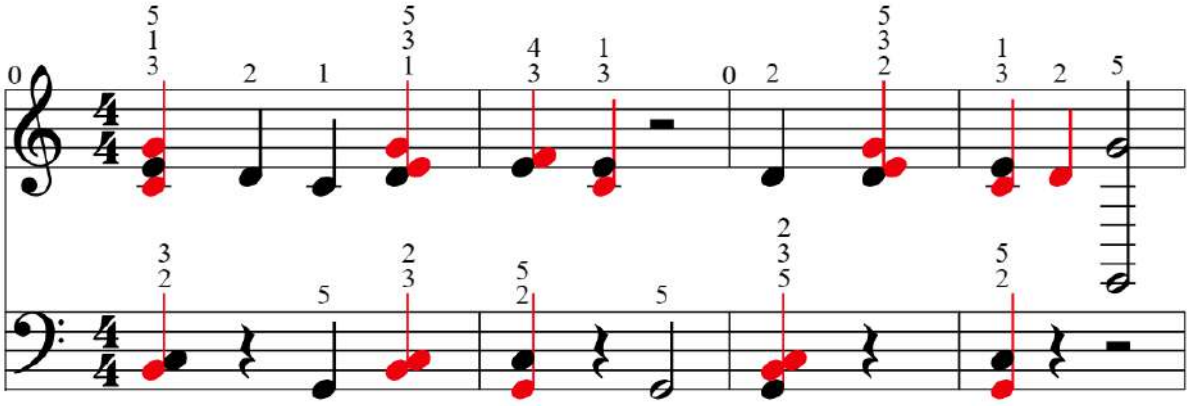


Figure 37: Sample of the transcribed piano score.

This format is chosen to resemble the piano score as much as possible since it is how most pianists "read" music. It allows them to quickly understand how the performance was executed, and possibly re-create them. The wrong note colouring also highlights possible mistakes that require attention. The piano score is created using OSMD, a React library for rendering music scores [31], from a musicxml file generated by the analysis scripts.

The playback chart shows the local dynamics and tempo values against the time of the performance. Meanwhile, the vertical green lines highlight the onset time of notes. Moreover, the vertical red line is synchronized with the video playback, it will move along the x-axis when the video is playing. A sample playback chart is shown in Figure 38.

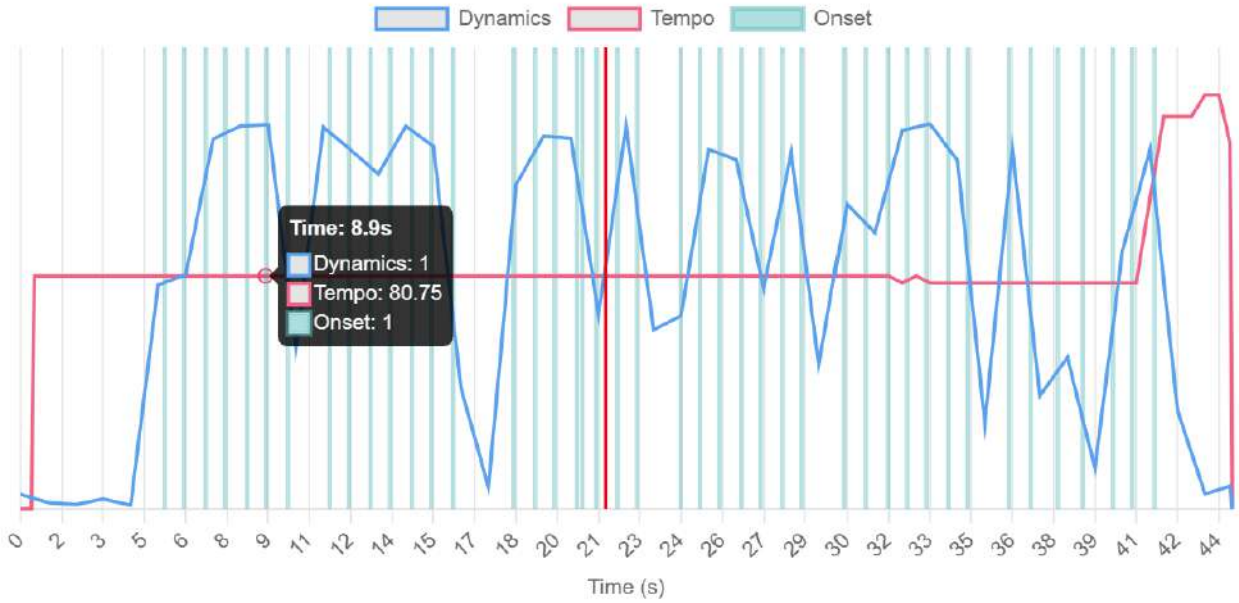


Figure 38: Sample of the playback chart.

This chart is placed below the video, so the position of the red line would approximately align with the video's progress bar. This makes it easier for users to map the chart values to the corresponding performance time and observe the changes in the charts throughout different sections of the performance.

Similar to other charts in the application, this is generated using Chart.js. The dynamics and tempo charts are passed as line graph data, while the onsets are reformatted as a bar chart with an extremely thin width. The red line is added as an annotation, by overlaying a vertical red on the x-axis using the annotation plugin.

The synchronization of the red line's position and video progress is achieved by implementing a callback function in the video player, which updates the red line's position every 0.1 seconds. The position of the line is calculated by:

$$\text{Number of timestamps in chart} * \frac{\text{Current video time}}{\text{Video length}}$$

.

Upon mouse hover, a box would appear near the cursor showing the timestamp and values of all three metrics corresponding to the cursor's x position. It is achieved by implementing a custom tooltip feature that would take the x position of the cursor, calculate the index of the data point it corresponds to (i.e. divided by 0.1), and return the y values of all three charts at that point.

3.5.4 Emotion Analysis

The main goal of this part is to automatically detect the emotion conveyed through a piece of music uploaded by the user. It can help users understand the variations in emotions throughout a music, or to evaluate their performance. A valence arousal graph is used to represent the emotions in a quantified and objective manner.

Figure 39 gives an overview of how different components in the system interact to support the actions of uploading new music files and viewing emotion analysis results.

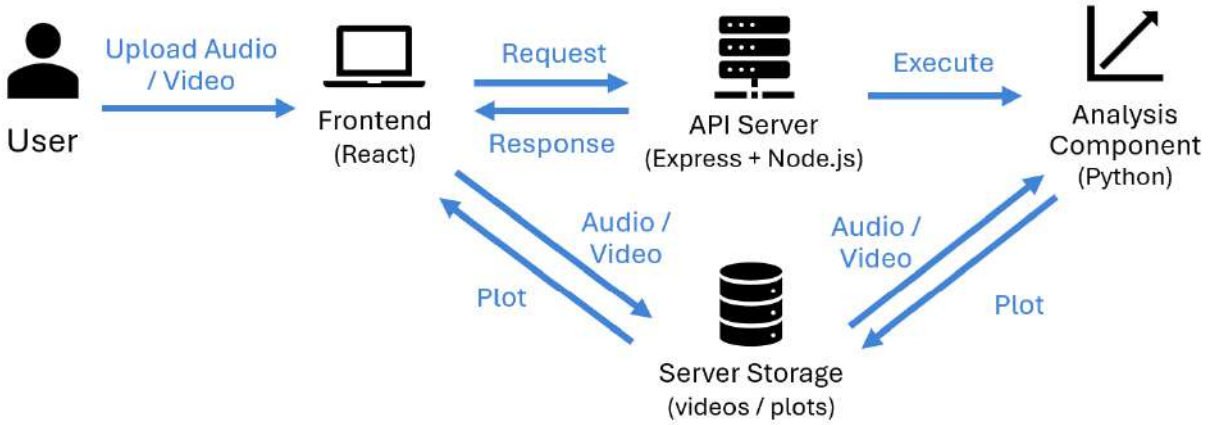


Figure 39: System flowchart of emotion analysis.

The expected user action flow of the function is shown in Figure 40.

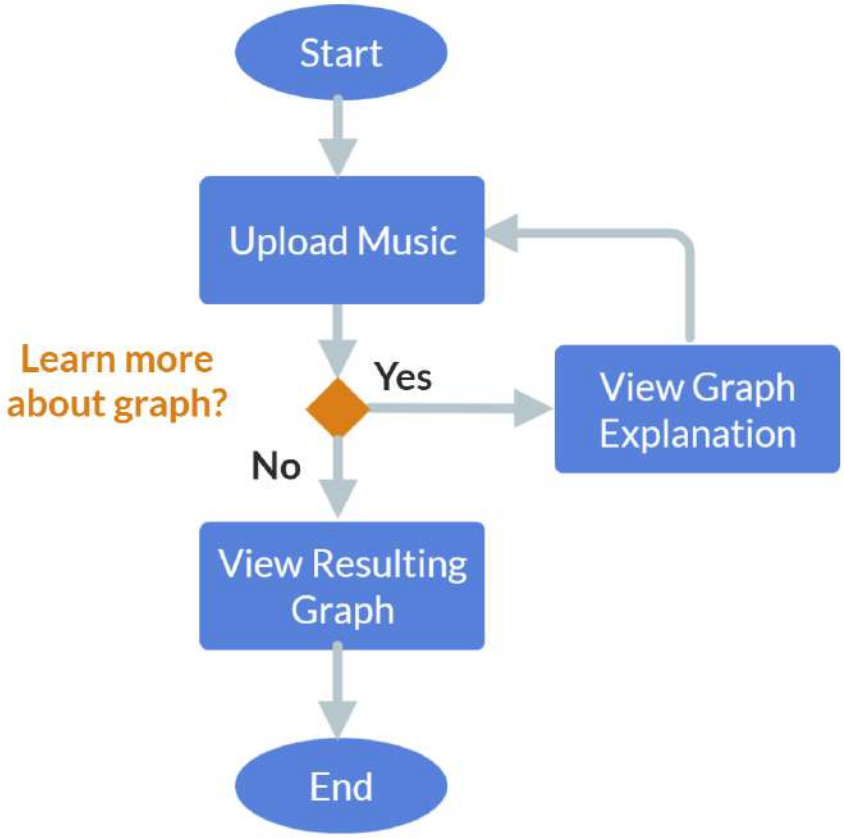


Figure 40: User flow for emotion analysis.

Since the user actions available are relatively simple, all relevant contents are displayed on the same page, as illustrated in Figure 41.

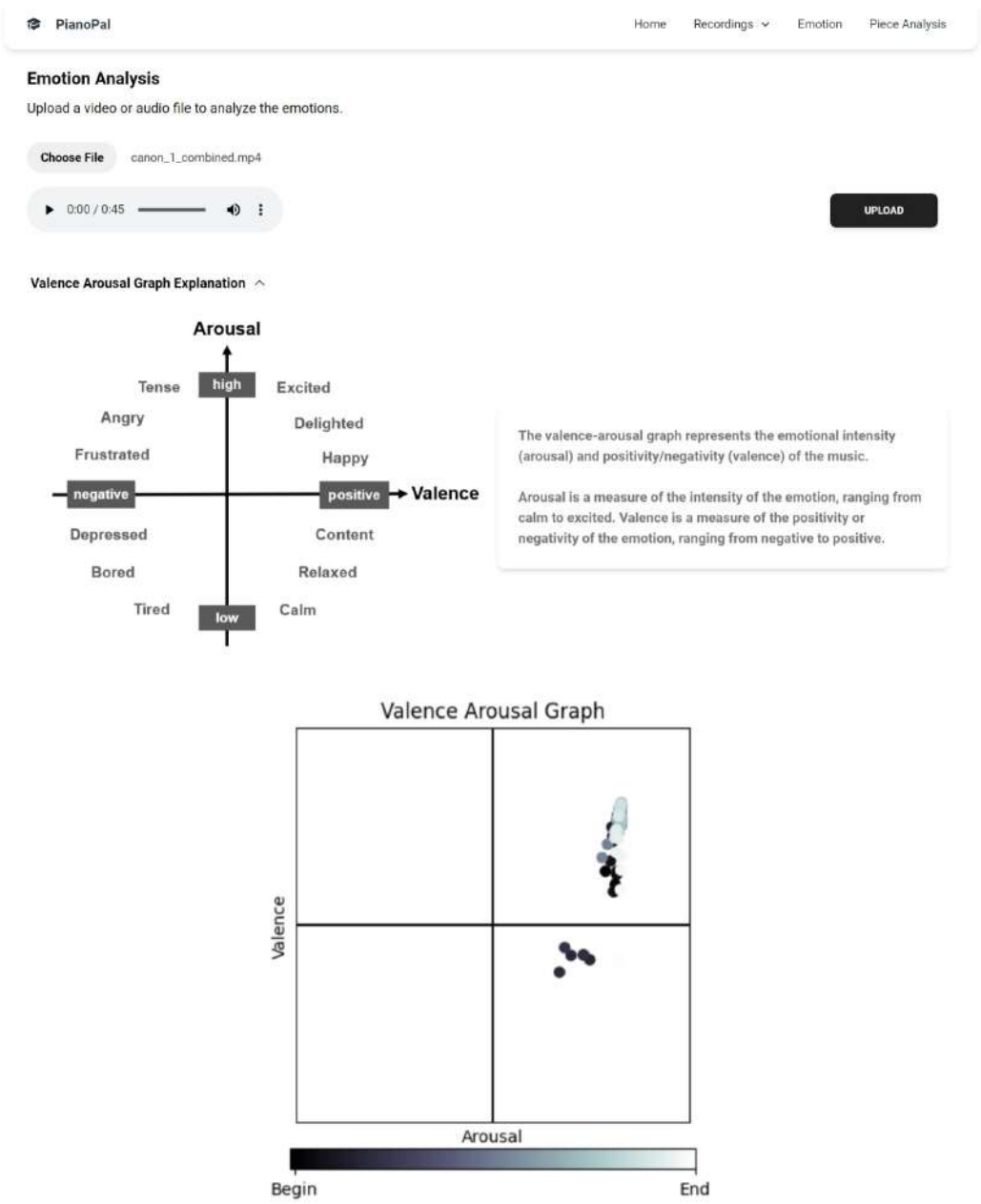


Figure 41: Interface of emotion analysis.

The page consists of three main parts: a brief description of the page with an upload component, an explanation of the valence arousal graph, and the detected emotion throughout the music plotted on the graph.

The waveform of music is taken for analysis, and it supports both audio and video input. After the file is selected, an audio preview is shown for the user to check if the correct file is selected. Upon clicking the "UPLOAD" button, the file is uploaded to the server, and the analysis result will appear at the bottom of the page once it is done. The sequence diagram

of the analysis process can be found in Figure 42.

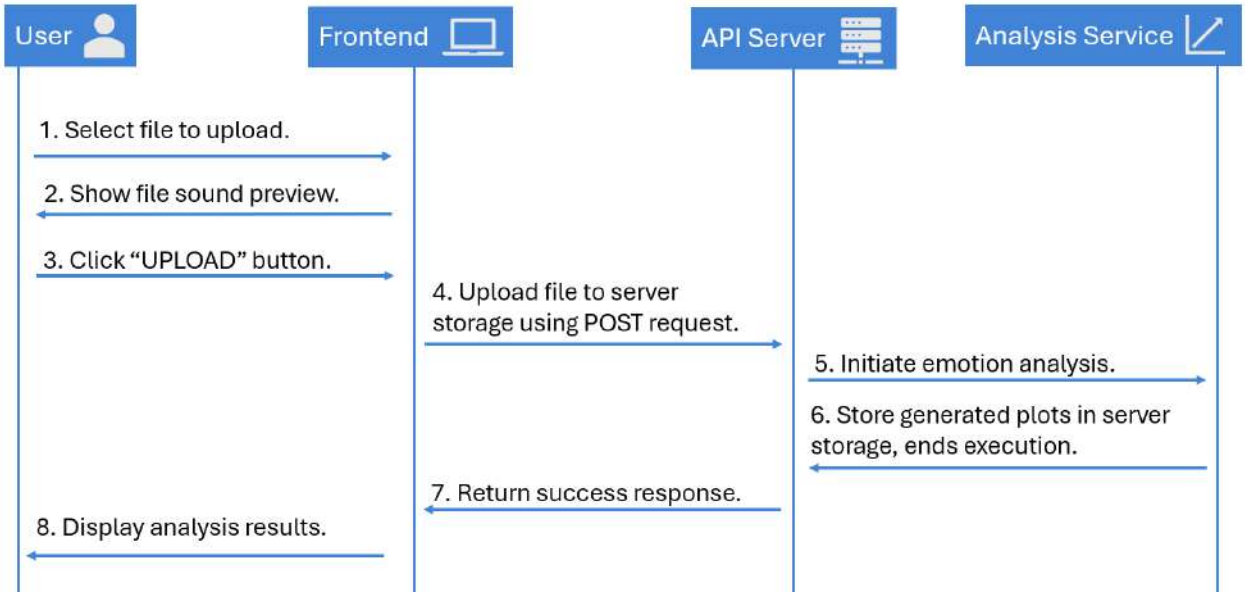


Figure 42: Sequence diagram of emotion analysis.

The analysis result is presented in the form of a valence arousal graph, with each point representing the emotion at each time point throughout the music. The emotional information is presented as a two-dimensional graph, with the level of valence as the x-axis and arousal as the y-axis. While the time information is represented by a colour transition from dark blue to white. The analysis details can be found in Section 5.

To provide users with an understanding of the valence arousal graph and its representation, a collapsible explanation component is placed above the results. This component includes a concise description of the two dimensions (valence and arousal) and a chart illustrating the corresponding emotions represented by different positions within the graph. This helps users interpret the graph accurately and facilitates their comprehension of the emotional analysis presented.

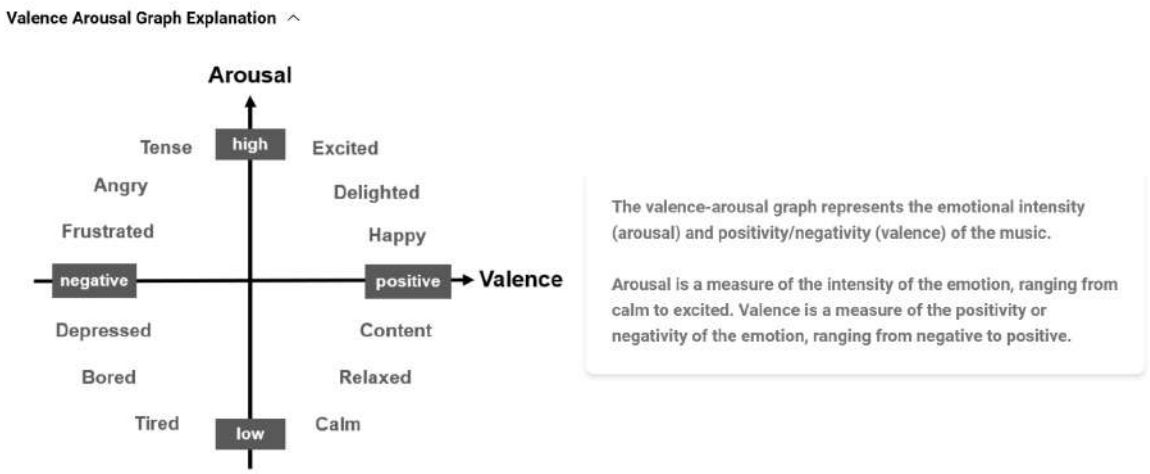


Figure 43: Explanation of the valence arousal graph.

Compared to alternatives like text labels such as "happy" or "sad," the valence-arousal graph offers a richer dimension of data. It goes beyond rigid class labels by providing a continuous scale of intensity along each axis. This enables a more objective and comprehensive understanding, even capturing subtle variations in emotion with greater precision.

Additionally, the valence-arousal graph captures the variation of emotions throughout the entire music piece, rather than providing a single global prediction. This allows for a more detailed and in-depth analysis by distinguishing different sections with varying emotions. Given that many music compositions consist of different sections with distinct modes (major/minor) and emotions, this type of analysis better reflects the structural variations in the music and provides more accurate results. It enables a more nuanced understanding of how emotions evolve and unfold within the context of the composition.

3.5.5 Music Piece Analysis

The main goal of this part is to facilitate users to analyze and play a new piano piece. It takes a MIDI file of the music piece uploaded by the user to generate a music score with suggested piano fingering and analyze the musical era the piece belongs to. Users can also choose to convert the piece to the style of another musical era. The details on how the analysis is carried out can be found in Section 6.

Figure 44 gives an overview of how different components in the system interact to support the actions of uploading new music files and viewing the generated analysis.

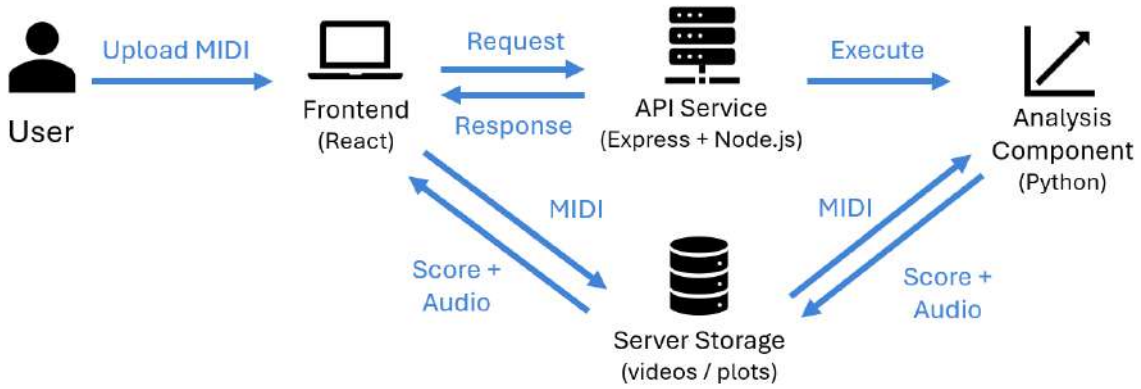


Figure 44: System flowchart of music piece analysis.

The expected user action flow of the function is shown in Figure 45. The user would first upload a MIDI file and start the piece analysis. After the analysis is done, the user can choose to convert the piece into a style of another musical era.

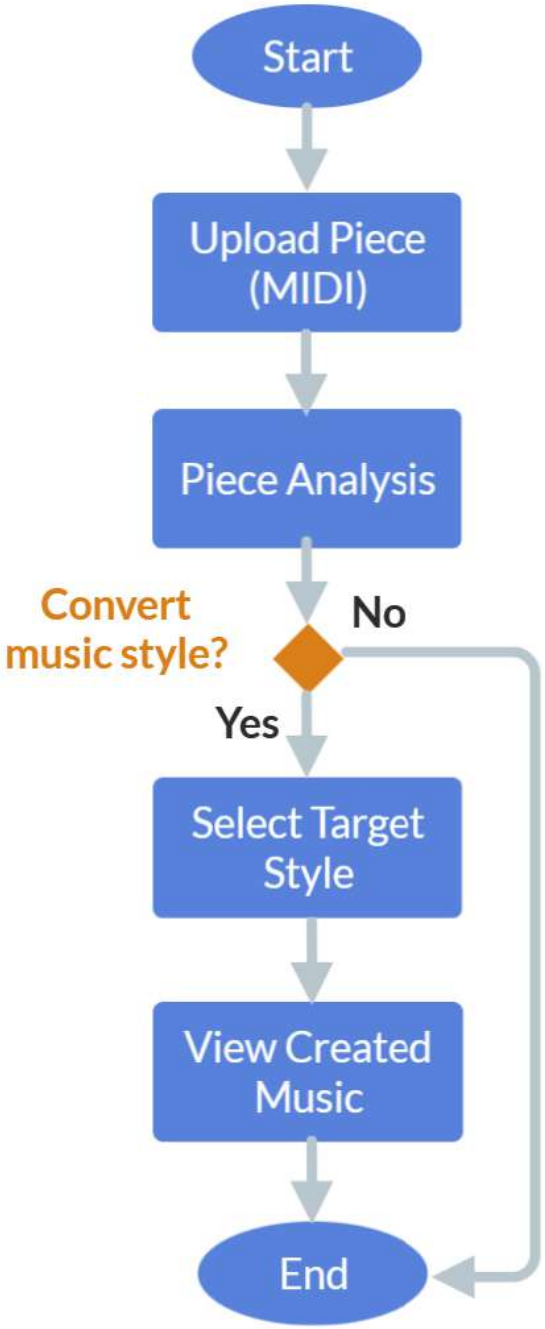


Figure 45: User flow for music piece analysis.

Similar to emotion analysis, all contents of this part are constrained to the same page for simplicity and ease of navigation. The interface of the page is shown in Figure 46.

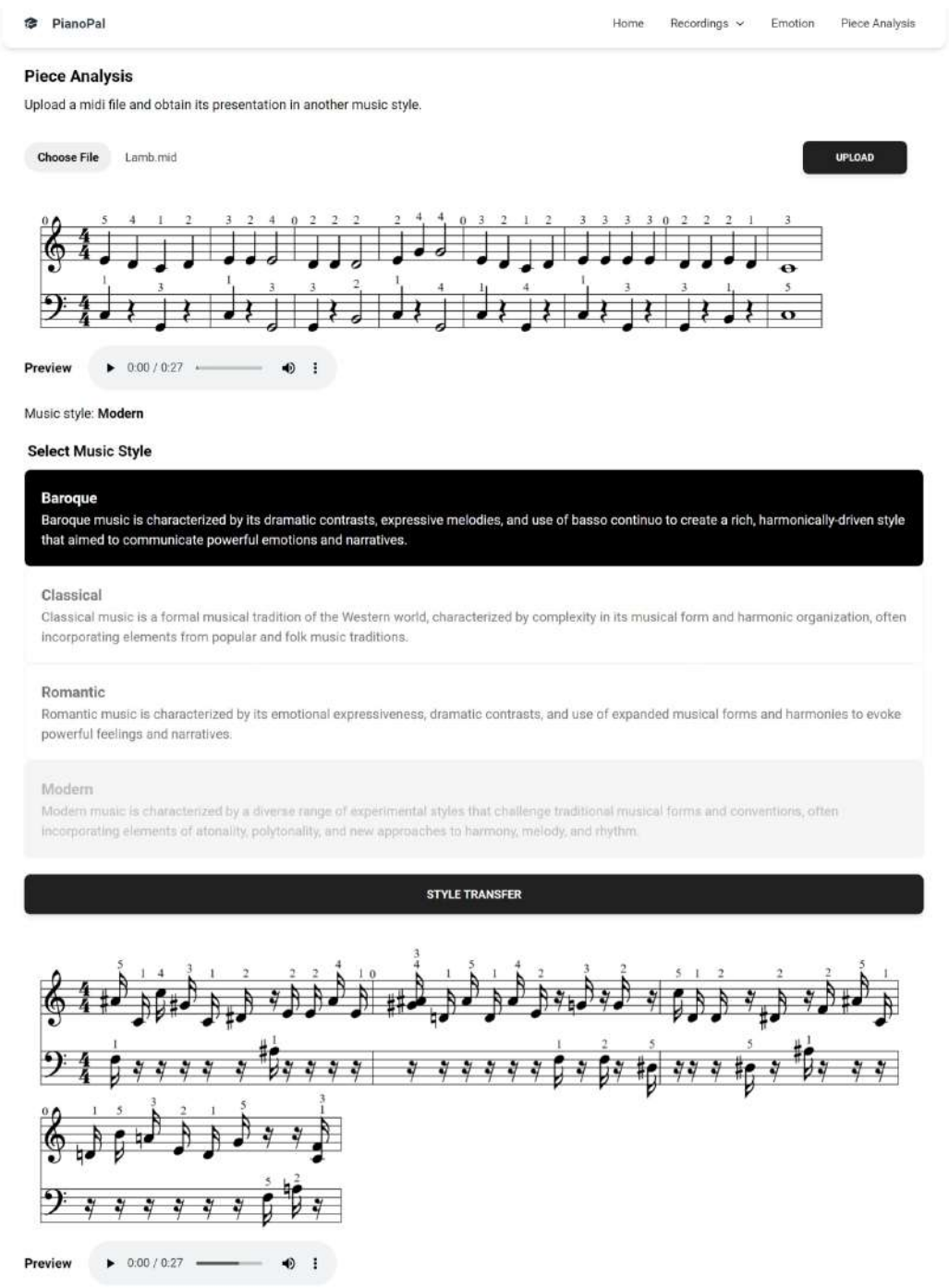


Figure 46: Interface of music piece analysis.

Initially, the page only contains the page description, file upload controls and music style descriptions with the "STYLE TRANSFER" button disabled. After the piece analysis is done, the analysis results will be presented. They include the piano score with suggested fingerings, an audio of the synthesized MIDI and which musical era the music is predicted to be in.

The menu in the centre allows users to select which music style they want the piece to be transferred to. It includes the musical era's name and a brief description of the characteristics of that era's music. It is wrapped in a set of radio buttons, so only one target style can be selected. Also, the option for the original music style would be disabled.

After the new music is generated, its piano score with suggested fingerings and synthesized audio is shown at the bottom. It allows users to preview the output music, and even try to play it themselves.

A more detailed sequence diagram of how the process is executed can be found in Figure 47.

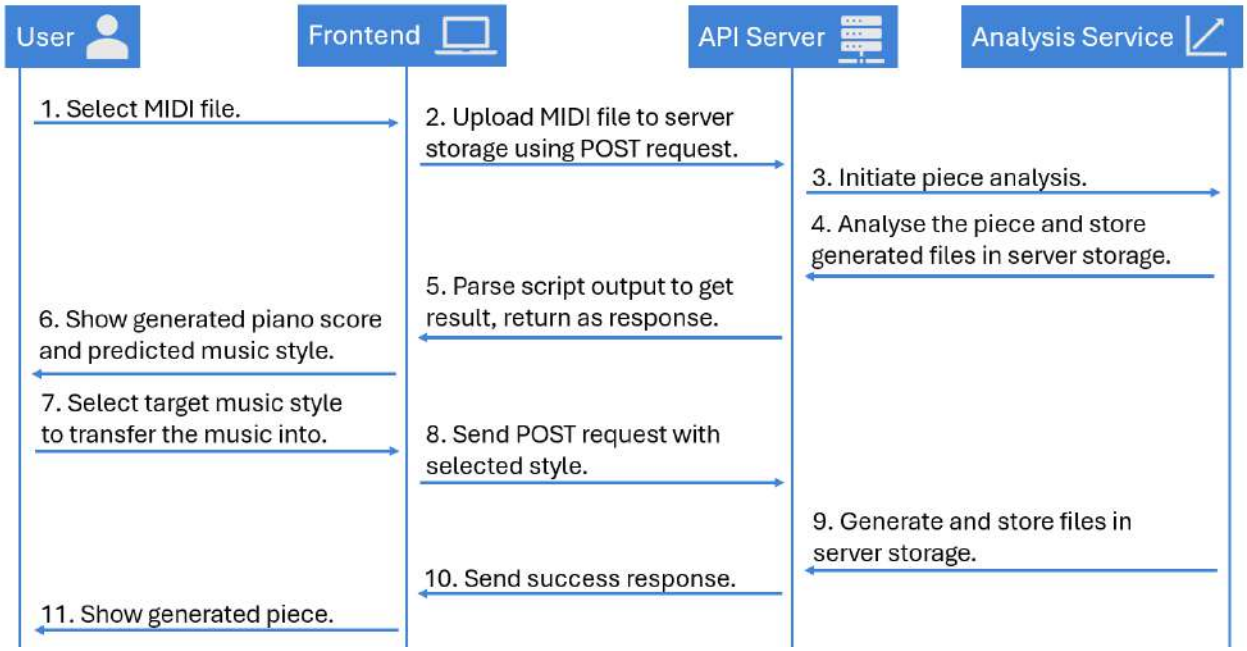


Figure 47: Sequence diagram of music piece analysis.

4 Performance Analysis

This section covers the technical details to compute the performance analysis. For the details of how the results are presented to users, please refer to Section 3. Sections 4.1 to 4.6 are mostly written in Term One, but some improvements were made in Term Two. Please refer to Sections 4.5.5 and 4.6.7 for details of the improvements made. Sections 4.7 and 4.8 are entirely written in Term Two, which covers how the results are integrated into the web application.

4.1 Function Overview

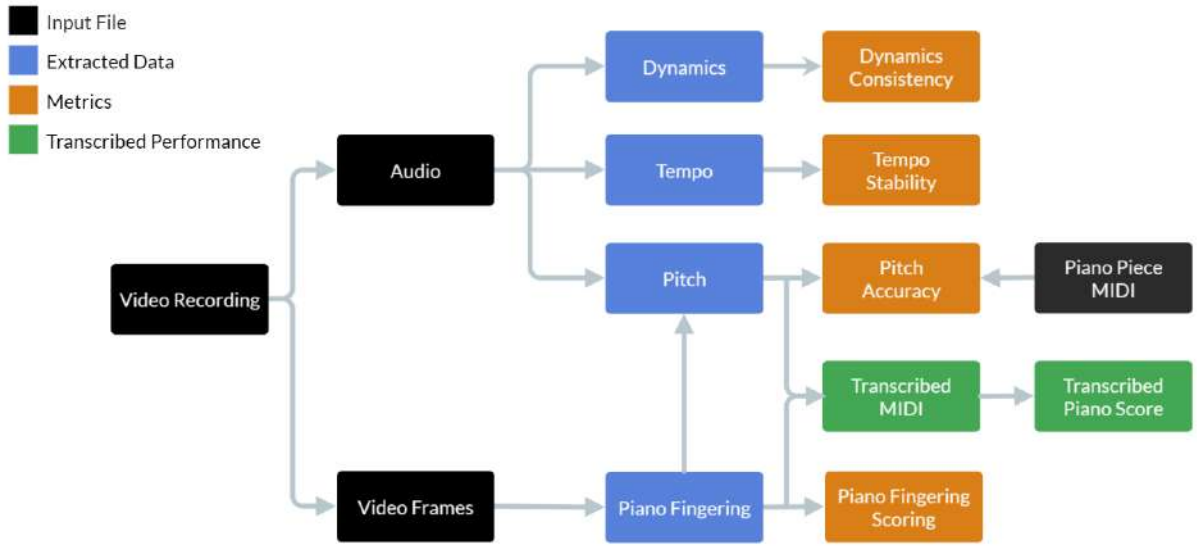


Figure 48: The proposed design of the performance analysis function.

Figure 48 shows the relationship between various components of the performance analysis function. The major input to this function would be videos of the user playing the piano. Its video frames would be used for piano fingering detection while its audio would be extracted for pitch, tempo, and dynamics information. The information extracted would be further processed to compute different metrics.

4.2 Sample Music Pieces and Recordings

Some sample music pieces are selected to support performance analysis. For each music piece, some sample performances are recorded to facilitate development and test the system's performance.

Since the samples are for evaluation but not training purposes, we have selected 10 musical pieces and recorded some of their performances in three different venues as our data.

4.2.1 Selected Pieces

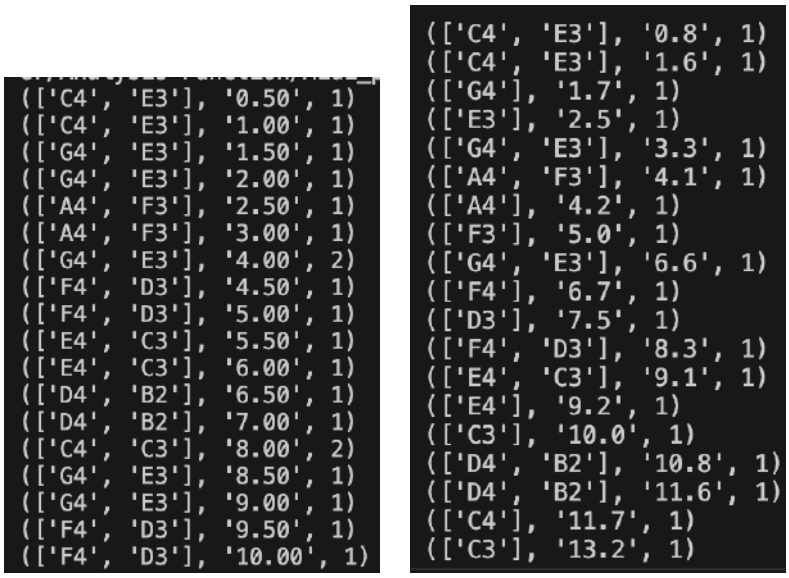
We have shortlisted 10 music pieces to be tested on in this project. Both the music score and the MIDI file of the music pieces are downloaded from MuseScore [25]. Different difficulties in playing and composing musical periods are the factors we took into consideration to evaluate the effectiveness of our proposed solution. By using the information provided in MuseScore, we can obtain basic music information about the pieces and the collective rating of the difficulty of the pieces by the users of MuseScore. The chosen pieces are listed in table 5 according to their difficulty in playing rated on MuseScore, tempo, musical period, and complexity of musical notation.

Name	Difficulty	Tempo	Period	Musical Notation
Twinkle Twinkle little star	Easy	Slow	N/A	Easy
Mary had a little lamb	Easy	Slow	N/A	Easy
Ode to Joy	Easy	Medium	Classical-Romantic	Easy
Minuet in D Minor	Easy	Medium	Baroque	Easy
Canon in D	Intermediate	Medium	Baroque	Easy
Air on the G String	Intermediate	Slow	Baroque	Medium
Für Elise	Intermediate	Fast	Romantic	Medium
Goldberg Variations Aria	Hard	Medium	Baroque	Hard
Moonlight sonata (1st movement)	Hard	Slow	Romantic	Hard
Clair de Lune	Hard	Expressive	Romantic	Hard

Table 5: Chosen music pieces for the project.

However, after arriving at the experimental and testing stage for our accuracy analysis function, we found out that using the MIDI files that were downloaded directly from MuseScore could severely affect the performance of the function for mapping MIDI data with the detected notes. The reason behind this is that we cannot obtain a correct sequence of notes on and off events from the MIDI file. The mapping between the notes on and off events with the piano score is incorrect as shown in Figure 49, hence we need to remake the whole MIDI file for every piece.

Therefore, in Term One, the choices of musical pieces were narrowed down since the aim was to generate some preliminary results to evaluate the performance of our pitch detector. In this term, the remaining pieces were also remade and all are available on the web application.



(a) Correct MIDI information

(b) MIDI information from MuseScore

Figure 49: Obstacle in extracting MIDI data.

4.2.2 Performances Recording

The input data used for the analysis would be the top-view video recorded from above the keyboard, which clearly shows the keyboard and pianist's hands. Since the main target of the project is to serve home users, unlike some other works, our system is robust enough to handle videos recorded without professional setups and cameras. An example of a simple setup is illustrated in Figure 51, in which we recorded some of our data with a mobile phone mounted on a simple rack.



(a) Top view

(b) Side view

Figure 50: Example of recorded frames.

We collected three sets of piano-playing videos in different venues with different piano keyboards and recording devices. Sample video captures are shown in Figure 6. As seen in the figures, our system can handle imperfect recordings, such as videos not including the entire piano keyboard, with the piano keyboard not perfectly aligned, and audio tracks recorded by mobile phones instead of studio mics. These allow an easy and low-cost setup, increasing the accessibility of our software.



(a) Venue 1 (b) Venue 2 (c) Venue 3

Figure 51: Recording setup example.

Venue 1 is a piano room in The Chinese University of Hong Kong. The recording was done by a mobile phone mounted similarly as in Figure 51. The piano is an electronic piano with internal speakers. We were able to collect videos with satisfactory audio.

Venue 2 is the studio of our collaborator Parsons Music. The video recording is done by a camera mounted on a tripod. It has the longest distance from the keyboard, which results in the most complete keyboard with the least distortion, at the same time, more background objects are captured. However, due to issues with the recording software, the audio collected was too noisy and distorted to be used. Therefore, we used the performance MIDI data collected from the piano to re-synthesize the video's soundtrack.

Venue 3 is in a room with a MIDI piano keyboard manually set up using a rack. It is also the setup shown in Figure 51, using a mobile phone to record the video. Since the MIDI keyboard does not have an internal speaker, and we do not have any speakers that are compatible with it, we were unable to play its sound in real time. Therefore, we also resorted to using the performance MIDI data collected from the keyboard to re-synthesize the video's soundtrack.

Venue	Recording Device	Audio
1	Mobile Phone	Recorded
2	Camera	Re-synthesized
3	Mobile Phone	Re-synthesized

Table 6: Recording venues for the project.

4.3 Dynamics Extraction

Dynamics in music refers to the sound level or in other words, the amplitude of the music playing. They are one of the expression elements that can manipulate the intensity of music performance. For instance, a louder volume of music may indicate a higher arousal level of expressions like anger, emotion, and happiness. On the other hand, a softer volume could indicate sadness, careful, and sneaky. Therefore, it is essential to extract dynamic information from the music for further musical analysis.

4.3.1 Dynamics marking

As volume change is impossible to determine without comparing to the volume in different parts of music, dynamics marking is always relative throughout the whole piece of music. In classical music notation, we use Italian terms to describe the dynamics marking. Common dynamic markings are listed in Table 7.

Name	Sign	Meaning
piano	*p*	Quiet
pianissimo	*pp*	Very Quiet
pianississimo	*ppp*	Very Very Quiet
mezzo-piano	*mp*	Moderately Quiet
mezzo-forte	*mf*	Moderately Loud
forte	*f*	Loud
fortissimo	*ff*	Very Loud
fortississimo	*fff*	Very Very Loud
crescendo	$<$	Getting Louder
diminuendo/ decrescendo	$>$	Getting Quieter

Table 7: Common dynamic markings.

4.3.2 Functionality

This function aims to extract the amplitude information from the audio such as the peak volume, the trough volume, changes in amplitude, etc. The result will be presented in graphics and dynamic markings noted with timestamps for better illustration and musical evaluation.

By applying 0.5-second windowing to the audio, we can split the original audio into equal parts of the audio with 0.5-second time length. Just like segmentation, we then calculate the peak of the absolute of the waveform for each of the segmented audio. The reason why we take a peak for the absolute value of the waveform is that a waveform can have both positive and negative values as it oscillates around the zero point. By taking the absolute values, we effectively disregard the direction of the displacement and only consider the magnitude or

absolute magnitude of the peak value. These peaks represent the max amplitude every 0.5 seconds of the audio.

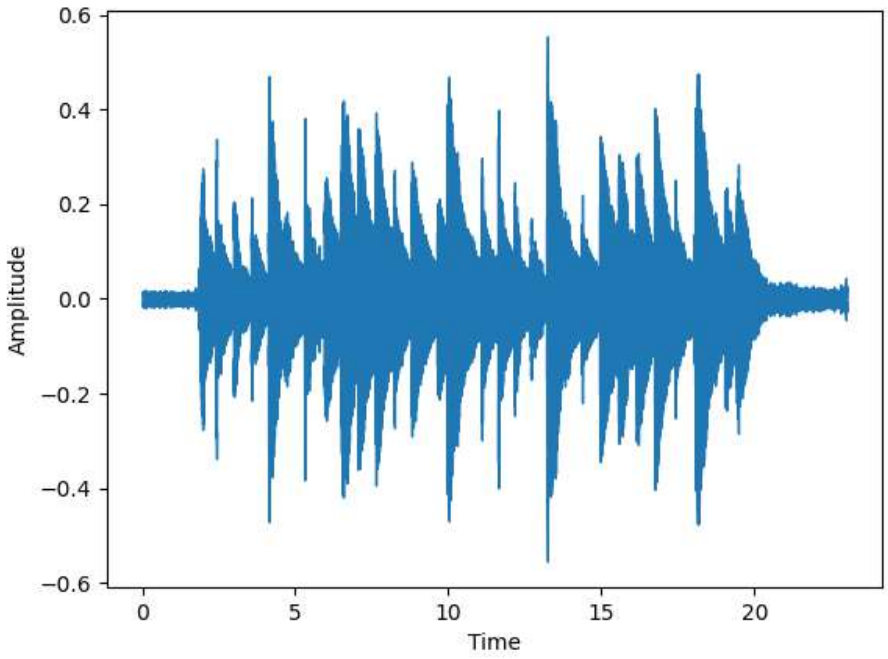


Figure 52: Sample waveform of an audio.

In order to find out the general change in dynamics, we find the peaks among the amplitude of each segmented audio such that we can find general changes in peak amplitude in different "regions" of music without considering the release at the end of a musical phrase. By plotting the peak of the amplitude, we can have a curve of changes in the amplitude which can be useful for further musical analysis in our project.

For the current progress, simple dynamic changes such as *crescendo* and *decrescendo* can be extracted by applying windowing to the audio and finding the peak amplitude of the segmented audio. By comparing the amplitude of the current window to that of the previous window, an increase or decrease in amplitude can be observed. The result of the demo is shown in Figure 53. The blue line represents the amplitude detected in each time frame while the orange line represents the general change in peak amplitude of each time frame.

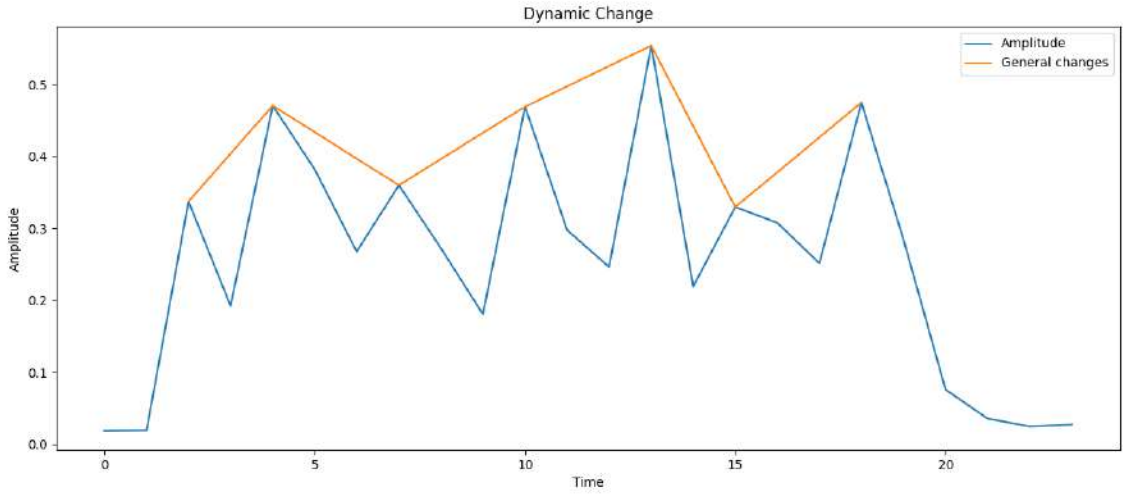


Figure 53: Dynamic change of a demo audio.

4.3.3 Summary

In conclusion, we successfully extract the basic dynamic change and general change from the audio. Our function may not be able to detect a sudden change in amplitude, or fortepiano, pianoforte, and sforzando in terms of Italian tempo markings. However, these sudden changes in amplitude and dynamics are not considered into account upon our scope of the project.

This function is also served as one of the feature extraction functions for classifying the emotion of user's playing. Mean of amplitude and stand deviation of the amplitude which representing the changes in amplitude are included as one of the feature in determining the emotion expressed in the user's input.

4.4 Piano Fingering Extraction

Piano key-finger mapping is the record of which finger is used to play which piano key. This section focuses on the automatic extraction of key-finger mapping from the visual component of the video using computer vision techniques.

4.4.1 Relevant Works

To build a more robust pitch detection model, both audio and visual information can be combined to reduce the effect of noise and suboptimal audio quality which deters the results of audio-based music transcription. Wan et al. [32, 33] proposed to eliminate irrelevant pitches by comparing detected pitches to the notes the pianist's hands are placed on, and removing notes out of the pianist's hands' range. However, their approach requires marking the piano keyboard manually and a strict limitation of lighting for hand detection by skin color.

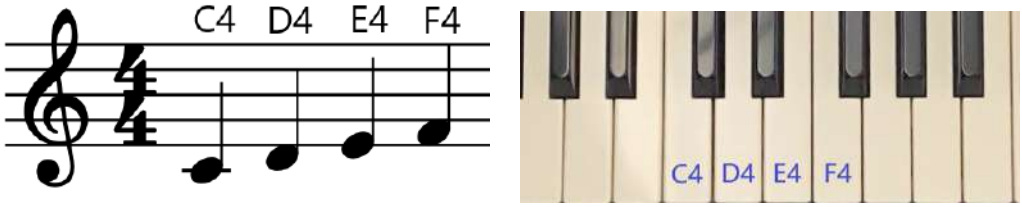
several works support automatic piano keyboard segmentation and piano key detection. Huang et al. [34] built a markerless AR piano application that can track the piano keys using a simple web camera, but still requires calibration using the keyboard size inputted by users. To cater to this problem, Goodwin and Green [35] suggested the use of Hough line transformation and binarization, which became popular in subsequent works. However, their approach uses keyboard dimension information for white piano key extraction, which may fail on non-conventional keyboards like MIDI keyboards with slightly different piano key widths. There are other works that combine image processing techniques, Hough line transform and manual rules based on piano keyboards [36, 37].

In recent years, there are also newer works based on deep learning. For example, Li et al. [38] finetuned the PSPNet [39] for piano segmentation on their custom dataset of 9443 images, which slightly out-performed the proposed CNN-SVM classification approach proposed in [40] that reached an average F1 score of 0.95 for piano key detection.

4.4.2 Piano key-finger Mapping

For example, in the sequence of notes shown in Figure 54, it is reasonable to play the four notes using the right-hand thumb(1), index(2), middle(3) and ring(4) finger respectively. This arrangement allows all fingers to be placed on the corresponding keys at once, without needing to rearrange them when playing.

On the other hand, a less ideal arrangement would be playing C4 and D4 with ring(4) and little(5) fingers, then playing E4 and F4 with thumb and index fingers. This arrangement requires the entire hand to shift right by almost an octave between two consecutive notes (D4 and E4), which requires extra time and effort.



(a) Piano score (b) Corresponding piano keys

Figure 54: Note sequence sample.

Recording the key-finger mapping allows users to review and enhance their performance by designing more efficient mappings. We also plan to build statistical models to rate and suggest better mappings in the next term. Moreover, as suggested by Wan et al. [32, 33], the placement of fingers is applied to filter out irrelevant pitches detected from the audio for more accurate music transcription (more in section 4.6).

4.4.3 Image Processing Techniques

This section introduces some image processing techniques that are applied to the piano segmentation and piano key detection steps. For ease of implementation, related functions in OpenCV [21] are used in our application. For better illustration, sample images are created with Microsoft Paint and the results of applying the image processing techniques to them are shown in Figures 55 and 56.

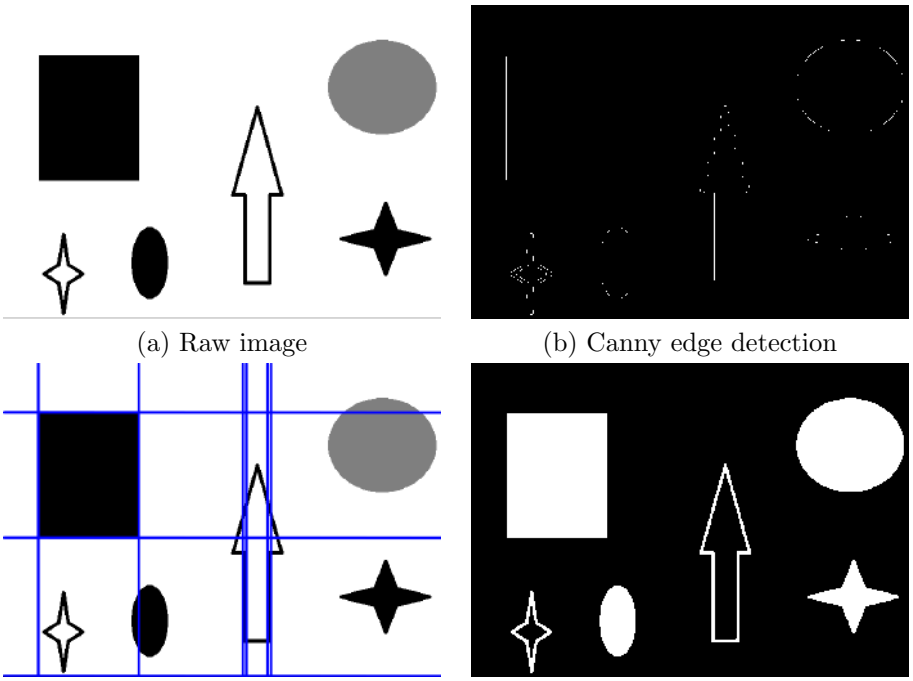
Canny Edge Detection Edge detection is a process that extracts the edges of objects in an image. The Canny edge detection algorithm is a multi-stage algorithm with steps shown as follows:

1. Reduce noise with Gaussian filter.
2. Apply gradient-based kernels (e.g. Sobel kernel) to find the intensity gradient.
3. Apply non-maximum suppression to remove non-edge pixels.
4. Apply double thresholding to extract strong and weak edges.
5. Perform edge tracking by hysteresis (extract strong edges and weak edges connected to strong edges).

Compared to methods only applying gradient-based kernels (e.g. Sobel edge detection), Canny edge detection has a lower error rate and is less susceptible to noise. Therefore, it is adapted in our work.

Hough Line Transformation The Hough line transformation is an algorithm for detecting straight lines in images. It converts edge points into a parameter space called the Hough space, where each point represents a straight line in the original image. For a point in the Hough space with the accumulated count over an arbitrary threshold, its corresponding line in the image is considered significant and will be extracted. The extracted lines represent long but not necessarily continuous lines in the image.

Binary Thresholding It works by thresholding the pixel value (0 to 255) in a grayscale image. For example, in Figure 55(d), the threshold value is set as 127, and only pixels with a value lower than 127 (i.e. darker) are extracted and shown in white in the resulting image. It is a convenient method to filter out dark and light pixels, which are useful for the extraction of black and white piano keys.



(a) Raw image (b) Canny edge detection

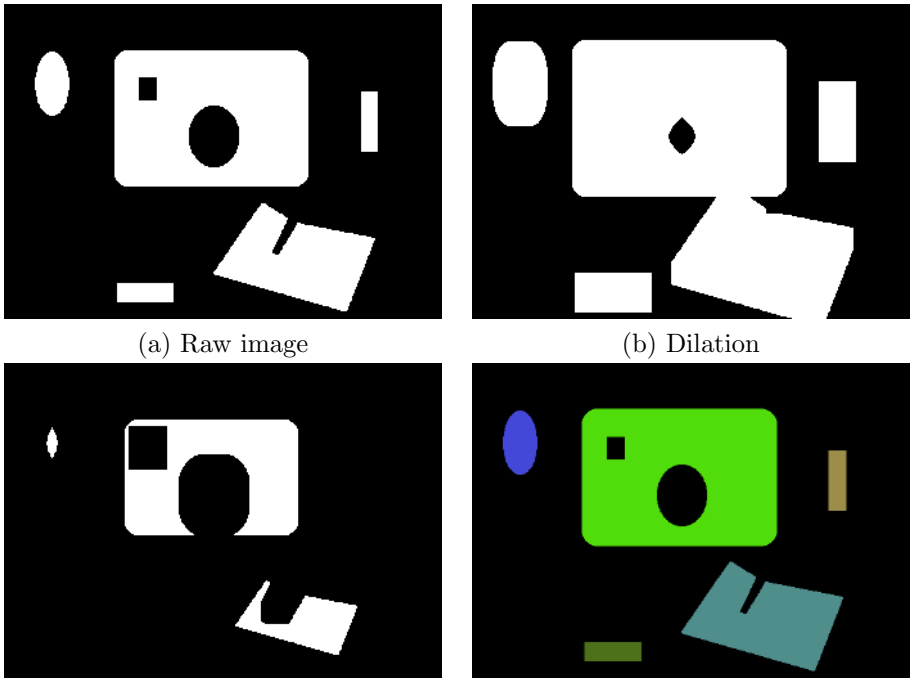
(c) Hough line transformation (d) Binary thresholding

Figure 55: Results of applying image processing techniques.

Dilation and Erosion They are morphological operations. Dilation adds pixels to the boundaries of objects in an image, thus filling in holes and gaps within an object. Whereas erosion removes pixels on object boundaries, thus eliminating noise and small objects. The exact number of pixels added or removed (i.e. how "strong" the effect is) is determined by the size of the structuring element used for the operation.

Connected Components Extraction Connected components in an image refer to continuous groups of pixels that are of the same color and intensity. For example, there are five connected components in Figure 56(d). The labelling of connected components is often

done using a depth-first search algorithm that exhausts all neighbouring pixels of the current group until no more neighbouring pixels of the same pixel value are found.



(a) Raw image (b) Dilation

(c) Erosion (d) Connected components extraction

Figure 56: Results of applying image processing techniques.

4.4.4 Overall Methodology

This function takes the visual component of the piano performance video and extracts which piano key each finger is on throughout the whole video. The function's overall flow is shown in Figure 57.



Figure 57: Overall flowchart for piano key-finger mapping.

4.4.5 Piano Segmentation

This module aims to extract the piano keyboard from video frames which may include background scenes or objects and are not horizontally oriented. In the application, users are required to select a frame without the pianist's hand on the keyboard, which would be

used as the basis for piano segmentation and piano key detection. The flowchart of keyboard segmentation is shown in Figure 58.



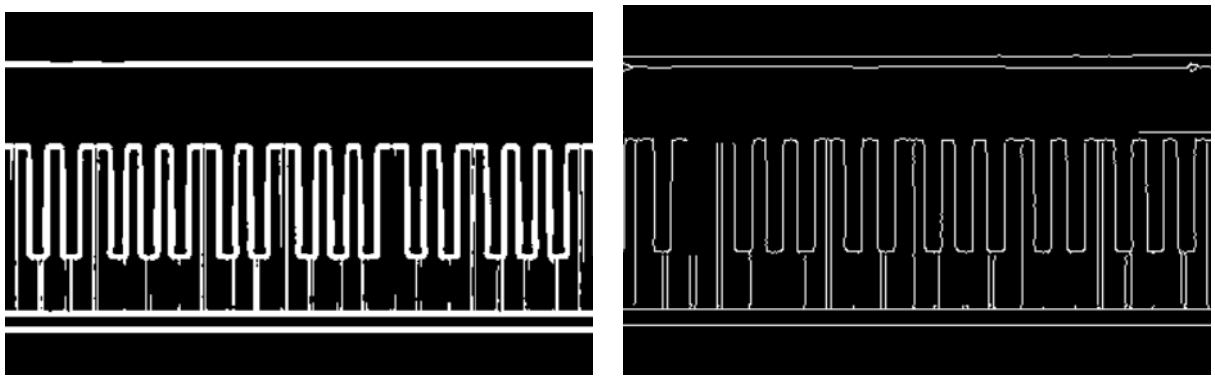
Figure 58: Flowchart for keyboard segmentation.

The main idea is to extract the keyboard by locating its top and bottom boundaries using a combination of image processing techniques and knowledge of the piano keyboard. While deep learning-based methods have been increasingly popular, they are not chosen due to the lack of public piano keyboard segmentation datasets and higher computation requirements.

Edge Detection Edge detection is performed by passing the captured image through a Canny edge detector followed by a denoising filter to remove small connected components.

Suteparuk used the Sobel edge operator to extract the edges of the piano keyboard [37]. However, the edges generated are thick and contain much noise, such as those around the vertical lines in Figure 59(a). These noises are hard to remove and affect the performance of the Hough transform.

Therefore, the Canny edge operator is used. It produces cleaner lines which is more desired since the aim is to extract only the top and bottom edges of the keyboard, it is fine to lose smaller edges such as those of individual keys. A comparison of the results of the two operators is shown in Figure 59.

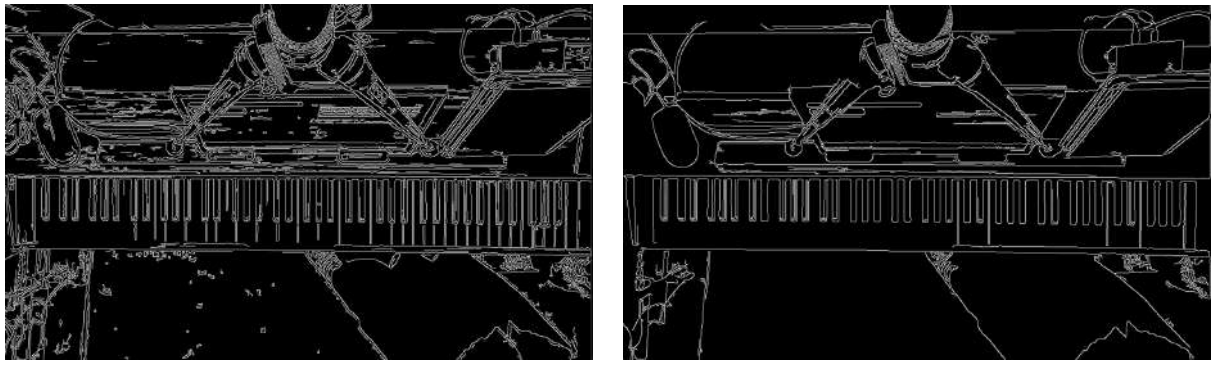


(a) Sobel edge operator (b) Canny edge operator

Figure 59: Comparison of Sobel and Canny edge operators.

In order to reduce the amount of noise, a denoising procedure is applied to the edge image. All connected components with an area smaller than 10 would be removed. As shown in

Figure 60, it successfully removes much noise from the background environment, which makes the image cleaner and improves the performance of the Hough transformation.

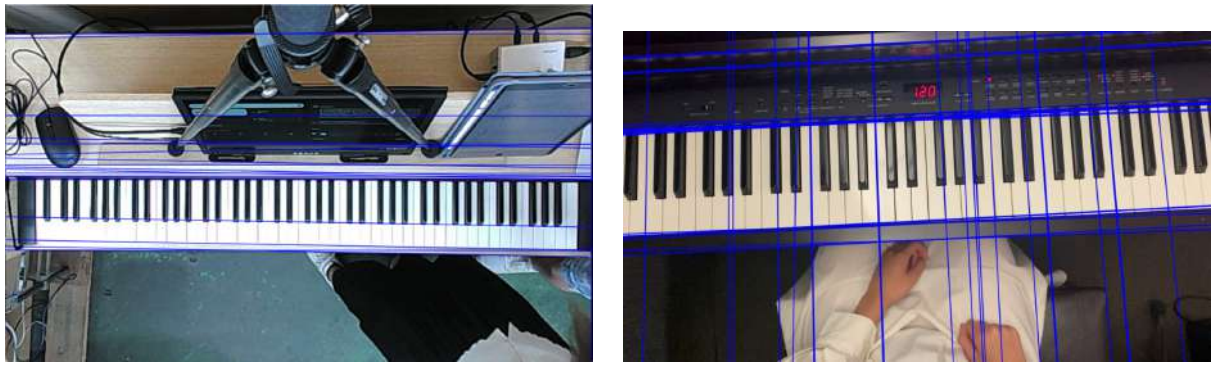


(a) Before denoising (b) After denoising

Figure 60: Edge detection result before and after applying denoising.

Hough line transformation After obtaining the edge image, Hough line transformation is then applied to detect straight lines which are potentially the top and bottom boundaries of the keyboard.

Under the assumption that the piano keyboard is the main object in the image and the keyboard boundary is clearly shown in the frame, the longest lines should be parallel to the keyboard (either the top and bottom boundaries of the keyboard, or the horizontal edges of other parts of the piano). The results of the Hough line transformation in two different venues we recorded are shown in Figure 61.



(a) (b)

Figure 61: Results of Hough line transformation in two different venues.

However, it is notable that if the color of the keyboard's background is similar to the keyboard's color, the edge detection operator may fail to extract the edge of the keyboard. Consequently, the keyboard boundary cannot be extracted in Hough line transformation. An example is Figure 61(a), in which the keyboard's bottom is not accurately extracted since its background color is light.

Orientation Correction To allow easier processing in subsequent steps, the image would be rotated such that the keyboard becomes horizontally aligned with the image. As shown above, most Hough lines detected are parallel to the piano keyboard. Therefore, the orientation is calculated as the dominating angle among the Hough lines. The image is then rotated accordingly.

Cadidate Boundary Extraction Afterwards, all Hough lines that are approximately parallel to the keyboard's horizontal orientation are selected. Each line's orientation is calculated and compared to the keyboard's orientation, if the difference is lower than one degree, the line will be selected. These selected lines are the candidate keyboard's top and bottom boundary.

Keyboard Boundary Selection All pairs of selected are then exhausted to check if they satisfy an arbitrary set of rules we designed based on the piano keyboard's structure. Pixels between the two lines (i.e. a long horizontal strip) are extracted for checking, this strip is the potential keyboard. To ensure that only the keyboard is checked, but not other parts of the piano or background, only the middle 50% of the strip is cropped out and checked against the rules below:

1. The strip's height is greater than 20% of the image height.
2. In the bottom 30% of the keyboard, over 90% of the pixels should be white.
3. In the top 60% of the keyboard, over 30% of the pixels should be black.
4. In the top 60% of the keyboard, over 40% of the pixels should be white.

Rule 1 ensures that the strip's height is large enough to be the keyboard, which should be a significant object in the image. Meanwhile, rules 2-4 are designed based on the typical arrangement of black and white keys in a piano keyboard.

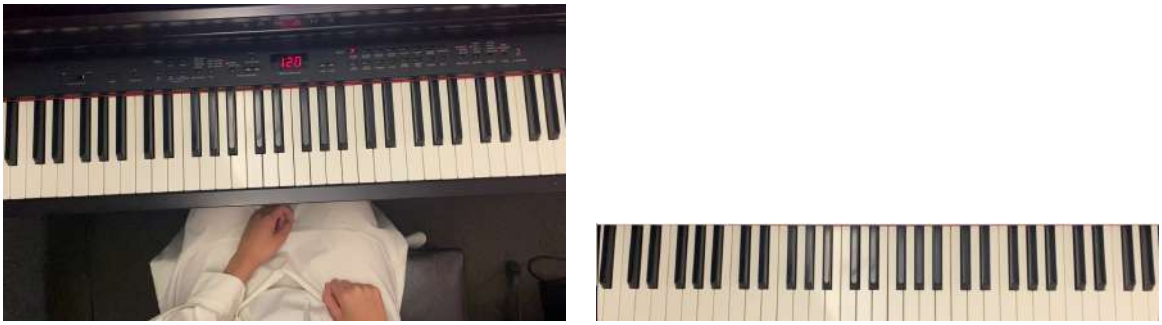
Binary thresholding is applied to isolate the black and white pixels in the image, which supposedly correspond to the black and white keys. Other color filtering methods such as HSV filtering and adaptive thresholding were tested too, but binary thresholding worked the best in this case.

The first pair of lines satisfying all rules would be considered the top and bottom keyboard boundary. The keyboard will then be cropped out based on these two lines and passed to the piano key detection module. Figure 62 shows a keyboard image extracted by the algorithm.



Figure 62: Extracted keyboard.

To conclude, this keyboard segmentation module takes a top-view input frame from the performance video and extracts the piano keyboard with a corrected orientation, as shown in Figure 63.



(a) Input frame (b) Extracted keyboard

Figure 63: Input frame and the corresponding keyboard extracted.

4.4.6 Piano Key Extraction

This module takes the keyboard extracted from the previous section as the input and labels the pixels of the piano keys in the original image. The overall procedure used to extract the notes is shown in Figure 64.



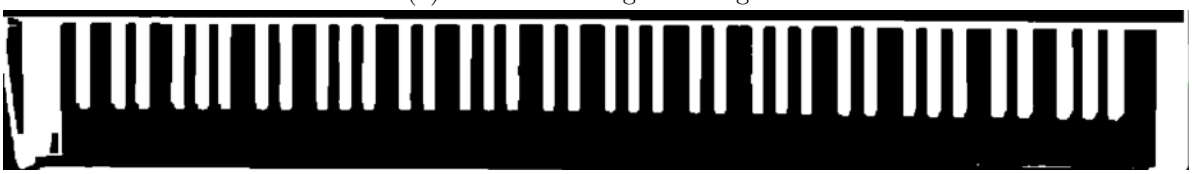
Figure 64: Flowchart for piano key extraction.

The main idea is to first extract the black keys since they are usually more clearly visible than white keys amid sub-optimal lighting. They then act as anchors for marking the piano key and the location of each white piano key.

Filter Black Region Binary thresholding is first applied to the keyboard image to extract the black region only (white area in Figure 65). However, it is notable that there are some holes near the bottom of the black keys, which may be caused by light reflection. Therefore, an extra step of dilation followed by erosion is applied to fill up the gaps.



(a) After extracting black region



(b) After filling up gaps

Figure 65: Input frame and the corresponding keyboard extracted.

Extract Connected Component Afterwards, connected component extraction is applied to extract individual black piano key candidates. Since the top and bottom edges of the keyboard may be included in the image (e.g. the white bar at the top of Figure 65(b)), the top and bottom 20% of the image are removed.

Also, the dark gaps between the white keys may be extracted from the previous steps as well. Therefore, an extra step of erosion followed by dilation is applied to remove these kinds of thin and small noises. All connected components in the resulting binary image are then extracted as the potential black keys.

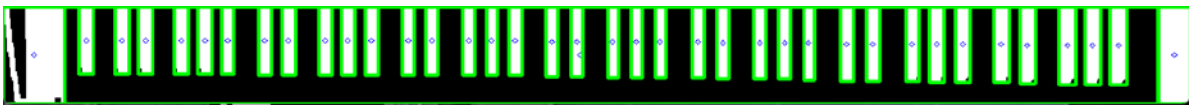


Figure 66: Extracted connected components.

Extract Black Keys From Figure 66, it is notable that some connected components highlighted are not black keys, such as the edges at the two sides. Irrelevant components are then removed by comparing their width and bottom height with the remaining components. The remaining ones are labelled as black keys.

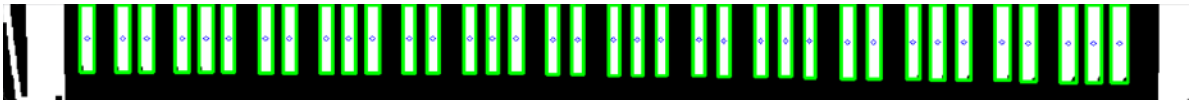


Figure 67: Filtered connected components.

All pairs of C# and D# notes (two consecutive black keys) are extracted, and the pair closest to the centre is labelled as C#4 and D#4 (same octave as "middle C"). The notes of the remaining black keys are then computed subsequently based on their horizontal order.



Figure 68: Extracted black keys.

Extract White Keys The boundaries of white keys are then computed based on the midpoints of adjacent black keys. Their notes are also deduced based on the notes of the black keys. While the detected notes are not visible in this report due to image size limitation, the notes of all detected keys bounded in orange or blue boxes are correctly recognised.

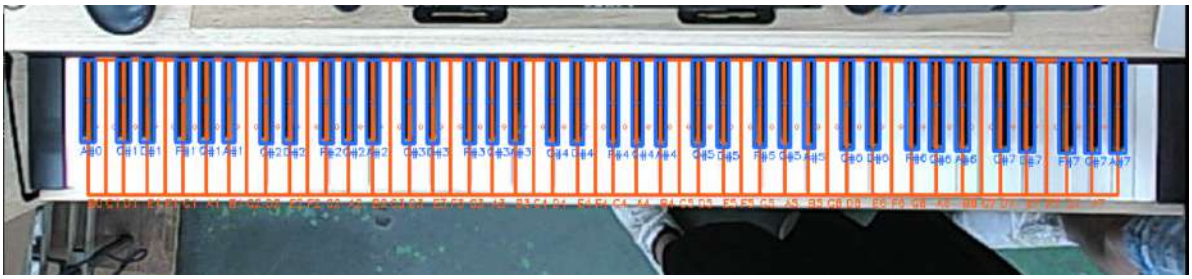


Figure 69: The results of notes detection.

Map Keys to Frame Finally, the piano key location and note detected are mapped back to the original frame before all the cropping and rotation for hand tracking. A 2D array with the same dimension as the image is built, where each grid stores the corresponding keynote of the piano key the pixel is in (e.g. "C4", "G#6" or NULL). The final result is shown in Figure 70.

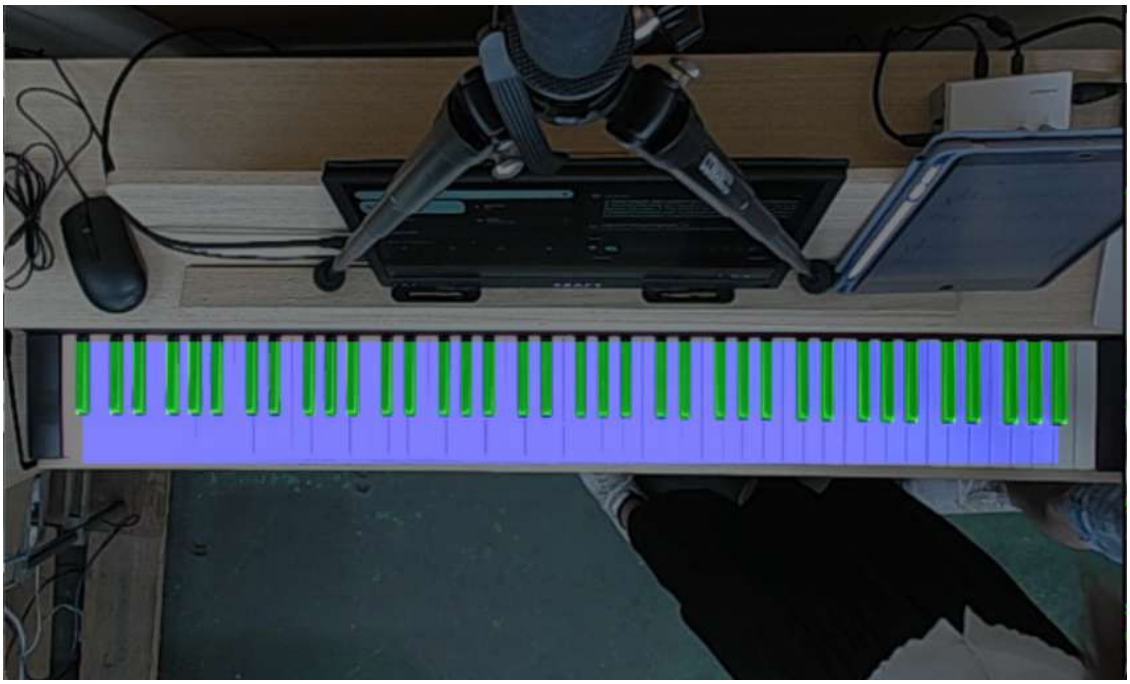


Figure 70: Note segmentation results on the input image.

4.4.7 Hand Tracking

In order to achieve a higher accuracy under a sufficiently short processing time, pre-trained models are applied to extract the coordinates of the landmarks of the pianist's hands. The hand detector of CVZone [41] was applied, which accurately tracks the finger joints of both hands.

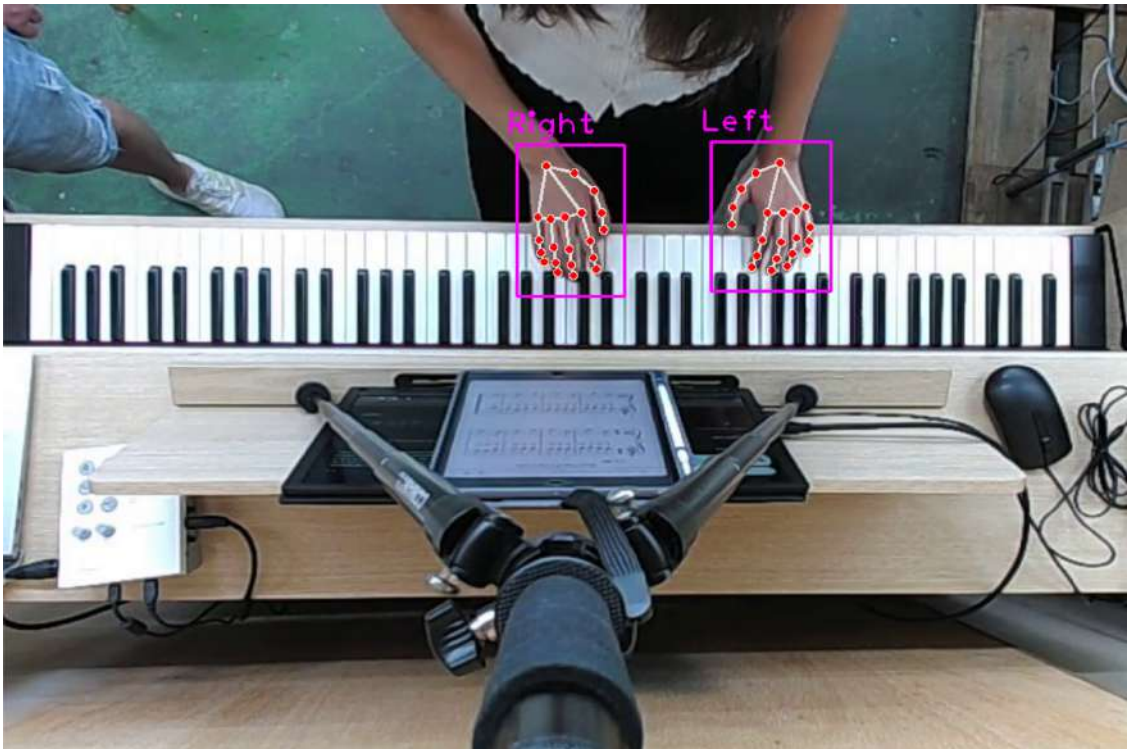


Figure 71: Hand tracking result.

The hand tracker is used to capture the coordinates of all fingertips at each video frame, which will be passed to the piano key-finger mapping module.

4.4.8 Piano key-finger Mapping

This module merges the fingertip coordinates obtained from hand tracking and the piano key coordinates obtained from the piano key extraction module. Under the assumption that the camera remains fixed and stable, the set of piano key locations will be applied throughout the whole video. Also, since a pianist normally only uses his/her fingertips to press piano keys, only the fingertip coordinates are concerned.

For each frame, the coordinate of each fingertip is rounded off and then checked against the array recording each pixel's corresponding keynote. This result (blue words in Figure 72), together with the tracked hand's skeleton overlaid on the original video, is outputted to a video to be loaded for playback in the GUI.

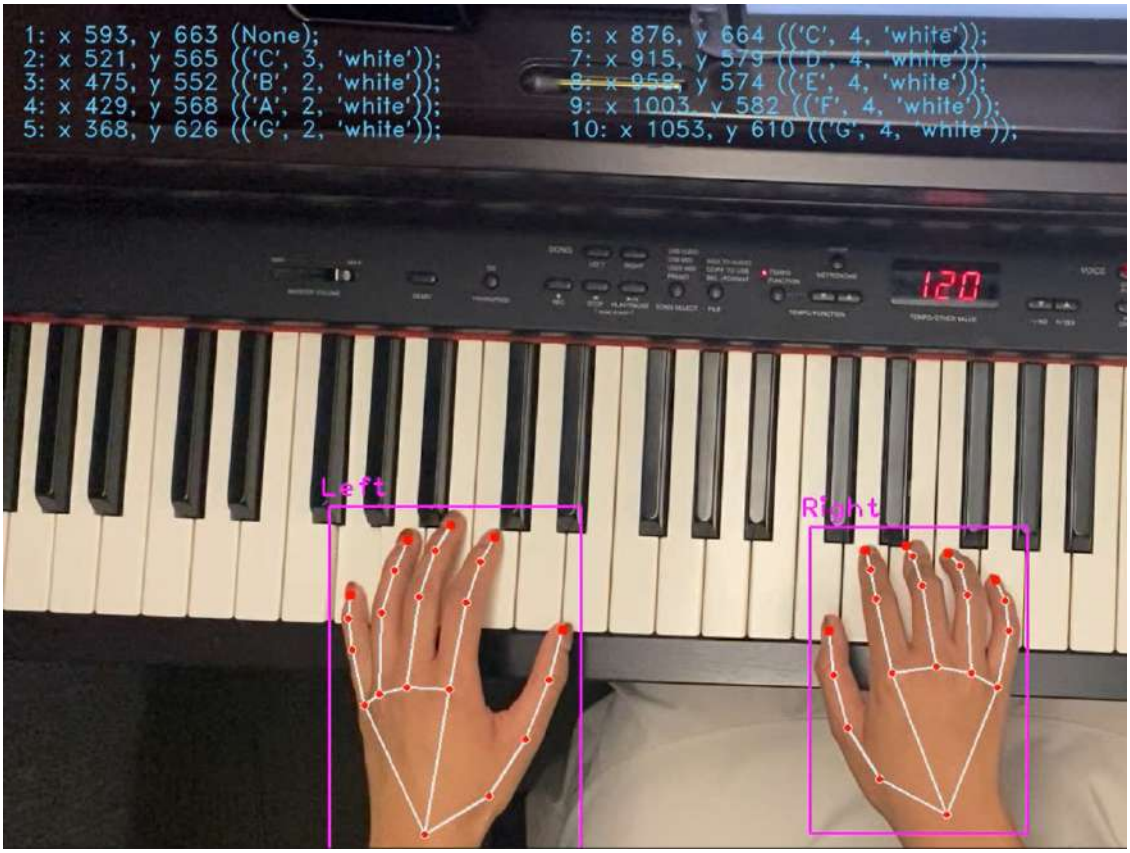


Figure 72: Sample result of key-finger mapping.

Moreover, since the audio analysis uses the number of seconds elapsed instead of the number of frames to time and run the predictions, another array is generated with its entries in the format of `(timestamp (in seconds), [list of piano keys with fingertips present on it])` and stored in a text file. The entries are recorded approximately every 0.1 seconds, where the timestamp is calculated by:

$$\text{Timestamp} = \frac{\text{Number of frames elapsed}}{\text{Frame per second}}$$

4.4.9 Piano Key Detection Results

$$\text{Accuracy} = \frac{\text{Number of correctly extracted keys}}{\text{Total number of keys}} * 100\%$$

The detection accuracy of the piano image captured from videos recorded in the three venues is measured and compared based on the formula above. Since the images captured from all videos recorded in the same venue are nearly identical, only one image is randomly selected from each venue.

The results are shown in Figure 73, Figure 74 and Figure 75. It is seen that the accuracy is very high for all our setups.

Venue	#Correctly Extracted Keys	#Keys in Total	Accuracy
1	66	66	100%
2	85	88	96.59%
3	49	50	98%

Table 8: The accuracy of piano key detection.

Also, it is observed that the missed piano keys are mainly the boundary white keys, whose location cannot be deduced since they are not surrounded by two black keys. Therefore, a possible way to fix it is to deduce the size of the boundary white keys based on their adjacent white keys.

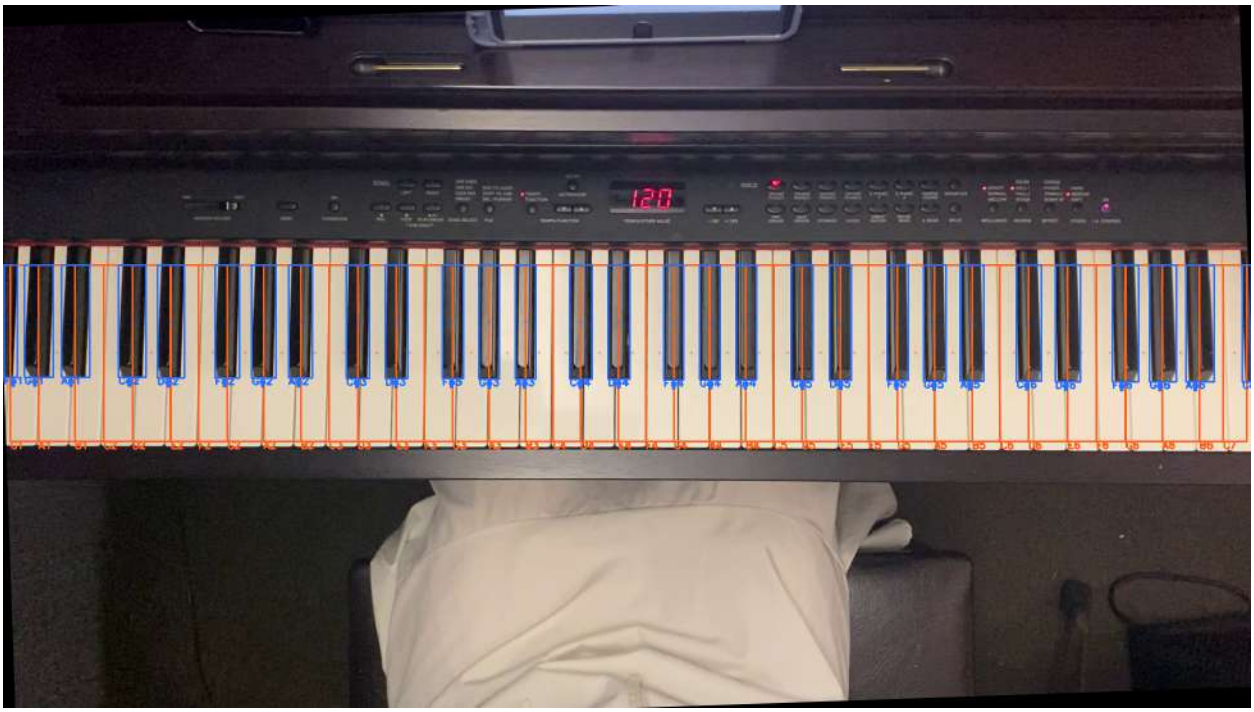


Figure 73: Result of key detection on a frame taken in venue 1.

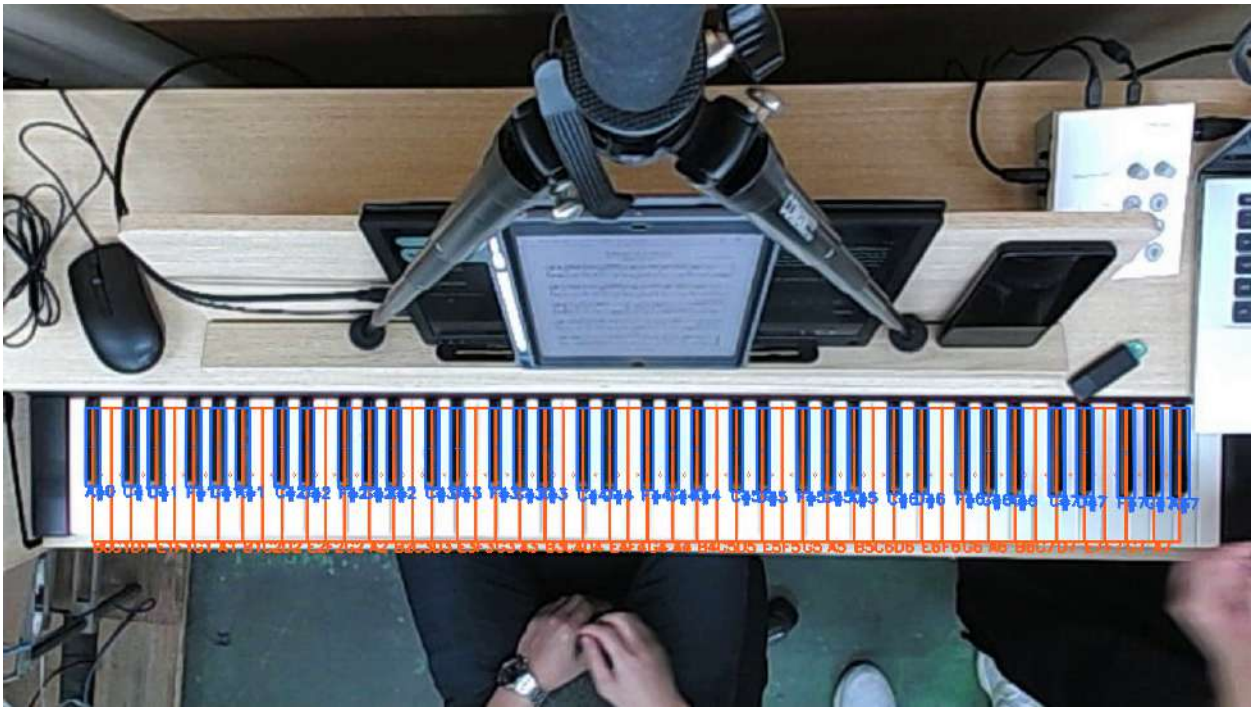


Figure 74: Result of key detection on a frame taken in venue 2.



Figure 75: Result of key detection on a frame taken in venue 3.

4.4.10 Key-Finger Mapping Results

It is observed that the hand-tracking module is very accurate, so the mapping is basically correct as long as the piano keys are correctly extracted in the previous steps. Figures 76 to 79 show the detected mappings under finger placements more complicated than usual. Yet still, all mappings are detected correctly even under more complicated placements.

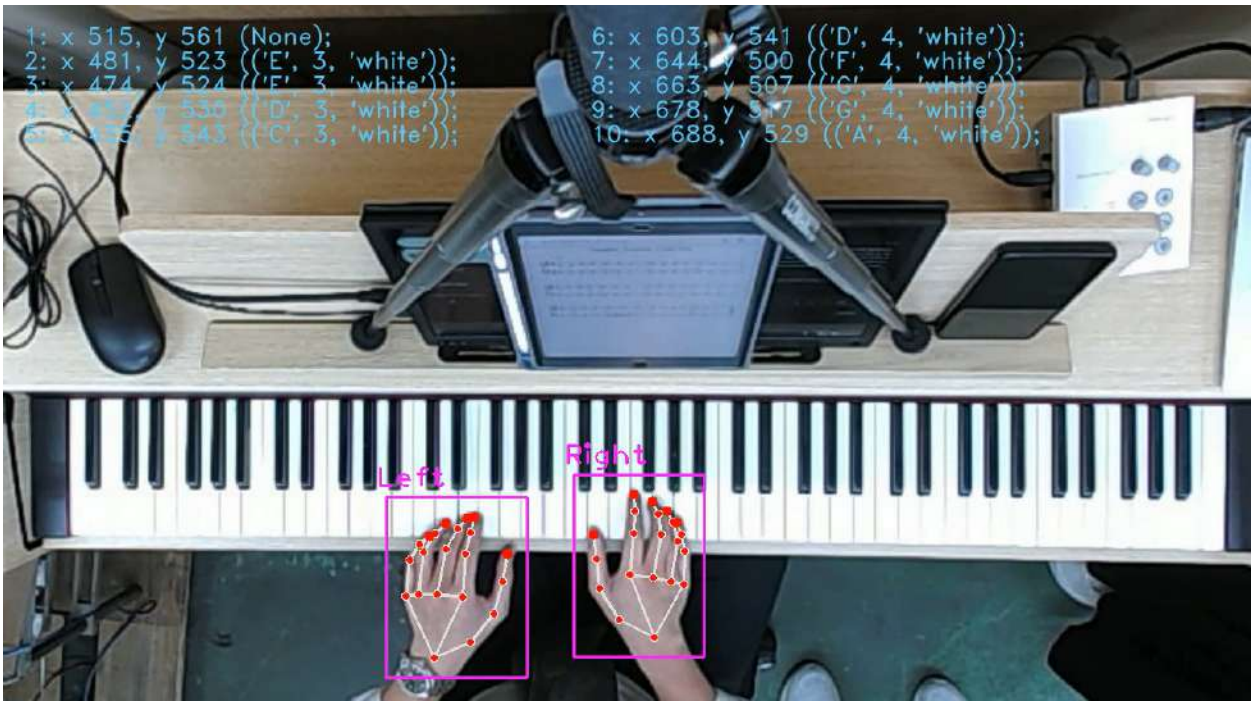


Figure 76: The left thumb is right out of the keyboard.

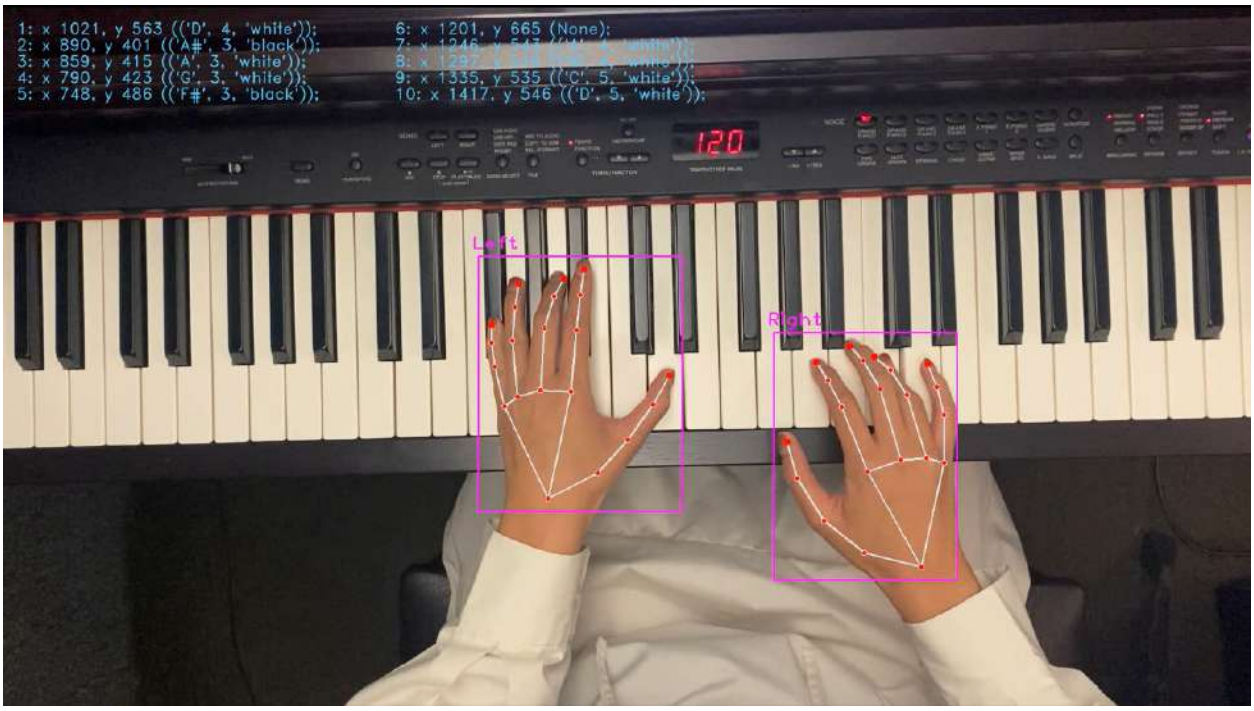


Figure 77: The left-hand fingers are located on or between consecutive black keys.

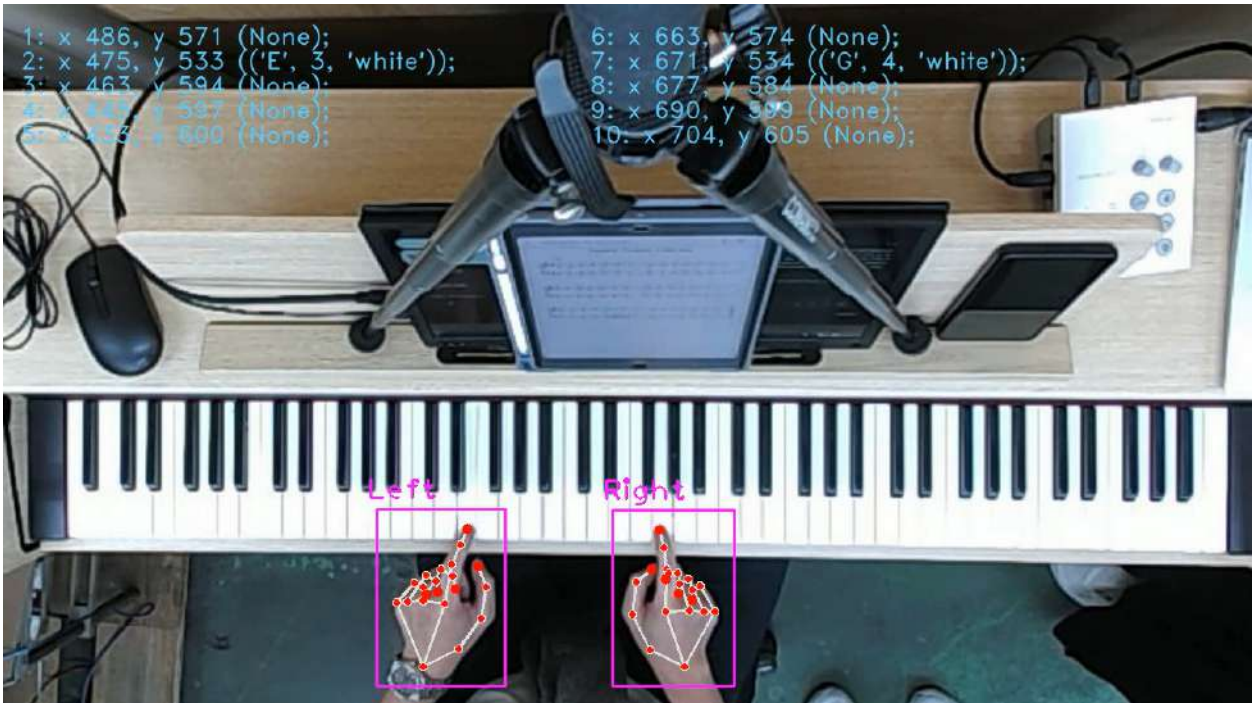


Figure 78: Only the index(2) fingers are on the keyboard, some remaining fingers are not visible.

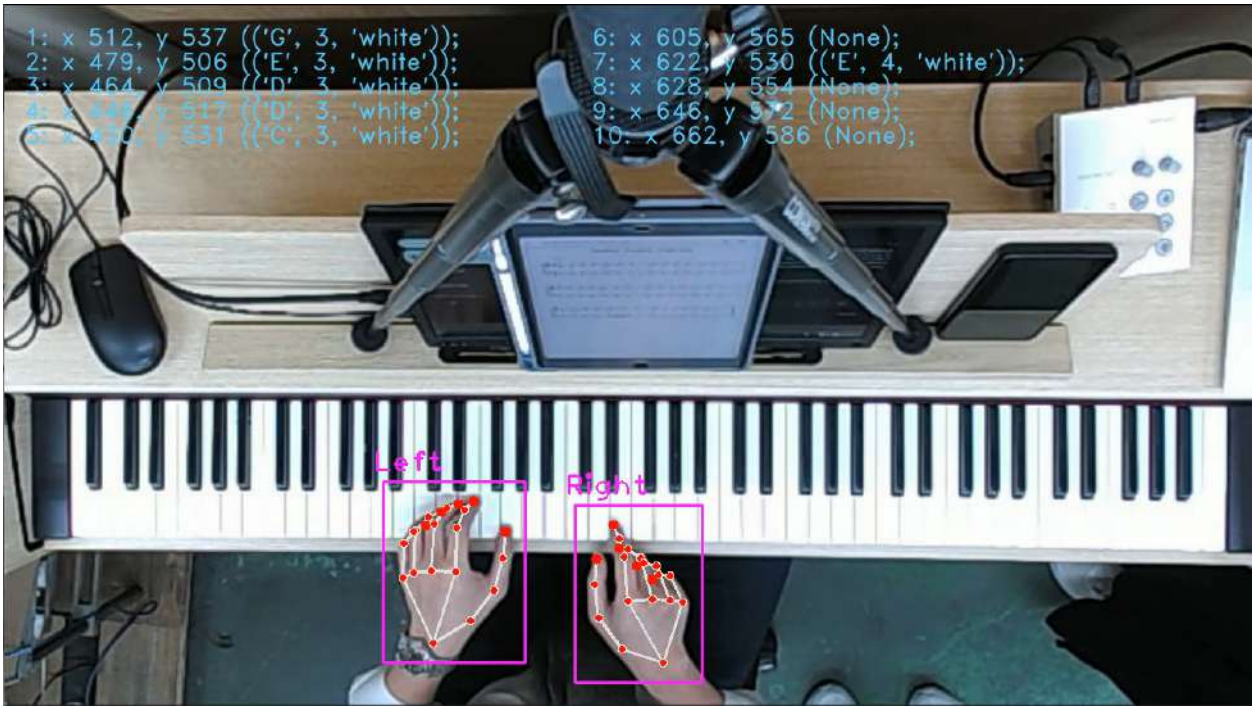


Figure 79: Four left-hand fingers are placed on three consecutive white keys, with only the index(2) finger used in the right hand.

4.4.11 Summary and Further Development

We have successfully extracted most piano keys and mapped them to the fingertip coordinates correctly using the top-view video. There are some directions to further improve the system's performance to work on in the future.

First, although the system can cater to rotations to different degrees in the captured video, it does not handle a tilted camera well. When the camera is tilted, the keyboard is no longer a rectangle (as shown in Figure 80), so the top and bottom edges of the keyboard are no longer parallel. This would seriously hinder the performance of the keyboard extraction step. To handle this, a perspective warp can be applied. However, it would either require users to manually enter the keyboard range, or employ more advanced keyboard extraction methods like machine learning.

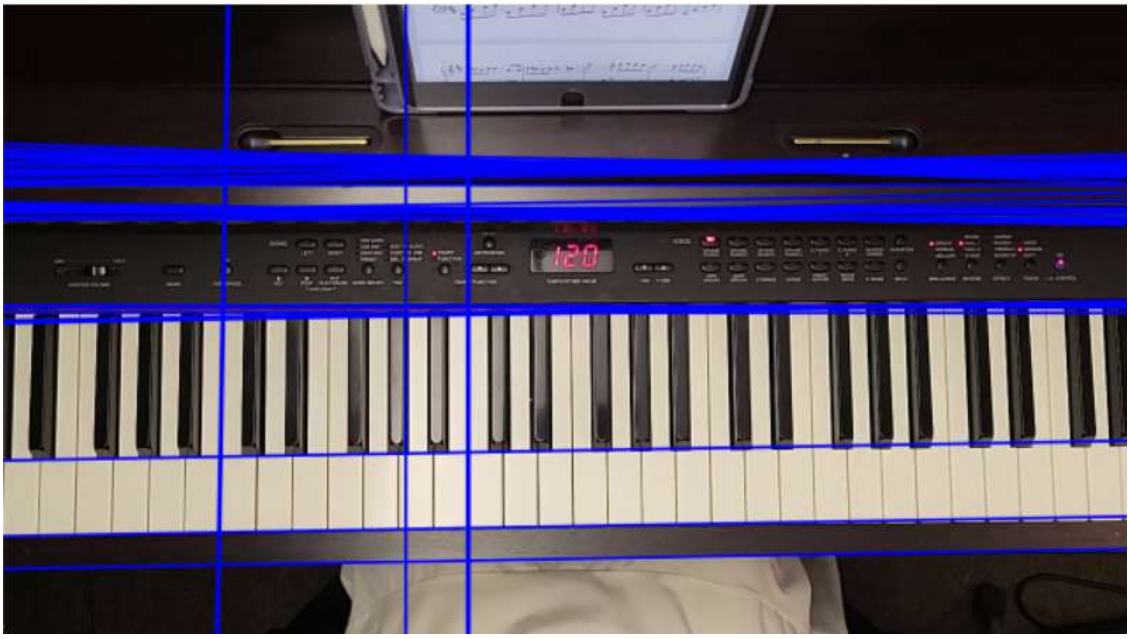


Figure 80: The Hough lines detected in a tilted piano image.

Secondly, while the current use of binary thresholding is able to extract the black and white keys in all our test cases, it is not guaranteed that they can succeed in more extreme lighting (e.g. very dim, or the use of intense coloured lights). A possible solution is to use an adaptive threshold instead of an absolute value for the binary thresholding, which requires calibration or some computations based on the video's luminosity and hue.

4.5 Tempo Detection

Tempo of music basically refers to the speed of music playing which is an essential tool that musicians use to express their feelings and interpretation of the music. Moreover, tempo and rhythm are one of the evaluation metrics of musical performance and can further be used in different musical analyses. Therefore, tempo detection has played an important role in our project.

4.5.1 Representation of tempo

The representation of tempo can basically be defined into three formats, namely Beat per Minute (BPM), time signature, and Italian terminology for tempo markings.

Beat per Minute indicates the number of beat within a minute. For instance, 120 BPM refers to the speed of 120 beats of sound being played within a minute, or in other interpretations, 2 beats are being played within a second. The formula for calculating the BPM for a piece of music is:

$$\text{BPM} = \frac{\text{Total number of beats}}{\text{Length of the music in minutes}}$$

Note value is the length of time that a musician sounds a particular pitch. There are five common note values in music which are the whole note, the half note, the quarter note, the eighth note, and the sixteenth note. In a music bar that contains 4 beats, a whole note represents a 4 beat playing of a pitch, a half note represents a 2 beats playing of a pitch, a quarter note represents a 1 beat playing of a pitch, an eighth note represents a 1/2 beat playing of a pitch, and a sixteenth note represents a 1/4 beat playing of a pitch.

Name	Sign	Meaning
Whole Note	𝅝	4 beats
Half Note	𝅗𝅥	2 beats
Quarter Note	♩	1 beat
Eighth Note	♪	1/2 beat
Sixteenth Note	𝅘𝅥𝅯	1/4 beat

Table 9: Common note value.

Time signatures serve as a universal language for rhythm. They are represented by two numbers stacked vertically at the beginning of a score. The top number refers to the number of beats in each music bar while the bottom number refers to the note value for each beat in the bar. For instance, a 4/4 time signature represents the music bars in the piece are the

4-beat bar and each beat has a note value of the quarter note. In our project, we need to extract the time signature value to find out how many notes are contained in one bar to map the detected notes with a bar value.



Figure 81: A 4/4 time signature.

For many years, Italian has been the language in describing the musical notations in classical music. In terms of tempo, Italian words convey tempo change and description through specific information about the speed of music playing. These tempo markings share a common ground between classical musicians and often can be represented by a range of BPM. Some common tempo marking has been listed in Table 10. By referencing the range of BPM of these musical notation, we can find out the optimal BPM of playing for evaluation and further analysis.

Name	Meaning	Corresponding BPM
Larghissimo	very, very slow, almost droning	slower than 20 BPM
Grave	slow and solemn	20–40 BPM
Lento	slowly	40–60 BPM
Largo	the most commonly indicated "slow" tempo	40–60 BPM
Larghetto	rather broadly, and still quite slow	60–66 BPM
Adagio	at ease tempo	66–76 BPM
Adagietto	rather slow	70–80 BPM
Andante	at a walking pace	76–108 BPM
Moderato	moderately	108–120 BPM
Allegro moderato	moderately quick	112–124 BPM
Allegro	lively, cheerful	120–168 BPM
Vivace	lively and fast	168-176 BPM
Presto	very fast	168–200 BPM
Prestissimo	extremely fast	faster than 200 BPM

Table 10: Common Italian tempo marking.

4.5.2 Onset detection

A note onset always goes along with a sudden increase in the signal's energy level. Let $x : \mathbb{Z} \to \mathbb{R}$ be a discrete-time signal. Moreover, let $w : [-M : M] \to \mathbb{R}$ for some $M \in \mathbb{N}$ be a bell-shaped window function centered at time zero. By transforming the signal into local energy $E_w^x(n) : \mathbb{Z} \to \mathbb{R}$ by this function:

$$E_w^x(n) := \sum_{m=-M}^{M} |x(n+m)w(m)|^2 = \sum_{m\in\mathbb{Z}} |x(m)w(m-n)|^2$$

We can take a derivative of the local energy function to find the energy change. In the discrete case, the easiest way to realize such a derivative is to take the difference between two subsequent energy values. Furthermore, since we are interested in energy increases (and not decreases), we keep only the positive differences while setting the negative differences to zero. Altogether, we obtain an energy novelty function:

$$\Delta_{Energy}(n) := |E_w^x(n+1) - E_w^x(n)|_{\geq 0}$$

By applying the peak extraction function to the novelty function, we can find the sudden change in signal energy level. In fact, these peaks are the onset of audio which is very useful in different functions of our project such as mapping the piano key pressing information with the pitch information and finding note value for each pitch which will be discussed in the later session.

4.5.3 Predominant local pulse

The task of tempo and beat tracking has been a challenging one, especially for those highly expressive performances in which the assumption of local periodicity is often violated [42]. Therefore, the detection of local pulse using predominant local pulse (PLP) was proposed to reduce the impact of the subjective notion of tempo and beats. Intuitively, the PLP function can be regarded as a pulse tracker that can adjust to continuous and sudden changes in tempo as long as the underlying novelty function possesses local periodic patterns.

The principle behind this function is out of the scope of discussion in our project, but in short, we first compute a Fourier tempogram from the complex Fourier coefficient of the signal. Then, by obtaining the phase information, we can define an optimal windowed sinusoid for the signal. With the application of the overlap-add technique, we can compute the PLP function and detect the peaks of the function to obtain the beat of audio.

However, the performance of the PLP function is dependent on the window length for the optimal windowed sinusoid. Using longer windows, one obtains more accurate and robust tempo estimates but loses temporal flexibility. In contrast, using shorter windows increases the temporal resolution, but makes the tempo estimates more vulnerable.

4.5.4 Functionality

This function calculates the local pulse of the audio using the Predominant local pulse (PLP) function. Moreover, it will extract the onset information from the audio which is essential for further musical analysis such as pitch detection, Emotion classifier, etc.

We observed that there is some noise at the beginning and the end of the audio such as noise from the chair, background, etc., which could lead to false detection of onset and local pulse. Therefore, by doing some preprocessing to the audio, we can increase the accuracy of tempo detection and analysis. After trimming away the part of the beginning and the end of the audio in which their top decibel is higher than 10dB, we can reduce the unnecessary noise.

After the preprocessing stage, we compute the onset envelope that basically calculates the short-term spectral energy or the magnitude of the audio signal over time. These techniques aim to capture the energy or intensity variations in the audio signal, which are often associated with the occurrence of onsets.

After finding the onset envelope, we conduct onset detection which returns the sample position of onset in the audio. However, the result of onset detection may not be very accurate in our first few trials which multiple onsets have been detected in a very short time.

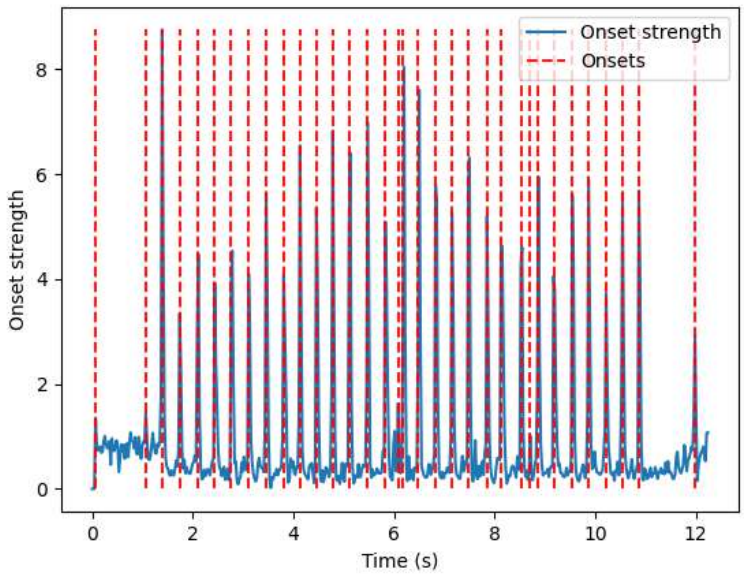


Figure 82: Multiple onset detected in a very short time.

Therefore, we have implemented some optimization in peak picking by setting some arguments such as the number of samples to wait after picking a peak and threshold offset for

mean. As a result, the above problem of detection of multiple onsets in a very short can be tackled and get a better result of onset detection. Then we can present the result by plotting the onset position along with a mel-spectrogram as shown in Figure 83.

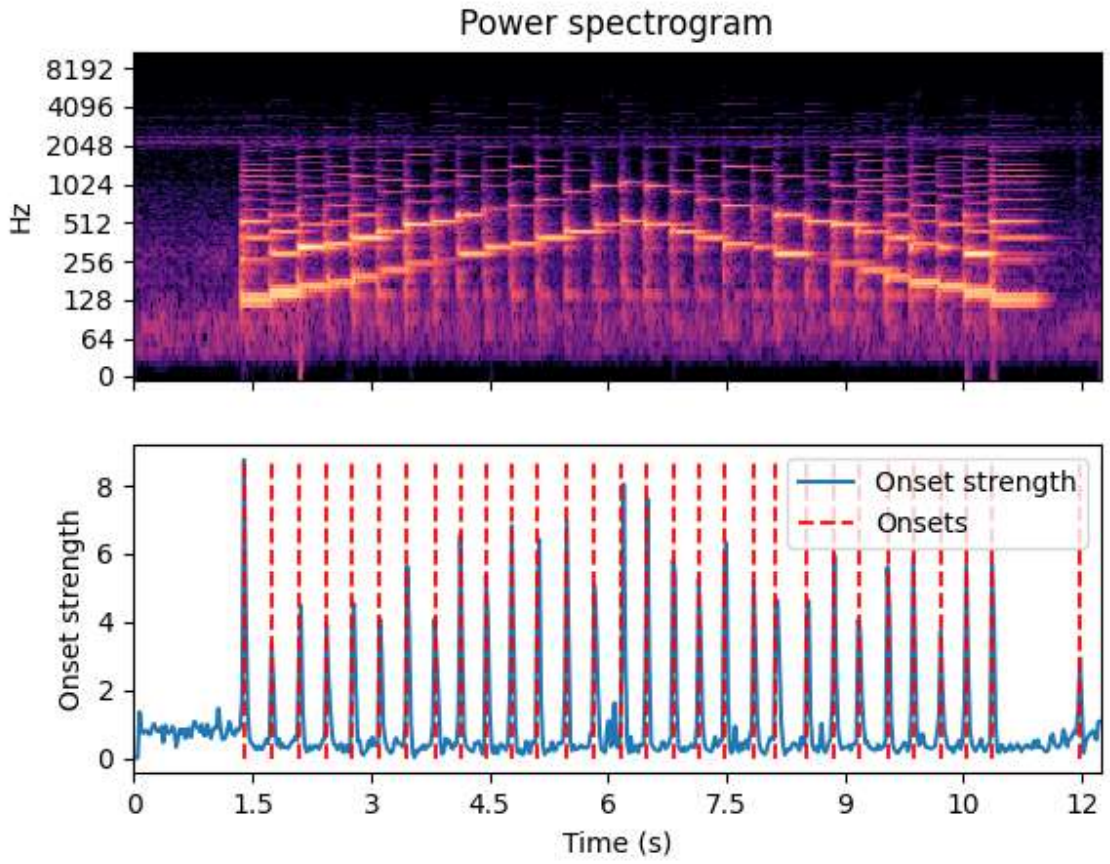


Figure 83: Demonstration of onset detection result.

Apart from the onset detection, we can also compute the local pulse detected by the PLP function by inputting the onset envelope into the function. We then compute the local maxima of the pulse curve as these local maxima can be taken as estimated beat positions in the audio. Then we present the result by plotting the beat position against the pulse curve and time in the x-axis as shown in Figure 84.

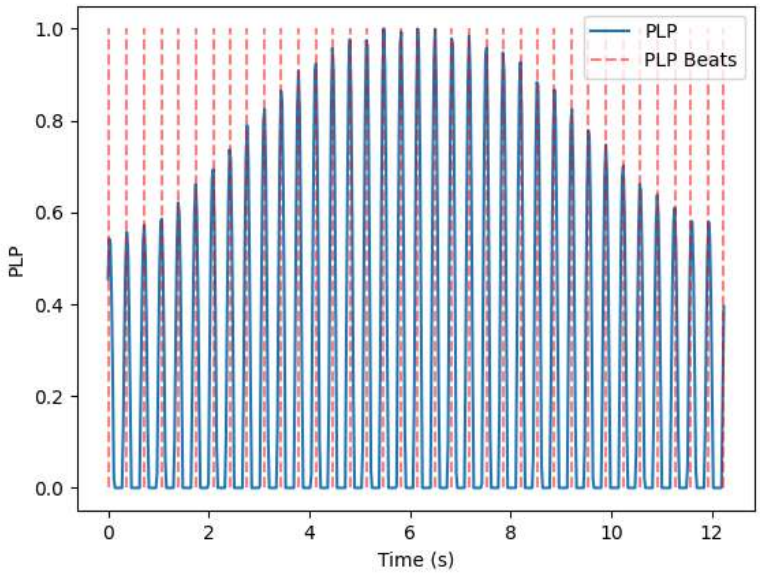
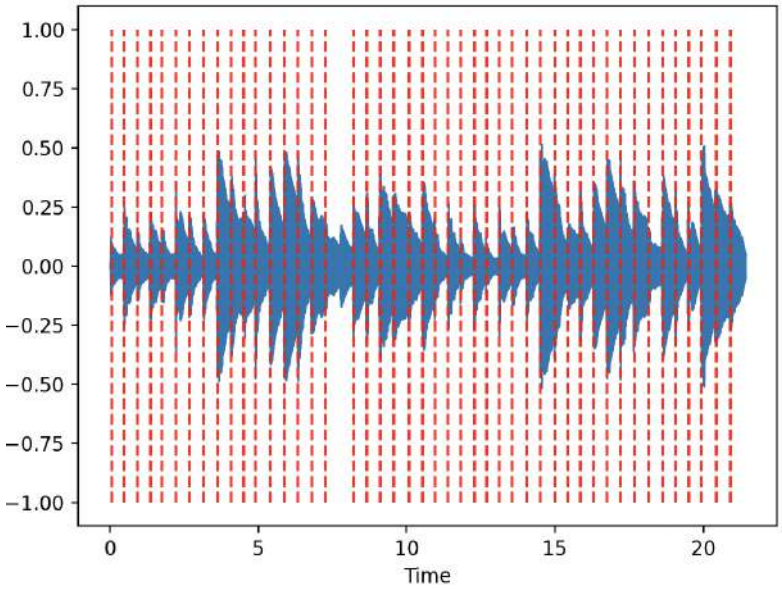


Figure 84: Sample graph for PLP function.

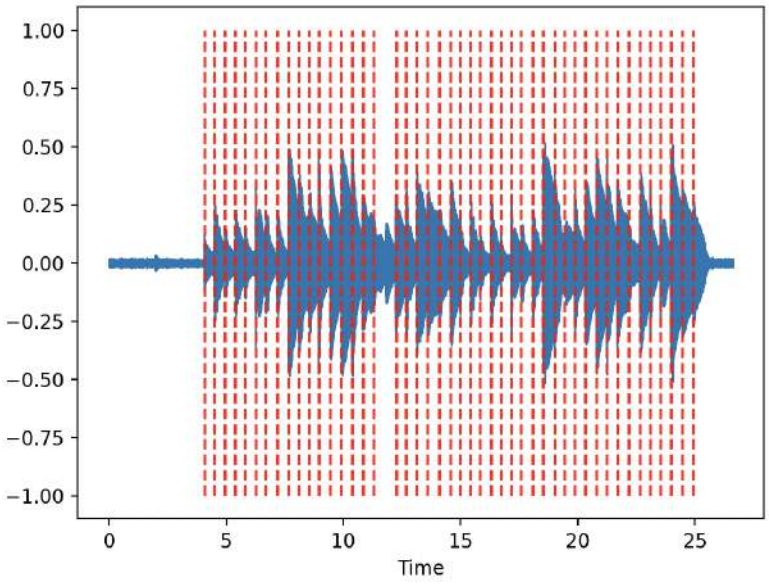
4.5.5 Improvements in Term 2

In the first term, we are defining a customized threshold for each of the inputs that are used for peak picking which prevents multiple onsets detected in a very short time. However, this method may take much more time to cherry-pick an optimized threshold that is used for peak picking. Therefore, in order to cater to the difference in threshold offset for mean, we have implemented a for-loop to find a threshold that the best result is obtained by comparing it with the length of the target piece MIDI file. By sacrificing a very short computation time, we avoid the problem of setting the customized threshold for each input.

Moreover, as the audio is trimmed in the preprocessing stage, the result is unfavorable for displaying the result or plotting the onset detection result eventually. The timestamps are not matched with the original audio and hence we have restored the audio in the post-processing stage of onset detection and now we can plot an accuracy result on the original audio.



(a) Before post-processing.



(b) After post-processing.

Figure 85: Onset detection result with post-processing.

Apart from the above developments, we have implemented a new function for detecting the local/ dynamic tempo throughout the whole audio. By using the onset detection function with librosa, we could only obtain the global BPM of the audio. However, even if we tried to calculate the local BPM by splitting the audio into frames, the result fluctuates as there is insufficient onset information within one frame. If we enlarge the frame size, there will be insufficient local BPM information for displaying and analysis. Therefore, eventually, we came up with an approach that ensures sufficient onset information for calculating the local BPM while preserving an amount of dynamic tempo data for displaying. That is designing

overlapping windows for the calculation of the local BPM. By default, the frame duration is 6 seconds and local BPM will be calculated for every 0.5 seconds. The graph is then output to the website for displaying purposes.

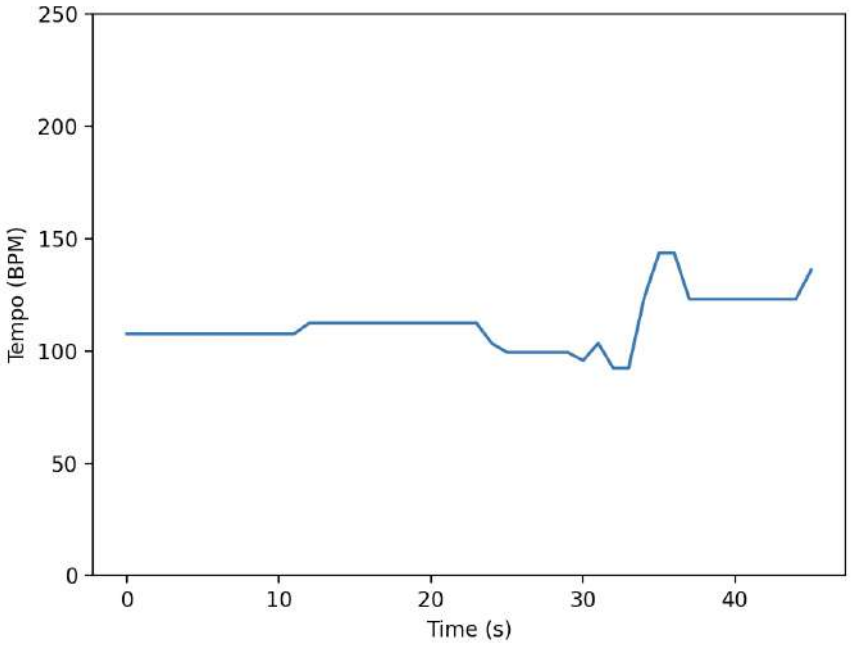


Figure 86: Output for dynamic tempo.

4.5.6 Summary

This function is aimed at providing basic tempo information extracted from the user's playing. By exploiting the function of onset detection and PLP detection, we could obtain tempo changes and beat information for further analysis and mapping in the pitch detection function or as the data for advanced model training. After further optimization of the function, the function is more user-friendly in the way that we are not required to customize the threshold in onset detection. In order to facilitate better tempo detection result displaying to the user, we have post-processed the result to map with the original audio before the trimming process as stated in the pre-processing stage. Moreover, local/ dynamic tempo detection is implemented to provide more tempo information for analysis and display purposes in the web application section.

4.6 Pitch Detection

This function extracts the pitch and time of each note played, and compares it to the MIDI score by mapping the pitch collected and that of the MIDI score to determine the correctness of the notes played. The traditional pitch extraction methods which solely depend on the audio are prone to noises, significantly lowering its accuracy, or resulting in many false detections. To tackle this problem, we plan to use a robust approach combining the visual and audio information extracted from the performance video. The proposed approach is shown in Figure 87.

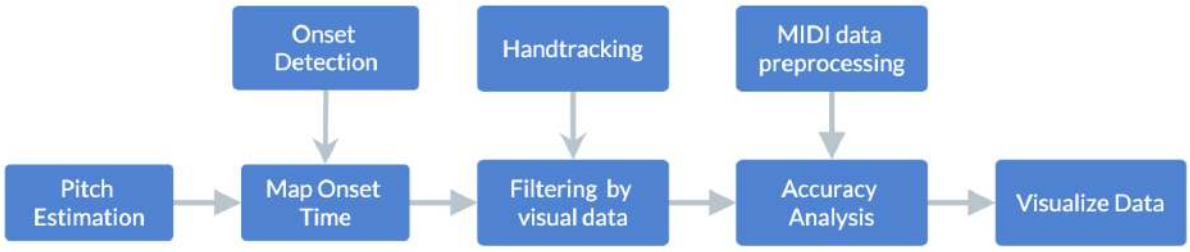


Figure 87: Flowchart for pitch detection

Fundamental frequency is defined as the lowest frequency of a periodic waveform. In terms of a sound wave, it often refers to the "pitch" that we are hearing. Therefore, the fundamental frequency is what we try to find in our pitch detector. However, there is harmonic frequency which is the integer multiple of the fundamental frequency. While playing different musical instruments, it is nearly impossible to obtain a pure wave of frequency as each instrument produces its own harmonic spectra. These harmonic frequencies could lead to false detection of notes which are an octave, or even multiple octaves higher than the fundamental note that we would like to find. In short, background noises and noises from harmonic frequency are a huge hurdle to this function, thus, pre-processing of data and post-processing of the result should be implemented in this function.

For pitch detection, we basically can define the problem into two typical types. The first one is monophonic pitch detection and the other is polyphonic pitch detection. As mentioned in the previous section, monophonic music in piano refers to playing one note at one time while polyphonic music in piano refers to playing multiple notes at one time such as left-hand, and right-hand playing simultaneously, and playing chords.

4.6.1 Related Work

Automatic Music Transcription (AMT) has been a heated topic in Music Information Retrieval. Given a polyphonic audio of instrument playing, AMT aims to turn it into a readable music score that contains pitch information, dynamic markings, tempo marking, etc. AMT is composed of multiple tasks such as pitch detection, onset detection, and score visualization, which are related to our project.

While the older methodology separated the tasks as mentioned above first and then merged them back in a later stage, recent scientists have proposed using deep learning to solve the task. Sigtia et al. [43] proposed an end-to-end neural network composed of an Acoustic model and a Music Language model. By taking input of time-frequency representation forms in Constant Q Transform (CQT) format, the model will output posteriograms that then will be transformed into a binary piano-roll representation.

Liu et al. [44] has proposed a multitask learning model that is composed of convolutional stack and separated sequential models. By combining Gated Recurring Units (GRUs), Autoencoder, and Convolutional Neural Networks (CNN), the model has outperformed a single-task model in score transcription.

The method for monophonic pitch detection has been studied for more than a half-century, and many reliable methods have been proposed since. The earlier research methods on monophonic pitch detection, often employ candidate-generating functions such as cepstrum [45] and autocorrelation [46] to generate pitch curves. For instance, the YIN algorithm is an autocorrelation-based method and refines the estimated period of frequency by using parabolic interpolation for pitch detection [47]. A further application of the YIN algorithm is the probabilistic YIN (pYIN) algorithm which employs the Hidden Markov Model (HMM) to estimate the most probable sequence of pitch which enhances the accuracy [48]. A more recent approach is to use a deep learning model to produce pitch estimates such as Convolutional REpresentation for Pitch Estimation (CREPE) that directly operates on the time-domain waveform [27].

Nonetheless, for polyphonic pitch detection whose difficulty is far beyond that of monophonic pitch detection, only a few methods have been proposed to achieve the estimation. When multiple notes are played together at the same time, all tones are mixed and there is more noise which makes it harder to distinguish the fundamental frequency from the partials especially when they overlap with another fundamental frequency. Deep learning models such as ConvLSTM, and ConvGRU which take the input of spectrograms can be trained to find multiple fundamental frequencies (f0) from the audio [49].

4.6.2 Monophonic Pitch Detection

Monophonic pitch detection is a rather simpler task to tackle as we are just detecting one certain fundamental frequency within a short time frame. Several algorithms and deep learning models have already been proposed and there are sufficient resources can be obtained on the internet. Hence, this task is not our main focus in this project as most of the piano playing is in a polyphonic style.

Currently, a monophonic pitch detector has been developed to extract the time and frequency information of monophonic audio of piano playing. The detector is composed of a pre-trained CNN model from CREPE which can detect the frequency of the notes played by taking just the input of the waveform. We can adjust the stepsize (100 stepsize is equal to

0.1 second) to determine how long the segmented signal the model would run the detection for every segmented signal.

For experimental purposes, we have used the playing of the C Major scale on the piano to test the detector's accuracy. The result is satisfactory as both the time of each note played and the frequency of each note have been extracted successfully. The demo result is shown in Figure 88 and Figure 89 .

```
['C4' 'C4' 'C4' 'C4' 'C4' 'D4' 'D4' 'D4' 'D4' 'E4' 'E4' 'E4' 'E4' 'E4'
 'F4' 'F4' 'F4' 'F4' 'G4' 'G4' 'G4' 'G4' 'G4' 'A4' 'A4' 'A4' 'A4' 'B4'
 'B4' 'B4' 'B4' 'C5' 'C5' 'C5' 'C5' 'C5' 'B4' 'B4' 'B4' 'B4' 'A4' 'A4'
 'A4' 'A4' 'A4' 'G4' 'G4' 'G4' 'G4' 'G4' 'F4' 'F4' 'F4' 'F4' 'F4' 'E4'
 'E4' 'E4' 'E4' 'E4' 'D4' 'D4' 'D4' 'C4' 'C4' 'C4' 'C4' 'C4']
```

Figure 88: Detected notes with a stepsize of 100.

```
Time: 1.14 Note: C4
Time: 1.17 Note: C4
Time: 1.2 Note: C4
Time: 1.23 Note: C4
Time: 1.26 Note: C4
Time: 1.29 Note: C4
Time: 1.32 Note: C4
Time: 1.35 Note: C4
Time: 1.38 Note: C4
Time: 1.41 Note: C4
Time: 1.44 Note: C4
Time: 1.47 Note: C4
Time: 1.5 Note: C4
Time: 1.53 Note: C4
Time: 1.56 Note: C4
Time: 1.59 Note: C4
Time: 1.62 Note: C4
Time: 1.65 Note: C4
```

(a) Stepsize 30

```
Time: 1.2 Note: C4
Time: 1.3 Note: C4
Time: 1.4 Note: C4
Time: 1.5 Note: C4
Time: 1.6 Note: C4
Time: 1.8 Note: D4
Time: 1.9 Note: D4
Time: 2.0 Note: D4
Time: 2.1 Note: D4
Time: 2.3 Note: E4
Time: 2.4 Note: E4
Time: 2.5 Note: E4
Time: 2.6 Note: E4
Time: 2.7 Note: E4
Time: 2.8 Note: F4
Time: 2.9 Note: F4
```

(b) Stepsize 100

Figure 89: Detected notes and played time with a stepsize of 30 and 100.

4.6.3 Polyphonic Pitch Detection

Polyphonic pitch detection task is known to be a complicated and complex task in the field of music information retrieval. In the polyphonic pitch detection function, we aim to extract the pitch information from recorded polyphonic audio instead of synthesized polyphonic audio that has been widely used for experimental use in the field of music information retrieval.

There have been some well-known pitch-detecting algorithms in recent years and we want to find out which is the best algorithm for our project. YIN algorithm is based on the concept of the autocorrelation function, which measures the similarity between a signal and a delayed version of itself. The audio signal is divided into overlapping frames, typically using a sliding window. For each frame, the difference function is computed by calculating the squared difference between the signal and its shifted version at different time lags. The algorithm performs the cumulative mean thresholding to filter peaks caused by noise and harmonic overtones. The fundamental frequency is then estimated by locating the location of the first reliable peak and its subharmonic peaks.

For the probabilistic YIN (pYIN) algorithm, it is based on the YIN algorithm and the observation of a hidden Markov model (HMM). HMM is a well-known algorithm whose basic idea is to model the probabilistic relationships between the hidden states and the observable features. Therefore, from the feature extracted by the YIN algorithm, pYIN can modify YIN to output multiple pitch candidates with associated probabilities which can be used in polyphonic detection.

Another famous pitch detection method is to use parabolic interpolation methods. In simple terms, interpolation helps to fill in the gaps between data points by estimating values for intermediate positions. This is particularly useful when you have discrete data points and want to obtain values at positions that are not explicitly given in the data set.

One of the parabolic interpolation methods is to use quadratic interpolation of the spectral magnitude. The method first divides the audio signal into overlapping frames and computes the Fourier Transform to obtain the frequency spectrum. Then the algorithm will locate the highest magnitude bin and its neighbouring bins in the spectrum. By applying a quadratic equation (or a parabola) for the interpolation of the above three points, we can obtain the interpolated peak position which then can be used for estimating the pitch or the fundamental frequency by:

$$p = \frac{1}{2}\frac{\alpha - \gamma}{\alpha - 2\beta + \gamma}$$

We aim to make a polyphonic pitch detector that can turn the polyphonic audio into a MIDI file for further analysis of correctness by comparing it with the original MIDI score that has been developed. After trying different pitch detection algorithms that were mentioned in the previous part, which are the parabolic interpolation method, YIN, and pYIN to extract

the polyphonic pitch respectively, we have gained some directions and insight from using these algorithms.

For the development of our polyphonic detector, we have decided to use the parabolic interpolation method to detect the pitches in the audio as the accuracy of this method has outperformed the YIN and pYIN methods that are mainly used for monophonic pitch detection. We first apply a sliding window to split the audio into equal parts. Then we obtain the pitch information by applying parabolic interpolation to each of the mel-spectrograms that is computed from each window of audio.

$$\text{Number of window} = \frac{\text{Length of audio time series}}{\text{Length of window duration} * \text{Sampling rate}}$$

By applying a threshold to the magnitude of the audio data, we can filter the background noise from the audio. We then assign a "NULL" value for the filtered sample to maintain the structure of the audio time series. Moreover, we have set a range of possible pitches detected from C1 to C7 notes to eliminate very low or very high-frequency noise. Apart from the pitch information, time information can be obtained by multiplying the position of the window with the window size and sampling rate.

4.6.4 Accuracy Analysis

After obtaining the raw pitch information, we only have the information of which note is presented in which time frame of the audio but not the exact piano key pressing time of the notes. Apart from the audio data, we proposed a fusion method that integrates the information from both the vision data and audio data. However, due to time constraints, we have not developed the function of detecting piano key pressing and hence we need to map the piano key pressing information from audio.

In order to obtain the piano key pressing information, we decided to extract the onset from the audio. By using the onset detection function from the tempo analysis functions, we find out exactly when each note or chord was being played. However, we observed that the accuracy of onset detection may not be good enough for mapping the pitch just like what was shown in Figure 90.

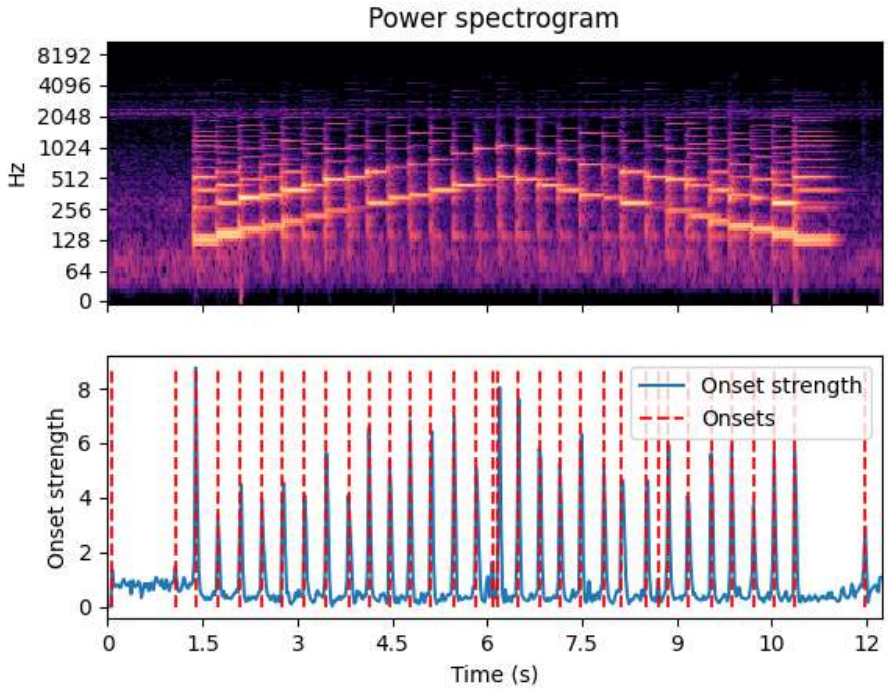


Figure 90: Onset was detected before any note was played.

Therefore, we filtered away the onset that was detected before the first detected note by the polyphonic pitch detector. After mapping the onset time with the pitch information using the first note detected and the first onset detected, we now know when the piano was being played and what notes were being played by the user.

As mentioned before, we also proposed to map the hand-tracking information that is extracted from the video recording of playing the piano. The hand-tracking information can be used to filter the false notes detected that were out of the piano key positions of each finger in that time stamp. This could possibly eliminate the octave error induced by the harmonic spectra. In the current stage, our detector may not handle the pedaling and sustained note well. Therefore, to obtain a better result, we have implemented several methodologies to remove sustaining and pedal notes. For the notes detected that were detected in the previous time frame which does not exist in the current correct note stated in MIDI information will be identified as the pedal or sustaining note. These notes are automatically ignored and not added to the detected note array.

After obtaining a filtered array of notes and corresponding timestamps, we map these with the original MIDI file of the pieces. By using the MIDI processing library Mido [23], we obtain the information on note-on and note-off events of the musical piece as well as the tempo that is being set from the musical score. By calculating the duration of a quarter note by:

$$\text{Quarter note duration} = \frac{60}{\text{default tempo of the musical piece}}$$

We can differentiate the note value of the notes in the MIDI data. After pre-processing the MIDI data into a 2D array that contains the information of what notes are being played and the rhythmic symbol (e.g. Quarter note, Double note, Eighth note, etc.) that indicates the duration of playing. By mapping the first note being played between the detected information and MIDI data, we can find out what notes should be played and what notes were being played in each timestamp as shown in Figure 91. By calculating the count of notes being correctly played, we can calculate the error rate of students' playing.

$$\text{Accuracy rate} = \frac{\text{Correct note}}{\text{Total number of note}} * 100\%$$

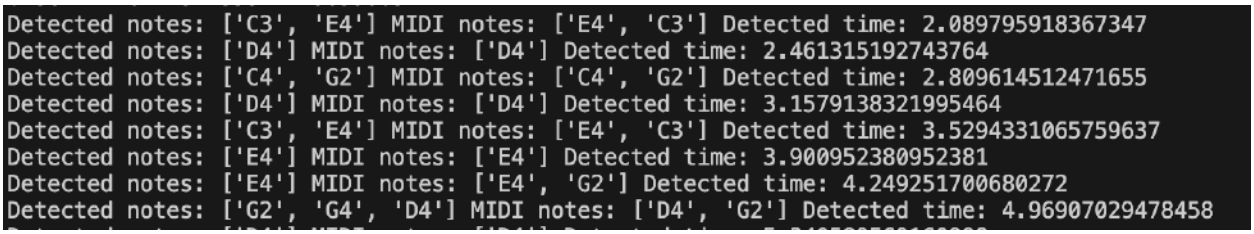


Figure 91: Detected note and MIDI data and corresponding timestamp.

The result with visual filtering has outperformed the result without visual filtering as finger-tracking data can be used to filter away the notes that have an octave error and other pitches due to background noises. With visual data, the accuracy of tracking is much higher than that without visual data. In Figure 92 and Figure 93, the MIDI notes are referring to the ground truth of the notes. Therefore, by comparison of Figure 92 and Figure 93, the result filtered with visual data is much more accurate than that without filtering with visual data as more notes are detected correctly after visual filtering.

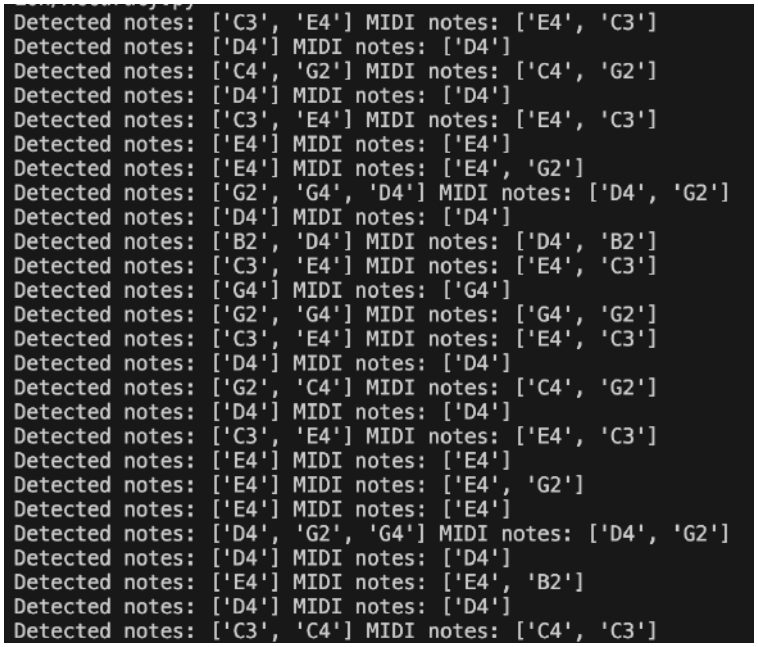


Figure 92: Result with visual data.

```
Detected notes: ['C3', 'E4'] MIDI notes: ['E4', 'C3']
Detected notes: ['D4'] MIDI notes: ['D4']
Detected notes: ['C5', 'G5'] MIDI notes: ['C4', 'G2']
Detected notes: ['D4', 'D5'] MIDI notes: ['D4']
Detected notes: ['C3', 'E4'] MIDI notes: ['E4', 'C3']
Detected notes: ['E4'] MIDI notes: ['E4']
Detected notes: ['E4', 'E5'] MIDI notes: ['E4', 'G2']
Detected notes: ['D5', 'D♯4', 'F♯3', 'G2', 'G3', 'G4'] MIDI notes: ['D4', 'G2']
Detected notes: ['D4'] MIDI notes: ['D4']
Detected notes: ['B2', 'B4', 'D4', 'D5', 'D♯4'] MIDI notes: ['D4', 'B2']
Detected notes: ['C3', 'E4'] MIDI notes: ['E4', 'C3']
Detected notes: ['G4'] MIDI notes: ['G4']
Detected notes: ['G2', 'G4', 'G♯4'] MIDI notes: ['G4', 'G2']
Detected notes: ['C3', 'E4'] MIDI notes: ['E4', 'C3']
Detected notes: ['D4', 'D5', 'D♯4'] MIDI notes: ['D4']
Detected notes: ['C5', 'G2', 'G5'] MIDI notes: ['C4', 'G2']
Detected notes: ['D4', 'D5'] MIDI notes: ['D4']
Detected notes: ['C3', 'E4'] MIDI notes: ['E4', 'C3']
Detected notes: ['E4'] MIDI notes: ['E4']
Detected notes: ['E4'] MIDI notes: ['E4', 'G2']
Detected notes: ['E4'] MIDI notes: ['E4']
Detected notes: ['B4', 'D4', 'D5', 'F♯3', 'G2', 'G4'] MIDI notes: ['D4', 'G2']
Detected notes: ['D4'] MIDI notes: ['D4']
Detected notes: ['E4'] MIDI notes: ['E4', 'B2']
Detected notes: ['D4'] MIDI notes: ['D4']
Detected notes: ['C3', 'C5'] MIDI notes: ['C4', 'C3']
```

Figure 93: Result without visual data.

4.6.5 Visualization of Result

By using a Python Library Music21 [24], we can append the note or the chord being played in each time stamp. By mapping this information with the rhythmic symbol that indicates the duration of the note or chords being played. We can turn the result into a MusicXML score which is a file type for musical score and visualize it by turning the score into an image file using the command line function from MuseScore [25]. The sample result is shown in Figure 94.

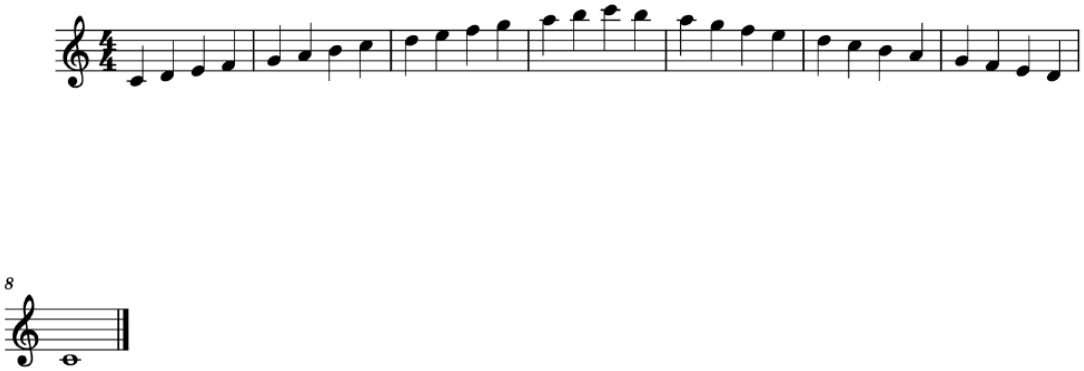


Figure 94: Visualized score display.

4.6.6 Evaluation of Pitch Detection

To evaluate the performance of our pitch-detecting algorithm, the algorithm detecting accuracy is defined as:

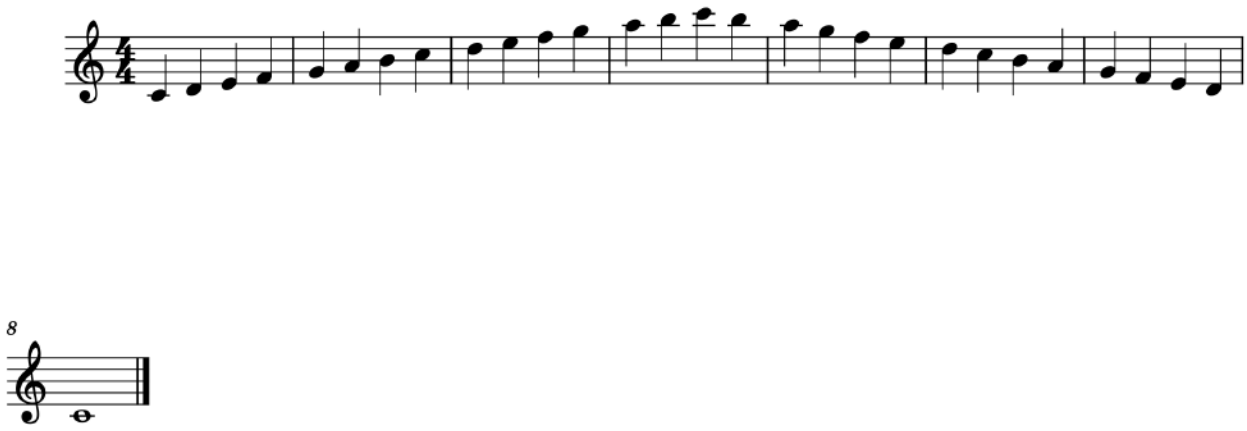
$$\text{Accuracy} = \frac{\text{Number of correctly detected notes}}{\text{Total number of notes}} * 100\%$$

Below is the result of different recordings which was recorded in a different venue (Refer to Section 4.2 for the detailed setup of the venue). Due to the problem derived from using the MIDI file directly from MuseScore for accuracy analysis as mentioned in Section 4.2.2, we have cherry-picked some simpler musical pieces and playing of C-major scale to obtain some preliminary evaluation of our function.

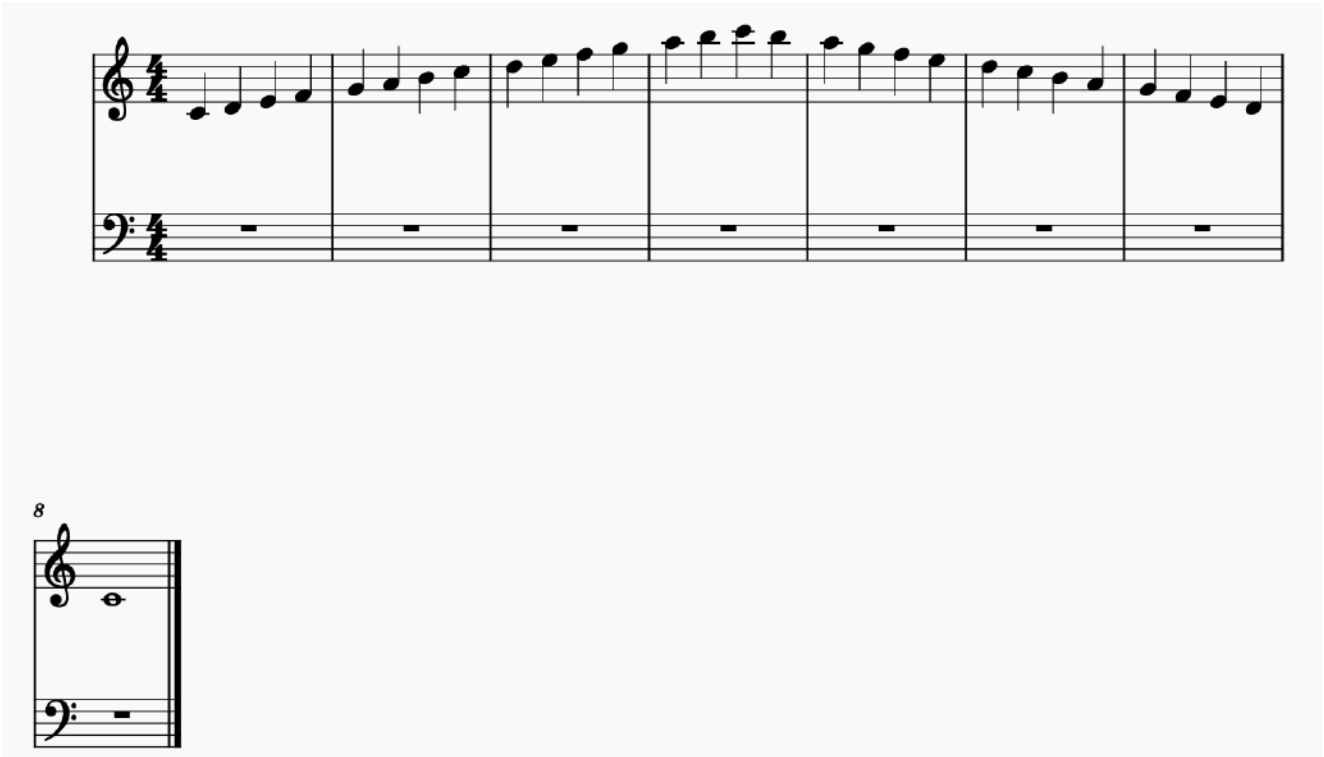
Case	Name	Venue	Best Accuracy
1	C Major Scale (right hand)	1	100%
2	C Major Scale (both hands)	1	89.66%
3	Mary had a little lamb	1	92.68%
4	Mary had a little lamb	2	100%
5	Twinkle twinkle little star	2	85.37%
6	C Major Scale (right hand)	2	100%
7	C Major Scale (right hand)	3	100%
8	Mary had a little lamb	3	100%

Table 11: Evaluation of Polyphonic detector accuracy.

Below is the result of converting the detected notes into a piano score and the corresponding ground truth of the piano piece. Figure 95 has shown a correctly detected monophonic sound of playing of C major scale.



(a) Detected notes of C Major Scale with the right hand only in Venue 1



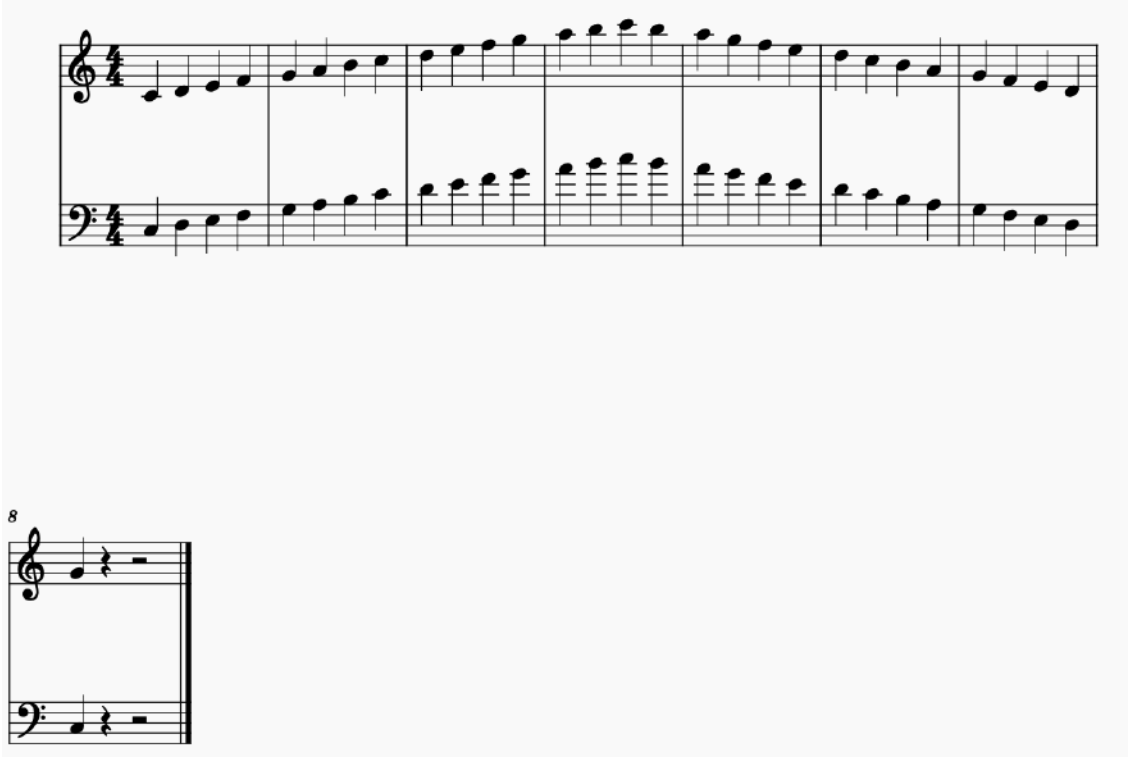
(b) Ground truth of C Major Scale with the right hand only in Venue 1

Figure 95: Result of pitch detection of C Major Scale with the right hand only in Venue 1.

From what can be observed in Figure 96, there is still some inaccurate detection of the polyphonic pitch by our algorithm as labelled by a red box in the figure. To put it precisely, some of them are still suffering from octave errors induced by the harmonic spectra which lead to extra or missing octave notes.



(a) Detected notes of C Major Scale with both hands in Venue 1



(b) Ground truth of C Major Scale with both hands in Venue 1

Figure 96: Result of pitch detection of C Major Scale with both hands in Venue 1.

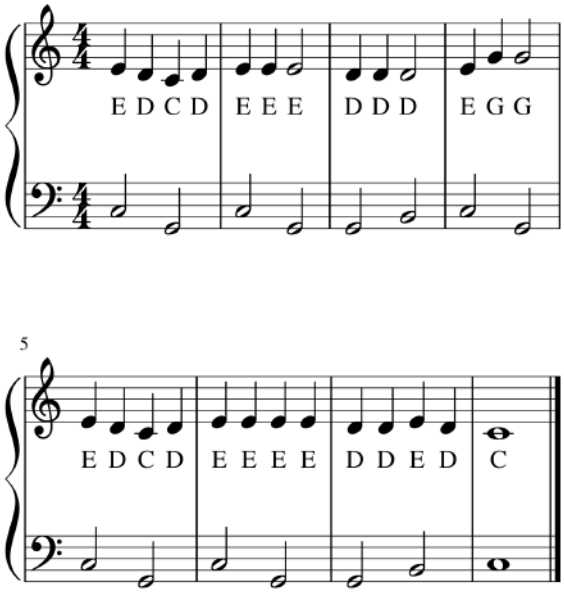
By comparing the result of the sample using recorded data and synthesized data in Figure 97, we observe that using synthesized data can achieve a relatively higher detecting accuracy than using recorded data. The reason behind this is probably due to the better quality and resolution of synthesized data.



(a) Detected notes of Mary had a little lamb in Venue 1



(b) Detected notes of Mary had a little lamb in Venue 2



(c) Ground truth of Mary had a little lamb in Venue 1

Figure 97: Result of pitch detection of Mary had a little lamb in Venue 1.

4.6.7 Improvements in Term 2

As mentioned in the first term, we have adopted an approach of running polyphonic pitch detection in each of the separated windows. However, the pitch detection result is still affected by several factors such as the windowing width and the accuracy of onset detection. In this term, we have further enhanced the accuracy of onset detection as explained in the previous section to facilitate the searching of the optimal threshold for onset detection.

Apart from enhancing the accuracy in onset detection, we have developed another method for determining the windowing length for pitch detection. As we are running the pitch detection function for the user's pre-recorded playing, we are not processing the function in a real-time scenario. Therefore, we have proposed another more optimized method that takes onset detection results into account. Similar to the original approach, we run polyphonic pitch detection on each window frame. However, the window methodology is now dependent on the onset detection. We run polyphonic detection to the window frame which between each onset detected to obtain a more accurate result.

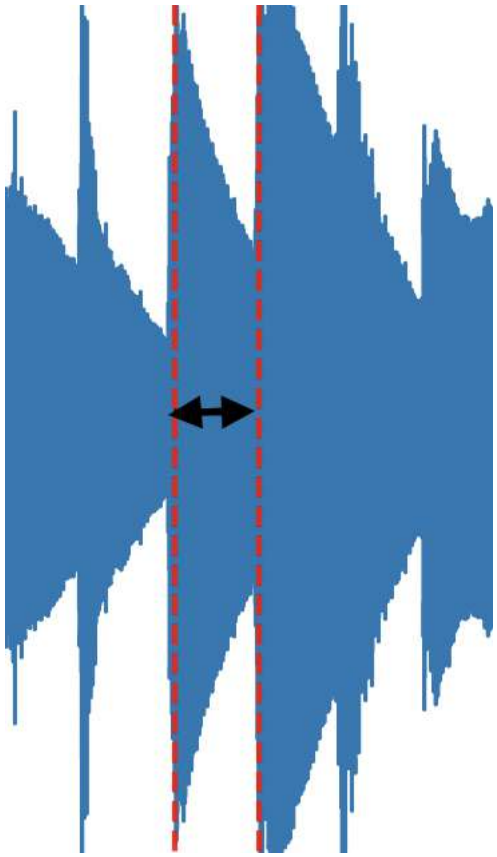
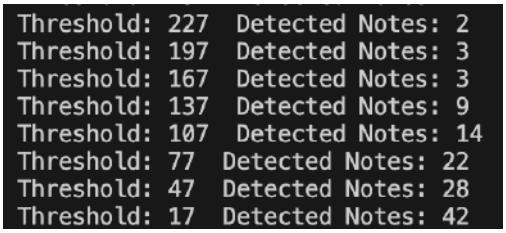


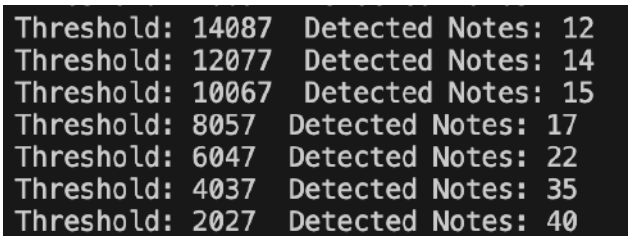
Figure 98: New methodology of windowing.

Apart from the new windowing method, we applied dynamic thresholding which depends on the different magnitudes of each audio data. In the first term, a customized threshold is required for different audio data. In fact, the optimized threshold can be varied from power = 10, all the way up to power = 2000. Therefore, we have implemented a for-loop, to decrease the threshold from the maximum value of the magnitude with a step-size that also varies with the maximum magnitude, until sufficient pitch information is obtained by comparing it to the ground-truth MIDI file.

In terms of sustaining notes which is a complicated obstacle for polyphonic pitch detection due to the complexity of continuous mel-spectrograms, we proposed one possible method that



(a) Sample 1 with small threshold

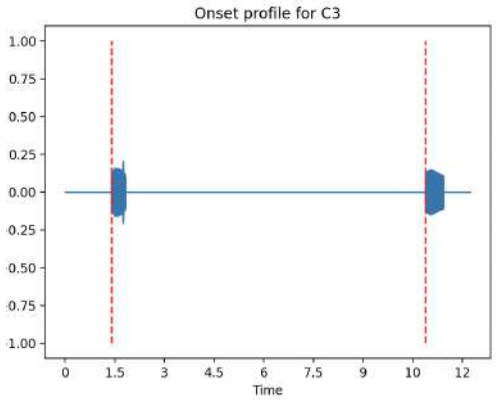


(b) Sample 2 with large threshold

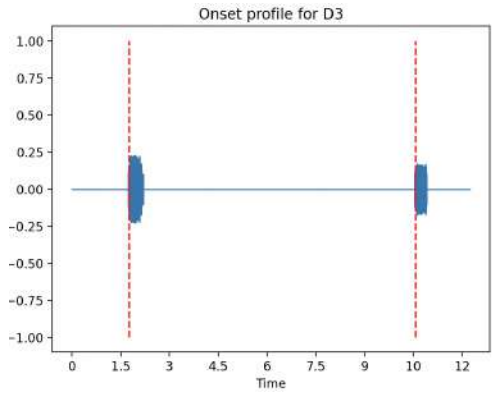
Figure 99: The optimized threshold for each recording can be varied.

could separate the audio into pitch onset profiles. An onset profile is a term that we defined as the onset detection result for each possible note in the audio. By using the visual result from piano fingering, we obtain a set of possible notes that could be played by the user. Then, a bandpass filter is applied to only allow the frequencies that fall in the frequency range of each possible note. As a result, we could know if a note is played continuously or only once when a sustained note is detected. Some samples of onset profile are shown in Figure 100.

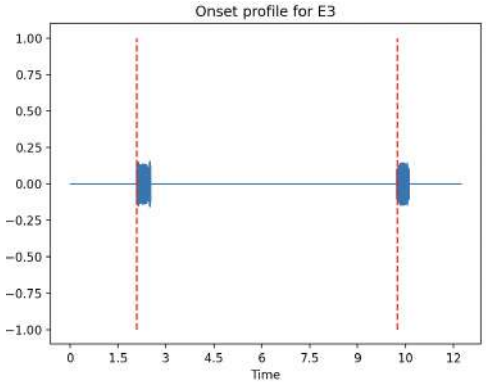
However, due to limited time and our planned scope of assessment for this term, the above detection for sustaining notes has not merged into the pitch detection function. In this project, we neglect the effect of sustaining notes by comparing the detected notes with the ground truth MIDI notes. This methodology could be considered for any possible continuous development in the future.



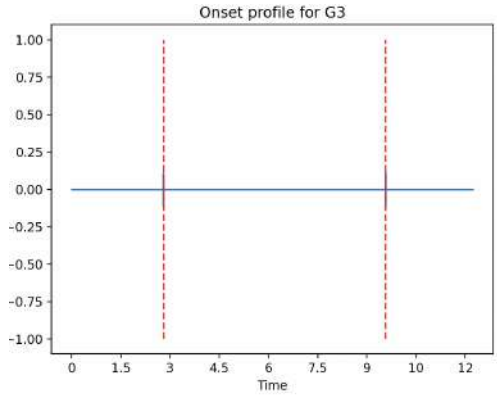
(a) Onset profile for C3



(b) Onset profile for D3



(c) Onset profile for E3



(d) Onset profile for G3

Figure 100: Detected onset profiles for playing of C Major Scale

4.6.8 Summary

By using the method of parabolic interpolation to obtain the pitch and time information, we successfully map the detected pitch with the correct MIDI notes with the aid of onset detection. By calculating the error rate of the detected note and plotting the music score of the detected note, we have provided some information for the students to know which part of the music they played wrong.

After optimizing the pitch-detecting function, by passing only necessary arguments to the function, a satisfactory result of pitch detecting can be obtained without fine-tuning thresholds for each audio. Moreover, we have tidied up the flow of logic and transmission of results to facilitate further interpretation or analysis in the later functions such as user interaction in the web application, data for style transfer as well as emotion classification.

4.7 Analysis Result Unification

This section explains how the detected data of pitch detection is shared and transmitted across each section of our project

4.7.1 Note Time Representation

In order to facilitate the data transmission to the web application section and piano fingering analysis section, pitch detection output and accuracy output are written into CSV format indicating which note is played correctly, is detected but not included in the ground truth, and is not detected but included in the ground truth. In other words, for each row of the CSV file, we include either one detected note or an undetected note that exists in the ground truth, and its corresponding time stamp. Then an index indicating the class of the note from 0 to 2 (played correctly = 0, is detected but not included in the ground truth = 1, and is not detected but included in the ground truth = 2) is included in the row too.

Time stamps	Note	Class index
0.1	A3	1
0.1	B3	2
0.2	C3	3

Table 12: Example of the data in CSV

4.7.2 Chart Resampling

In the web application, there is a graph in the recording playback that overlays multiple charts onto the same chart canvas (Figure 101). Therefore, the x-axis format and number of points sampled should be aligned.

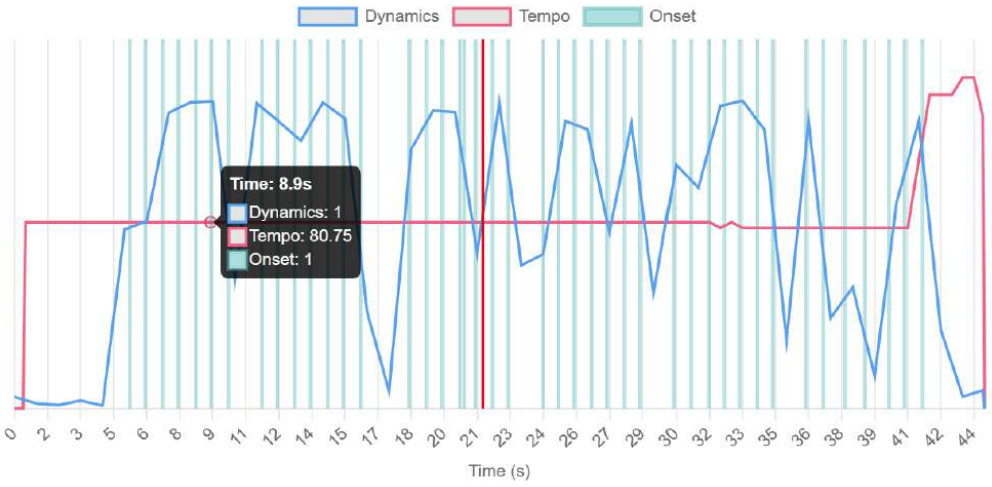


Figure 101: Table showing the span range of different fingering transition pairs.

Since the original chart data generated from different components in the analysis have different sampling rates, a resampling is needed to ensure the data points align. The sampling rate is set as 0.1 samples per second to achieve the balance between the data size and chart smoothness. It is done by performing interpolation between the original chart, and the new x-axis scale.

4.8 Performance Metrics

This section introduces the methods used to convert the analytics in the previous part into meaningful and comparable metrics for a more intuitive understanding of the performance.

4.8.1 Pitch Accuracy

This metric measures the percentage of notes correctly played. This metric focuses on the pitch, so "correct" is determined solely by whether the correct pitch is played. More details regarding pitch detection and pitch-to-score mapping can be found in Section 4.6.

$$\text{Pitch Accuracy} = \frac{\text{Number of correct notes}}{\text{Total number of notes on piece}} * 100\%$$

4.8.2 Dynamics Consistency

This metric measures how consistent the dynamics of the audio played by the user are. Assuming there are no changes in dynamics marking, we define a dynamics consistency score by calculating the standard deviation of the global dynamics changes of the user's playing. If the standard deviation is higher than 60, according to statistics, we would consider the user has got a zero mark for dynamics consistency. The dynamics consistency score is calculated by this equation:

$$\sigma_{\text{global dynamics}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

$$\mu = \text{mean of global energy in decibel}$$

$$N = \text{total number of global energy value}$$

$$\text{Dyanmics Consistency Score} = \begin{cases} \frac{60-\sigma_{\text{global dynamics}}}{60} * 100, & \text{if } \sigma_{\text{global dynamics}} < 60 \\ 0, & \text{otherwise} \end{cases}$$

4.8.3 Tempo Stability

This metric measures how stable the tempo of the audio played by the user is. Assuming there are no changes in tempo marking, we define a tempo stability score by calculating the standard deviation of the local tempo of the user's playing. If the standard deviation of tempo is higher than 80, according to statistics, we would consider the user has got a zero mark for tempo stability. The tempo stability score is calculated by using the local tempo by this equation:

$$\sigma_{\text{local tempo}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

$$\mu = \text{mean of local tempo value}$$

$$N = \text{total number of local tempo values}$$

$$\text{Tempo Stability Score} = \begin{cases} \frac{80-\sigma_{\text{local tempo}}}{80} * 100, & \text{if } \sigma_{\text{local tempo}} < 80 \\ 0, & \text{otherwise} \end{cases}$$

4.8.4 Piano Fingering

This metric measures how optimal the piano fingering played by the user is. The idea is to first use a cost function to compute the costs of the played and generated fingerings respectively. Afterwards, a score (out of 100) can be computed based on the percentage difference of their costs.

Denote the cost of the detected and generated piano fingerings as c_{det} and c_{gen} respectively. The piano fingering score is defined as follows:

$$\text{Piano Fingering Score} = 100\% - \frac{c_{det} - c_{gen}}{c_{gen}}$$

Cost Function

The cost function from the work of Parncutt et al. [50] is adopted with modifications to cater to polyphonic music. In their work, 13 rules were employed to assign cost values to different fingering transitions.

Many of the rules are built on a table showing the ranges of semitone differences for all pairs of finger transitions under relaxed, comfortable or barely practical posture (Figure 102).

Assumptions of the Model: Minimum and Maximum Practical, Comfortable, and Relaxed Spans (in Semitones) for All Pairs of Right-Hand Fingers

Finger Pairs	MinPrac	MinComf	MinRel	MaxRel	MaxComf	MaxPrac
1-2	–5	–3	1	5	8	10
1-3	–4	–2	3	7	10	12
1-4	–3	–1	5	9	12	14
1-5	–1	1	7	10	13	15
2-3	1	1	1	2	3	5
2-4	1	1	3	4	5	7
2-5	2	2	5	6	8	10
3-4	1	1	1	2	2	4
3-5	1	1	3	4	5	7
4-5	1	1	1	2	3	5

Figure 102: Table showing the span range of different fingering transition pairs.

The values in this table are for the right hand, with notes going from the lower finger to the higher finger. All values are 0 if the start and end fingers are identical. Under the assumption that reversing a sequence shall yield an identical cause, both the fingerings and start and end pitches are simply swapped when the notes in the subsequence go from the higher finger to the lower finger. When processing the left hand, a nearly identical table is built with the range values flipped in sign.

The ranges shown in the table are based on a normal male adult pianist. The actual range varies among individuals and shall be tailor-made for each user ideally, as in the work of Al Kasimi et al. [51]. However, since this application just serves as a proof-of-concept, we decided to exclude such a level of customization from this project due to limited development time.

The cost function of a piano fingering sequence is computed based on the 13 rules proposed, with the situation covered by each rule shown in Table 13. The first, second and third sections contain rules for evaluating a single, a pair and a triplet of notes respectively.

Rule	Condition
Weak finger Rule	When a weak finger (4 or 5) is used.
Stretch Rule	When finger pair spans further than the comfortable span.
Small-span Rule	When finger pair spans closer than relaxed range.
Large-span Rule	When finger pair spans wider than relaxed range.
Three-to-Four Rule	When finger 3 is immediately followed by finger 4.
Four-on-Black Rule	When fingers 3 and 4 occurs consecutively with finger 3 on white key and finger 4 on black key.
Thumb-on-Black Rule	When finger 1 plays on a black key.
Five-on-Black Rule	When finger 5 plays on a black key.
Thumb-Passing Rule	When finger-passing occurs.
Position-Change-Count Rule	When the hand position changes.
Position-Change-Size Rule	When the span of fingers across three notes is out of comfortable span.
Three-Four-Five Rule	When fingers 3, 4, and S occur consecutively in any order.

Table 13: Rules for computing piano fingering cost.

While the rules were arbitrarily defined by Parncutt et al., we believe that it is still a decent representation that gives a thorough evaluation that considers various types of transition and can quantitatively measure how optimal a fingering is.

The robustness of the rules can be illustrated with an example. Consider a C major scale $C_4 - D_4 - E_4 - F_4 - G_4 - A_4 - B_4$ shown in Figure 103 being played with the right hand.

If only consider the first five notes $C_4 - D_4 - E_4 - F_4 - G_4$ the fingerings $1 - 2 - 3 - 4 - 5$ yields a cost of 4. While $1 - 2 - 3 - 1 - 2$ yields a cost of 10 due to the finger crossing and change in hand position (the entire hand shifted right).

If the remaining two notes are included, the set of fingerings $1 - 2 - 3 - 4 - 5 - 1 - 2$, a reasonable extension of $1-2-3-4-5$, costs 28. However, the cost of $1-2-3-1-2-3-4$, the reasonable extension of $1 - 2 - 3 - 1 - 2$, is only 12.

While the first sequence has a much lower cost when only the first five notes are considered, its cost is higher after considering the entire sequence. The second sequence is in fact the more commonly-used fingering when playing the C major scale.

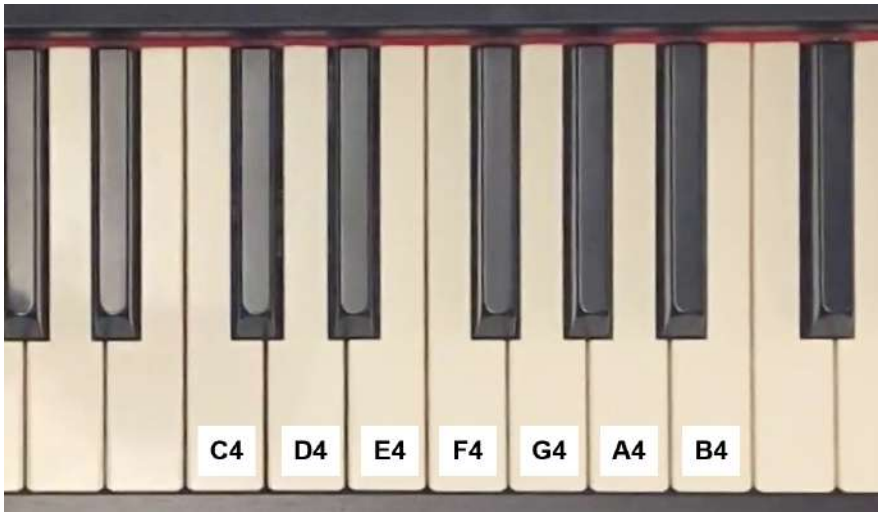


Figure 103: C major on keyboard.

It reflects that this cost function is capable of identifying fingerings that may not be the optimal solution in some subsequences but are better as a whole.

Modification for Polyphonic Music The major limitation of the cost function above is that it is for monophonic music only. Therefore, the cost function is modified to cater to polyphonic music. As inspired by Kasimi et al. [51] the notes are grouped into chords and the vertical and horizontal cost of each chord is computed.

The notes are modified into a sequence of chords. Where a chord is defined by all notes that are played at the same time. For example, if $C - E - G$ are played simultaneously, they will be grouped as a chord. Under this definition, if only one note is played at a time, it would be grouped as a chord with a single note.

The vertical cost of a chord is defined as the cost of the weak-finger rule plus the maximum pairwise cost among all pairs of notes in the chord. For example, for a chord $C4 - E4 - G4$ played by $1 - 3 - 5$, the candidate pairs are $C - E$ by $1 - 3$, $C - G$ by $1 - 5$ and $E - G$ by $3 - 5$.

The horizontal cost, which represents the transition between chords, is divided into costs of two-chord and three-chord transitions.

The two-chord transition refers to the transition of adjacent chords, they are computed as the pairwise maximum cost between all pairs of notes between the starting and ending chords. The rules for evaluating pairs of notes are used here. All sets of transitions are evaluated, including the overlapping ones. For example, in the note sequence $[C4, E4] - [D4] - [C4 - F4]$, the cost of both transition pairs $[C4, E4] - [D4]$ and $[D4] - [C4 - F4]$ are calculated.

The three-chord transition refers to the transition of three consecutive chords. Since these rules focus on the change of hand position, the note played by the smallest finger number (i.e. nearest to the thumb) of each chord is used for computing the cost instead. Similarly, the costs of all transition triplets, including overlapping ones, are included.

5 Emotion Classification

This function mainly aimed to generalize how people perceive and understand the feeling expressed in music. To provide some insights for the user as reference of how general people would feel when they listen to the playing. The whole section is written in Term 2.

Emotion has always been a subjective concepts for each individuals. There is no absolute definition for any kind of emotion. In terms of music, it is acted as a channel for both ways of expression and perception of one's emotion. Therefore, through this function, we hope to quantize the features from the input in a rather objective way to determine or classify the user's emotion when they express their feelings in music. However, as aforementioned, emotion is a subjective concept. Not everyone of us will agree with the prediction made by our model.

This function integrates the features and metrics of user's input evaluated in Section 4, including features in dynamics, tempo, and pitches. The whole flowchart for emotion classification is shown in Figure 115.

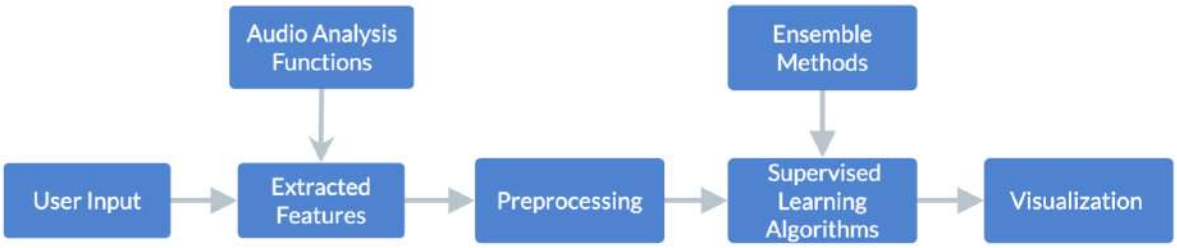


Figure 104: Flowchart for emotion classification

5.1 Related Works

Music emotion recognition (MER) has been a heated topic in the field of music information retrieval. However, most MER tasks target emotion classification in pop music including both vocal and acoustics lines. Zhao & Yoshii [52] proposed a multi-modal neural network approach to achieve a state-of-the-art performance with a valence-arousal classification accuracy of 69.2% using the EMOPIA dataset [53]. They used the bidirectional gated recurrent unit (Bi-GRU) to extract dynamics, melody, and rhythm features from the MIDI files. Then they used two CNN models to extract tone color features and harmony features from MFCCs and Chromagram respectively. Eventually, by combining all the features using the multi-head self-attention (MHSA) mechanism, they achieved an accuracy of 86.9% in predicting valence value and 69.2% in the prediction of four quadrants of valence/arousal values.

ER & ESİN [54] proposed a machine learning approach by using the audio features to classify the emotion of Turkish music into four classes (happy, sad, angry, and comfortable). They have extracted 34 features from the audio including Root Mean Square (RMS) Energy, Low energy, Tempo, Spectrum, Zero Crossing rate, etc. By using Support Vector Machines (SVM), K-Nearest Neighbor (K-NN), and Artificial Neural Network (ANN), they have achieved accuracy of 77.46%, 75.76%, and 77.37% respectively. In terms of the relationship between audio features and emotion, Chau et al. [55] have proven that dynamics and pitches actually correlate with emotion categories such as Heroic, Scary, Romantic, Shy, and Sad. It is notable that dynamic levels and pitches could lead to similar changes in emotion that have similar arousal or valence value. Such that it is possible that we could exploit both dynamics features and pitch features to classify different emotion categories.

5.2 Dataset

Upon looking into datasets for emotion classification, we noted that there are mainly two types of datasets. One contains labels of dimensional data of time-continuous arousal-valence values, while another one contains categorical labels for emotion. We have chosen the latter type of dataset as the purpose of our function is to provide a general classification of the input's emotion instead of giving an exact arousal-valence value based on these subjective data.

Features	Examples
Timing	Tempo, tempo, variation, duration, contrast.
Dynamics	Overall level, crescendo/decrescendo, accents.
Articulation	Overall (staccato, legato), variability.
Timbre	Spectral richness, harmonic richness.
Pitch	High or low.
Interval	Small or large.
Melody	Range (small or large), direction (up or down).
Tonality	Chromatic-atonal, key-oriented.
Rhythm	Regular, irregular, smooth, firm, flowing, rough.
Mode	Major or minor.
Loudness	High or low.
Musical form	Complexity, repetition, disruption.
Vibrato	Extent, range, speed.

Figure 105: Table of audio feature types related to music emotion classification [56]

As a result, we have chosen the 4Q audio emotion dataset which was generated by Panda et al. [56][57]. This dataset contains 900 audio clips, annotated into 4 quadrants, according to Russell's circumplex model [2]. The dataset also provides the extracted 100 novel features of the audio clips such as onset, MFCCs, sharpness, spectral centroid, etc., covering rhythm features, dynamics features, melody features, harmony features, and tonal color as shown in Figure 106. However, due to the concern about the processing time for feature extraction, we would generate our own audio features from the audio files and classify the audio according to their labels.

Before we dig into the details of emotion classification, let us recap the arousal-valence model that has been mentioned in the previous sections. Valence ranges from positive valence (indicating positive emotions) to negative valence (representing negative emotions). Arousal, on the other hand, spans from low arousal (indicating calmness or relaxation) to high arousal (representing excitement, fear, or intense stimulation). By combining valence and arousal, we could creates a two-dimensional emotional space. For more details regarding Russell's arousal-valence model, please kindly refer to Section 2.1.10.

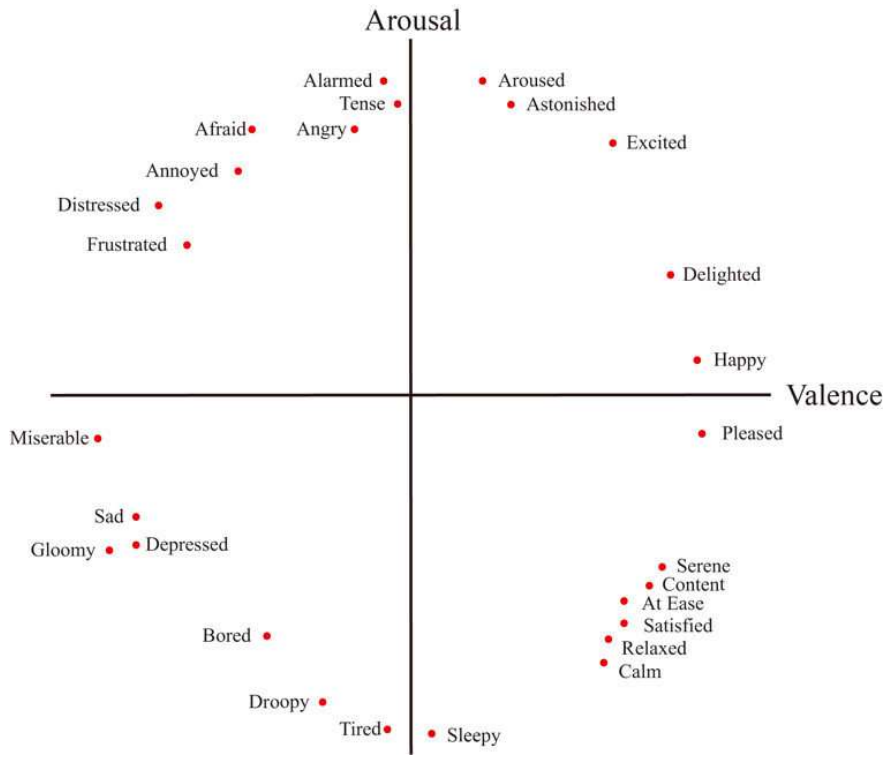


Figure 106: Arousal-Valence model by James Russell [2]

5.3 Feature extraction

As mentioned in the previous sections, we would like to merge the output from performance analysis functions to cover both dynamics features, melody features, rhythmic features, and even tonal features. For dynamics features, we have computed the mean and variance of

Root Mean Square (RMS) energy which represents to global energy of the audio and the variability in dynamics. In terms of rhythmic features, we have included the onset time, the global tempo, as well as the variance of the dynamic tempo which are generated by our tempo analysis function. As for the melody or harmony features, we have generated the chroma features, major or minor key values, the corresponding estimated scale, and harmonic values. Apart from the above, tonal features such as zero crossing rate, Mel-frequency cepstral coefficients (MFCCs), spectral contrast, and tonnetz feature. Details and explanations of each feature are included in Table 14.

Feature	Category	Description
Onset	Rhythm	Estimated starting time of notes
Global tempo	Rhythm	Global mean of tempo in BPM
Variance of tempo	Rhythm	Fluctuation of dynamic tempo
Mean of chromagram	Pitch	Energy distribution among pitches
Variance of chromagram	Pitch	Intensity of changes in chromagram
Harmonic elements	Pitch	Harmonic component of the audio
Major/Minor Key	Pitch	Relative key of the audio
Scale of Key	Pitch	The pitch scale of the key
Mean of RMS Energy	Dynamics	Global energy of signal
Variance of RMS Energy	Dynamics	Intensity of changes in audio energy
Volume	Dynamics	Amplitude of audio in dB
Mean of MFCCs	Tonal	Measure of spectral shape
Variance of MFCCs	Tonal	Intensity of changes in spectral shape
Spectral Contrast	Tonal	Contrast of the spectrum of audio
Zero Crossing Rate	Tonal	waveform sign-change rate
Mean of Spectral Centroid	Tonal	Brightness of sound
Variance of Spectral Centroid	Tonal	Intensity of changes in brightness
Tonnetz	Tonal	Conceptual lattice diagram of tonal space

Table 14: Extracted features for emotion classification

1x17 feature vectors are then generated for every audio in the dataset. Then the vectors are saved in .npy format and transmitted to the directory with their corresponding label (Q1, Q2, Q3, Q4). Then 80% data is split into training dataset and rest is put into testing dataset.

5.4 Model Training

As with many other existing methods for emotion classification, we exploit the usage of machine learning methodology for the multi-class classification task. Apart from using the standalone model approach, we adapt the ensemble method to combine multiple individual models to improve predictive performance and enhance generalization ability. Moreover, we would compare the performance between Random Forest (RF), Support Vector Machine (SVM), and Gradient Boosting Decision Trees (GBDT). The best machine learning technique would be used for the emotion classification task.

Random Forest Random Forest is a combination of multiple decision trees, which therefore is using the ensemble method itself already (Bagging technique). Decision tree learning employs a divide-and-conquer strategy by conducting a greedy search to identify the optimal split points within a tree. This process of splitting is then repeated in a top-down, recursive manner until all, or the majority of records have been classified under specific class labels. As a result, a decision tree could already give out predictions on its own. By getting multiple predictions from uncorrelated decision trees which were trained separately and applying major-voting methods, a final prediction can be made by the Random Forest.

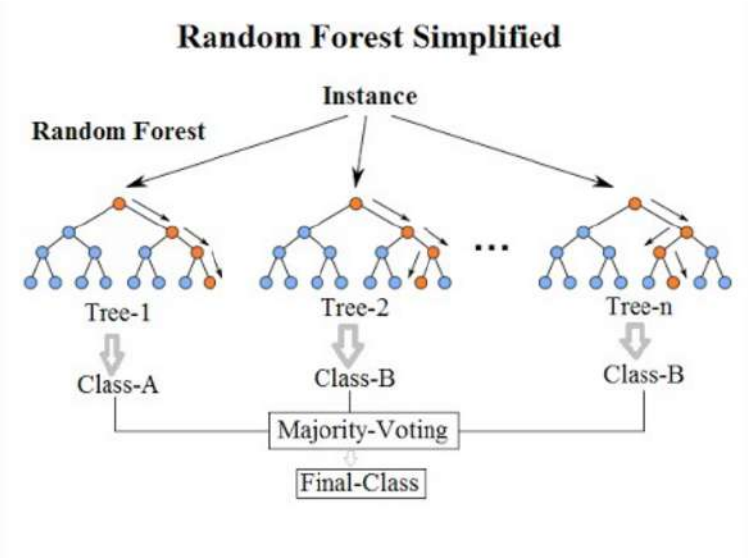


Figure 107: Explanation of Random Forest [58].

Support Vector Machine SVM is a kind of supervised machine learning algorithm that classifies data by finding an optimal line or hyperplane that maximizes the distance between each class in an N-dimensional space. As shown in Figure 108, we are finding the best plane by maximizing the margin. In fact, instead of finding a straight linear line, we could set up different types of kernels such as a polynomial line or a radial basis line to tackle cases of non-linear datasets.

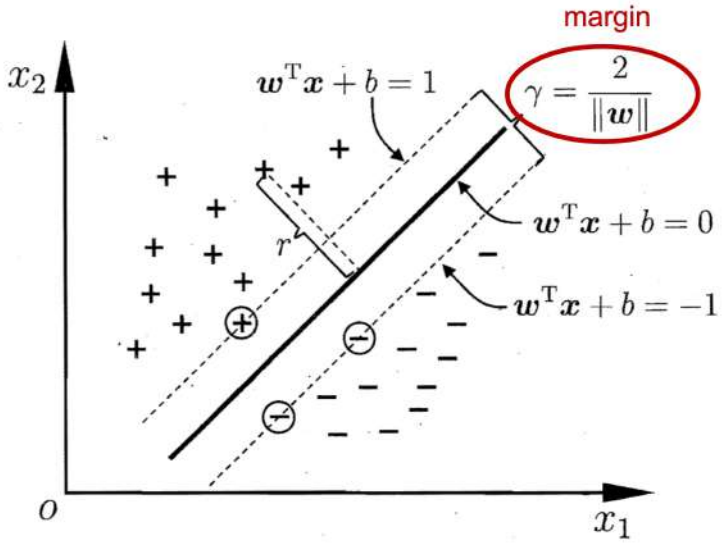


Figure 108: Explanation of SVM.

However, this kind of SVM can only give a binary prediction, then how do we apply SVM for multi-class problems? There are two approaches for multi-class classification problems. The first one is the one-to-one scheme, in which binary SVMs are trained for each pair of classes by ignoring the data points of other classes. Another approach is the one-to-rest scheme, where binary SVMs are trained to identify one class and the rest of the data points. By comparing and combining the prediction of each SVM, an ultimate prediction can be made by the group of SVMs. In our function, a one-to-one scheme would be adopted.

Gradient Boosting Decision Trees GBDT is a decision tree ensemble learning algorithm similar to random forest, for classification and regression. Both random forest and GBDT build a model consisting of multiple decision trees. The difference is in how the trees are built and combined. Unlike how uncorrelated the decision trees in RF, GBDT uses the Boosting technique to improve a single weak model by combining it with a number of other weak models in order to generate a collectively strong model. GBDTs iteratively train an ensemble of decision trees, with each iteration using the error residuals of the previous model to fit the next model.

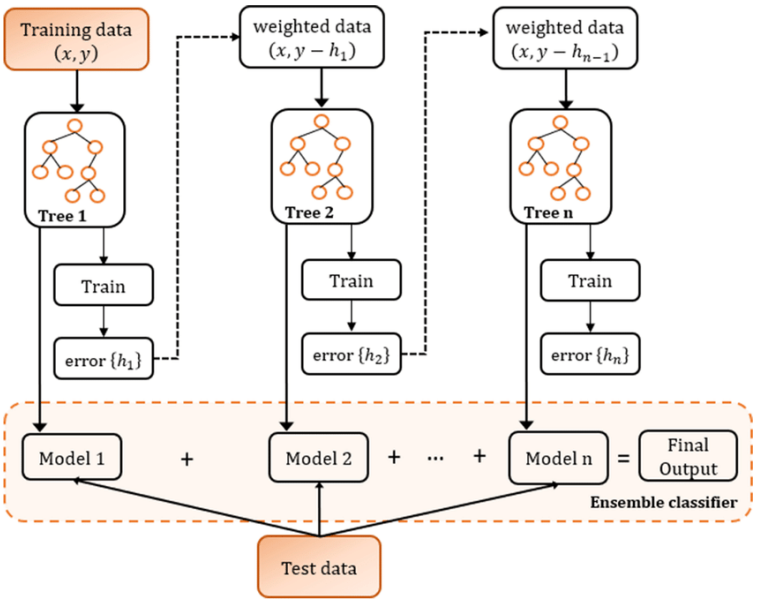


Figure 109: Explanation of GBDT [59].

Model Configuration For random forest, we have tried different numbers of estimators as well as the maximum depth for each of the tree estimators. The reason is to avoid overfitting to the training dataset and conduct fine-tuning to find a better model. For SVM, we are using different kernel methods to find the best shape of the optimized lines to differentiate our feature from audio as our data. For GBDT, we are training versions with different learning rates of 0.01 and number of estimators. It is the same reason for avoiding overfitting as that of the configuration of the random forest when we are setting a low learning rate. We are using the famous Scikit learn library [60] to train the random forest model and the SVM. While we are using the XGBoost library [61] for the training of GBDT model.

5.5 Result

In general, the GBDT has achieved the best overall accuracy of 71.21% with a 0.005 learning rate and 200 estimators followed by the random forest model achieving the best result for an overall accuracy of 69.70% with 50 estimators and a maximum depth of 10, and for SVM, it achieves an overall accuracy of prediction by 53.03% with a kernel type of linear line.

For the random forest model, we have tested with different estimators including 100, 200, and 400, with different maximum depths in terms of 5, 10, 20. The best result among all different configurations is achieved by 50 estimators with maximum depth. In fact, with a larger maximum depth, the model could overfit to the training data and this could worsen the performance of the model when evaluating with the testing dataset. The result is shown in Table 15.

Number of estimator	Maximum depth	Accuracy
20	5	62.12%
20	10	66.67%
20	20	59.09%
50	5	63.64%
50	10	69.70%
50	20	60.61%
100	5	68.18%
100	10	65.15%
100	20	63.64%

Table 15: Result of RF with different parameters

For the SVM model, we have tested with different kernel types namely linear, polynomial, sigmoid, and radial basis. The best result among all the different types of kernel is achieved by the linear kernel. The corresponding result is shown in Table 16.

Kernel type	Accuracy
Linear	53.03 %
Polynomial	30.30%
Sigmoid	25.76%
Radial Basis	31.82%

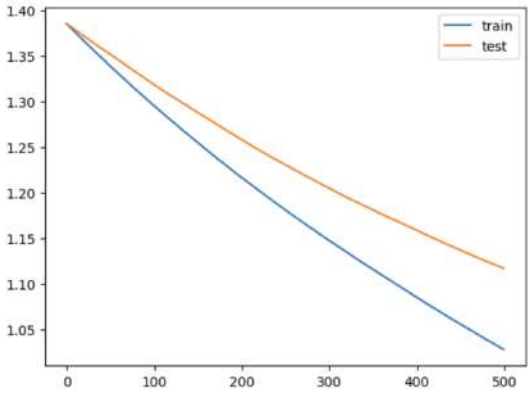
Table 16: Result of SVM with different types of kernel

Lastly for the GBDT model, we have tested with different learning rates namely 0.001, 0.005, and 0.01 with different numbers of estimators in terms of 100, 200, and 500. The reason why we are choosing a larger number of estimators is that the learning rate of GBDT can be fine-tuned through a parameter when using the XGBoost library while there is no customized option for the learning rate when training the random forest model with Scikit Learn. The result is shown in Table 17.

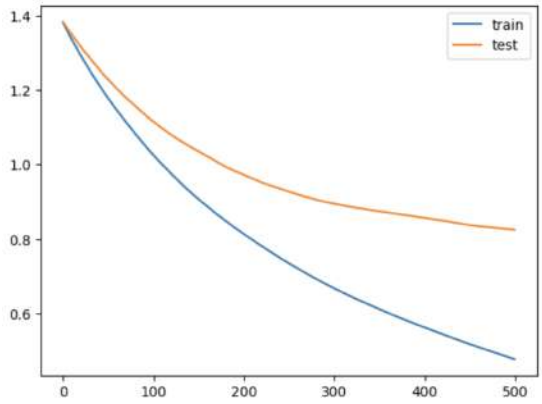
Number of estimator	Learning rate	Accuracy
100	0.001	62.12%
100	0.005	66.67%
100	0.01	68.18%
200	0.001	63.64%
200	0.005	71.21%
200	0.01	65.15%
500	0.001	65.15%
500	0.005	65.15%
500	0.01	62.12%

Table 17: Result of GBDT with different parameters

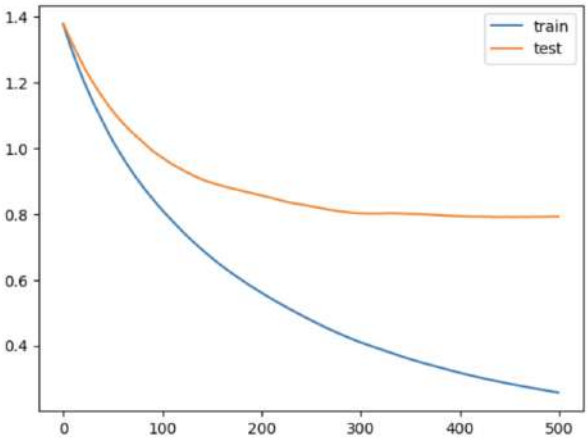
In fact, we have plotted the validation loss in terms of multiclass log loss to observe if there are any overfitting problems. Figure 110 has shown the effect of tuning the learning rate when the number of estimators remains unchanged. It shows that when trained with a learning rate of 0.01, the model started to overfit with a lower training loss and a higher testing loss. The situation is also applicable when tuning the number of estimators in the GBDT model. In this case, there are higher chance of overfitting when we increase the number of estimators and keep a constant learning rate.



(a) Learning rate of 0.001.



(b) Learning rate of 0.005.

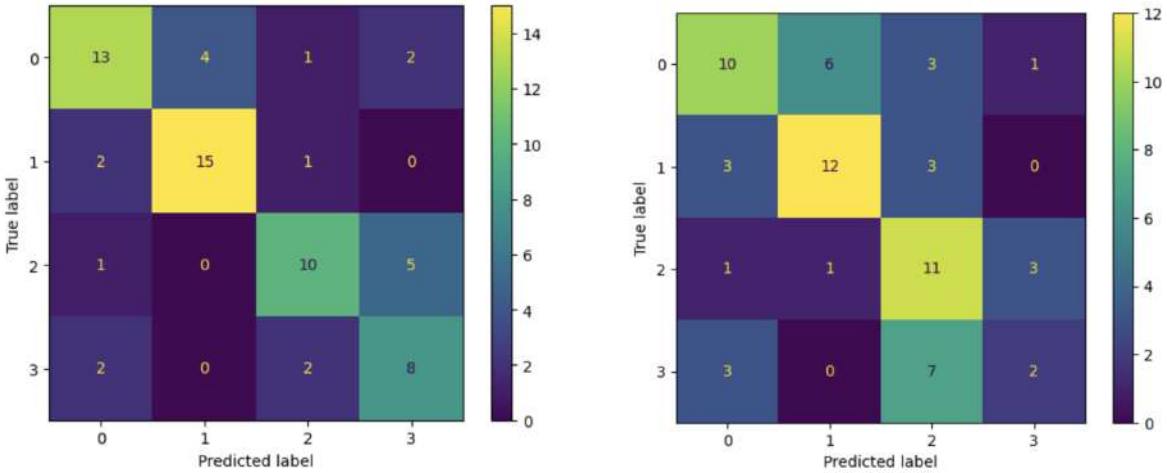


(c) Learning rate of 0.01.

Figure 110: Loss curves when tuning learning rate.

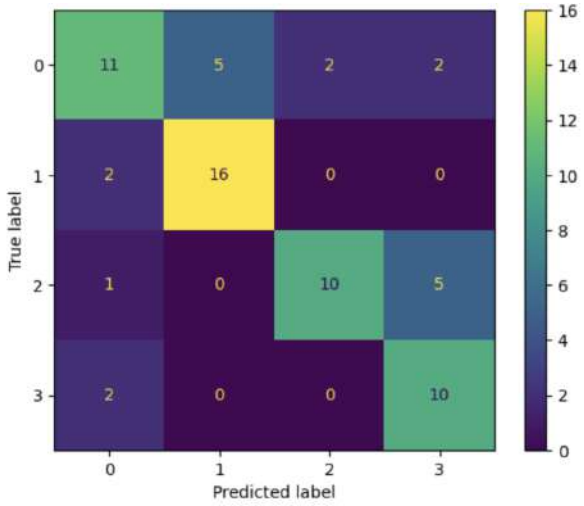
By comparing the confusion matrix of each model, we find out that SVM is not performing well in classifying the data in quadrant four which generally refers to emotions like calm, sleepy, and tired. However, it achieves the best accuracy in predicting quadrant three data. For the GBDT model, it achieves the best accuracy in predicting both quadrants two and four data. While the random forest model performs the best result in predicting the quadrant one data.

In general, all of the three models would misclassify quadrant three data referring to emotions like sad, depression, and boredom into quadrant four emotions as mentioned before. On the other hand, all of the three models would also misclassify quadrant one emotions like excited and delighted into quadrant two emotions like tensed and angry.



(a) Confusion Matrix of RF.

(b) Confusion Matrix of SVM.



(c) Confusion Matrix of GBDT.

Figure 111: Confusion matrix of each model.

By calculating the accuracy of predicting high-arousal or low-arousal, positive-valence or negative-valence emotions respectively, we could conclude that our model is performing better in classifying high or arousal emotions than classifying valence differences between two data with similar arousal values. Eventually, this may indicate that the features we extracted could show differences in arousal value but are not very effective in showing differences in valence value.

Model	High-arousal/Low-arousal	Positive-valence/Negative-valence
SVM	81.82%	65.15%
RF	89.39%	77.27%
GBDT	89.39%	77.27%

Table 18: Accuracy in classifying different arousal and valence respectively

5.6 Visualization of result

After training the model, we have decided to use the best weight from the GBDT model as it has achieved the best overall accuracy in predicting all of the four quadrants data. When the users input their audio into the web application, the function splits the audio into 6-second audio segments every 0.5 seconds by default. The audio features would be extracted from each of the audio segments with the same method as we prepared for the dataset of training. The array of feature vectors is then fed into the GBDT model to obtain the prediction. In order to visualize the prediction on an arousal-valence graph as introduced by James Russel [2], we obtain the probability of prediction for each class instead of the class with the highest probability. Therefore, by this simple equation, we would be able to calculate the x and y coordinates of the prediction on the 100 x 100 arousal-valence graph.

$$\text{x} = 100 * (\text{Probability of Q1} - \text{Probability of Q2} - \text{Probability of Q3} + \text{Probability of Q4})$$

$$\text{y} = 100 * (\text{Probability of Q1} + \text{Probability of Q2} - \text{Probability of Q3} - \text{Probability of Q4})$$

By plotting the coordinates on the graph, we obtain dynamic emotion changes along the user's playing. To indicate the emotion change in time, we added a color dimension for the user to track how their expressed emotion changed along with their continuous playing. As an example, we have tested our model by inputting a playing of Canon in D by Pachelbel. The prediction indicates that the song mainly belongs to quadrant one with both high arousal value and positive valence value, representing it is predicted as a song with emotions like happy, excited, and pleasing. To put it more precisely, the beginning of Canon in D is predicted as quadrant four which represents comfortable or calming emotion. Soon when the intensity of the piece is raising to the climax, the model predicts these audio segments belong to quadrant one which represents emotion like happy and excited. In our own opinion, we consider these changes in emotion is reasonable and could be taken as a reference for the user.

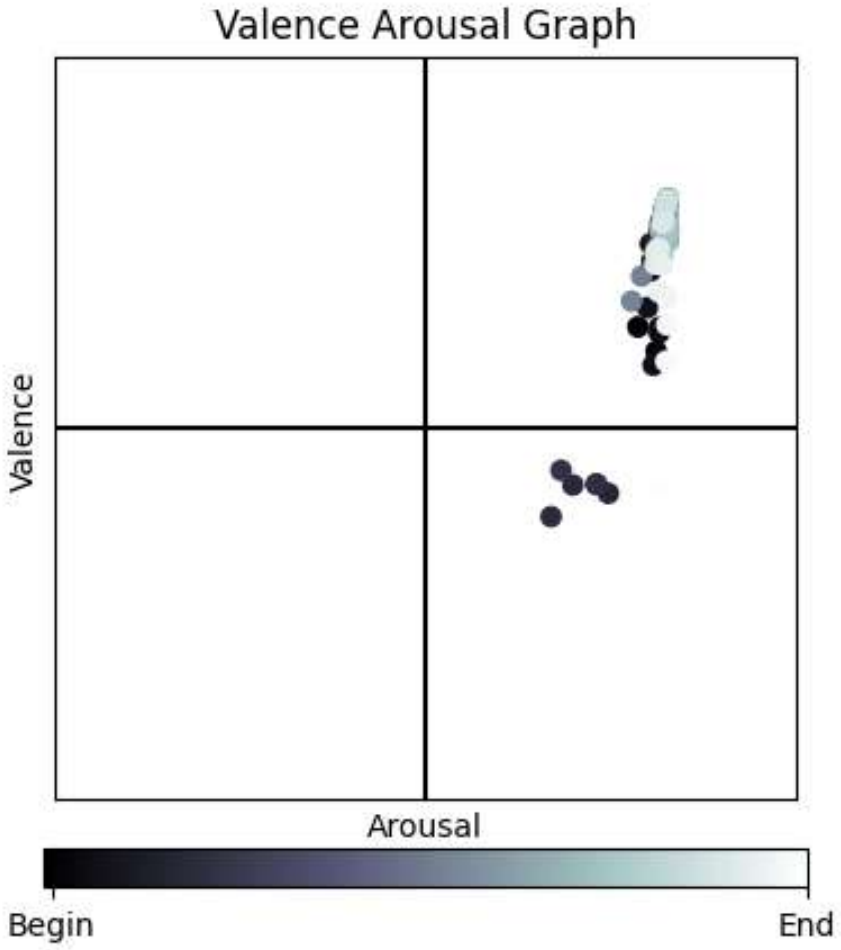
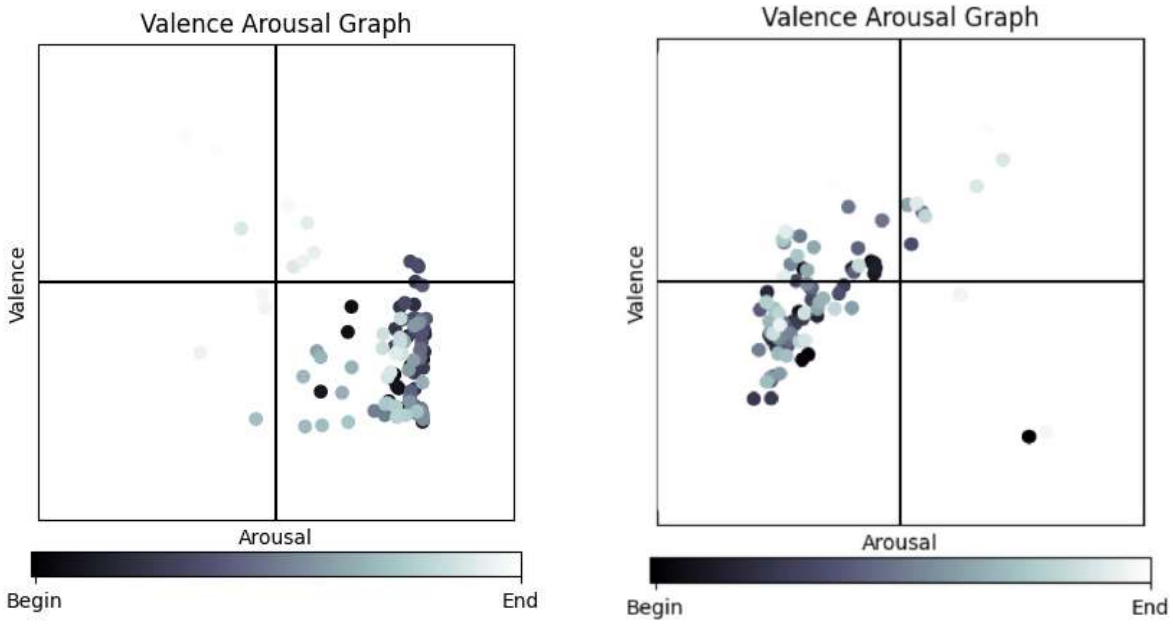


Figure 112: An arousal-valence graph for Canon in D.

5.7 Conclusion

In order to provide some reference on the expression of emotion in music for our user, we have trained three models using machine learning techniques namely Random Forest, Support Vector Machine, and Gradient Boosting Decision Tree. The audio features are extracted dynamically from the input audio by splitting the audio into segments with 6-second duration with a 0.5 seconds sampling rate. Then the predictions of these dynamic audio features are visualized on an arousal-valence graph and being shown to the user in the web application section.

Let's keep in mind that emotion is a subjective feeling. Everyone has their own interpretation of emotion even if they are listening to the same piece of music. It is possible that the composer would like to express their feeling of excitement toward something but the descendants thought that the composer had been mad about someone they hated. In fact, the annotations or the labels of existing datasets are all tagged by human beings or collective opinions by questionnaires. Therefore, we would like to emphasize that this function is just for referencing and education purposes for the users. The result is also presented in a more generalized way only indicating that this song is more or less having high arousal energy and positive valence.



(a) Result of Minuet in D Minor by Bach, J.S.

(b) Result for Moonlight Sonata 1st Movement by Beethoven, L, van.

Figure 113: More sample result of emotion classification.

6 Music Piece Analysis

This section covers the architecture, functionality and implementation of the advanced functions in musical analysis. For technical details for the web application and performance analysis functions, please refer to sections 3 and 4. The entire section is written in term two.

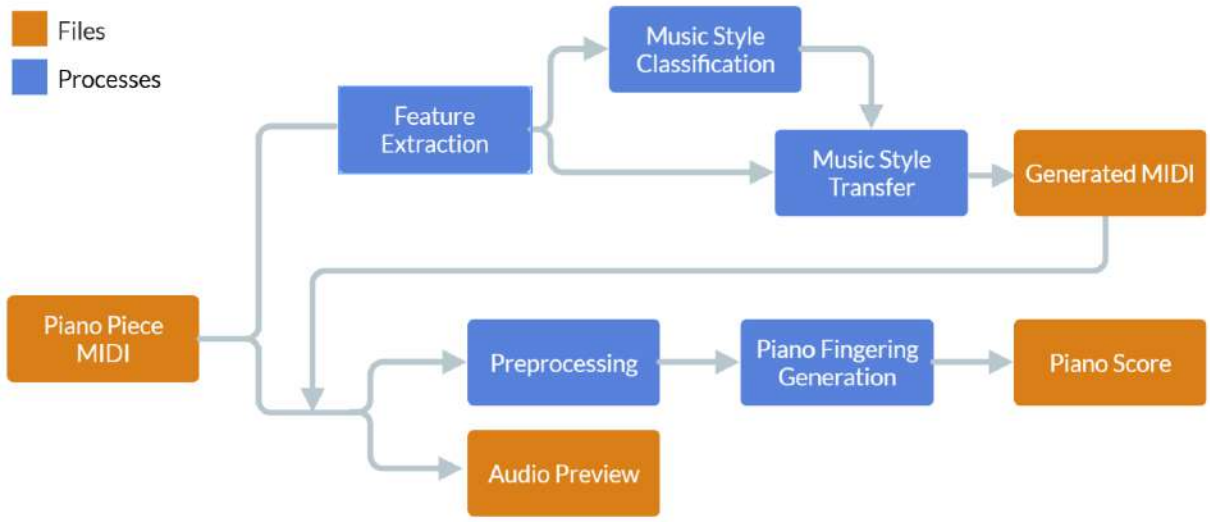


Figure 114: Flowchart of music piece analysis.

6.1 Style Classification

This function is aimed to classify the music era that the user's input possibly belongs to and provides a supplementary purpose for the style transfer function. Moreover, this function could provide more insight for the user to see if they could play in certain music era styles when they are improvising different pieces. In our function, we are using a multi-modal approach by combining a CNN model and a Bi-LSTM model. The whole flowchart of the style classification function is shown in Figure 115.

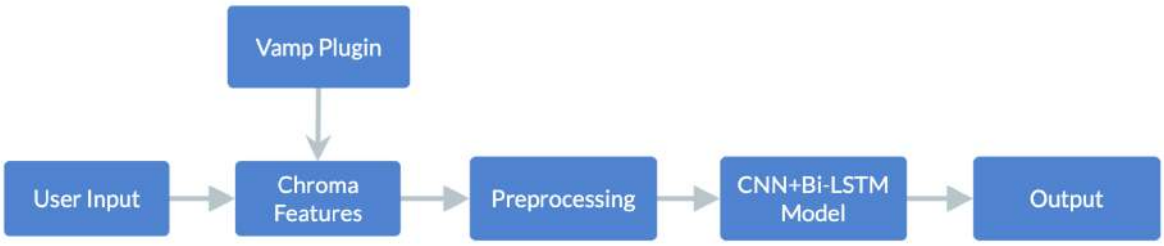


Figure 115: Flowchart for style classification

6.1.1 Related Works

There is not much work has been done on this topic of music era classification. The main reason is due to the lack of dataset regarding different music era pieces. In fact, even some of the famous pieces are still arguably classified into a certain classical music era. Spijker, B.C. van 't [62] proposed to use Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Support Vector Machines as Machine Learning algorithms for classifying pieces into Baruoque era, Classical era, Romantic era, and Modern era. By using MAESTRO dataset from Magenta [63], which consists of approximately 200 hours of piano performance recordings, Spijker extracted Mel-Frequency Cepstral Coefficients and Chromagrams as the evaluating features. The best result is achieved with the CNN model with an accuracy up to 39%.

In our function, we are using time series data for training. As mentioned in the previous paragraph, LSTM is a heated model architecture for machine learning tasks and audio classification task. In fact, Lezhenin et al. [64] have proposed using LSTM model to identify urban sound. Mel-spectrograms are extracted for the training data of the model. Apart from using the LSTM model, the CNN model is also commonly used to analyze the image of the spectrogram. Massoudi et al. [65] have achieved 91% on the testing dataset by training their proposed CNN model with images of mel-spectrogram of urban sound such as horn, drilling, dog bark, etc.

6.1.2 Dataset

For this function, we are using the Cross-era dataset generated by the Audio Lab [66]. It is compiled from commercial audio recordings, totaling 2000 tracks, where a track refers to the movement level of a piece. The dataset contains 1000 tracks for piano and orchestra (without singing voice / solo instruments), respectively. Furthermore, the set is balanced with respect to the historical periods by containing each 400 pieces that are representative of the four historical periods Baroque, Classical, Romantic, and Modern (20th century), as well as an "Add-On" class comprising transitional composers between those periods. Originally, we thought that the orchestral music should be excluded from our dataset. After considering that the data only contains the chroma features and to enlarge our dataset size, we believe that the data of orchestral music could also provide information regarding on their music eras.

Class	Tracks	Example of Composers
orchestra_baroque	200	Albinoni, T.; Bach, J. S.; Handel, G. F.; Vivaldi, A.
orchestra_classical	200	Bach, J. C.; Boccherini, L. R.; Mozart, W. A.;
orchestra_romantic	200	Brahms, J.; Dvorak, A.; Grieg, E.; Liszt, F.; Schumann, R
orchestra_modern	200	Antheil, G.; Bartók, B.; Berg, A.; Britten, B.; Hindemith, P.;
orchestra_addon	200	Bach, C. P. E.; Beethoven, L. van; Debussy, C.; Schubert, F.;
piano_baroque	200	Bach, J. S.; Couperin, F.; Giustini, L.; Platti, G. B.; Rameau, J.-P.
piano_classical	200	Cimarosa, D.; Clementi, M.; Dussek, J. L.; Haydn, J.; Mozart, W. A.
piano_romantic	200	Brahms, J.; Chopin, F.; Liszt, F.; Schumann, C.; Tchaikovsky, P. I.
piano_modern	200	Hindemith, P.; Milhaud, D.; Shostakovich, D., Stravinsky, I.;
piano_addon	200	Bach, C. P. E.; Beethoven, L. van; Debussy, C.; Ravel, M.;

Table 19: Cross-era Dataset

As there are commercial recordings in the dataset, they cannot directly provide audio files. Instead, by using the NNLS chroma algorithm as published in [66], which is freely available as a VAMP plugin, they provide the chroma features of the pieces. Concerning the parameters, a window size of 8192 samples and a step size of 4410 samples are used which lead to a chromagram resolution of 10 Hz. They use the NNLS approximate transcription and no normalization to compute the chroma features from Chroma A to Chroma G# with a sample rate of 0.1 seconds.

6.1.3 Feature Extraction

The type of feature that is being used for training is a time series of chroma features from the audio. The Chroma feature is a descriptor, which represents the tonal content of a musical audio signal in a condensed form. In most scenarios, pitch classes are categorized into 12 different semitone classes, from note A to note G#.

NNLS Chroma analyses a single channel of audio using frame-wise spectral input from the Vamp host which is an easy system to develop plugins for. The NNLS Chroma plugin was originally developed to extract treble and bass chromagrams for subsequent use in chord extraction methods. The spectrum is transformed to a log-frequency spectrum (constant-Q) with three bins per semitone. The processed log-frequency spectrum is then used as an input for NNLS approximate transcription (using a dictionary of harmonic notes with geometrically decaying harmonics magnitudes). The output of the NNLS approximate transcription is semitone-spaced. Therefore, in our function, we are using the Vamp NNLS Chroma plugin to generate the chroma feature from user input.

Many machine learning models, such as CNN and LSTM, require inputs of a fixed shape. Moreover, batch training will be implemented during our training process. To form valid batches, all input samples within a batch must have the same dimensions. As we have different lengths of input audio and we need to unify the length of training data when we feed into our proposed model, preprocessing of the data is required. By considering the average input size of the user and preserving temporal order, we group 100 time stamps of audio data into 2D arrays where the sampling rate is 0.1 seconds. One time stamp of data includes the current time stamp with 12 semitone chroma features from A to G# leading to training data with shapes of [100, 13].

Timestamp	Chroma features (A-G#)
0.1	0.4876, 1.2604, ..., 0.4993
0.2	0.4126, 1.2304, ..., 0.4123
0.3	0.5878, 2.6404, ..., 0.6197

Table 20: Sample of training data

After the preprocessing of the features, we split 80% of the data into the training dataset while the rest into the validation dataset. As a result, the training dataset has a length of 1600 data while the validation dataset has a length of 400 data.

6.1.4 Model Training

As mentioned in the above section, we have proposed a multimodal approach for training which is combining a CNN model with a Bi-LSTM model.

CNN Architecture There are a total of 13 features from input. The CNN model contains 2 convolutional layers, and the first convolutional layer consists of 32 filters with a kernel size of 3. After the convolution layer, a Rectified Linear Unit (ReLU) activation function was applied. The main purpose of using ReLU is to introduce non-linearity in the network. By applying the ReLU function element-wise to the output of a layer, negative values are set to zero, while positive values remain unchanged as shown in Figure 116. This allows the network to learn and model complex patterns and relationships in the input data.

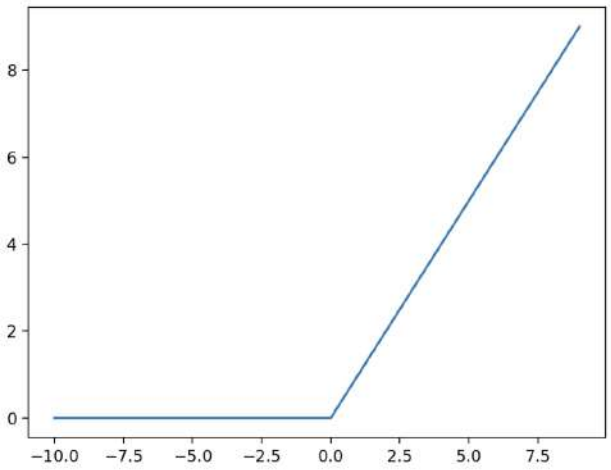


Figure 116: A ReLU Function

After the ReLU function, we use max pooling to downsample the representation of the input by half, where the spatial dimensions are reduced. Following by the second convolutional layer consists of 64 filters with a kernel size of 3. A ReLU activation layer and max pooling layer are applied again after the convolutional layer. The architecture of the CNN model is as shown in Figure 117.

Bi-LSTM Architecture In order to capture temporal information, we use a variation model of the Long Short-Term Memory (LSTM) model which is called the Bi-directional LSTM (Bi-LSTM). LSTM is good for time series prediction because of its ability to remember previous inputs for a long period of time using a mechanism called the "memory cell" that can preserve information for a long period. This makes it well suited for handling sequences with long-term dependencies, where earlier time steps may have a significant impact on later time steps.

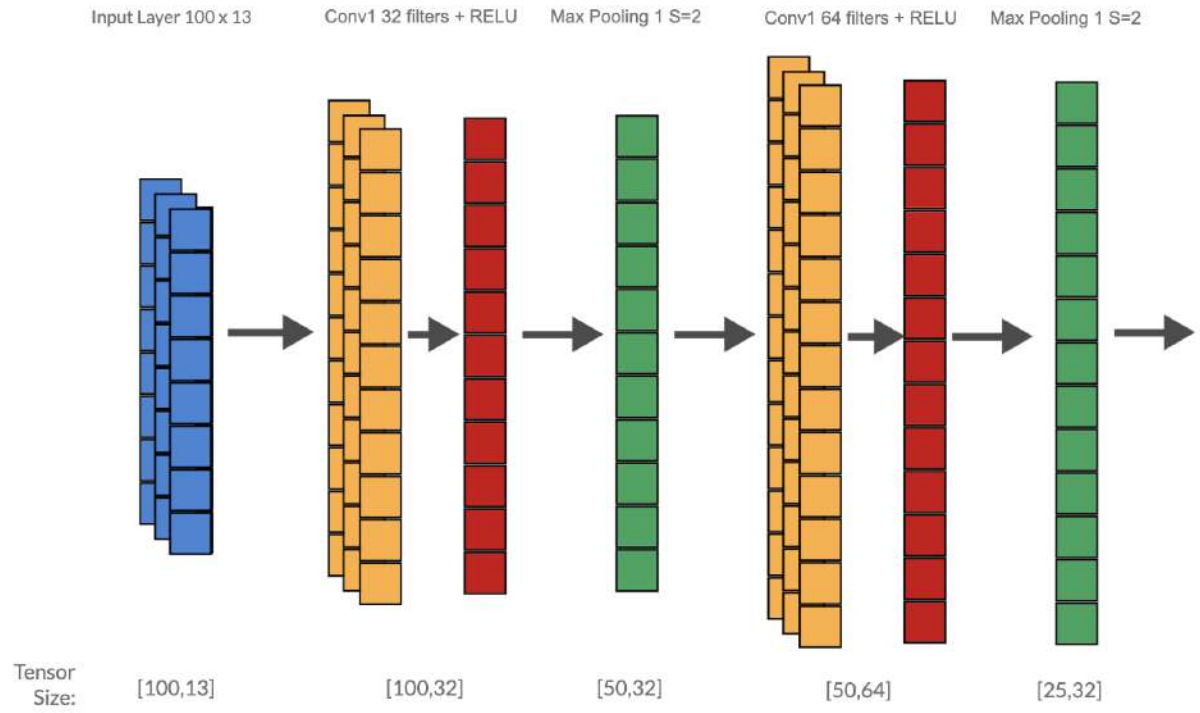


Figure 117: Architecture of the CNN Model

For Bi-LSTM, the main difference between the Bi-LSTM model and the original LSTM model is that, instead of "remembering" the previous inputs for a long period of time, a Bi-LSTM cell could obtain future information by a backward layer flowing from future. By combining two LSTM layers, one flowing forward while the other one flowing backward, the Bi-LSTM model preserves information from both the past and the future. In our case, the model could obtain both the past chroma information and the future chroma information from the audio input.

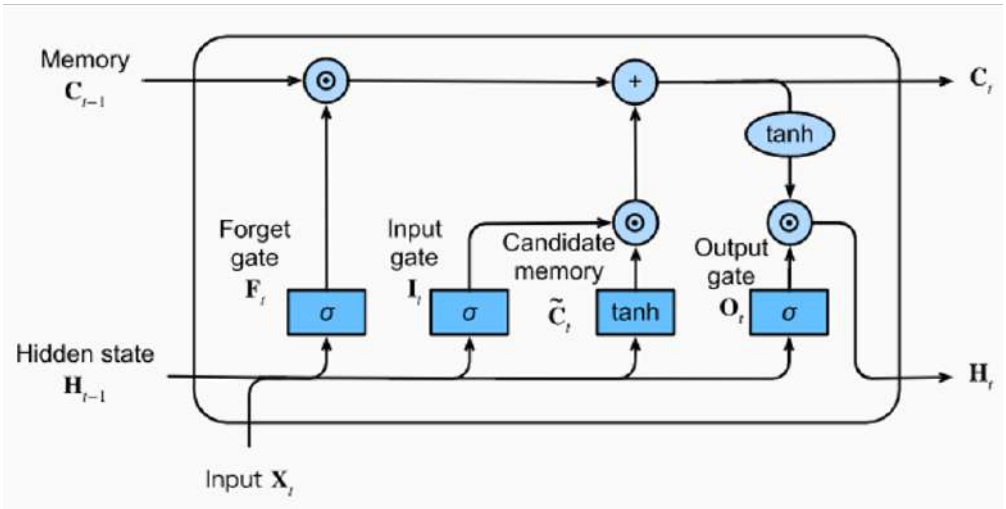


Figure 118: Architecture of the LSTM Model [67]

Overall Architecture In our proposed architecture, we first extract spatial features from the 12 classes of chroma features by using a CNN model. Then we pass the extracted features into the Bi-LSTM model to preserve both past and future information to tackle the time dependency due to the nature of music. The overall architecture of our multimodal approach is as shown in Figure 120.

Training configuration The model is constructed with the famous machine learning library PyTorch [69] in Python [70] version 3.10. The training was running with a P100 GPU to accelerate the progress. As mentioned before, we are using batch training for this model. For fine-tuning, the model is trained with batch sizes of 64, 96, and 128. Adam optimizer with learning rates of 0.0001, 0.0005, and 0.001, are used for training different versions of the model. The loss function used for the model is the Cross-Entropy Loss function which is commonly used for multi-class classification tasks. Early stopping function is implemented to avoid overfitting and longer training time.

Hyperparameter	Value
Batch size:	32, 64, 96, 128
Optimizer:	Adam
Learning rate:	0.0001, 0.0005, 0.001
Loss Function:	Cross-entropy Loss

Table 21: Hyperparameter for style classification

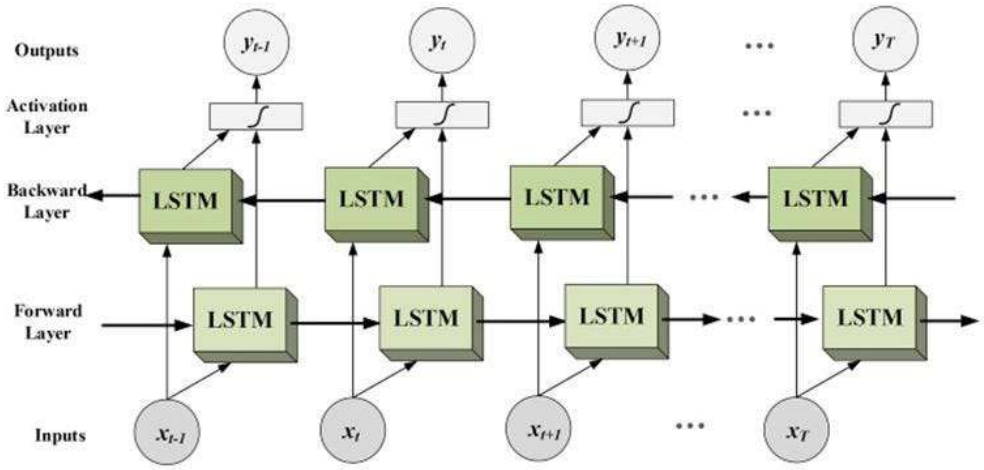


Figure 119: Architecture of the Bi-LSTM Model [68]

6.1.5 Result

To compare the performance of the multi-modal approach, we have trained Bi-LSTM as a baseline for this function. First, we compare the results of models that are trained with different batch sizes, keeping other hyperparameters the same (Optimizer: Adam, Learning rate: 0.0005, Loss function: Cross-entropy loss). The best result in terms of validation accuracy is achieved by training the model with a batch size of 32. The corresponding accuracy is 50.07%. The main reason behind this could be larger batch size is leading to overfitting eventually or converging to a local minimum. However, a smaller batch size means that the model needs to process more batches in one epoch which would lead to a longer training time.

Batch size	Bi-LSTM validation Accuracy	CNN+Bi-LSTM Validation Accuracy
32	43.62%	50.07%
64	42.84%	49.34%
96	40.77%	44.58%
128	42.32%	45.71%

Table 22: Result with different batch size

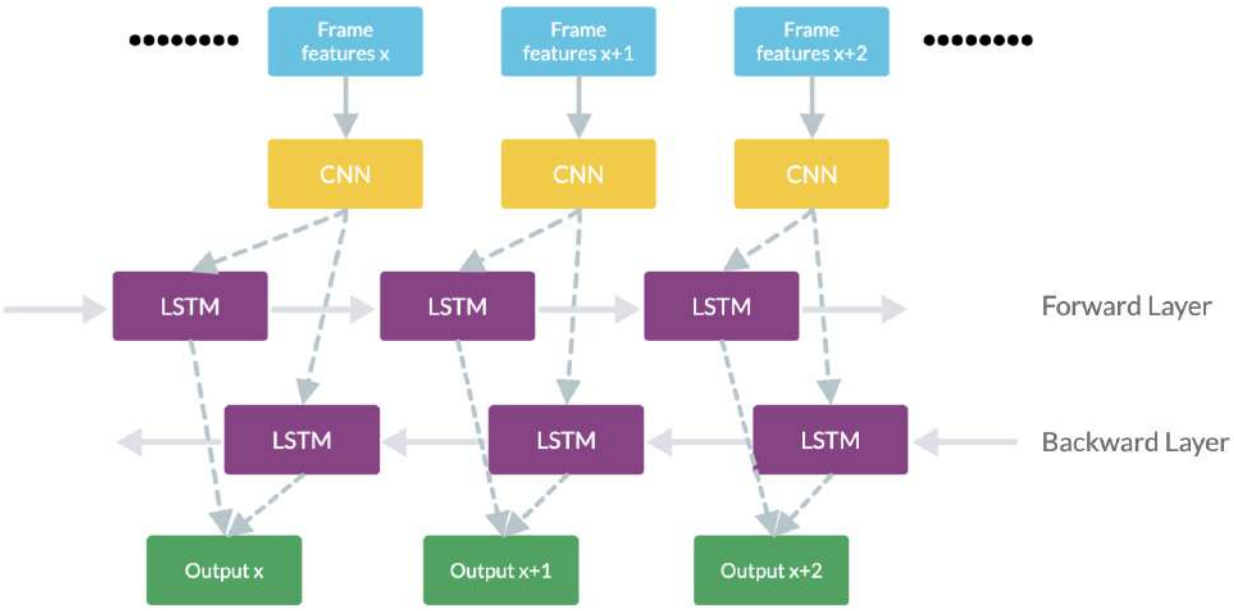
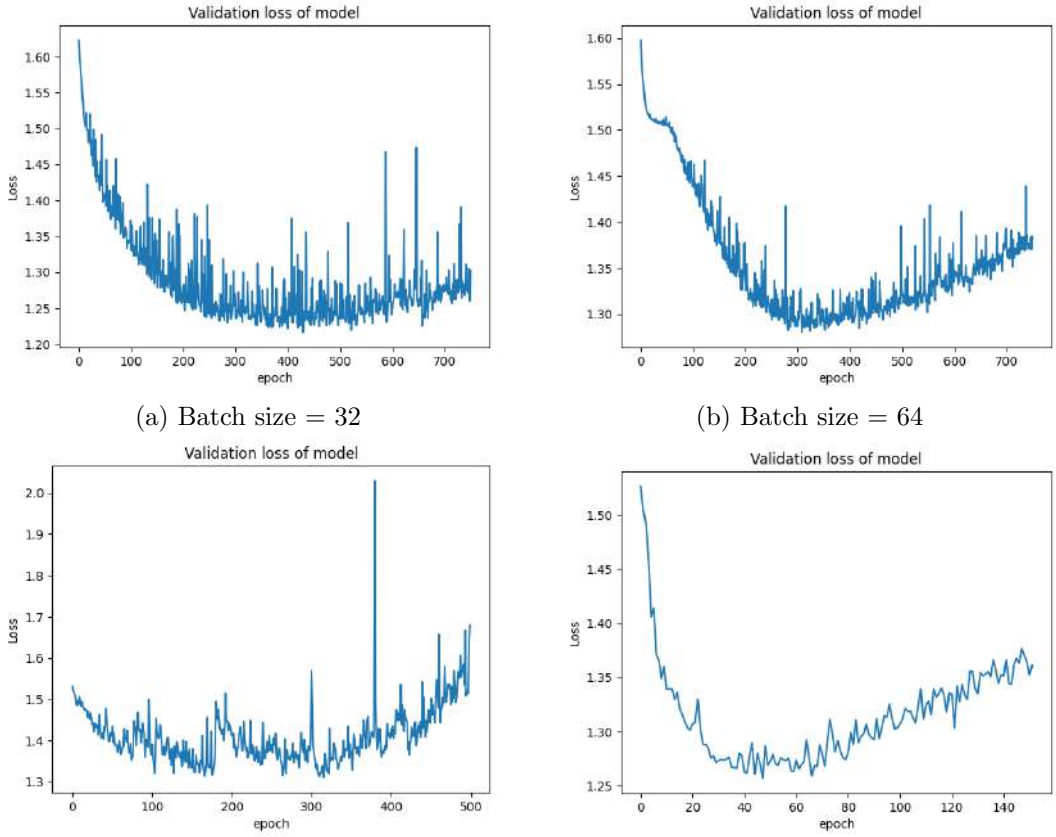


Figure 120: Architecture of the Overall Model

For evaluation purposes, validation loss of the model with different batch sizes are plotted as shown in Figure 121. For larger batch size, the model would trigger the early stopping function in order to reduce the training time. The validation loss is converged to the lowest value for a batch size of 32. However, the result for batch size in 32 is fluctuating because smaller batch sizes give noisy gradients but they converge better to the global minimum.



(a) Batch size = 32

(b) Batch size = 64

(c) Batch size = 96

(d) Batch size = 128

Figure 121: Validation loss with different batch size

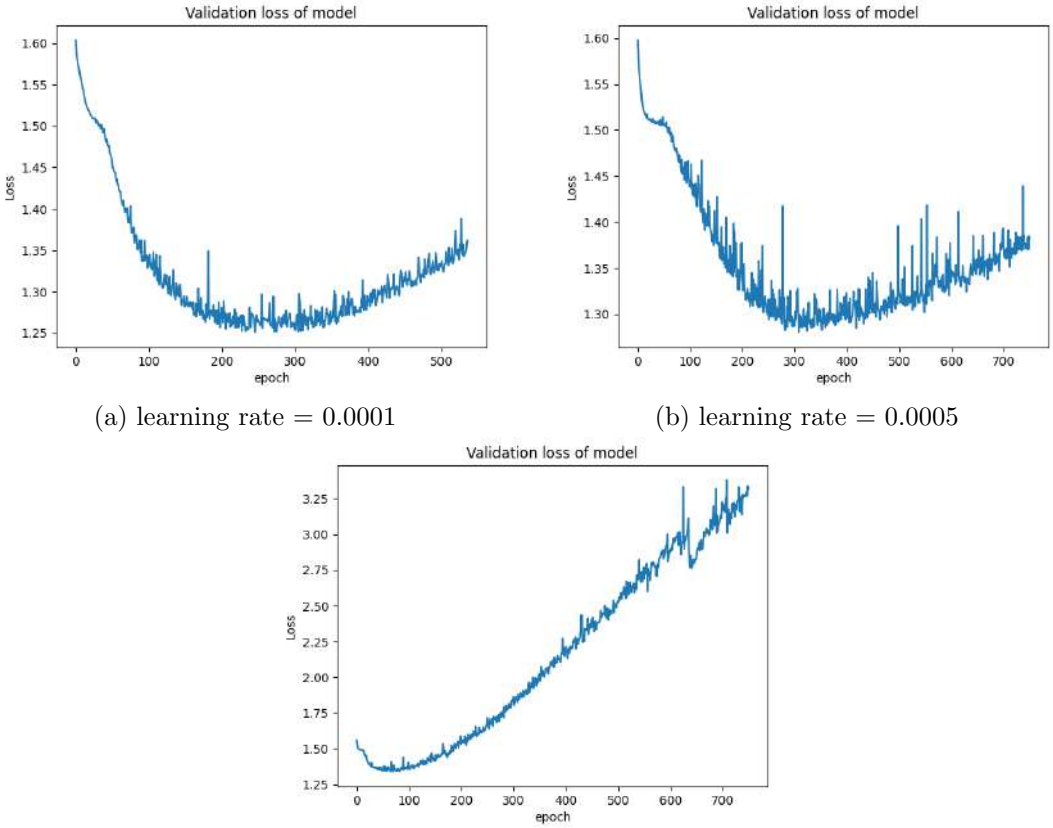
For the evaluation of the effect of the learning rate, we trained our models with different learning rates using the same hyperparameter (Optimizer: Adam, Batch size: 64, Loss function: Cross-entropy loss). The best result is achieved when the model is trained with a learning rate of 0.0001.

Learning rate	Bi-LSTM Accuracy	CNN+Bi-LSTM Accuracy
0.0001	47.83%	54.19%
0.0005	44.84%	49.34%
0.001	39.45%	40.46%

Table 23: Result with different learning rate

As shown in Figure 122, we observe that when the learning rate too large, overfitting occurs with increasing validation loss if the training progress keeps going.

After comparing the results of the model trained with different batch sizes, learning rates, and architecture, we have obtained the best CNN+Bi-LSTM model trained with a batch size



(a) learning rate = 0.0001

(b) learning rate = 0.0005

(c) learning rate = 0.001

Figure 122: Validation loss with different learning rate

of 32 and a learning rate of 0.0001, achieving a validation accuracy up to 57%. It is clear that the multi-modal approach performs better than the baseline Bi-LSTM model. However, this model took almost 12 hours for training which has consumed a lot of computational resources.

6.1.6 Conclusion

Due to the unique subjectivity of individuals, differentiating the music eras of a piece has been a quite challenging task even for a musical expert. Regarding some of the pieces, they could still raise heated controversy regarding which music era they belong to. Therefore, by using a machine-learning approach, we hope to seek more objective insights from an AI model that analyzes music pieces by their dynamic chromatic features.

Possible future improvements can be made by including tempo information of the audio. Even though dynamic chroma features already provide some temporal information, one of the essential criteria for differentiating the music era of a piece is the tempo structure. We have also taken tempo information into account when we planned for the development of the model. However, after digging into the available datasets on the internet, we found that we lack datasets that include such data. One possible reason is that generating these datasets could be much labour intensive and computationally intensive by comparing them to the

computation process of the chroma features. Therefore, if there are complete datasets that include both the chroma features and tempo structure of the pieces, we are quite sure that our model performance can be further improved to a much higher validation accuracy.

6.2 Style Transfer

Style transfer in audio is yet a popular task due to its complexity and time cost. Especially in the music era style, there is no existing work that explored the feasibility of this task. This function is aimed at performing style transfer to the audio from one music era into another music era. We planned to provide a playground for the user to experiment with how well their playing sounds when it is transferred to another music era style. In our function, we first obtain the result of style classification and then perform style transfer to the musical piece by passing the preprocessed transcribed MIDI file of the user's playing generated by our pitch detection function. The whole flowchart of the style transfer function is shown in Figure 123.

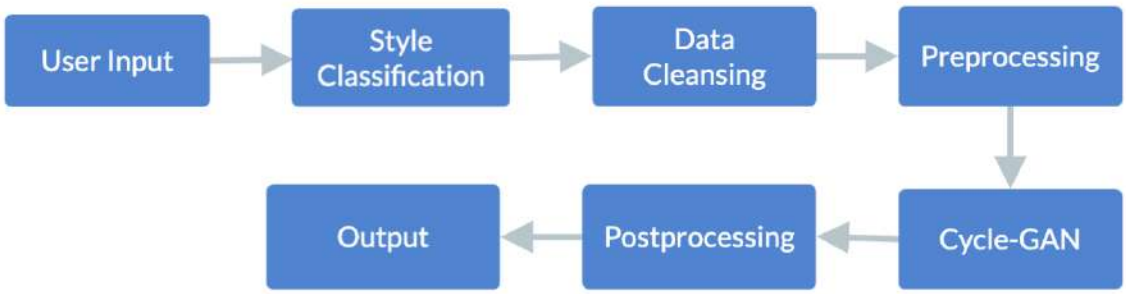


Figure 123: Flowchart for style transfer

6.2.1 Related Works

As aforementioned, there are only a few works that have dug into the field of style transfer in music by comparing it to other tasks in music information retrieval. Existing work on music style transfer includes Brunner et al. [71] proposal of MIDI-VAE, a multitask Variational Autoencoder model with a shared latent space that is capable of changing the style of complete compositions from, e.g., Classic to Jazz. MIDIVAE models most other aspects of music contained in MIDI files, i.e., velocities, note durations, and instrumentation. Malik et al. [72] introduce a model that learns to play music in the style of a human musician. However, it can only change note velocities and does not learn the characteristics of different musical styles/genres.

Style transfer in images has been a heated field recently. Various sequential models like a transformer, Bi-LSTM, or generative models like AutoEncoder (AE) and Generative Adversarial Network (GAN) are widely adopted for style transfer. Deng et al. [73] proposed a transformer-based approach called StyTr2. Similar to tasks in Natural Language Processing, they split the content images and style images into patches and then use a linear projection to obtain patch sequences. Then, the content sequences are fed into a content transformer encoder, while the style sequences are fed into a style transformer encoder. Following the

two transformer encoders, a multi-layer transformer decoder is adopted to stylize the content sequences according to the style sequences. Finally, they use an upsampling decoder to obtain the final output. This approach requires one-to-one pair images for training the model.

Another approach called CycleGAN proposed by Zhu et al. [74] does not require the extraction of explicit style and content features, but instead uses a pair of generators to transform data from a domain A to another domain B. The nature of the two domains implicitly specifies the kinds of features that will be extracted. In other words, the data does not require one-to-one matching and being fed into the model in pairs. However, there will be a one-to-one pair of style-transferring generators. For example, there are two datasets that one of them contains horse images while the other one contains zebra images. CycleGAN can directly transfer the style of horse images into zebra images by learning the characteristics of zebra and apply to any of the horse images. However, the two generators are then restricted to either transferring images from horse to zebra or zebra to horse.

Brunner et al. [75] were then inspired by the method of CycleGAN and applied it to music genre transfer. They proposed to apply CycleGAN in transferring the symbolic representation (Piaonroll) of one genre to that of another genre. CycleGAN would also be implemented in our function for attempting to transfer the audio from one music era to another era. More details regarding the whole process shall be discussed in the following sections.

6.2.2 Dataset

After searching for datasets that contain MIDI files of classical music, we were not able to find a dataset that contained both the MIDI files of the classical music and music era labels. The main reason is that most of these MIDI files have commercial copyrights and are unavailable for sharing or disclosing to the public. Eventually, Giant-MIDI Piano [76] is used as the dataset for this function. Giant-MIDI Piano is a classical piano MIDI dataset that contains 10,855 MIDI files of 2,786 composers. The curated subset by constraining composer surnames contains 7,236 MIDI files of 1,787 composers. GiantMIDI-Piano is transcribed from live recordings with a high-resolution piano transcription system [77].

As the other way round, we need to label the data with music era tags by ourselves. The dataset has provided us with a CSV file that contains the composers' names, birth years, and death years. By assuming their composing style is settled at their 30 years old, we classify the composers into the music era of Baroque, Classical, Romantic, and Modern. However, some of the composer's birth and death years are labeled as unknown, data cleansing is applied to obtain rough CSV files of the composer who belongs to the above four music eras. By searching the MIDI files using the composer's name as the key, we have built a new dataset that classifies the MIDI files into Baroque, Classical, Romantic, and Modern eras.

Music era	Range of Year	Number of MIDI file
Baroque	1600A.D. - 1730A.D.	619
Classical	1731A.D. - 1815A.D.	786
Romantic	1816A.D. - 1890A.D.	3365
Modern	1891A.D. - 1975A.D.	2926

Table 24: Labeled MIDI file for different music eras

6.2.3 Data Preprocessing

As we are feeding the symbolic representation of music into the CycleGAN model, we need to convert the data from MIDI files to the matrix that represents a piano roll. By using two Python libraries called pretty_midi [78] and pypianoroll [79], we could load the MIDI files and convert them into piano rolls. Since every MIDI file has its own arbitrary length, it is necessary to re-sample the MIDI file in order to unify the format of the training data. We use a sampling rate of 16 time steps per bar, a common choice in the literature [71][80], which means that the shortest possible note value is a sixteenth note. Therefore, for a piece with 4/4 time signature, a measure would contain 16 time steps.

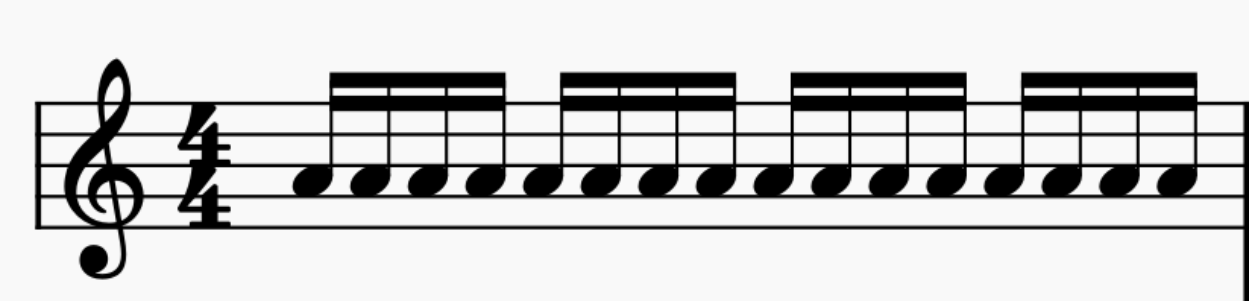


Figure 124: A bar contains 16 time steps (in sixteenth notes)

For a MIDI note, it could contain 128 values representing notes from C-2 up to G8. In considering it is extremely rare to see pitches below C1 or above C8, we will limit the possible range of pitch value to facilitate the training of our model. As a result, one bar would contain a piano roll matrix with a size of 16 (time steps) x 84 (pitch values). However, we also need to take temporal structure into account when we train our model, by referencing the practice of MuseGAN [81], we combine four consecutive bars to include the temporal information of music in our training data, leading to a data size of 64 x 84.

Before any data cleansing, the data could be noisy in the way that their first beat indicated in the MIDI file does not start at 0. Moreover, if we want to unify the time steps for each MIDI, we need to further filter away pieces that do not have a time signature of 4/4. Furthermore, we need to omit the velocity information of each MIDI note by setting them to 127, such that every note has the same loudness. This could improve the model performance in the training progress. After these data cleansing steps, we could now save these matrices in .npy format which is a unique file type for storing the Numpy array. The processed data sizes for each music era are shown in Table 25.

Music era	Number of piano roll matrix
Baroque	19269
Classical	47810
Romantic	83913
Modern	79281

Table 25: Number of piano roll matrix of each music era

In order to accelerate the training process, a decent GPU could shorten the training time by more than 10 times compared to using a CPU for training only. We transmit our data to the GPU server of the CSE Department at the Chinese University of Hong Kong for training of the CycleGAN. Due to the tremendous amount of data, it took more than one hour to transmit all the data to the server.

6.2.4 Model Training

Generator Resnet [82] is used to compose the structure of a generator. The Resnet generator networks utilize skip connections, which prevent vanishing gradient by allowing information to flow through the network without going through all layers. In short, vanishing gradient is referring to a situation that when the gradients become extremely small, they convey little information for updating the earlier layers of the network. As a result, the earlier layers receive weak or no updates, leading to slower learning or ineffective training. In extreme cases, the gradients can become so close to zero that they effectively "vanish," preventing the network from learning meaningful representations or converging to an optimal solution. The structure of a ResNet block is as shown in Figure 125(a). The convolutional layers in the ResNet blocks have 3x3 kernels, stride 1x1, and padding 3.

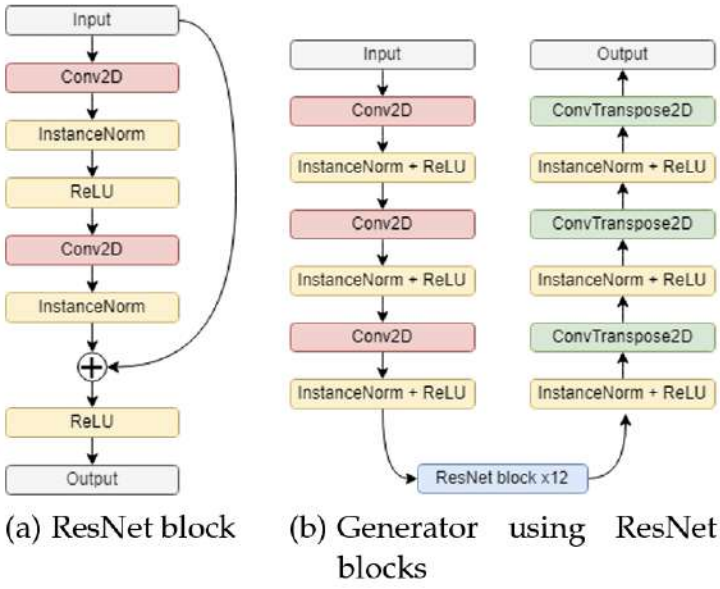


Figure 125: Structure of ResNet based Generator [83].

The generator consists of three convolutional layers, followed by 12 ResNet blocks with different kernel sizes. Finishing with three convolutional transpose layers where the padding of these layers is set to the same.

Layer	Kernel	Channel	Stride	Norm	Activation
Conv2D	7 x 7	64	1	Instance	ReLU
Conv2D	3 x 3	128	2	Instance	ReLU
Conv2D	3 x 3	256	2	Instance	ReLU
ResNet Block x 5	3 x 3	256	1 Instance	ReLU	
ResNet Block x 1	3 x 3	512	1 Instance	ReLU	
ResNet Block x 1	3 x 3	512	1 Instance	ReLU	
ResNet Block x 1	3 x 3	256	1 Instance	ReLU	
ResNet Block x 4	3 x 3	256	1 Instance	ReLU	
ConvTranspose2D	3 x 3	256	2	Instance	ReLU
ConvTranspose2D	3 x 3	128	2	Instance	ReLU
ConvTranspose2D	7 x 7	64	1	Instance	ReLU
Sigmoid	-	1	-	-	-

Table 26: Full structure of generator.

Discriminator The discriminator has a simple convolutional architecture. Moreover, in order to prevent the discriminator from overpowering the generator, additional noise is added to each input to the discriminator.

Layer	Kernel	Channel	Pad	Norm	Activation
Conv2D	7 x 7	64	3	-	LeakyReLU
Conv2D	7 x 7	256	3	Instance	LeakyReLU
Conv2D	7 x 7	1	3	-	Sigmoid

Table 27: Full structure of discriminator.

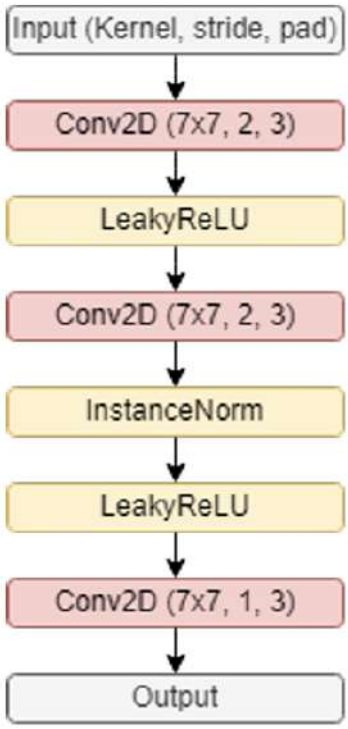


Figure 126: Structure of the discriminator [83].

GAN Architecture The architecture of our model is referencing the original architecture of CycleGAN. The architecture utilized in our model is based on Generative Adversarial Networks (GANs) [84]. GANs consist of two main components: a generator, denoted as G, and a discriminator, denoted as D. The generator's objective is to produce realistic data by transforming input noise, while the discriminator aims to differentiate between the generator's output and real data. G and D are trained iteratively in a competitive manner, forming a two-player minimax game. In our case, as the objective is to transfer music between domains, the generator does not receive noise as input but actual samples from the source domain. For simplicity, we focus on translating between two domains at a time, denoted as domains A and B, representing different music eras.



Figure 127: Structure of GAN.

CycleGAN Architecture Unlike other generative tasks that use one GAN solely, CycleGAN consists of a pair of generators and a pair of discriminators. One generator G_A is utilized in style transferring data from style domain A which was generated by G_B back to style domain B. While another generator G_B is utilized in transferring data from style domain B which was generated by G_A back to style domain A only. For the discriminators, D_A is trying to classify if the data from G_A is real or generated data while D_B is trying to classify if the data from G_B is real or generated data. The basic structure of the model is shown in Figure 128.

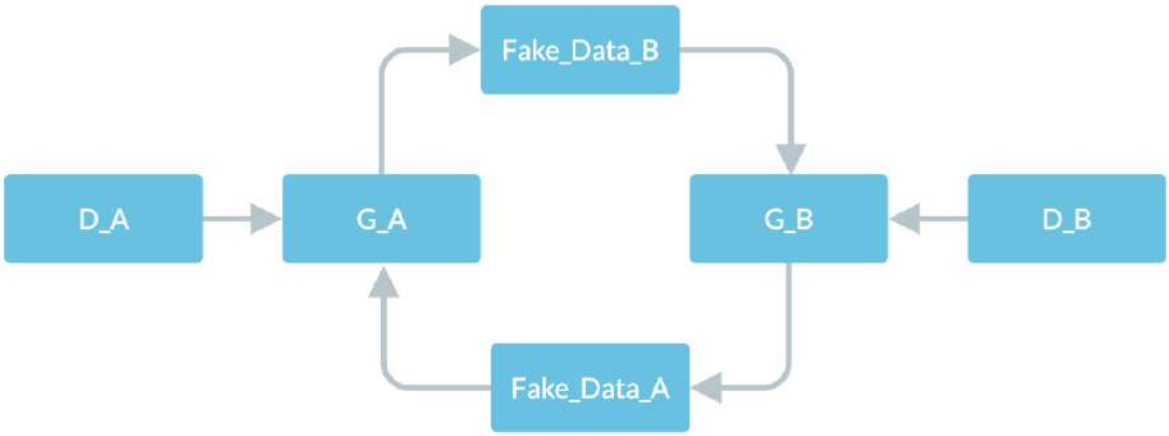


Figure 128: Basic structure of CycleGAN.

6.2.5 Result

There are several metrics for evaluating the performance of the CycleGAN. As proposed in [84], we use the L2 norm for the adversarial loss to evaluate our generators, the equations are as below:

$$L_{G_{A\to B}} = \|D_B(\hat{x}_B) - 1\|_2$$

$$L_{G_{B\to A}} = \|D_A(\hat{x}_A) - 1\|_2$$

Apart from the adversarial loss induced from the discriminator, Zhu et al. [74] also introduce an extra L1 loss called cycle consistency loss. By incorporating the cycle consistency loss, we guarantee that the input, upon traversing both generators and completing the cycle, is accurately mapped back to itself. Or in other words, for every complete cycle, we are calculating the loss between the original input to G_A and the generated output of G_B by taking input of the generated output of G_A.

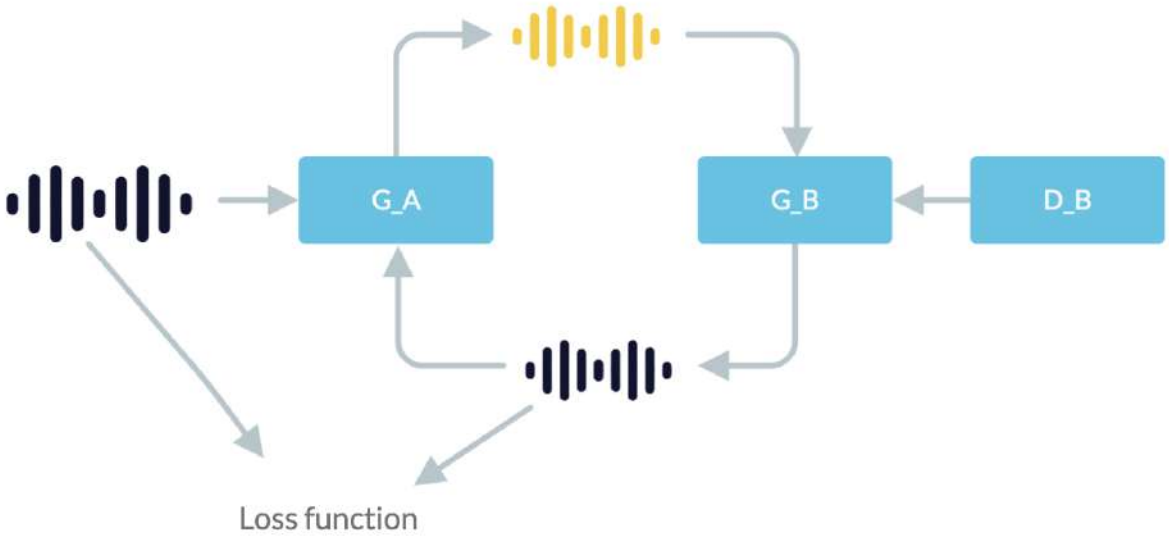


Figure 129: Explanation of Cycle Consistency Loss.

Therefore, the Cycle Consistency Loss can be calculated as :

$$L_c = \|\hat{x}_A - x_A\|_1 + \|\hat{x}_B - x_B\|_1$$

And the total generator loss can be obtained by doing addition of all the above losses:

$$L_G = L_{G_{A\to B}} + L_{G_{B\to A}} + L_c$$

As one CycleGAN model can only cater to the style transfer from one style to another, in our case, we have to train 6 CycleGAN models to cater to all kinds of style transfer of music eras. For each of the CycleGAN models, we downsample the data size of the era that has a larger dataset than that of the other one to avoid introducing bias due to an imbalanced dataset.

Type of style transfer	Balanced data size
Baroque, Classical	19269
Baroque, Romantic	19269
Baroque, Modern	19269
Classical, Romantic	47810
Classical, Modern	47810
Romantic, Modern	79281

Table 28: Six types of CyclGAN and their data size

We have trained 20 epochs for each of the CycleGAN models. The average training time of one epoch is varied with the data size of the two music eras. For example, it took around 2 hours to finish the training of the Baroque, Classical model, while it took nearly 8 hours to train the Romantic, Modern model. The best result in terms of total generator loss is the CycleGAN model of Baroque and Modern eras with a total loss of 1.27. While the worst result among the models is 3.07 of the models for Romantic and Modern. A possible reason is that the overall data size is too big and hence it is extremely difficult for the model to converge and learn from the tremendous features of audio. Therefore intuitively, when an audio is trying to transfer to a music era that could have any pattern of notes, the result would be disappointing.

Type of style transfer	L_c	$L_{G_{A\to B}}$	$L_{G_{B\to A}}$	L_G
Baroque, Classical	0.87	0.52	0.40	1.79
Baroque, Romantic	1.23	0.45	0.31	1.99
Baroque, Modern	0.65	0.35	0.27	1.37
Classical, Romantic	1.04	0.33	0.59	1.96
Classical, Modern	1.42	0.56	0.38	2.36
Romantic, Modern	1.78	0.71	0.58	3.07

Table 29: Total Generator Loss of the models

Another possible reason for the disappointing result is that the dataset or data type is not that good for training. We are not saying that the quality of the MIDI is not that good, instead the nature of the audio may not suit the requirement of uniformity for training data. It is difficult to let the machine to understand the music in segments without sequential information. Just like tasks in natural language processing, we need to consider the past and future information of the audio in order to learn the pattern of different musical styles.

6.2.6 Post-processing

After obtaining the trained weights of the six CycleGAN models, we merge the function with the style classification function and the web application. By knowing the music era that the user's input is, we could let the user choose which kind of music era they want to transfer their style to. For example, the user has played the piece Minuet in D Minor by Bach. The classifier would then classify the input as Baroque style. Then it provides options for the user to change from Baroque to the era of either the Classical era, Romantic era, and Modern era. In case the user has chosen the style transfer to the Modern era. The function will then split the audio into segments of four measures in order to fit into the CycleGAN model. Then after getting the result from the model, we merge the segments back together to form the transferred audio.



Figure 130: Sample of the transferred result of the Minuet in D Minor.

6.2.7 Conclusion

Notice that the model is trained with data of 16-timesteps per measure, and therefore the output model is also pitched in sixteenth notes. This may not be very user-friendly for people who want to try and play the transferred piece. Moreover, the MIDI file is auto-transcribed by MuseScore and this may also affect the readability of the generated piano score. However, we hope to provide some insights or information about how machines could interpret music from different eras.

As a matter of fact, it is better to reduce the data size or even resample the data by choosing some of the famous and signature composers from each era, as their music could better represent the uniqueness of the music era than grouping and, classifying musical pieces that were composed between the range of a certain music era.

Apart from resampling the data, we should be using a sequential model instead of a generative model even though CycleGAN provides huge usability in training without one-to-one matching. We can not neglect the probability that CycleGAN could eventually trained to become an outstanding model in music era style transfer. However, it is better to provide more information on the temporal structure of the model instead of giving only four measures of music as the training data.

6.3 Piano Fingering

The main goal of this module is to automatically generate piano fingering for any given piano pieces. A long short-term model (LSTM) is trained to take a sequence of notes as input and generate their corresponding piano fingerings.

The model is first trained on a piano fingering dataset, then the trained model is used to generate the fingerings of a new music piece given as the input of the music piece analysis function. The overall training pipeline is shown in Figure 131. Feature extraction and data preprocessing are performed on the data, then it is passed to the model for training. Finally, the accuracy of the trained model is evaluated.

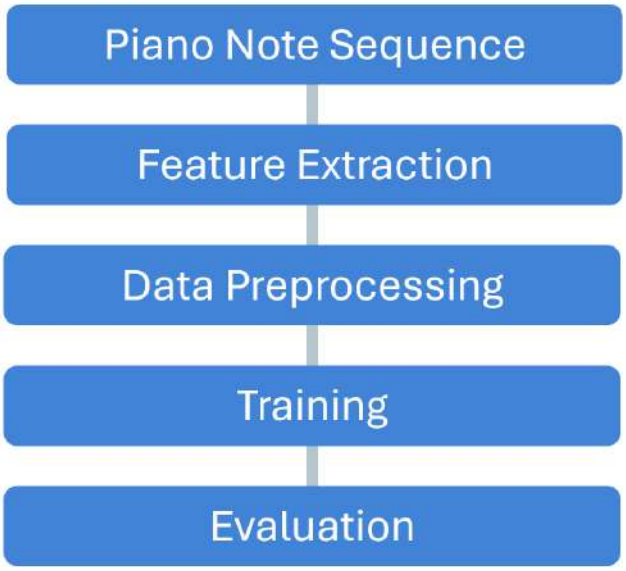


Figure 131: Training pipeline for piano fingering generation.

6.3.1 Related Works

The topic of estimating piano fingering has been long around. An early approach was the constraint-based method proposed by Parncutt et al. [85], which focuses on monophonic pieces. Various improvements have been made based on this method, such as an online extension of the method by Lin et al. [86], and a polyphonic version by Balliauw metal [87].

Apart from constraint-based methods, Hart el a. [50] also proposed a cost-based approach combined with dynamic programming to generate monophonic music. This method was later improved by Al Kasimi et al. [51] who proposed the modelling of fingering of chords into trellis graph and the usage of horizontal and vertical costs to evaluate chord transitions.

Later, the Hidden Markov Model (HMM) started to emerge, in which the works by Yonebayashi et al. [88] and Nakamura et al. [89] are all based on HMM, with a trend of shifting to higher order HMMs.

In recent years, under the rapid development of neural networks, Guan et al. explored the usage of LSTM to generate monophonic music, and then convert the monophonic music's fingering back to chords by using sets of heuristic rules.

6.3.2 Dataset

The piano fingering dataset (PIG dataset) prepared by Nakamura et al. is used for this task [89]. It contains 150 piano pieces by 27 Western classical music composers with fingerings annotated by pianists.

The raw data is a set of text files, where each file represents a sequence of notes. Each file contains a tab-separated table, in which each row represents a single note. The provided attributes of each note (i.e. table columns) are as follows: names=["id", "onset_time", "offset_time", "spelled_pitch", "onset_velocity", "offset_velocity", "channel", "finger"]

Name	Description
onset_time	Note onset (key press) time in seconds.
offset_time	Note offset (key release) time in seconds.
spelled_pitch	Pitch containing pitch class and octave (e.g. A4).
onset_velocity	Velocity (note intensity) of key press.
offset_velocity	Velocity (note intensity) of key release.
channel	Indicate it is played with the left (1) or right (0) hand.
finger	Indicate it is played with which finger (1-5).

Table 30: Features in the PIG dataset.

6.3.3 Feature Extraction

Feature extraction is performed on the data to convert the raw data into more meaningful features. It helps reduce the dimension and noise in the data, which increases the training efficiency and possibly improves the model's performance. It also enables a better training outcome under limited data size.

The feature extraction converts the features from the dataset ("onset_time", "offset_time", "spelled_pitch", "onset_velocity", "offset_velocity", "channel", "finger") to a new set of features ("onset_time_difference", "note_distance", "previous_key_color", "current_key_color", "finger").

Velocity

Velocity refers to the speed or force with which a piano key is pressed. While it affects the volume and timbre of the note, it does not affect the physical positioning and movement of fingers, which are the key factors affecting piano fingering. Also, many MIDI pieces do not contain the onset and offset velocity data, making it hard to build a model based

on it that is applicable to most piano pieces. Therefore, the features "onset_velocity" and "offset_velocity" are removed.

Time-related Features

The time-related features in the raw data is the "onset_time" and "offset_time". However, the "offset_time" is removed due to the difficulty in accurately determining its value in piano performance transcription.

This function is applied to generate fingerings for the transcribed score of the user's performance recordings. However, the detection of offset time has a high error rate because the release of note does not yield a sudden energy change like onset, the gradual decrease in energy, which is also affected by the velocity, makes it hard to determine its boundary and distinguish cases such as whether the note is pedalled or sustained. Moreover, the decay of the note would overlap with subsequent notes in continuous music and could hardly be identified due to its low energy level. Hence, the offset time detection can hardly be relied upon, and including it in the model may yield unexpected results during inference.

In our work, we used the difference between the onset times of adjacent notes instead of the absolute onset time. The onset time difference of note i is defined as $onset_time[i] - onset_time[i-1]$, where $onset_time[i]$ refers to the onset time of note i.

This is selected because the ideal fingerings for the same note sequence should be the same even when the sequence is time-shifted (i.e. starting the entire sequence earlier or later). For example, many piano pieces contain repeating sections. Supposedly, since the note sequence (pitch, sequence, length) is the same, the optimal fingerings should remain the same. In such a case, using the relative time between notes instead of the absolute time can help remove noise and result in consistent feature values across repeated sections.

Meanwhile, the onset time difference is used because the time allowed to adjust one's hand and finger positions greatly affects the fingering decision. Transitions such as hand position shifting (i.e. shifting the entire hand left or right) are more time-consuming than spanning the hand to reach further notes.

Case	Onset time of 1st note (s)	Onset time of 2nd note (s)
1	0	1
2	2	3
3	0	0.1

Table 31: Example for the note sequence C4-C5

Consider the sample shown in Table 31. Suppose the sequence consists of two notes, C4-C5. They are exactly one octave (12 semitones) different. In case 1, the sequence can be played by spanning the hand (e.g. 1-3) or hand shifting (e.g. 1-1). In case 2, the onset times are shifted two seconds behind compared to case 1, and the same sets of fingerings are possible. Meanwhile in case 3, while the onset time of the starting note is the same as in case 1, the onset time of the second note is much earlier, changing from 1 second to 0.1 seconds, and hand shifting becomes much less feasible in such a case.

The example above illustrates the effects of time shifting and relative onset times on making piano fingering decisions. Therefore, the onset time difference is selected as the only time-related feature.

Pitch-related Feature

The only pitch-related feature in the raw data is "spelled_pitch". It is first converted into its corresponding MIDI note number, such as from "C4" to 60.

Instead of the note number, we used the note distance between adjacent notes. For example, a note distance of 1 means that the two notes differ by one semitone. More precisely, the note distance of note i is defined as $note_number[i] - note_number[i-1]$, where $note_number[i]$ refers to the MIDI note number of note i.

This is because the note distance is correlated with the physical distance of two notes on the piano keyboard since the note number of the piano keyboard is arranged in ascending order from left to right. Also, the repeating arrangement of the keyboard makes the shifting of pitch less significant.

The keyboard repeats once every octave (12 semitones), so the fingerings should be the same also, as long as the relative pitch remains the same. For example, the sequences C4-D5 should be treated identically as C5-D6, which shifts the whole sequence up by an octave. Meanwhile, within each octave, the note difference approximates the horizontal distance generally well, with an exception of the pairs of keys E-F and B-C which does not have a black key in between, resulting in a double in horizontal distance despite having a horizontal distance of 1.

Therefore, we also included the label of whether the previous and current piano key is black or white as inspired by [90]. In such a case, the input includes all four types of transition between black and white keys. There should be sufficient input to identify the previously mentioned exception. The significance of the piano key color in piano fingerings is also reflected by the fact that it is included in one of the rules.

Therefore, the pitch-related features extracted are note distance and black/white key labels. In order to investigate the significance of the black/white key labels, another model is trained without these labels.

To sum up, four features are extracted to be the input to the model. They are "onset_time_difference", "note_distance", "previous_key_color" and "current_key_color".

Fingerings

In the dataset, "channel" and "fingers" are used to indicate the hand and finger index respectively. In our model, we reduce it to "fingers" only by converting the left-hand fingering into the same format as the right-hand fingering.

An important property is the symmetry of the two hands. As shown in Figure 132, the structures of the left and right hands are reflections of each other.

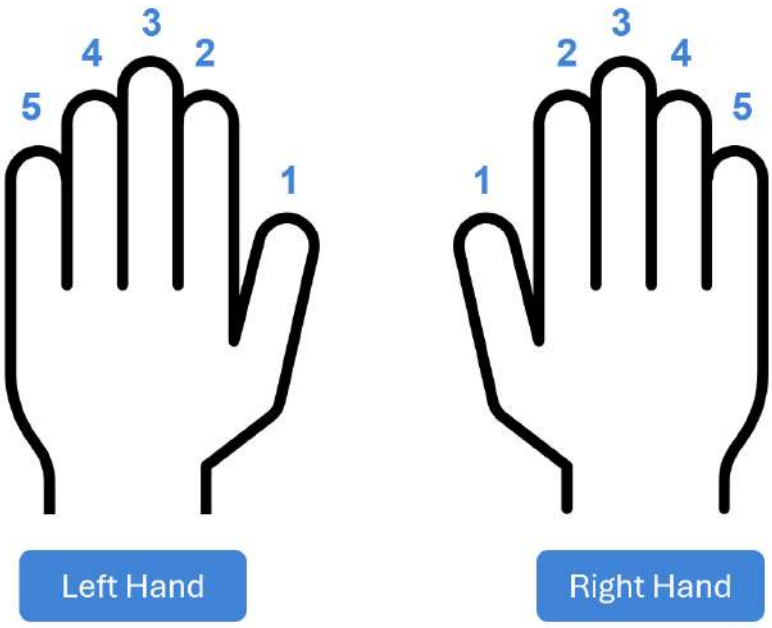


Figure 132: Structure and finger indices of the two hands.

Under the assumption that the key color feature can cater for the asymmetry of a piano keyboard, a sequence played by the left hand can be mapped to its right-hand equivalent by "flipping it horizontally", which means flipping the signs of the note distance.

Also, we assumed that the piece provided is not designed to contain hand-crossing, which means that the hands will overlap with each other, and at least one of them is playing out of its normal note range. Under this assumption, the note sequence of the two hands can be considered independently.

Therefore, when performing feature extraction, each sequence provided in the dataset is first separated into two sequences representing the two hands based on the "channel" feature.

Afterwards, the features mentioned above are computed for each sequence. Finally, the sign of the note distance in the left-hand sequence is flipped.

6.3.4 Data Preprocessing

After the features of each sequence are extracted, this module converts it into the suitable format for model input. The procedures include feature normalization and sequence padding.

Feature Normalization

Normalization is performed on note distance. A clipping is first applied to clip the absolute note distance to smaller than or equal to 12. The value 12 represents an octave, and it is assumed that if two consecutive notes are distanced over an octave, it likely requires a hand position shifting. In such a case, the entire fingering position is reset, and whether shifting two or three octaves does not matter.

Afterwards, the note distance value is normalized to the range of $[0, 1]$ by this formula: $\textbf{normalized note distance} = \frac{\textbf{note distance}+12}{24}$.

Normalization is not performed on onset time difference because, in a normal music piece that is continuous, most time difference between notes falls lower than one second. Meanwhile, the maximum values can be large, such as one hand taking a long rest when it is the other hand's solo section. Hence, dividing all time differences by the maximum value may yield unreasonably small values for the majority of data. As for the key colour attribute, they are originally binary labels valued 0 or 1, so they do not require normalization either.

Sequence Padding

After normalization is done, the sequence's shape should be (sequence length, 4). However, the sequence length varies among sequences but the model would require a fixed input dimension.

Therefore, sequence padding is used to either pad short sequences with [-1, -1, -1, -1], or trim longer sequences to unify the sequence lengths. In the training, both target lengths 128 and 256 are tested. The pad_sequences function of Keras is used.

After the sequence padding, each input sequence is shaped (128/256, 4), while each output sequence is shaped (128/256).

Train-test-split

Afterwards, train-test-split is performed to separate the data into two subsets for training and testing. The training subset is used for the model to learn and adjust its weights. The testing subset is solely for evaluating the model throughout training, to check its performance on data out of the training set.

The proportion of training and testing data is 3:1, and are randomly selected from the dataset. It is implemented using the train_test_split function in scikit-learn [91], a machine-learning library in Python.

6.3.5 Model Architecture

The model takes in sequences of dimensions (128/256, 2/4) and outputs the predicted class (fingering) of each item, so the output dimension is (128/256, 5). Its architecture is shown in Figure 133. It consists of a total of 6 layers and is built using Keras.

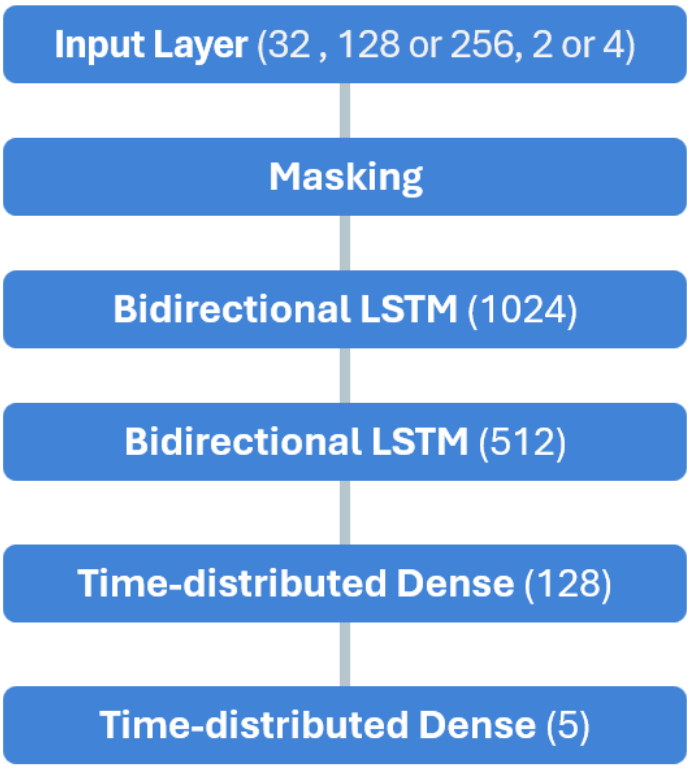


Figure 133: Piano fingering model architecture.

The input layer defines the dimension of the input data, which is (batch size, sequence length, number of features). As mentioned above, we have experimented with the sequence length and number of features, so the values may be different among models.

The second layer is a masking layer. It is a layer for masking the pads in a padded sequence. It would create a binary mask, which is set to false if that part of the sequence is

padded.

The key component of the model is the two sequence-to-sequence bi-directional LSTM layers. LSTM refers to a Long Short-Term Memory model, it is a type of recurrent neural network (RNN) architecture designed to learn from sequences of data by maintaining a memory of previous inputs through internal states and gates. This ability to capture long-term dependencies and contexts makes it useful for handling our data. It resembles how pianists make their fingering decisions based on their past decisions (i.e. the current fingering position).

A bi-directional layer is wrapped around each LSTM layer, this enables LSTMs to access both past and future context. This also resembles the behaviour of pianists as they would also consider notes ahead to determine the piano fingering. For example, a fingering choice optimal for the current time step may not be the best after a few more notes are played. A more detailed description and comparison of the architectures and differences between LSTMs and bi-directional LSTMs can be found in Section 6.1.4.

The last two layers are time-distributed dense layers. A dense layer is independently added to each time step of the sequence (i.e. each note). The first appended layer is a dense layer with 128 neurons, while the second layer is a dense layer with 5 neurons, which is exactly the number of output classes (corresponds to 5 fingers).

A Rectified Linear Unit (ReLU) activation function is applied to the first layer, which increases model complexity by introducing non-linearity in the network. By applying the ReLU function element-wise to the output of a layer, negative values are set to zero, while positive values remain unchanged.

A softmax activation function is applied to the second layer to convert the output logits into probabilities by exponentiation and normalizing them. It is chosen since the task is a multi-class classification task, the the i-th output can be interpreted as the probability that the class is i. This facilitates the loss calculation using cross-entropy loss.

6.3.6 Model Training

Table 32 shows the hyperparameter used for the training.

The batch size is set at 32, which is the largest possible value under the training environment. This is because LSTM models need to be updated sequentially, so they cannot utilize the parallel computation performed by GPU. Hence, the training is done on the CPU, and the resource allocated is very much limited. The training speed is also slow due to this reason, taking over 6 hours for around 30 epochs.

Hyperparameter	Value
Batch size	32
Optimizer	Adam
Learning rate	0.001
Loss Function	Cross-entropy Loss
Epoch	200 (with early stopping)

Table 32: Hyperparameter for training the piano fingering model.

The Adam optimizer is used with its default learning rate of 0.001. It is an optimizer that supports an adaptive learning rate and is commonly chosen for its robustness and efficiency in converging towards a lower loss. On the other hand, the loss function is the sparse cross-entropy loss, which measures not only the accuracy but also if the model is able to give the correct class a higher probability (close to 1) while minimizing the probability of incorrect classes.

When training, the number of epochs is configured to be 200 with an early stopping callback which will halt the training when the validation loss does not decrease for over five epochs. This helps prevent the model from overfitting the training data.

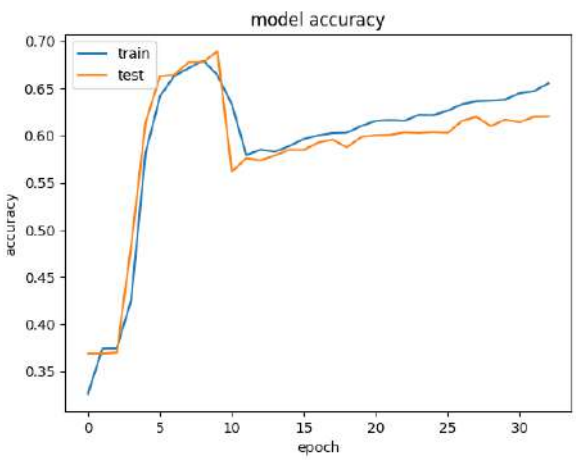
6.3.7 Result

As mentioned above, three different model configurations were used for comparison. The generated results are shown in Table 33.

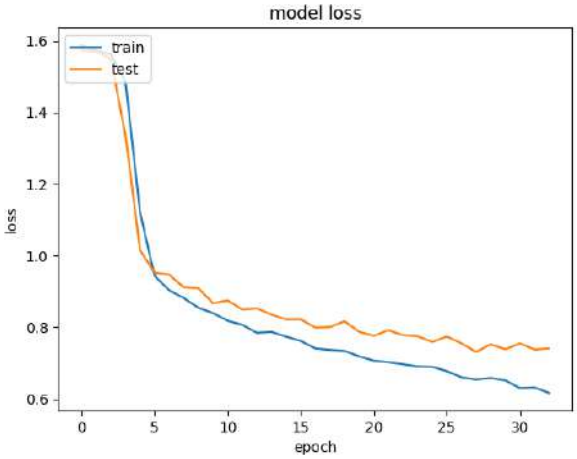
Model	#Features	Sequence Length	Training Accuracy	Validation Accuracy
1	4	128	0.6639	0.6201
2	2	128	0.6372	0.6248
3	4	256	0.3916	0.3989

Table 33: The results of different model configurations.

Among the three settings, models 1 and 2 have a similar performance on the validation set, with a validation accuracy of around 0.62. While model 3 performed significantly worse than them, it achieved a validation accuracy of 0.4 only. The loss and accuracy curves of the three models are shown in Figure 134, Figure 135 and Figure 136.

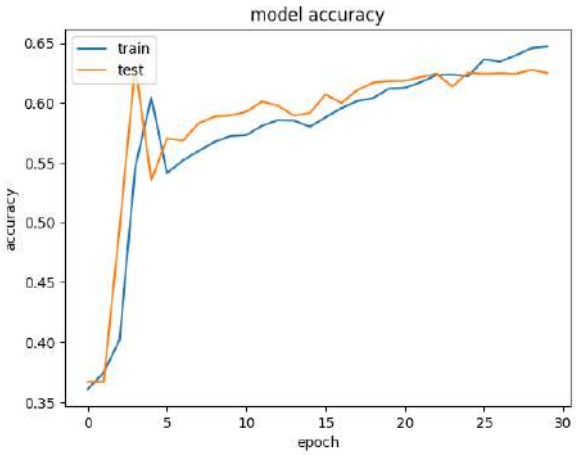


(a) Accuracy

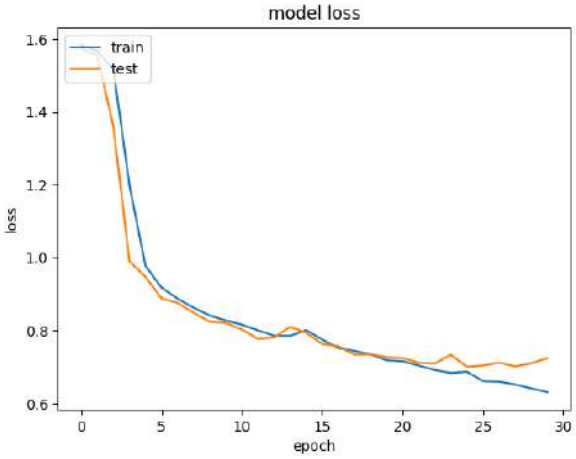


(b) Loss

Figure 134: Model 1 (4 features and 128 sequence length).

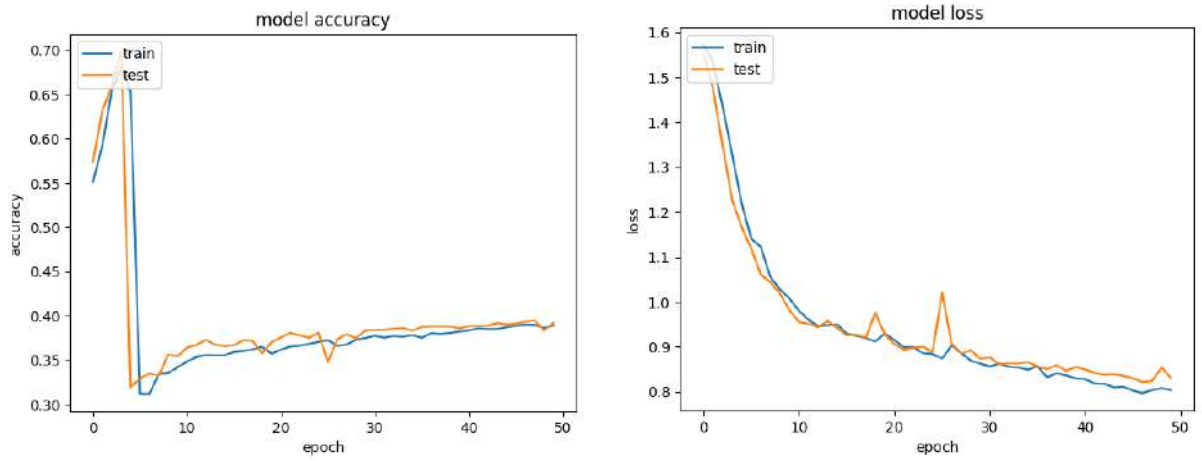


(a) Accuracy



(b) Loss

Figure 135: Model 2 (2 features and 128 sequence length).



(a) Accuracy (b) Loss

Figure 136: Model (4 features and 256 sequence length).

Sample Output A sample output is shown in Figure 137. It can be observed that the model is capable of recognizing the general pitch movement and assigning fingers that match the general trend. It does have some issues such as generating different fingerings for consecutive notes of the same pitch (measure 2) or introducing some unnecessary hand changes (measure 1). However, it appears to be feasible to play and does cater to the hand's ergonomics most of the time.



Figure 137: Sample fingering prediction result.

Result Interpretation

We found two key points worth investigating in this result.

The first one is the poor performance of the model with 256 sequence length. Compared to the other two models with 0.6 validation accuracy, its validation accuracy is only 0.4. Upon observing the accuracy curve, a drastic drop is obvious at around epoch 5. This model

was tuned with different hyperparameters and model architecture (adjusting the number of LSTM layers and their sizes), and the drop appears consistently across all variations. At the same time, the model loss descends smoothly without any significant peaks when the accuracy drops.

A possible reason is that the sequence length of 256 is too long for most snippets in the dataset. Having too many masked values in the sequence may introduce unnecessary complexity and confuse the model, causing a poor result. It may also lead to a tendency to overfit classes mapped for the pads.

The second point is the surprisingly similar performances for models 1 and 2. It seems that the addition of the key color features does not bring any observable advantage to model 2 at all. A possible explanation is that the remaining two features (onset time difference, and note distance) are way more significant in determining the piano fingering. Among the three types of features, the onset time difference is the only time domain feature, the note distance corresponds to the horizontal movement of hands (left and right on the keyboard), while the key color corresponds to the vertical movement of hands (forward and backwards on the keyboard).

However, due to the structure of our hands, it is relatively easier to accommodate vertical differences without changing fingerings, as we can adjust the curvature of our fingers accordingly. As a result, the distinction between black and white keys may be subtle, as both types of keys can be played by the same finger as long as their horizontal positions are similar. In contrast, reaching a significant horizontal distance with a single finger often requires adjusting the hand's position, so it would impact piano fingering to a greater extent.

Another possible explanation is that the model is not complex enough, so it fails to learn the information captured by this feature. However, due to computation resource limitations, we are unable to significantly expand the model to test this hypothesis.

The third point is that the accuracy metrics may not be the best evaluation criteria for this dataset. The dataset contains different annotations for the same piece labelled by different pianists. This means that there may be multiple ground truth labels for the same note sequence.

In the work of Guan et al. [90], several levels of strictness are applied to determine the correctness of a prediction. The strictest measures the average match rate with all ground truths, which is quite similar to the "accuracy" metrics. The second one returns the match rate with the closest ground truth, and the last one considers an estimated fingering as a correct estimation if it matches with any ground truth.

However, validation accuracy, despite being stricter than necessary, is still capable of providing objective evaluation and comparison of model performance. This metric is kept

in this project due to development time constraints, and other developments or features are prioritized.

6.3.8 Inference

Figure 138 shows the inference pipeline.

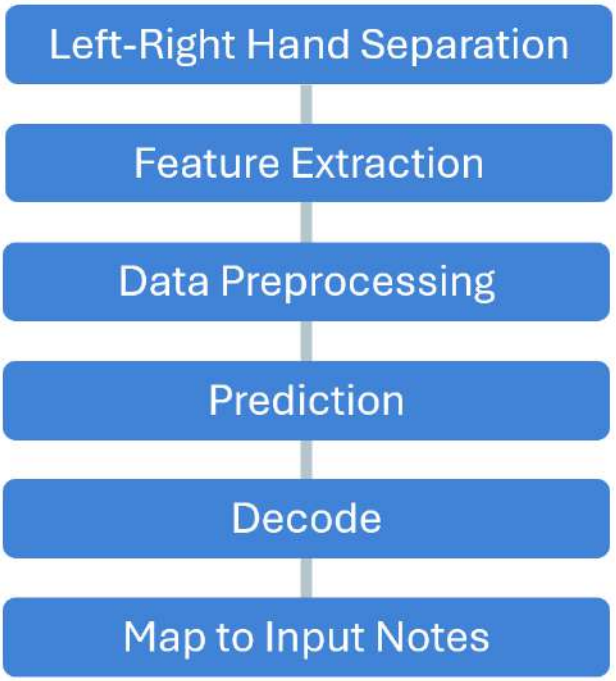


Figure 138: Piano fingering inference pipeline.

Unlike the training data, which provides the hand annotation for us to directly separate the two hands, a manual hand separation is needed for the piece analysis function. A simple separation is performed by grouping all notes below MIDI note number 60 (C4) to be played by the left hand, and the remaining parts are played by the right hand.

The next part is similar to the training pipeline, which converts the note sequences into features and performs data preprocessing. Afterwards, it will be passed through the trained model to predict the piano fingering. The results would be decoded, such as reshaping and removing the paddings, and mapped back to the inputted note sequence. The output piano score would be generated based on this data.

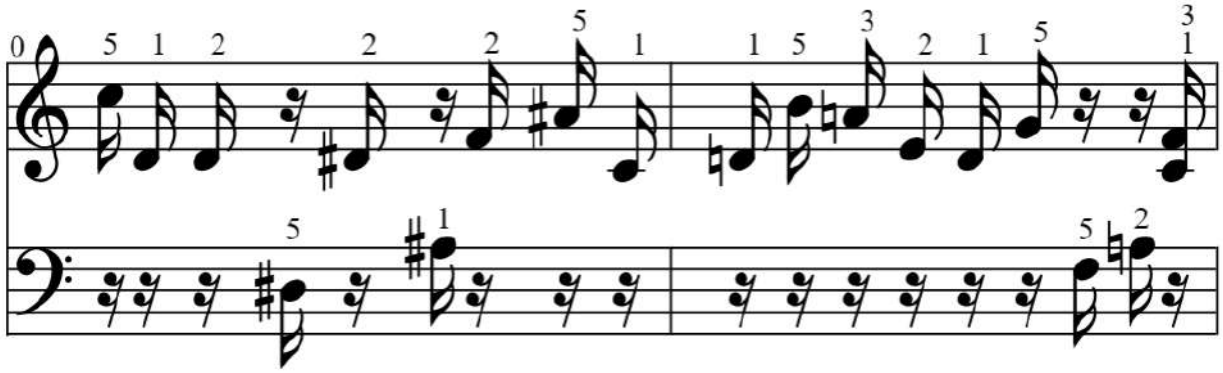


Figure 139: Example score for a music piece generated from style transfer.

Figure 139 shows the generated music score with hand separation and generated piano fingerings. It can be observed that the generated fingerings are generally good and match the note distances even for music generated by the style transfer function.

7 Conclusion

7.1 Summary

To conclude, we successfully built a web application which contains performance, emotion and piece analysis features. The project demonstrated the possibility of automating parts of piano education. It requires both the technical skills to perform in-depth analysis on a broad spectrum of musical features and a thoughtful user interface design to ensure that the complicated analysis can be presented to users in an understandable and meaningful manner.

7.2 Evaluation

Across the whole journey of the final year project, I have explored the fields of computer music, music education, UIUX, music information retrieval, machine learning, and AI, etc. From this project, I have gained a lot of hands-on practical experience which I could implement and exploit the knowledge I have obtained in the four years of studying for my Bachelor's degree in artificial intelligence. Unlike other final-year project groups, we could collaborate with Parsons Music and discuss the topic of music education onsite with Angus and Matthew. However, this has provoked my thinking on how to balance the opinions of others and our development plan. Surprisingly, people who are not in the field of engineering could provide many new perspectives on enhancing user interaction which we never think of. Therefore, sometimes when we are stuck in a bottleneck, we shall go out of our comfort zone and seek more opinions from our supervisors, or even people in different industries.

As a matter of fact, I think that we have been ambitious about the whole planning of our final year project. We thought we could go deep into every function or topic we have planned in our final-year proposal. However, most of the time we have been stopped by some hurdles from one of the functions we plan. For example, we thought that by using some existing algorithms could already achieve a decent accuracy in pitch detection. It turns out that we have overestimated the power of these algorithms. In fact, they work well for synthesized audio data or for monophonic pitch detection tasks. Nonetheless, we have overlooked the effect of noises in recording audio which could lead to a great distortion to the accuracy of these algorithms. We have faced more hurdles like this throughout these two terms. Not to mention we have rebuilt a whole web application in replace to the GUI we made in Term 1. One obstacle like this is not a problem, however, we have faced multivarious troubles when we dug down into the development of each function.

In a nutshell, a well-considered plan and feasibility proposal are very crucial when we are planning a long-lasting project like this final-year project. We shall not overestimate our ability to finish the development in a short time, nor neither underestimate the possible challenges we are going to face when we start working step by step. All in all, I have gained a lot of experience in both the practical, and application side, as well as the more research-based side of developing some AI models. A huge credit to our supervisor Dr. Chau

Chuck-jee , Angus and Matthew from Parsons Music who have given us a chance to develop a smart AI music teacher this year. Thank you!

8 References

[1] J. Ellinger, "Musc 101-04 notation 2." Webpage, 2012.

[2] L. Feldman Barrett and J. A. Russell, "Independence and bipolarity in the structure of current affect.," *Journal of personality and social psychology*, vol. 74, no. 4, p. 967, 1998.

[3] I. B. Arshinova, "Elements of piano education and the evolution of piano pedagogy," *Journal for the Interdisciplinary Art and Education*, vol. 3, no. 2, pp. 81–97, 2022.

[4] D. D. Coffman, "Effects of mental practice, physical practice, and knowledge of results on piano performance," *Journal of Research in Music Education*, vol. 38, no. 3, pp. 187–196, 1990.

[5] E. I. C. Dyal, *An examination of factors which associate with a successful outcome in piano lessons*. Teachers College, Columbia University, 1991.

[6] L. M. Maynard, "The role of repetition in the practice sessions of artist teachers and their students," *Bulletin of the Council for Research in Music Education*, pp. 61–72, 2006.

[7] R. A. Duke, A. L. Simmons, and C. D. Cash, "It's not how much; it's how: Characteristics of practice behavior and retention of performance skills," *Journal of Research in Music Education*, vol. 56, no. 4, pp. 310–321, 2009.

[8] H. Kim, P. Ramoneda, M. Miron, and X. Serra, "An overview of automatic piano performance assessment within the music education context," 2022.

[9] G. Rubin-Rabson, "Studies in the psychology of memorizing piano music: Ii. a comparison of massed and distributed practice.," *Journal of Educational Psychology*, vol. 31, p. 270–284, Apr. 1940.

[10] "MongoDB: The Developer Data Platform — mongodb.com." `https://www.mongodb.com/`.

[11] "Express - Node.js Web &x61C9;&x7528;&x7A0B;&x5F0F;&x67B6;&x69CB; — expressjs.com." `https://expressjs.com/zh-tw/`. [Accessed 14-04-2024].

[12] "React — react.dev." `https://react.dev/`.

[13] "Node.js — Run JavaScript Everywhere — nodejs.org." `https://nodejs.org/`.

[14] "GitHub - axios/axios: Promise based HTTP client for the browser and node.js — github.com." `https://github.com/axios/axios`.

[15] "Mongoose ODM v8.3.1 — mongoosejs.com." `https://mongoosejs.com/`.

[16] "NumPy." `https://numpy.org/`.

[17] "SciPy." https://scipy.org/.

[18] "User Guide — MoviePy 1.0.2 documentation." https://zulko.github.io/moviepy/. Accessed: November 27, 2023.

[19] "FFmpeg." https://ffmpeg.org/.

[20] "Librosa: audio and music processing in python." https://librosa.org/. Website.

[21] "OpenCV - open computer vision library." https://opencv.org/. Website.

[22] "Matplotlib — Visualization with Python." https://matplotlib.org/.

[23] "Mido - MIDI objects for python." https://mido.readthedocs.io/. Website.

[24] "music21: a toolkit for computer-aided musicology." http://web.mit.edu/music21/. Website.

[25] "Free music composition and notation software — MuseScore." https://musescore.org/en.

[26] "PyPI · The Python Package Index." https://pypi.org/.

[27] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, "Crepe: A convolutional representation for pitch estimation," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 161–165, IEEE, 2018.

[28] "Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. — tailwindcss.com." https://tailwindcss.com/.

[29] "Socket.IO — socket.io." https://socket.io/.

[30] "Chart.js — chartjs.org." https://www.chartjs.org/.

[31] "HOME - Open Sheet Music Display — opensheetmusicdisplay.org." https://opensheetmusicdisplay.org/.

[32] Y. L. Wan, Z. G. Wu, R. H. Zhou, and Y. H. Yan, "Automatic transcription of piano music using audio-vision fusion," *Applied Mechanics and Materials*, vol. 333, pp. 742–748, 2013.

[33] Y. Wan, X. Wang, R. Zhou, and Y. Yan, "Automatic piano music transcription using audio-visual features," *Chinese Journal of Electronics*, vol. 24, no. 3, pp. 596–603, 2015.

[34] F. Huang, Y. Zhou, Y. Yu, Z. Wang, and S. Du, "Piano ar: A markerless augmented reality based piano teaching system," in *2011 Third International Conference on Intelligent Human-Machine Systems and Cybernetics*, vol. 2, pp. 47–52, 2011.

[35] A. Goodwin and R. Green, "Key detection for a virtual piano teacher," in *2013 28th International Conference on Image and Vision Computing New Zealand (IVCNZ 2013)*, pp. 282–287, IEEE, 2013.

[36] D. O. Gorodnichy and A. Yogeswaran, "Detection and tracking of pianist hands and fingers," in *The 3rd Canadian Conference on Computer and Robot Vision (CRV'06)*, pp. 63–63, IEEE, 2006.

[37] P. Suteparuk, "Detection of piano keys pressed in video," *Dept. of Comput. Sci., Stanford Univ., Stanford, CA, USA, Tech. Rep*, 2014.

[38] J. Li, W. Xu, Y. Cao, W. Liu, and W. Cheng, "Robust piano music transcription based on computer vision," in *Proceedings of the 2020 4th High Performance Computing and Cluster Technologies Conference & 2020 3rd International Conference on Big Data and Artificial Intelligence*, pp. 92–97, 2020.

[39] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2881–2890, 2017.

[40] M. Akbari, J. Liang, and H. Cheng, "A real-time system for online learning-based visual transcription of piano music," *Multimedia tools and applications*, vol. 77, pp. 25513–25535, 2018.

[41] cvzone, "GitHub - cvzone/cvzone." `https://github.com/cvzone/cvzone`.

[42] P. Grosche, M. Müller, and C. S. Sapp, "What makes beat tracking difficult? a case study on chopin mazurkas.," in *ISMIR*, pp. 649–654, 2010.

[43] S. Sigtia, E. Benetos, and S. Dixon, "An end-to-end neural network for polyphonic piano music transcription," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 5, pp. 927–939, 2016.

[44] L. Liu, V. Morfi, and E. Benetos, "Joint multi-pitch detection and score transcription for polyphonic piano music," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 281–285, IEEE, 2021.

[45] A. M. Noll, "Cepstrum pitch determination," *The journal of the acoustical society of America*, vol. 41, no. 2, pp. 293–309, 1967.

[46] B. Yegnanarayana, K. Murty, B. Y. Guruprasad, K. S. R. Murty, M. Wu, and D. Wang, "Supplementary materials to: Herbst & dunn, fundamental frequency estimation of low-quality electroglottographic signals," 2017.

[47] A. De Cheveigné and H. Kawahara, "Yin, a fundamental frequency estimator for speech and music," *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.

[48] M. Mauch and S. Dixon, "pyin: A fundamental frequency estimator using probabilistic threshold distributions," in *2014 ieee international conference on acoustics, speech and signal processing (icassp)*, pp. 659–663, IEEE, 2014.

[49] C. Thomé and S. Ahlbäck, "Polyphonic pitch detection with convolutional recurrent neural networks," *arXiv preprint arXiv:2202.02115*, 2022.

[50] R. Parncutt, J. A. Sloboda, E. F. Clarke, M. Raekallio, and P. Desain, "An ergonomic model of keyboard fingering for melodic fragments," *Music Perception: An Interdisciplinary Journal*, vol. 14, no. 4, pp. 341–382, 1997.

[51] A. K. Alia, "A simple algorithm for automatic generation of polyphonic piano fingerings," *The International Society for Music Information Retrieval (ISMIR-2007), Vienna (Austria)*, 2007.

[52] J. Zhao and K. Yoshii, "Multimodal multifaceted music emotion recognition based on self-attentive fusion of psychology-inspired symbolic and acoustic features," in *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, pp. 1641–1645, IEEE, 2023.

[53] H.-T. Hung, J. Ching, S. Doh, N. Kim, J. Nam, and Y.-H. Yang, "Emopia: A multimodal pop piano dataset for emotion recognition and emotion-based music generation," *arXiv preprint arXiv:2108.01374*, 2021.

[54] M. B. ER and E. M. ESİN, "Music emotion recognition with machine learning based on audio features," *Computer Science*, vol. 6, no. 3, pp. 133–144, 2021.

[55] C.-J. Chau, R. Mo, and A. Horner, "The emotional characteristics of piano sounds with different pitch and dynamics," *Journal of the Audio Engineering Society*, vol. 64, no. 11, pp. 918–932, 2016.

[56] R. Panda, R. Malheiro, and R. P. Paiva, "Novel audio features for music emotion recognition," *IEEE Transactions on Affective Computing*, vol. 11, no. 4, pp. 614–626, 2018.

[57] R. Panda, R. Malheiro, and R. P. Paiva, "Musical texture and expressivity features for music emotion recognition," in *19th International Society for Music Information Retrieval Conference (ISMIR 2018)*, pp. 383–391, 2018.

[58] W. Koehrsen, "Random forest simple explanation," Aug 2020.

[59] H. Tao, S. Salih, A. Y. Oudah, S. Abba, A. M. S. Ameen, S. M. Awadh, O. A. Alawi, R. R. Mostafa, U. P. Surendran, and Z. M. Yaseen, "Development of new computational machine learning models for longitudinal dispersion coefficient determination: case study of natural streams, united states," *Environmental Science and Pollution Research*, vol. 29, no. 24, pp. 35841–35861, 2022.

[60] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[61] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, ACM, Aug. 2016.

[62] B. Spijker, "Classifying classical piano music into time period using machine learning," Master's thesis, University of Twente, 2020.

[63] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019.

[64] I. Lezhenin, N. Bogach, and E. Pyshkin, "Urban sound classification using long short-term memory neural network," in *2019 federated conference on computer science and information systems (FedCSIS)*, pp. 57–60, IEEE, 2019.

[65] M. Massoudi, S. Verma, and R. Jain, "Urban sound classification using cnn," in *2021 6th international conference on inventive computation technologies (icict)*, pp. 583–589, IEEE, 2021.

[66] C. Weiß, M. Mauch, S. Dixon, and M. Müller, "Investigating style evolution of Western classical music: A computational approach," *Musicae Scientiae*, vol. 23, no. 4, pp. 486–507, 2019.

[67] O. Calzone, "An intuitive explanation of lstm," Apr 2022.

[68] Y. Verma, "Complete guide to bidirectional lstm with python codes," Mar 2024.

[69] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[70] "Welcome to python.org." `https://www.python.org/`. Website.

[71] G. Brunner, A. Konrad, Y. Wang, and R. Wattenhofer, "Midi-vae: Modeling dynamics and instrumentation of music with applications to style transfer," *arXiv preprint arXiv:1809.07600*, 2018.

[72] I. Malik and C. H. Ek, "Neural translation of musical style," 2017.

[73] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, "Stytr2: Image style transfer with transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11326–11336, June 2022.

[74] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[75] G. Brunner, Y. Wang, R. Wattenhofer, and S. Zhao, "Symbolic music genre transfer with cyclegan," in *2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 786–793, 2018.

[76] Q. Kong, B. Li, J. Chen, and Y. Wang, "Giantmidi-piano: A large-scale midi dataset for classical piano music," 2022.

[77] Q. Kong, B. Li, X. Song, Y. Wan, and Y. Wang, "High-resolution piano transcription with pedals by regressing onset and offset times," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[78] C. Raffel and D. P. Ellis, "Intuitive analysis, creation and manipulation of midi data with pretty_midi," in *15th international society for music information retrieval conference late breaking and demo papers*, pp. 84–93, 2014.

[79] H.-W. Dong, W.-Y. Hsiao, and Y.-H. Yang, "Pypianoroll: Open source python package for handling multitrack pianoroll," *Proc. ISMIR. Late-breaking paper*, 2018.

[80] L.-C. Yang, S.-Y. Chou, and Y.-H. Yang, "Midinet: A convolutional generative adversarial network for symbolic-domain music generation," *arXiv preprint arXiv:1703.10847*, 2017.

[81] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "Musegan: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, Apr. 2018.

[82] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition*, CVPR '16, pp. 770–778, IEEE, June 2016.

[83] L. Sulaiman and S. Larsson, "Genre style transfer: Symbolic genre style transfer utilising gan with additional genre-enforcing discriminators," 2022.

[84] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems* (Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, eds.), vol. 27, Curran Associates, Inc., 2014.

[85] R. Parncutt, J. A. Sloboda, E. F. Clarke, M. Raekallio, and P. Desain, "An Ergonomic Model of Keyboard Fingering for Melodic Fragments," *Music Perception*, vol. 14, pp. 341–382, 07 1997.

[86] C.-C. Lin and D. S.-M. Liu, "An intelligent virtual piano tutor," in *Proceedings of the 2006 ACM international conference on Virtual reality continuum and its applications*, VRCIA06, ACM, June 2006.

[87] M. Balliauw, D. Herremans, D. Palhazi Cuervo, and K. Sörensen, "A variable neighborhood search algorithm to generate piano fingerings for polyphonic sheet music," *International Transactions in Operational Research*, vol. 24, no. 3, pp. 509–535, 2017.

[88] Y. Yonebayashi, H. Kameoka, and S. Sagayama, "Automatic decision of piano fingering based on hidden markov models," p. 2915 – 2921, 2007. Cited by: 26.

[89] E. Nakamura, Y. Saito, and K. Yoshii, "Statistical learning and estimation of piano fingering," *Information Sciences*, vol. 517, pp. 68–85, 2020.

[90] H. Guan, Z. Yan, and T. Hsu, "Automatic piano fingerings estimation using recurrent neural networks," 11 2021.

[91] "scikit-learn: machine learning in Python &x2014; scikit-learn 1.4.2 documentation — scikit-learn.org." `https://scikit-learn.org/stable/index.html`.

[92] "PyQt6 · PyPI." `https://pypi.org/project/PyQt6/`.

9 Appendix

9.1 Python Application with Graphical User Interface (Term One)

Our project aims to create a software prototype that allows piano learners to upload their practice or performance videos and receive automatic analytics and feedback. We believe that a positive user experience is key to effective learning, so we chose to build software with a graphical user interface (GUI) that is more user-friendly to make the process of learning piano more approachable and enjoyable.

Therefore, a Python program with a GUI is built to illustrate the system flow and how the analytical results are laid out in a way that is interactive and easy to understand. Since we hope to put the project's focus on analyzing the music instead of building complicated software, a simple interface is built for prototyping.

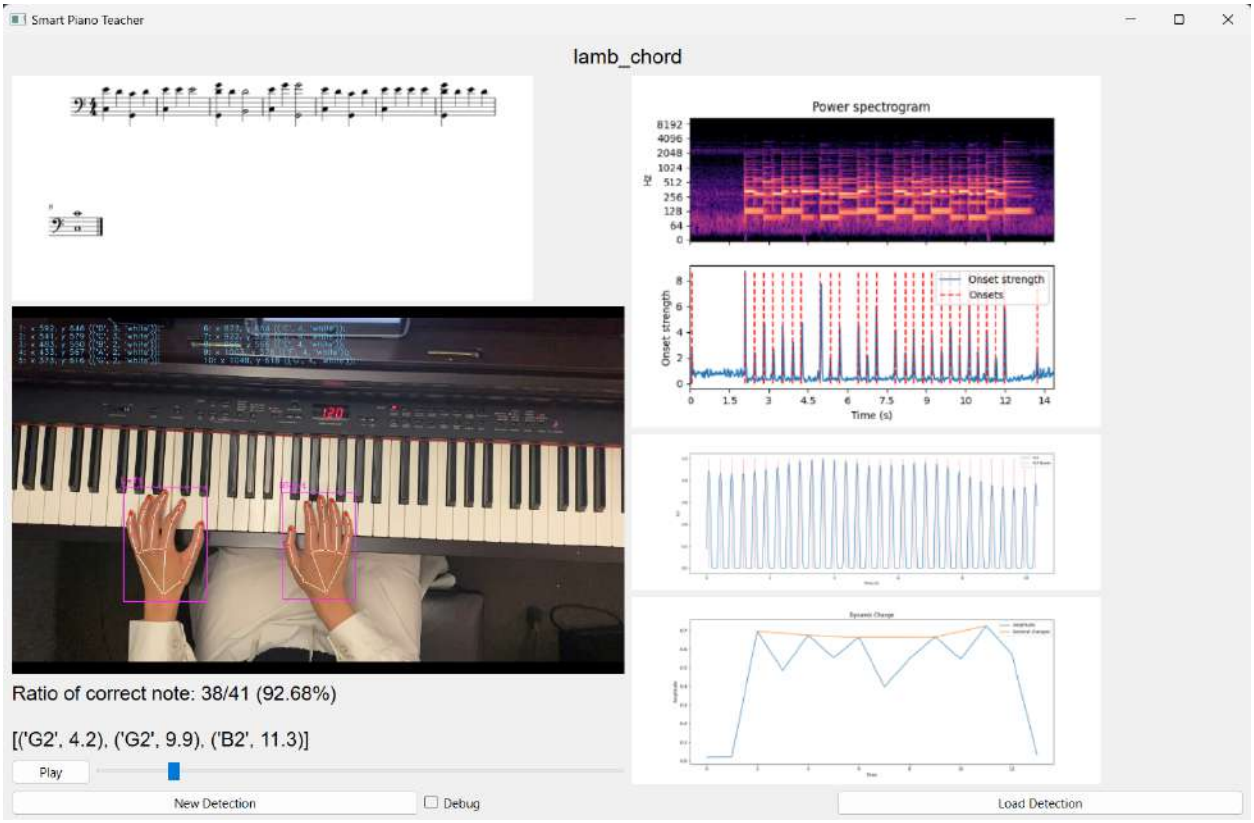


Figure 140: Graphical user interface of the application.

The GUI is mainly built using the PyQt6 [92] package in Python [70]. It provides all the GUI components needed for this prototype and some useful functionalities such as signal firing. More details of each component will be explained in the subsequent sections.

9.1.1 Overall Flow

The flow of the whole application is shown in Figure 141. Users can choose to start a new project and run the analysis on a video and its corresponding midi score specified by the user. The detection results would be stored in a local directory with the same name as the project. They can also choose to revisit previous analytics by loading their detection results from the project path in local storage.

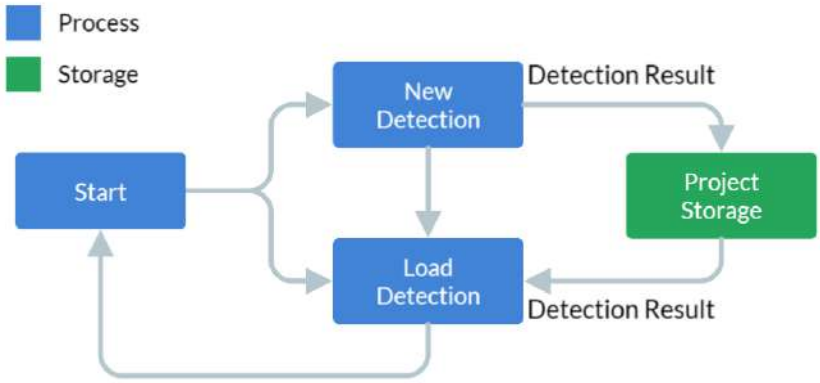


Figure 141: Flowchart of the application.

9.1.2 Detection Result Files

Table 34 shows the detection result files that are generated and stored in the project directory after the detection process is completed.

Filename	Explanation
background.jpg	Piano frame selected for piano key detection.
hand_detection.mp4	Annotated video produced from key-finger mapping.
hand_detection.txt	Key-finger mapping results.
audio.wav	Audio extracted from the input video.
onset_detection.png	Onset plotted with a mel-spectrogram.
pitch_detection.txt	Detected pitch by the algorithm.
overall_accuracy.txt	Error rate of playing and wrongly played notes with timestamps.
output_score.png	Image of piano score generated by the detected notes.
output_score.xml	musicXML score generated by the detected score.
tempo_analysis.png	Graph of PLP function.
dynamics_analysis.png	Graph of dynamics change.

Table 34: Detection result files.

9.1.3 Class Structure

Figure 142 shows the attributes and methods of each class. The piano key extraction functions are implemented in PianoKey and Piano classes, the hand tracking and key-finger

mapping functions are implemented in HandTracker and Hands, and the audio analysis functions are implemented in AudioAnalyser.

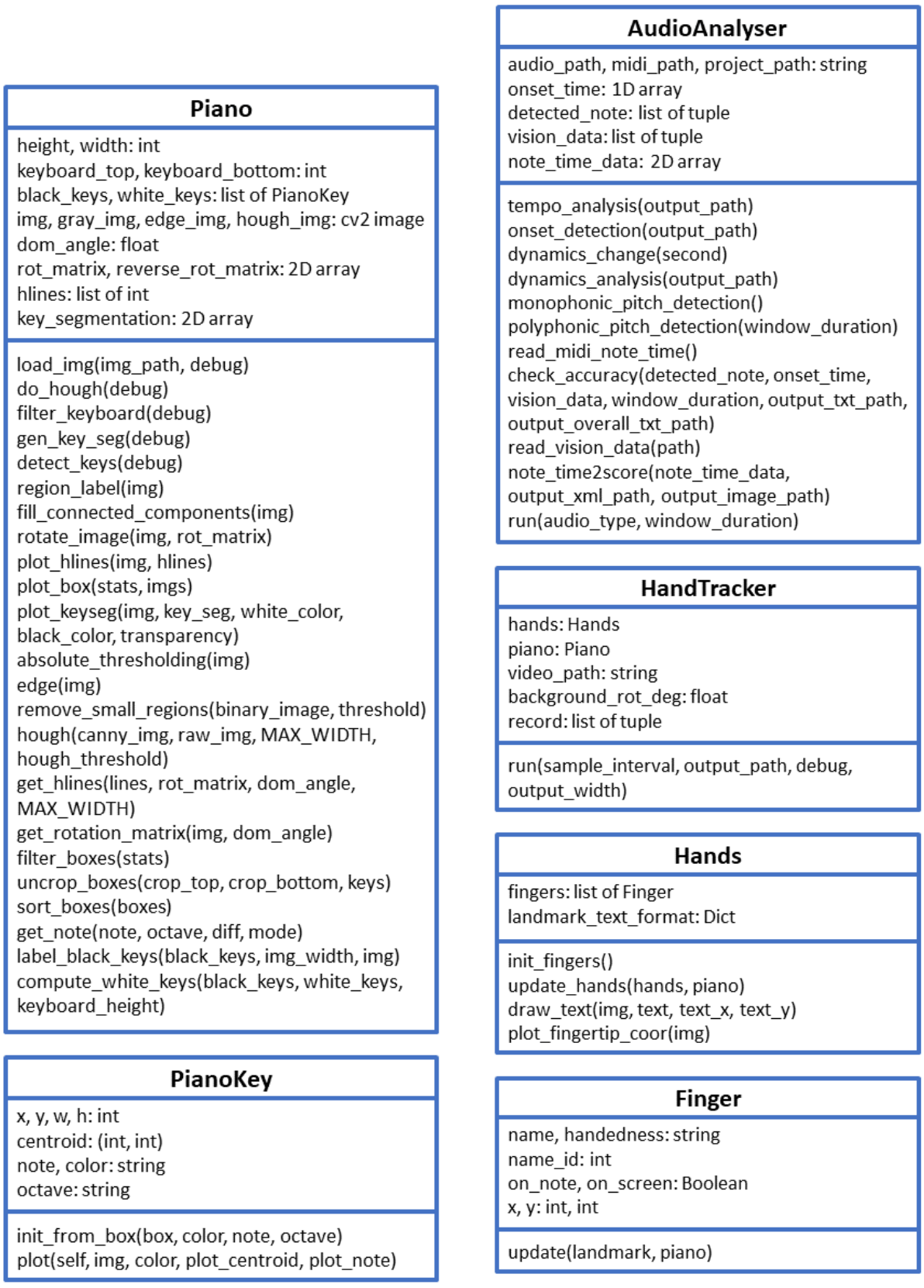


Figure 142: The class diagram of the application.

9.1.4 New Detection

The steps for running the analysis on a video to create a new project are shown in Figure 143. The green words indicate the project files generated in each step.

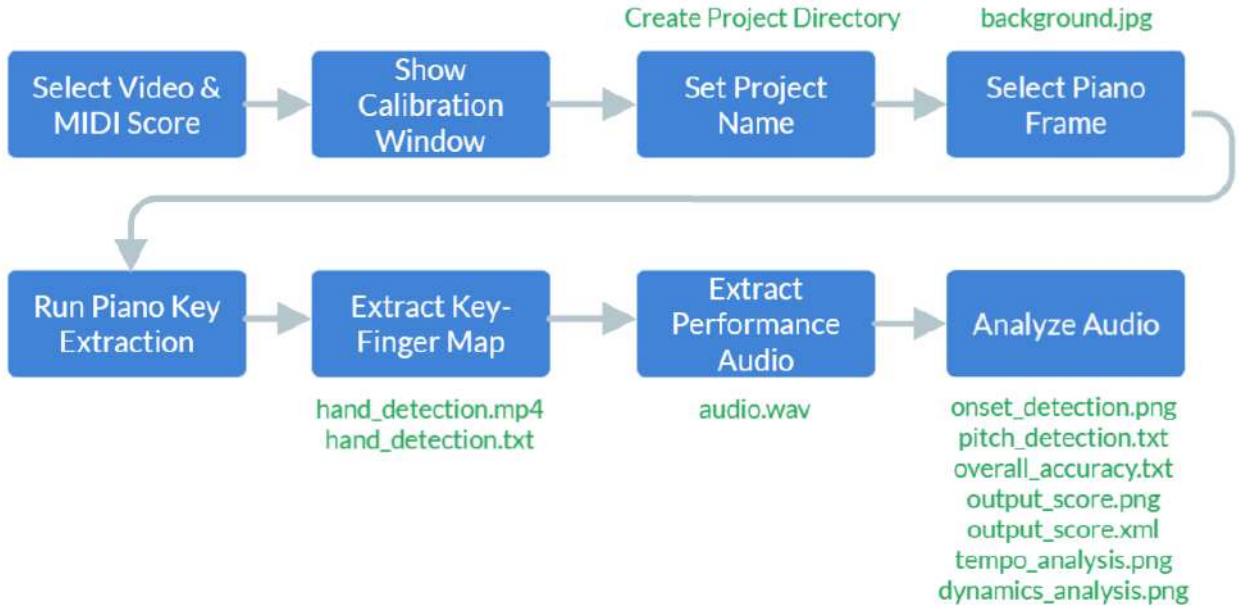


Figure 143: Flowchart for generating new detection.

Select Video and MIDI Score Users are prompted to first select the video of their piano performance to be analyzed. It is followed by the MIDI file of the piano score played to perform accuracy analysis.

Calibration Window A calibration window is then shown (as in Figure 144) to obtain other necessary inputs from the user to create the project. Users can name the project at the input box located at the top. Afterward, they can use the slider to navigate the input video and select a frame where the piano keyboard is not covered by the pianist's hands. The video is rendered and played using the QMediaPlayer() object of PyQt6 [92], and we implemented the slider logic for video seekback. The dropdown menu at the bottom left lets users select the video's orientation, so they do not need to manually rotate their videos before loading them into the application.

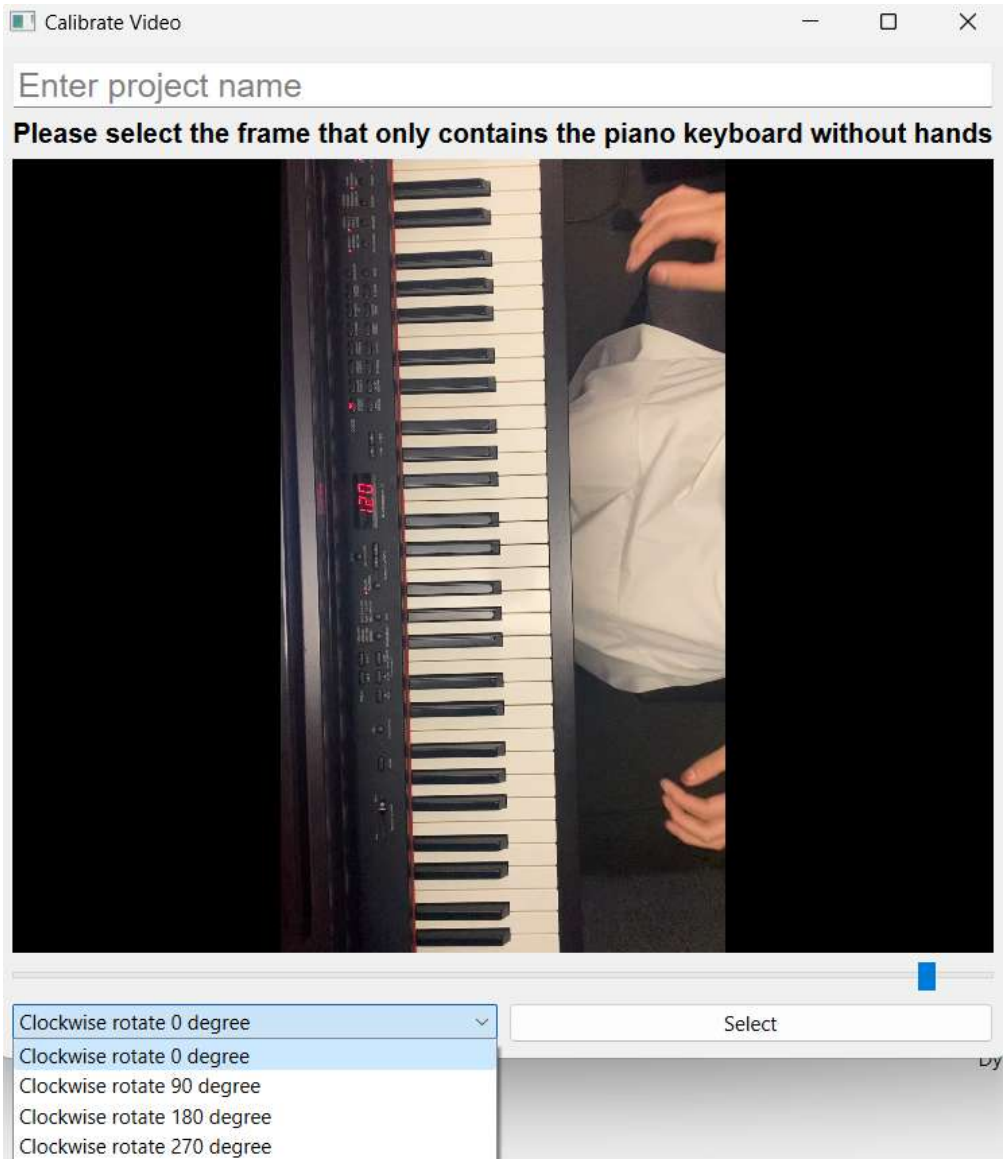


Figure 144: Calibration window.

Key-Finger Mapping Extraction The piano key extraction module is implemented in the Piano class, it extracts the location of each piano key. Afterward, the results are passed to the HandTracker object, which tracks the hand and produces the key-finger mapping.

Extract Performance Audio Afterwards, the MoviePy [18] library is applied to extract the audio part of the performance video for the subsequent audio analysis functions.

Audio Analysis All the audio analysis functions are executed in this step, they include pitch detection, tempo detection, and dynamics extraction which were introduced in previous sections.

The detection flow is then completed and all detection result files are added to the project directory. The new functions built in the next term will be integrated into this flow.

9.1.5 Load Detection

This function loads all files stated in table 34 and displays them for users to have a clear overview of his/her performance. An example of the page is shown in Figure 145.

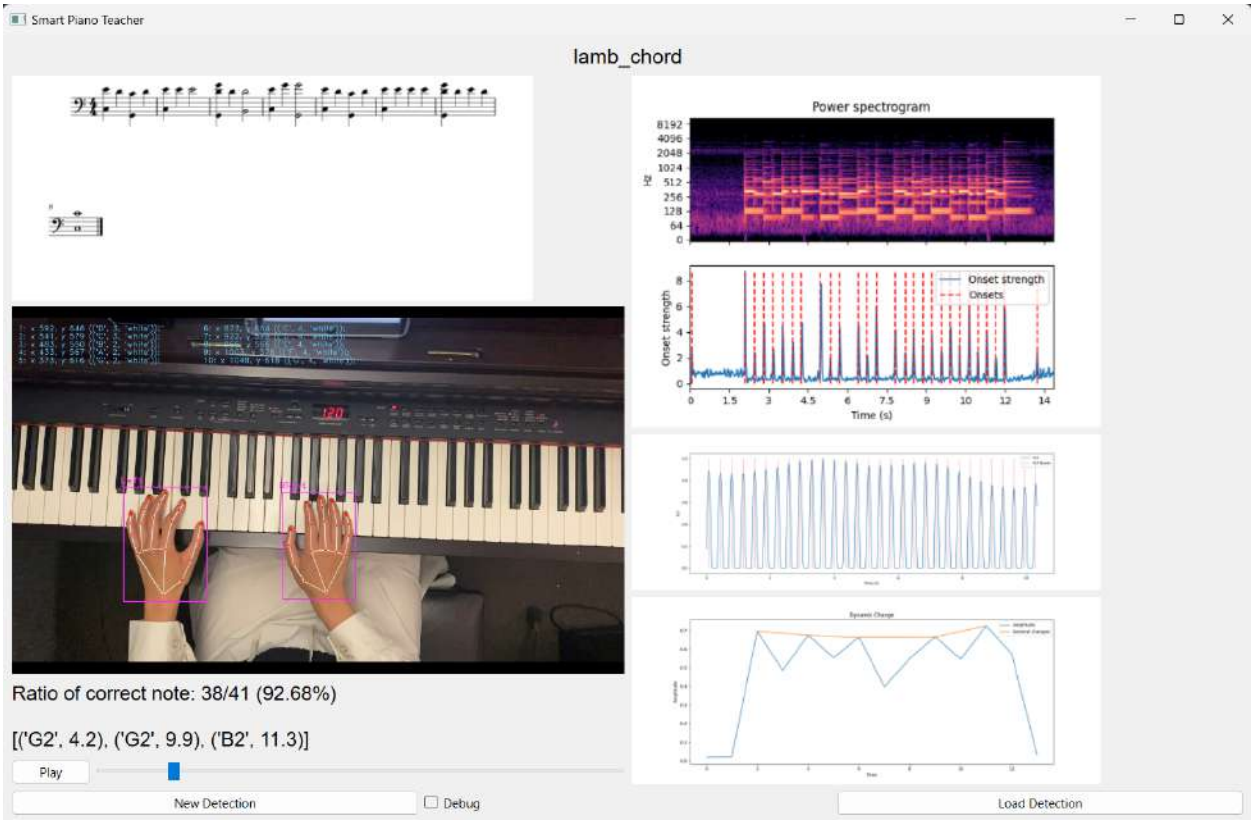


Figure 145: Application page showing the generated analytics.

The four graphs in the right column are the tracks of the power spectrogram, onset, tempo, and dynamic throughout the whole performance, which are conveniently aligned for an easy review.

The music score at the top-left is the music score generated based on pitch detection results of the notes the user played. Meanwhile, the video at the centre of the left column is the annotated hand-tracking playback with the hand skeleton and the mapping of what note each finger is lying on. The play/pause button and the slider are for seeking back the video.

Finally, the bottom left displays the overall accuracy of the notes played, and highlights which notes were played wrong with the timestamp. We plan to update this to display the key-finger mapping results in sync with the video as controlled by the progress bar.

These objective and quantified measurements help users review their performance and locate areas for improvement more precisely. By comparing the analytics of different practices, users can visualize their improvements and explore more possible aspects to work on.

9.1.6 Summary and Further Development

The GUI was successfully built, which was able to connect all the functions we implemented inside one application. While the current layout is able to support all the analytical graphs generated, it is foreseeable that using one single page is insufficient if we'd like to add more features in the next term. Therefore, we will explore the use of multi-page layouts or popup windows to display different analytical information.